Deployment-Oriented Neural Network State-Estimation Methods for Battery Management Systems:

Ageing Estimation, Robustness Benchmarking, Embedded Optimization, and Hardware Integration

submitted by
M. Sc.
Florian Rzepka

Chair of Electrical Energy Storage Technology
Institute of Energy and Automation Technology
Faculty IV, Technische Universität Berlin
in partial fulfillment of the requirements for the academic degree of

Doctor of Engineering
Dr.-Ing.

Doctoral thesis

Doctoral Committee:
Prof. Dr.-Ing. Julia Kowal
Prof. Dr.-Ing. Frank Opferkuch
Prof. Dr.-Ing. Weihan Li
Date of the scientific defense: tbd

Berlin, 2026

Preface

This research work took shape during my time at the Chair of Electrical Energy Storage Technology at Technische Universität Berlin. Its questions emerged gradually from the research environment there and from my own interest in how artificial intelligence can support battery-state estimation and battery management under real operating conditions rather than in idealized settings. This direction was reinforced by my involvement in the MG-FARM project, an international research collaboration on smart, stand-alone micro-grids for the electrification of remote agricultural regions. Working alongside university and industry partners in Morocco, France, and Algeria, I was able to see at first hand how decentralized energy systems and their battery storage are deployed and operated in places where grid access, connectivity, and maintenance are limited. Taken together, these influences shaped the central question of this thesis: how battery-state estimation can be made not only accurate, but also robust against imperfect measurements and executable on the modest embedded hardware that such systems actually rely on.

I am grateful for the support I received in many forms throughout this work. My sincere thanks go above all to my supervisor, Prof. Dr.-Ing. Julia Kowal, and to the Chair of Electrical Energy Storage Technology at Technische Universität Berlin, for the guidance and the freedom to pursue this topic in my own way. I thank my colleagues at the chair for countless discussions and for a productive working atmosphere, and the partners of the MG-FARM consortium across Morocco, France, and Algeria for their experience and a genuinely international perspective on decentralized energy. My path to this point began long before this work. I am deeply grateful to my parents, who supported me from my school years through university and gave me the confidence to follow this path. I also thank my family and friends for their understanding and companionship. I am especially grateful to my partner Hannah, whose optimism, patience, and close support helped me regain confidence and perspective during the most difficult phases of this work.

Abstract

Reliable estimates of state of charge (SOC) and state of health (SOH) are central to the safe and efficient operation of lithium-ion battery systems. At the same time, many neural battery estimators are still evaluated mainly as offline models under clean laboratory signals, whereas practical battery management systems (BMS) impose additional requirements through disturbed measurements, limited embedded resources, firmware constraints, and the physical measurement chain. This research therefore develops a deployment-oriented perspective on artificial intelligence for BMS applications. Predictive accuracy is treated as one requirement among others, and is assessed together with robustness, embedded feasibility, and hardware integration.

The work is organized around three connected contribution studies grounded in NMC and LFP ageing datasets and supported by a custom laboratory BMS platform. First, a neural ageing-estimation study shows that a compact multilayer perceptron can estimate SOH with competitive accuracy when cumulative-current information and lag-sequence history are used to represent ageing-relevant temporal context. Second, a common benchmark compares direct-measurement, hybrid, equivalent-circuit, and data-driven SOC estimators. Its robustness branch separates nominal accuracy, resistance to disturbed or incomplete measurements, and recovery after state inconsistency. The data-driven representative is a structurally pruned GRU, and its hardware branch measures numerical equivalence, inference latency, flash occupancy, and peak runtime RAM on an STM32 microcontroller. Exact rolling-window recomputation, continuous hidden-state forwarding, and periodic state reset are compared to distinguish network compactness from the consequences of recurrent-state handling. Third, an embedded-AI study transfers stateful recurrent SOC and SOH estimators to the microcontroller through structured pruning and INT8 weight quantization, measuring retained accuracy and resource demand after compression and deriving a clearly identified constant-power energy

proxy. The custom BMS platform provides the hardware and firmware context in which estimator exchangeability, sensing, and embedded validation can be investigated reproducibly.

Taken together, the studies show that neural battery-state estimators can combine expressive temporal modelling with the compactness required for embedded BMS deployment when representation, architecture, evaluation, and implementation are developed as one connected design problem. Useful temporal context does not necessarily require increasingly large networks. Lag-sequence and cumulative-current features can extend a compact MLP, while recurrent models can retain long-term context and still be reduced substantially through structured pruning and quantization. The resulting SOC and SOH estimators preserve strong estimation quality and sequential behaviour while meeting the memory and timing constraints of the selected microcontroller. The central contribution is therefore a deployment-oriented methodology for assigning model complexity where it improves battery-state representation, identifying weaknesses under realistic disturbances, and systematically reducing the remaining computational and memory demand for reliable integration into practical BMS hardware.

Table of Contents

Preface III

Abstract V

Abbreviations XIII

Glossary XVII

Symbols XXI

List of Figures XXVII

List of Tables XXX

1 Introduction and Motivation 1

1.1 Research Aim and Questions 1

1.2 Contribution, Scope, and Publication Basis 2

2 Battery-Management Requirement Framework 5

2.1 Performance requirements 6

2.2 Hardware requirements 6

2.3 Deployment requirements 7

3 State of the Art and Technical Foundations 9

3.1 Lithium-Ion Cell Fundamentals 9

3.1.1 NMC and LFP cell chemistry 10

3.1.2 Ageing Effects and State-of-Health Definitions 11

3.2 Battery Management Systems and Battery States 11

3.3 Model-Based Estimation and Kalman Filtering 12

3.4 Machine Learning and Neural-Network Foundations 14
3.4.1 Supervised regression, features, and labels 14
3.4.2 Artificial neurons and multilayer perceptrons 15
3.4.3 Recurrent neural networks 17
3.4.4 Comparison with Kalman-filter-based estimation 21
3.5 Embedded AI and Model Compression 23
3.5.1 Pruning 23
3.5.2 Quantization 34
3.6 Embedded Hardware and Model-Hardware Co-Design 43
3.7 Research Gap and Contribution Scope 46

4 Experimental Basis and Hardware Ecosystem 49
4.1 Data Sources and Ageing Campaigns 49
4.1.1 NMC Dataset for Stationary-Storage Ageing Estimation . . 50
4.1.2 LFP Dataset for Robustness Benchmarking and Embedded Deployment 52
4.1.3 SOC and SOH Reference-Label Construction 57
4.2 Embedded Targets and Execution Chain 58
4.3 Custom BMS Platform 59
4.3.1 System Objective and Laboratory Scope 60
4.3.2 Hardware Architecture 61
4.3.3 Firmware and Execution Concept 62
4.3.4 Experimental Relevance for Embedded AI 63

5 Temporal Feature Engineering for Neural SOH Estimation in Stationary Storage Systems 65
5.1 Study Objective and Relation to Prior Work 65
5.2 Temporal Feature Engineering and Data Preparation 67
5.2.1 Cumulative Current 67
5.2.2 Correlation Analysis 68
5.2.3 Formation of the Lag Sequence 69
5.2.4 Training, Validation, and Test Splits 71
5.2.5 Data Scaling 72
5.3 MLP Selection and Architecture 73
5.4 Evaluation Metrics and Hyperparameter Optimization 75
5.5 SOH Estimation Results and Effects of Temporal Representation . . 77

5.6 Conclusions on Feature-Based SOH Estimation 80

6 Performance and Robustness Benchmarking Across Battery State-of-Charge Estimator Classes 83

6.1 Study Objective and Benchmark Scope 83
6.2 Estimator Portfolio and Benchmark Methodology 84
6.2.1 Estimator representatives and class boundaries 84
6.2.2 SOC estimation methods 86
6.2.3 Shared SOH estimation method 89
6.2.4 Model and target-hardware preparation 89
6.2.5 Robustness benchmark design 90
6.2.6 Metrics and evaluation criteria 93
6.2.7 HECM lookup-table sensitivity analysis 94
6.3 Feature Reconstruction and Experimental Protocol 95
6.4 Performance and Robustness Results 97
6.4.1 Baseline performance 97
6.4.2 Current-gain sensitivity 98
6.4.3 Additive sensor offsets 100
6.4.4 Noise sensitivity 101
6.4.5 Initialization error and recovery 103
6.4.6 Missing samples and signal integrity 104
6.4.7 Spike sensitivity 107
6.4.8 ADC quantization 108
6.4.9 Diagnostic comparison with ground-truth SOH 109
6.4.10 HECM lookup-table sensitivity 110
6.4.11 Robustness synthesis across representatives 110
6.4.12 Embedded performance and recurrent execution modes . . . 112
6.5 Discussion 116
6.5.1 Structural interpretation across estimator classes 116
6.5.2 Accuracy, robustness, and recovery as separate requirements 118
6.5.3 Implications for BMS deployment 119
6.5.4 Limitations and future extensions 120
6.6 Conclusion 121

7 Efficient Neural SOC and SOH Estimation for Embedded Systems 123

7.1 Embedded Deployment Objective 123

7.2 Feature Engineering and Online Processing . . . 124
7.3 LSTM Architecture and Training Setup . . . 127
7.4 Structured Pruning Configuration . . . 130
7.5 INT8 Quantization and Manual STM32 Export . . . 133
7.6 Embedded Benchmark Methodology and Evaluation Metrics . . . 135
7.6.1 Transient Input-Buffer and Report-Loss Tests . . . 135
7.6.2 STM32 Measurement Scope and Evaluation Indicators . . . 136
7.7 Streaming Prediction Accuracy . . . 137
7.8 Temporal Stability and SOH Filter Interaction . . . 139
7.9 Error Distribution and Local Transient Behaviour . . . 140
7.10 Software-Level Fault-Sensitivity Results . . . 143
7.11 Resource Efficiency, Latency, and Trade-Off Summary . . . 144
7.12 Discussion of Embedded Compression Results . . . 148
7.12.1 Compression Behaviour of SOC and SOH Estimators . . . 148
7.12.2 Embedded Operating Points . . . 149
7.12.3 Latency, Utility Score, and BMS Deployment . . . 150
7.12.4 Limitations and Future Extensions . . . 150
7.13 Embedded Deployment Conclusions and Outlook . . . 151

8 General Discussion, Conclusion, and Outlook 153
8.1 Integrated Synthesis of the Research Questions . . . 153
8.2 Findings Across BMS Requirements . . . 155
8.3 Transferable Design Lessons . . . 156
8.4 Position Within the State of Research and Practical Implications . . . 158
8.5 Limitations and Generalization Boundaries . . . 158
8.6 Overall Conclusion . . . 160
8.7 Outlook . . . 161

Bibliography 163

A Supplementary Robustness and Embedded-Compression Analyses 185
A.1 Supplementary Robustness-Benchmark Material . . . 185
A.1.1 HECM lookup-table sensitivity . . . 187
A.1.2 Supplementary numerical results . . . 189
A.2 Supplementary Analyses for Embedded Model Compression . . . 191
A.2.1 Diagnostics of the Structured-Pruning Step . . . 192
A.2.2 Long-Horizon Behaviour of the Compressed Models . . . 193

A.2.3 Interaction Between SOH Compression and Filtering 194
A.2.4 Sensitivity of the Composite Utility Score 195
A.2.5 Transient Input-Buffer Fault Sensitivity 196
A.2.6 Model-Complexity Scaling Under Structured Pruning 197
A.2.7 Scope of the ℓ_2-Norm Pruning Criterion 198
A.2.8 Precision Boundary of the Quantized Network 199
A.2.9 Static Counts and Measured Quantized Runtime 200

Abbreviations

2RC	Two-resistor-capacitor equivalent circuit model
ADC	Analog-to-digital converter
AFE	Analog front end
AI	Artificial Intelligence
ASIC	Application-specific integrated circuit
BMS	Battery Management System
CC-CV	Constant-current constant-voltage charging
CPU	Central processing unit
CV	Constant voltage
DD	Data-driven estimator
DM	Direct-measurement estimator
DoD	Depth of discharge
DSP	Digital signal processor
DWT	Data Watchpoint and Trace
ECM	Equivalent Circuit Model
EFC	Equivalent Full Cycles
EMI	Electromagnetic interference
FP16	16-bit floating-point format

FP32	32-bit floating-point format
FPGA	Field-programmable gate array
GELU	Gaussian error linear unit
GPU	Graphics processing unit
GRU	Gated recurrent unit
HDM	Hybrid direct-measurement model
HECM	Hybrid equivalent-circuit model
HPPC	Hybrid pulse-power characterization
IC	Integrated circuit
INT8	8-bit signed integer representation
KPI	Key performance indicator
LFP	Lithium iron phosphate
LSTM	Long Short-Term Memory
MAE	Mean absolute error
MCU	Microcontroller Unit
MG-FARM	Smart micro-grid research project for remote agricultural regions
MLP	Multilayer Perceptron
MSE	Mean squared error
NMC	Lithium nickel manganese cobalt oxide
NPU	Neural processing unit
NTC	Negative temperature coefficient thermistor
OCV	Open-circuit voltage
ONNX	Open Neural Network Exchange

P95	95th percentile
PV	Photovoltaic
RAM	Random-access memory
RC	Resistor-capacitor
ReLU	Rectified linear unit
RGB	Red-green-blue color representation
RMSE	Root mean squared error
SRAM	Static random-access memory
STM32	STMicroelectronics 32-bit microcontroller family
STM32H7	STM32 high-performance microcontroller family based on Arm Cortex-M7
STM32H753ZI	STM32H7 microcontroller target used for the embedded benchmark
STM32N6	STM32 microcontroller family with integrated neural accelerator
SOC	State of Charge
SOC-GRU	SOC estimator based on a GRU recurrent core
SOH	State of Health
SOH-LSTM	SOH estimator based on an LSTM recurrent core
TPU	Tensor processing unit
TU	Technische Universität
UART	Universal asynchronous receiver-transmitter
`.bss`	Linker section for zero-initialized or uninitialized static and global data
`.data`	Linker section for initialized static and global data copied from flash to RAM

Glossary

Accuracy Agreement between an estimated battery state and its reference value. The required accuracy depends on the state variable, reference uncertainty, and the decision for which the estimate is used.

Baseline condition Evaluation condition with the nominal, undisturbed measurement stream. It provides the reference against which disturbance-induced changes are quantified.

Causal processing Processing that uses only information available at the current and previous samples. Future samples are unavailable during online execution.

Cell chemistry Combination of electrode and electrolyte materials that determines characteristic voltage behaviour, energy density, ageing, and safety properties. This work considers NMC and LFP cells.

Coulomb counting SOC estimation by integrating measured current over time and relating the accumulated charge to an assumed usable capacity.

Data-driven estimator Estimator whose input-output relationship is learned from representative measurements and reference labels rather than being prescribed completely by physical equations.

Deployment Transfer of an estimation method from model development into an executable target implementation, including numerical conversion, memory layout, timing, firmware integration, and validation.

Disturbance Deliberate or naturally occurring deviation from the nominal measurement stream, such as bias, noise, missing samples, irregular timing, dropout, or a transient spike.

Embedded inference Execution of a trained model on a resource-constrained controller as part of the operational signal-processing chain.

Feature Input quantity supplied to an estimator. A feature can be measured directly or derived causally from measurements, for example cumulative charge or a signal derivative.

Flash memory Non-volatile memory used for firmware, model parameters, and persistent lookup data. Its contents remain available when power is removed.

Label Reference target assigned to a training or evaluation sample. SOC and SOH labels are derived from measured quantities and a specified reference construction.

Model-based estimator Estimator that propagates battery states through an explicit physical or equivalent-circuit model and can correct them using measurement residuals.

Online estimation Sequential state estimation during operation, where each output is produced from information available up to the current sample.

Pruning Reduction of a trained neural network by removing weights or complete structures judged to be less important for the learned mapping.

Quantization Representation of model quantities with a restricted numerical grid, typically using fewer bits than the original floating-point representation.

Random-access memory Volatile runtime memory used for recurrent states, intermediate activations, buffers, stack, and other working data.

Recurrent state Internal memory propagated between consecutive samples of a recurrent neural network. It carries temporal information that is not contained in the current input alone.

Robustness Ability of an estimator to limit performance degradation under disturbances and non-ideal operating conditions.

State estimator Algorithm that reconstructs a battery state that cannot be measured directly from available measurements and prior information.

Structured pruning Pruning of complete channels, neurons, rows, columns, or other coherent groups so that the resulting tensors and runtime operations can be physically reduced.

Utility score Normalized combination of several evaluation dimensions used to compare deployment alternatives under an explicitly defined set of priorities.

Symbols

α	Smoothing factor of the causal SOH post-filter
ΔI	Current quantization step
ΔMAE	Difference in mean absolute error relative to the baseline case
Δt	Sampling interval
ΔT	Temperature quantization step
ΔU	Voltage quantization step
η	Coulombic efficiency factor in the equivalent-circuit observer
σ_I	Standard deviation of injected current noise
σ_T	Standard deviation of injected temperature noise
σ_U	Standard deviation of injected voltage noise
τ_1, τ_2	Time constants of the two RC branches
$\mathbf{A}$	State-transition matrix of the equivalent-circuit observer
$\mathbf{b}$	Input vector of the equivalent-circuit observer
$\mathbf{c}_t$	LSTM cell state at sample t
$\mathbf{F}_k$	Jacobian of the nonlinear state-transition function in the EKF
$\mathbf{f}_t$	LSTM forget-gate activation
$\mathbf{H}_k$	Jacobian of the nonlinear measurement function in the EKF
$\tilde{\mathbf{c}}_t$	LSTM candidate cell state at sample t

$\mathbf{h}_t$	Recurrent hidden state at sample t
$\tilde{\mathbf{h}}_t$	GRU candidate hidden state at sample t
$\mathbf{i}_t$	LSTM input-gate activation
$\mathbf{K}_k$	Kalman gain at discrete time index k
$\mathbf{o}_t$	LSTM output-gate activation
$\mathbf{P}_k$	State-estimation error covariance in the EKF
$\mathbf{Q}_k$	Process-noise covariance in the EKF
$\mathbf{r}_t$	GRU reset-gate activation
$\mathbf{R}_k$	Measurement-noise covariance in the EKF
$\mathbf{x}$	State vector or model input vector, depending on context
$\tilde{\mathbf{y}}_k$	Measurement innovation or residual in the EKF
$\mathbf{z}_k$	Measurement vector used by the EKF correction step
$\mathbf{z}_t$	GRU update-gate activation
b	Neural-network bias vector
$C(t)$	Capacity measured at time t
C_b	SOH-scaled capacity used by the equivalent-circuit observer
C_{app}	Apparent capacity used in SOC reference reconstruction
C_{chg}	Charge C-rate in the LFP ageing design
C_{dis}	Discharge C-rate in the LFP ageing design
C_{eff}	Effective capacity obtained from nominal capacity and estimated SOH
C_k	Measured discharge capacity during check-up k
C_{N}	Nominal capacity of the NMC cell
C_{nom}	Nominal cell capacity

E	Energy per inference used in the embedded utility score
F	Flash-memory footprint used in the embedded utility score
H	Recurrent hidden-state size
I	Current
I_k	Current sample at discrete time index k
$\lvert I\rvert_{\max}$	Maximum absolute current magnitude used to scale current-bias scenarios
k	Discrete time index
L_{SOC}	SOC training sequence length
m_k	Median of feature channel k used for robust normalization
n	Number of evaluated samples
q	INT8 quantized weight code
Q	Cumulative charge or charge throughput
Q_c	Online cumulative charge state
s_k	Stored scale of feature channel k used for robust normalization
R	Total RAM usage in the embedded utility score
R_0, R_1, R_2	Ohmic and polarization resistances of the equivalent-circuit model
R^2	Coefficient of determination
s	Quantization scale factor
s_h	Structured-pruning importance score of hidden channel h
SOC	State of charge
SOH	State of health
t	Time
T	Temperature

U	Voltage or composite utility score, depending on context
U_1, U_2	Polarization voltages of the two RC branches
W	Neural-network weight matrix
w	Individual neural-network weight
$\hat{w}$	Reconstructed floating-point weight after INT8 quantization
$x_{t,k}$	Feature value of channel k at sample t
y_i	Measured target value of sample i
$\hat{y}_i$	Predicted target value of sample i
$\mathrm{d}I/\mathrm{d}t$	Current derivative
$\mathrm{d}U/\mathrm{d}t$	Voltage derivative
DoD	Depth of discharge
EFC	Equivalent full cycles
MAE	Mean absolute error
MSE	Mean squared error
OCV	Open-circuit voltage
P95	95th percentile of the absolute error
RMSE	Root mean squared error

List of Figures

2.1 BMS requirement framework. 6

3.1 Families of battery-state estimators. 12
3.2 General structure and layer-wise mapping of a multilayer perceptron. 16
3.3 Detailed data flow through an LSTM cell. 18
3.4 Detailed data flow through a GRU cell. 20
3.5 Comparison of unstructured and structured pruning. 25
3.6 Dependency constraints in structured pruning. 27
3.7 Quantizable neural-network quantities and uniform 8-bit grids. . . . 35

4.1 SOH progression across the 14 NMC cells. 52
4.2 LFP DoE operating-point design. 54
4.3 Representative LFP trajectory structure. 54
4.4 SOH progression and data split across the LFP ageing campaign. . 55
4.5 Coverage of the independent LFP holdout test set. 57
4.6 Custom BMS Platform board-level view. 61
4.7 Custom BMS Platform system block view. 62

5.1 Correlation heatmap for SOH-related input variables. 69
5.2 Lag-sequence construction for supervised MLP samples. 70
5.3 Workflow from preprocessing to SOH evaluation. 72
5.4 MLP input and output structure for SOH estimation. 74
5.5 MAE matrix for lag length and time resolution. 78
5.6 Predicted vs. measured SOH trajectories. 79
5.7 Predicted vs. measured SOH scatter. 79

6.1 Methodology of the causal performance and robustness benchmark. 86
6.2 Taxonomy of benchmark disturbances. 91
6.3 Representative-window and causal evaluation protocol. 96

6.4 Baseline accuracy across the independent holdout test set. 98
6.5 Current-gain sensitivity across the holdout test set. 99
6.6 Current-gain behavior over a continuous ageing trajectory. 100
6.7 Local and global current-noise response. 102
6.8 Repeated noise robustness across six holdout cells. 103
6.9 Recovery from an SOC initialization mismatch. 104
6.10 Signal-integrity and timing stress. 106
6.11 Response to a one-hour burst dropout. 106
6.12 Voltage-spike response across the holdout test set. 107
6.13 Event-aligned voltage-spike transient. 108
6.14 ADC-quantization sensitivity. 109
6.15 Cross-scenario disturbance penalties. 111
6.16 Decision-oriented benchmark synthesis. 112
6.17 Numerical equivalence of hardware and software execution. 113
6.18 On-device inference-latency distributions. 114
6.19 Embedded memory requirements of the isolated SOC implementations. 115
6.20 DD inference-mode trade-off on the STM32. 116

7.1 Stateful LSTM+MLP estimator structure. 127
7.2 Task-specific SOC and SOH estimator architectures. 128
7.3 Embedded deployment pipeline. 130
7.4 Illustrative numerical example of gate-consistent LSTM pruning. . . 132
7.5 Row-wise weight quantization and embedded export. 134
7.6 Streaming SOC dashboard. 139
7.7 Streaming SOH dashboard. 140
7.8 SOC error distribution. 141
7.9 SOH error distribution. 142
7.10 Pulse-region SOC analysis. 142
7.11 Check-up-region SOC analysis. 143
7.12 Resource landscape of embedded variants. 147
7.13 End-to-end STM32 host-latency distributions. 148

A.1 Complete robustness-benchmark scenario matrix. 186
A.2 Latency distributions of the three DD execution modes. 187
A.3 HECM lookup-table sensitivity. 188
A.4 Diagnostics of the SOC pruning step. 192

A.5 Segment-wise temporal comparison. 193
A.6 SOH output at successive filtering stages. 194
A.7 Utility-score sensitivity to application priorities. 195
A.8 Response to one-sample input-buffer faults. 196
A.9 Analytical complexity scaling. 197
A.10 Scope of the selected pruning criterion. 198
A.11 Mixed-precision boundary of the quantized model. 199
A.12 Static and measured quantized-runtime comparison. 200

List of Tables

1.1 Research components and publication basis. 3

3.1 Estimation-relevant differences between NMC and LFP cells. 10
3.2 Comparison of Kalman-filter and neural battery-state estimators. . 22
3.3 Comparison of unstructured and structured pruning. 26
3.4 Principal pruning and sparse-training strategies. 31
3.5 Representative software support for neural-network pruning. 33
3.6 Granularity of neural-network quantization. 38
3.7 Principal neural-network quantization strategies. 40
3.8 Representative software support for neural-network quantization. . . 42
3.9 Functional overview of computing platforms for neural-network workloads. The categories describe their dominant implementation principle and can overlap within one device. 45

4.1 Overview of the two battery datasets. 50
4.2 Stationary-storage ageing scenarios. 51
4.3 Measured coverage and exact cell-to-set assignment for the fixed training, validation, and independent holdout test split. The 95th-percentile absolute C-rate is calculated over each complete 1-Hz trajectory. Load classes are defined only for the holdout test set. Its high-load class contains one cell and is exploratory. 56

5.1 Final tuned MLP layer configuration. 75

6.1 Estimator classes and evaluated representatives. 85
6.2 Disturbance configurations used in the robustness benchmark. . . . 91
6.3 Deployment-oriented estimator recommendation map. 120

7.1 STM32 KPIs for SOC deployments. 137

7.2 STM32 KPIs for SOH deployments. 137
7.3 Pruning and quantization summary. 148

8.1 Synthesis of contribution chapters and requirement dimensions. . . 155

A.1 HECM lookup sensitivity across the complete disturbance benchmark. The largest robustness interaction is selected from 22 measurement and signal-integrity subcases after equal-weight cell aggregation. Bracketed counts report subcases whose 95% interaction interval excludes zero. Recovery is the persistent recovery-or-censor time from the paired initialization analysis, with the equal-weight censored-cell percentage in brackets. 189
A.2 Baseline cell-macro statistics. 189
A.3 Paired baseline-MAE comparisons. 190
A.4 Selected disturbance penalties. 190
A.5 Robustness-score sensitivity. 190
A.6 Recovery and hardware summary. 191
A.7 Target-error change after input-buffer faults. 196

Chapter 1

Introduction and Motivation

Lithium-ion batteries are increasingly used in systems that must operate reliably for long periods with limited maintenance access. This includes stationary storage in decentralized energy systems, for example remote microgrids in regions where grid connection can be limited or unavailable. The same requirement appears in mobile and autonomous applications that have to continue operating when communication, remote control, or service access is restricted. In all of these cases, the battery-management system (BMS) needs reliable information about internal battery states that cannot be measured directly. The state of charge (SOC) affects usable energy and power limits, while the state of health (SOH) affects lifetime assessment, maintenance planning, and safe operating margins.

Artificial-intelligence-based estimation methods are attractive because battery behaviour is nonlinear and depends on ageing, load history, temperature, and operating point. Their performance is commonly assessed through estimation accuracy on clean laboratory test data. Nominal accuracy is essential because it shows how well a model represents the relationship between measurements and battery states under defined conditions. For BMS operation, however, accuracy is one part of a broader assessment that also includes sensitivity to imperfect measurements, causal online execution, memory and timing requirements, and the transfer from a trained model to firmware. This research combines these dimensions in a connected evaluation framework, extending accuracy-centred offline assessment from data representation and nominal performance to robustness under disturbed operation and measured embedded execution.

1.1 Research Aim and Questions

The aim of this work is to develop and evaluate AI-based battery-state estimation methods along the path from data representation to embedded execution. The

work is guided by three research questions:

1. How can simple, low-complexity models capture the complex and time-dependent ageing behaviour of lithium-ion batteries without resorting to large or elaborate architectures?

2. How do battery-state estimators behave under realistic operating and measurement conditions, and which factors determine their robustness in practice?

3. How can such estimators be transferred to embedded BMS hardware and optimized to run within its memory, latency, and energy limits?

Each question targets a different stage of this path. The first concerns model design. Ageing develops slowly and depends on prior operation, so a compact estimator must receive sufficient information about this history. The second concerns evaluation. An estimator that performs well on clean data must also be confronted with disturbed and incomplete signals before its operational reliability can be judged. The third concerns realization. A model is useful in a BMS only if it remains accurate while meeting the resource and timing budget of the target controller.

1.2 Contribution, Scope, and Publication Basis

The work makes three main technical contributions. First, it studies how lag-sequence construction and cumulative-current history can turn a memoryless multilayer perceptron (MLP) into a time-aware SOH estimator without increasing the architectural complexity unnecessarily. Second, it develops a common performance and robustness benchmark for direct-measurement, hybrid, equivalent-circuit, and data-driven SOC estimators. The benchmark combines nominal, disturbed, and recovery tests with measured latency, flash, and peak runtime RAM on the target microcontroller. For the pruned recurrent representative, it also isolates the influence of rolling-window recomputation, continuous state forwarding, and periodic state reset. Third, it transfers recurrent SOC and SOH models to STM32 microcontroller execution and evaluates the interaction between model architecture, compression, and measured embedded performance.

The ageing-estimation contribution presented in Chapter 5 is based on the published study by Rzepka et al.[1] The performance and robustness benchmark in

Chapter 6 forms the basis of a manuscript submitted to *Engineering Applications of Artificial Intelligence*.[2] The embedded-AI study in Chapter 7 is available as an SSRN preprint.[3] The chapter texts are integrated and expanded for the monographic research argument rather than reproduced as independent articles.

These contributions are supported by an experimental basis consisting of two ageing datasets and a custom BMS platform. The hardware platform is not treated as a separate algorithmic contribution in the same sense as the three estimator studies. It provides the measurement, firmware, and validation environment required to connect the methodological results to practical BMS implementation.

Table 1.1: Experimental basis, technical contributions, and associated publication basis of the research.

Research component	Role	Primary dimension	Publication basis
Experimental and hardware basis	Data, labels, embedded targets, and validation context	Shared experimental basis	Supports all three contribution studies
Ageing estimation in Chapter 5	Temporal representation for compact SOH estimation	Representation design and SOH estimation	Published journal article[1]
Performance and robustness benchmark in Chapter 6	Matched disturbance, recovery, and on-device comparison	Accuracy, robustness, recovery, and embedded performance	Manuscript submitted to *Engineering Applications of Artificial Intelligence*[2]
Embedded AI in Chapter 7	Compression and measured microcontroller execution	Deployment and compression	SSRN preprint[3]

Chapter 2

Battery-Management Requirement Framework

A BMS supervises a lithium-ion cell or pack and keeps it within safe and efficient operating limits by monitoring voltage, current, and temperature and by controlling charging, balancing, protection, and communication. SOC and SOH are central internal quantities in this control chain. They influence usable energy, power limitation, lifetime-aware operation, and maintenance decisions, but they cannot be obtained directly from one sensor channel. They must be inferred from the measurement stream and from knowledge about the cell or pack.[4, 5, 6]

For a BMS, state estimation is therefore not only an algorithmic accuracy problem. The estimator output has to be reliable enough for operating decisions, the implementation has to fit the controller and sensing chain, and the method has to be transferable into maintainable firmware. Figure 2.1 organizes these constraints into three connected dimensions.

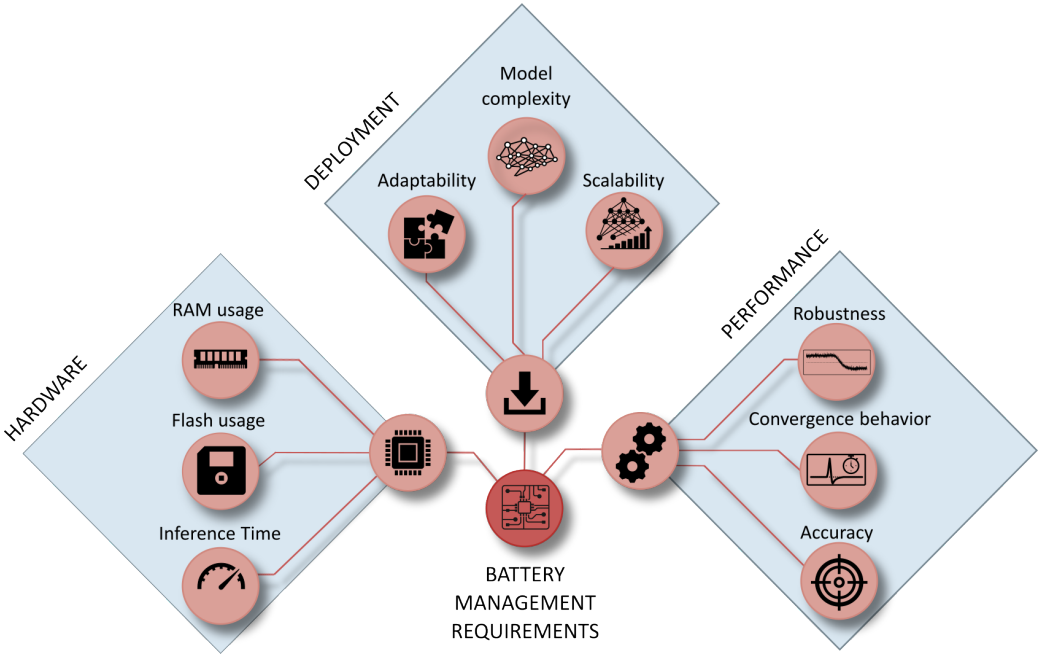


Figure 2.1: Battery-management requirement framework used throughout this work. The three connected dimensions cover estimator performance, the hardware resource envelope, and the engineering path to deployment.

2.1 Performance requirements

Performance requirements describe whether the estimated SOC or SOH signal is suitable for BMS decisions during operation. In this work, performance is not reduced to average error under clean measurements. It also includes robustness against disturbed input signals and convergence or recovery after an inconsistent initial state. This distinction is important because bias, noise, missing samples, timing irregularities, and communication interruptions can change the estimator ranking even when the clean-data error is low.[7, 8, 9]

2.2 Hardware requirements

Hardware requirements define the resource envelope in which state estimation can run on the BMS controller. Flash memory limits the amount of code, parameters, and lookup data that can be stored. RAM usage limits the size of recurrent states, intermediate buffers, stack, and runtime data. Inference time determines whether the estimator can meet the required update rate with sufficient timing margin.

Energy per inference becomes relevant when the controller has to operate under strict power budgets.[10, 11]

Selecting a controller with more internal RAM is possible in principle, but it is not a neutral remedy. A larger device can increase component cost, power consumption, package size, and qualification effort, and it can require changes to the board and firmware platform. External RAM introduces an additional memory interface, access latency, bus traffic, board area, and further power and electromagnetic-compatibility considerations. It can also weaken timing predictability compared with tightly coupled internal memory. A deployable estimator should therefore use the available memory efficiently instead of assuming that the hardware can always be enlarged around the model. The measurement chain belongs to the same requirement area because sensor resolution, noise, sampling timing, and communication behaviour define the input information available to the estimator.

2.3 Deployment requirements

Deployment requirements describe the engineering path from a trained or identified method to a reusable BMS implementation. Relevant aspects are model complexity, firmware transfer, calibration effort, adaptability to changed cells or operating conditions, and scalability to related estimation tasks or target platforms. A method is therefore not deployment-ready only because it runs once on a controller. It must also be reproducible, maintainable, and modifiable without rebuilding the full BMS software around it. Compression methods such as pruning and quantization can support deployment, but their effect depends on the architecture and must be verified on the target implementation.[12, 13]

Chapter 3

State of the Art and Technical Foundations

3.1 Lithium-Ion Cell Fundamentals

A lithium-ion cell stores energy through reversible movement of lithium ions between a negative and a positive electrode. During charging, lithium ions leave the positive electrode and are inserted into the negative electrode, which is commonly graphite. During discharge, the direction is reversed and electrons flow through the external circuit. A porous separator prevents direct electrical contact between the electrodes, while the electrolyte provides ionic transport. The resulting terminal voltage is not determined by SOC alone. It combines the equilibrium open-circuit voltage with ohmic, charge-transfer, diffusion, temperature, and hysteresis effects.[14, 4]

A reduced representation of this relation is

$$U_{\mathrm{t}}(t) = U_{\mathrm{OCV}}(z(t), T(t)) - \eta_{\mathrm{ohm}}(t) - \eta_{\mathrm{dyn}}(t), \tag{3.1}$$

where z denotes SOC, U_{OCV} is the equilibrium voltage, and the two overvoltage terms collect the immediate resistive drop and slower electrochemical dynamics. This relation explains why a measured voltage cannot generally be converted into an unambiguous SOC during load. Current integration provides temporal continuity, while voltage-based information can correct drift when the cell model and operating point are sufficiently observable.

Battery loading is commonly described by the C-rate and depth of discharge (DOD). A current of 1C would transfer the nominal capacity in one hour under idealized conditions. DOD describes the fraction of capacity removed within a cycle. Equivalent full cycles (EFC) aggregate absolute charge throughput into full-capacity equivalents and therefore provide a common loading coordinate when the

individual cycles have different depths. Temperature, C-rate, mean SOC, and DOD affect both the immediate voltage response and long-term degradation.

3.1.1 NMC and LFP cell chemistry

The experiments in this work use cells with nickel-manganese-cobalt oxide (NMC) and lithium iron phosphate (LFP) positive electrodes. Both are lithium-ion chemistries, but their voltage characteristics and typical application trade-offs differ. NMC cells combine comparatively high specific energy with a voltage curve that varies noticeably over much of the SOC range. LFP cells are valued for thermal stability and cycle life, but their open-circuit-voltage curve contains a broad, comparatively flat plateau. On this plateau, a voltage difference may correspond to a large SOC interval. Voltage-only SOC estimation is therefore weakly observable in this region, and reliable estimation places greater emphasis on current integration, dynamic voltage response, temperature, and history.[14, 4, 15]

Table 3.1: Qualitative differences between the NMC and LFP cell chemistries used in this work. The properties are typical tendencies rather than universal specifications for every commercial cell.

Aspect	NMC	LFP
Positive-electrode family	Layered transition-metal oxide	Olivine phosphate
Typical application strength	Higher specific energy and compact storage	Thermal stability and long cycle life
Voltage characteristic	More pronounced SOC-dependent slope over broad regions	Broad and comparatively flat voltage plateau
Estimation consequence	Voltage correction can be informative, but remains temperature- and model-dependent	Voltage alone provides limited SOC information on the plateau, increasing the importance of current history and dynamic features

3.1.2 Ageing Effects and State-of-Health Definitions

Lithium-ion ageing has calendar and cyclic components. Calendar ageing develops with time and depends strongly on temperature and storage SOC. Cyclic ageing is additionally shaped by charge throughput, current rate, DOD, and operating boundaries. The observable consequences include loss of usable capacity and an increase in internal resistance. These quantities do not necessarily change in a perfectly smooth or monotonic way from one diagnostic test to the next. Relaxation and reversible recovery, small differences in conditioning and temperature, and uncertainty in current integration and cut-off detection can cause a later capacity test to return a slightly higher value even though the long-term degradation trend remains downward.[16, 17, 18]

SOH is therefore a derived quantity whose exact meaning depends on the selected reference. Capacity-based SOH compares the currently measured usable capacity with a nominal or initial reference capacity. Resistance-based SOH instead tracks the increase in internal resistance. This work uses capacity-based SOH because the datasets contain periodic controlled capacity tests. The precise label construction for the NMC and LFP campaigns is specified in Chapter 4.

3.2 Battery Management Systems and Battery States

Practical BMS implementations combine measurement, protection, control, communication, and state estimation. A cell-monitoring front end records cell voltages and often controls passive balancing. Current and temperature sensors provide pack-level and thermal information. A microcontroller executes protection logic, communication, diagnostics, and higher-level estimation algorithms. Isolation, contactor or relay control, and external interfaces complete the system.[19, 20, 21, 22, 23]

Within this architecture, SOC describes the currently available charge relative to a defined usable capacity. SOH describes degradation relative to a capacity or resistance reference. State of power describes the short-term charging or discharging capability under voltage, current, and temperature limits. These states cannot be read from an individual sensor. They must be reconstructed from measured voltage, current, temperature, time, and a representation of the cell dynamics.[4, 5, 6]

The main estimator families are direct-measurement methods, model-based observers, data-driven models, and combinations of these approaches. Figure 3.1 summarizes this relationship. Coulomb counting is the principal direct method. Equivalent-circuit or electrochemical models provide the physical basis for observers. Neural networks learn a nonlinear mapping from measured and engineered variables to the target state. Hybrid estimators combine physical propagation with learned capacity, parameter, or state correction.[24, 25, 26, 27]

Figure 3.1: Main families of battery-state estimators. Hybrid methods combine elements of direct measurement, physical model structure, and data-driven adaptation.

3.3 Model-Based Estimation and Kalman Filtering

Equivalent-circuit models (ECMs) represent the terminal behaviour of a cell through an open-circuit-voltage source, an ohmic resistance, and one or more resistor-capacitor branches. The SOC and polarization voltages form the model state. Measured current is the input, while the predicted terminal voltage is compared with the measured voltage. This structure is computationally e cient and is therefore widely used in practical BMS observers.[4, 28, 29]

A nonlinear discrete state-space model can be written as

$$x_k = f(x_{k-1}; u_k) + w_k; \tag{3.2}$$

$$z_k = h(x_k; u_k) + v_k; \tag{3.3}$$

where x_k contains the hidden battery states, u_k contains known or measured inputs, and z_k contains the sensor measurements used for correction. In a battery observer, u_k commonly contains the cell current and z_k the terminal voltage. The

Software libraries normally stack the four input matrices and the four recurrent matrices for efficient evaluation. In the PyTorch gate order, for example, $\mathbf{W}_{ih} = [\mathbf{W}_i; \mathbf{W}_f; \mathbf{W}_g; \mathbf{W}_o]$ and $\mathbf{W}_{hh} = [\mathbf{U}_i; \mathbf{U}_f; \mathbf{U}_g; \mathbf{U}_o]$. PyTorch also stores separate input-side and recurrent-side bias vectors. This changes the stored parameter count to $4H(D+H+2)$, although the two bias contributions can be combined in the mathematical formulation above. The quadratic recurrent term explains why increasing the hidden dimension rapidly increases memory and arithmetic cost. In contrast to an MLP with a fixed lag vector, an LSTM learns how information is retained internally and can process sequences of varying duration. This flexibility requires explicit state management during training, validation, streaming evaluation, and embedded deployment. Resetting or misaligning either recurrent state changes the estimator trajectory even when the instantaneous input remains unchanged.

Gated recurrent unit

The gated recurrent unit (GRU) combines memory and exposed output in a single hidden state $\mathbf{h}_t$. An update gate $\mathbf{z}_t$ controls the interpolation between the previous state and a new candidate, while a reset gate $\mathbf{r}_t$ controls how strongly the previous state contributes when that candidate is formed:

$$\begin{aligned} \mathbf{z}_t &= \sigma(\mathbf{W}_z \mathbf{x}_t + \mathbf{U}_z \mathbf{h}_{t-1} + \mathbf{b}_z)\,, \\ \mathbf{r}_t &= \sigma(\mathbf{W}_r \mathbf{x}_t + \mathbf{U}_r \mathbf{h}_{t-1} + \mathbf{b}_r)\,, \\ \tilde{\mathbf{h}}_t &= \tanh(\mathbf{W}_h \mathbf{x}_t + \mathbf{U}_h(\mathbf{r}_t \odot \mathbf{h}_{t-1}) + \mathbf{b}_h)\,, \\ \mathbf{h}_t &= (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \end{aligned} \tag{3.14}$$

In this convention, an update-gate value close to one favours the candidate state, whereas a value close to zero retains the previous hidden state. The reset gate acts only on the history used to construct $\tilde{\mathbf{h}}_t$. A small reset value allows the candidate calculation to disregard selected components of the previous state. The symbol $\mathbf{z}_t$ follows conventional GRU notation for the update gate and is unrelated to the dense-layer pre-activation $\mathbf{z}^{(\ell)}$ used in Equation (3.8).

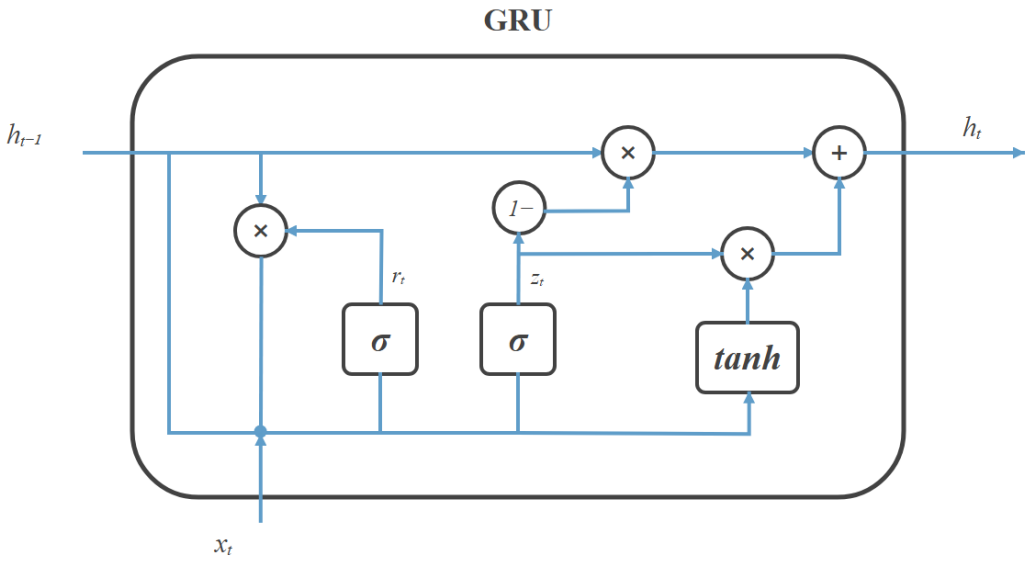


Figure 3.4: Detailed data flow through a GRU cell corresponding to Equation (3.14). The reset gate controls the historical contribution to the candidate state. In the convention used here, $\mathbf{z}_t$ weights the newly computed candidate and $1 - \mathbf{z}_t$ weights the previous hidden state. Unlike the LSTM, the GRU propagates only one recurrent state.

Equation (3.14) and Figure 3.4 use the candidate-weight convention and the reset-before formulation. The update gate therefore weights $\tilde{\mathbf{h}}_t$ directly, and $\mathbf{r}_t$ is applied to $\mathbf{h}_{t-1}$ before the recurrent matrix multiplication. An equally common notation lets the update gate weight the previous state instead. With the complementary gate $\mathbf{z}'_t = 1 - \mathbf{z}_t$, the same update can be written as

$$\mathbf{h}_t = (1 - \mathbf{z}'_t) \odot \tilde{\mathbf{h}}_t + \mathbf{z}'_t \odot \mathbf{h}_{t-1}. \tag{3.15}$$

Implementations may also apply the reset gate either before or after the recurrent affine transformation. These alternatives leave the tensor dimensions unchanged, but the equations, trained parameters, and recurrent states must be interpreted using the same convention.

For input dimension D and hidden dimension H, the update gate, reset gate, and candidate each require one input and one recurrent transformation. With one combined bias per transformation, the parameter count is

$$N_{\text{param}}^{\text{GRU}} = 3H(D + H + 1). \tag{3.16}$$

An implementation that stores separate input-side and recurrent-side biases instead contains $3H(D + H + 2)$ parameters. For identical D and H, a GRU consequently uses approximately three quarters of the recurrent parameters and affine operations of an LSTM. This does not imply a universal accuracy advantage. The LSTM offers a distinct cell-state path and independent forget, write, and output control, whereas the GRU uses a more compact interpolation in one state. The suitable architecture depends on the required memory behaviour, available data, optimization, and deployment constraints.

3.4.4 Comparison with Kalman-filter-based estimation

Neural and Kalman-filter-based estimators solve related state-estimation problems with different assumptions. Table 3.2 summarizes the principal trade-offs. The comparison does not imply that one family is universally superior. A Kalman filter is attractive when a sufficiently accurate model and parameterization are available. A neural network is attractive when a representative dataset can capture nonlinear dependencies that are difficult to express explicitly.

Reported model-based SOH approaches reach low nominal errors when identification and operating conditions are well controlled, while measurement-driven and neural methods avoid some explicit model assumptions but depend more strongly on feature validity and dataset coverage.[30, 34, 35, 36, 37] Recent hybrid and sequence-based methods increasingly combine physical constraints, filtering, and learned correction.[38, 39, 40, 41, 42]

Table 3.2: General strengths and limitations of Kalman-filter-based and neural battery-state estimators. Hybrid methods can combine several of these properties.

Aspect	Kalman-filter-based estimator	Neural estimator
Main prior knowledge	State-space model, parameters, OCV relation, and noise assumptions	Representative measurements, engineered features, and reference labels
Online correction	Explicit residual between predicted and measured voltage	Learned response to the current feature and recurrent history
Interpretability	States and correction equations are physically structured	Internal representation is less directly interpretable
Adaptation challenge	Parameters and covariance tuning must remain valid over temperature and ageing	Training coverage must include the relevant cells and operating conditions
Typical strength	Causal physical propagation with transparent uncertainty handling	Flexible nonlinear approximation and multi-signal fusion
Typical weakness	Model mismatch and weak observability can degrade correction	Domain shift, label quality, and unseen disturbances can degrade generalization
Embedded cost	Usually compact, but depends on model order and matrix operations	Strongly architecture-dependent and may require compression

3.5 Embedded AI and Model Compression

Embedded deployment requires more than transferring a trained model from a workstation framework to a constrained controller. Architecture, numerical representation, storage layout, recurrent-state handling, preprocessing, and execution kernels jointly determine whether the implementation preserves the intended model behaviour while meeting flash, RAM, and timing limits.[43, 10, 11]

This section introduces two complementary families of model compression. The pruning subsection distinguishes removal granularity and structural dependencies, explains criteria and ranking scopes, and discusses when pruning can be integrated into training or applied to an existing model. The quantization subsection describes which quantities can cross a precision boundary, how floating-point ranges are mapped to integer grids, and how range selection, granularity, calibration, arithmetic, and hardware support affect the resulting implementation. These foundations provide the terminology for the application-specific compression methods examined later in Chapter 7.

3.5.1 Pruning

Pruning reduces a neural network by removing parameters or complete model components that contribute comparatively little to its mapping. The underlying assumption is that trained networks commonly contain redundancy and that not every parameter is equally important for the final prediction. Its objective is therefore to identify a smaller subnetwork that retains an acceptable level of predictive accuracy. The design of a pruning procedure requires several separate decisions. These concern the structure to be removed, the criterion used to rank candidates, the scope of the ranking, the point in the training process at which pruning occurs, and the way in which the remaining network is adapted.[44, 45, 46]

Let the parameters $\boldsymbol{\theta}$ be partitioned into K candidate groups $\mathcal{G}_1, \ldots, \mathcal{G}_K$. A binary variable m_k determines whether group $\mathcal{G}_k$ remains active. The general constrained problem can be expressed as

$$\underset{\boldsymbol{\theta}, \boldsymbol{m}}{\text{minimize}} \quad \mathcal{L}(\boldsymbol{\theta}, \boldsymbol{m} \mid \mathcal{D}) \quad \text{subject to} \quad m_k \in \{0, 1\}, \qquad \sum_{k=1}^{K} m_k \leq K', \tag{3.17}$$

where $\mathcal{L}$ is the task loss on data $\mathcal{D}$ and K' is the permitted number of retained groups. The pruning fraction is $p = 1 - K'/K$, while $1 - p$ is the retained density.

This formulation does not prescribe how the discrete selection is solved. Practical methods replace the combinatorial search by ranking rules, continuous mask approximations, regularization, or repeated removal and retraining.

Parameter count, stored model size, arithmetic effort, and measured inference time remain distinct quantities. A high pruning fraction does not automatically produce an equivalent hardware speed-up because the practical benefit depends on the removed structure, its representation, the execution kernels, and memory traffic. This distinction is especially important for embedded targets, where a mathematically sparse model may still be executed as a dense model.

Pruning structures and granularity

The most direct classification concerns the granularity of the removed object. Unstructured pruning removes individual connections. For a weight matrix W, this operation can be represented by a binary mask M,

$$\widetilde{W}_{\text{unstructured}} = M \odot W, \qquad M_{ij} \in \{0, 1\}, \tag{3.18}$$

where a zero entry in M suppresses the corresponding weight. This fine granularity offers many possible pruning patterns and can produce high numerical sparsity while preserving important connections. However, the dimensions of $\widetilde{W}_{\text{unstructured}}$ remain identical to those of W. Standard dense kernels still execute the zero-valued entries. A reduction in storage or arithmetic therefore requires a sparse representation, index metadata, and execution kernels that can exploit the resulting irregular access pattern. At moderate sparsity, this overhead can offset part of the theoretical benefit.[12, 44, 45]

Structured pruning removes complete groups such as neurons, channels, filters, attention heads, or recurrent states. If $\mathcal{I}_{\text{out}}$ and $\mathcal{I}_{\text{in}}$ denote the retained output and input indices, the resulting matrix is

$$\widetilde{W}_{\text{structured}} = W[\mathcal{I}_{\text{out}}, \mathcal{I}_{\text{in}}]. \tag{3.19}$$

The tensor dimensions are reduced and the remaining values form a smaller dense matrix. Conventional dense kernels can therefore benefit directly, and the reduction can extend to intermediate activations and recurrent states. The coarser granularity provides fewer possible pruning choices than element-wise removal and can cause a larger accuracy loss if an important group is removed. Connected layers must

also be modified consistently because removing one output component changes the corresponding input dimension of the following operation. Figure 3.5 illustrates this difference between individual-weight removal and dimension-reducing structured pruning.[47, 48, 46]

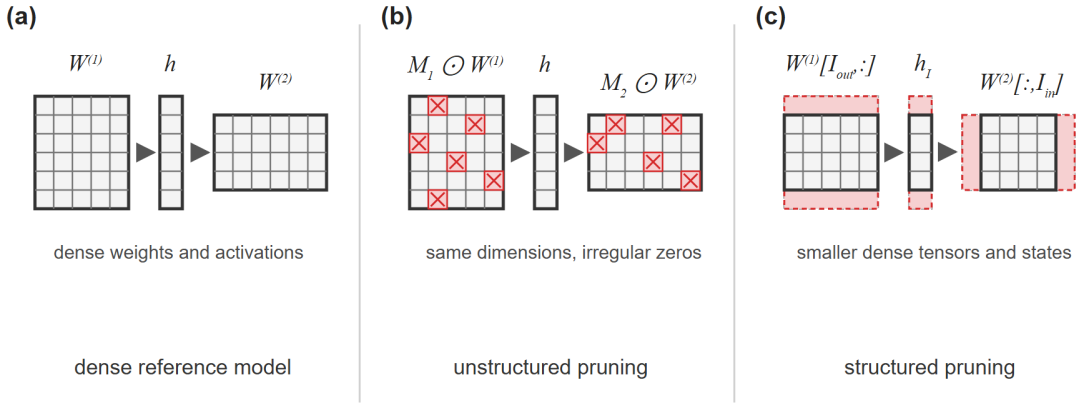


Figure 3.5: Comparison of pruning granularities. Panel (a) shows two dense weight matrices connected through an intermediate representation. In panel (b), unstructured pruning suppresses individual weights while retaining the original tensor dimensions. Panel (c) illustrates structured pruning, where complete output structures and the corresponding inputs of the following layer are removed. The resulting tensors and intermediate representation are smaller but remain dense.

Table 3.3 summarizes the practical trade-off. Unstructured pruning provides greater flexibility in selecting individual connections, while structured pruning more directly translates the removal decision into conventional embedded memory and compute reductions. Neither strategy guarantees a proportional reduction in measured latency because the realized gain still depends on storage format, kernel implementation, memory traffic, and the target processor.

Table 3.3: Practical advantages and limitations of unstructured and structured pruning.

Aspect	Unstructured pruning	Structured pruning
Removed object	Individual weights or connections	Complete neurons, channels, filters, or recurrent states
Granularity	Fine and highly flexible	Coarser and constrained by the network architecture
Resulting tensors	Original dimensions with irregular zero entries	Reduced dimensions with dense retained values
Main advantage	High numerical sparsity can be reached without removing complete functional groups	Parameter memory, intermediate states, and operation counts can be reduced using standard dense representations
Main limitation	Practical acceleration requires sparse storage and suitable sparse kernels	Coupled layers must be changed consistently and coarse removal can affect accuracy more strongly
Embedded suitability	Hardware- and sparsity-format-dependent	Generally easier to map to predictable dense execution

Semi-structured pruning occupies an intermediate position by enforcing small regular sparsity patterns, such as retaining a fixed number of weights within each block. A common example is an N:M pattern in which exactly N of every M weights remain active. It sacrifices some of the freedom of unstructured pruning to obtain predictable local patterns. Practical acceleration still requires a kernel and target processor that support the selected pattern.[45, 49]

Architectural dependencies. Structured pruning changes tensor dimensions and therefore cannot always be applied independently to every layer or path. In a sequential network, removing outputs of layer i requires removal of the corresponding inputs of layer $i + 1$. Parallel and residual paths impose an additional constraint. For

$$\boldsymbol{y} = f_A(\boldsymbol{x}) + f_B(\boldsymbol{x}), \tag{3.20}$$

the two outputs must have compatible dimensions and coordinate semantics. If the branches are independently reduced to $\mathbb{R}^{H'_A}$ and $\mathbb{R}^{H'_B}$ with $H'_A \neq H'_B$, the addition is not defined. Even if both branches retain the same number of components, independently selected channel identities may no longer correspond. Safe structured pruning therefore coordinates retained index sets across dependent paths or inserts an explicit projection that restores compatibility.

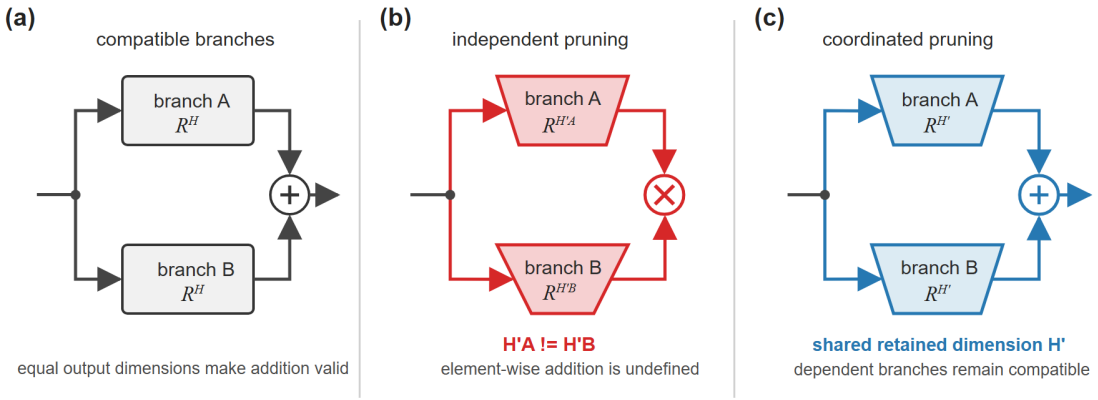


Figure 3.6: Dependency constraints in structured pruning. Panel (a) shows two compatible branches whose outputs can be added. Panel (b) illustrates an invalid result when the branches are independently reduced to different output dimensions. Panel (c) uses a shared retained structure so that both branches remain compatible. Similar consistency constraints apply between successive layers and between the coupled gates of recurrent networks.

Figure 3.6 illustrates why a structured pruning decision is not solely a local ranking problem. The removable unit is defined by the computation graph. For recurrent architectures, one hidden state participates in several gate matrices, the recurrent state vector, and the input of a subsequent output head. Treating these occurrences as one group preserves a valid recurrent model after removal. This architectural coupling is revisited for the LSTM estimator in Section 7.4.

Pruning criteria and ranking scope

After the pruning structure has been defined, an importance score is required for every candidate group. The score estimates the expected effect of removing the candidate. No inexpensive score can prove global optimality because parameters interact and the loss changes after each removal. Criteria therefore trade computational cost against task-specific information.

Magnitude criteria. The simplest criterion assumes that parameters with small magnitudes contribute less to the network mapping. For an individual weight, $s_{ij} = |w_{ij}|$. For a structured group $\mathcal{G}_k$, the same idea can be extended through a grouped norm,

$$s_k^{\text{mag}} = \|\boldsymbol{\theta}_{\mathcal{G}_k}\|_q = \left(\sum_{i \in \mathcal{G}_k} |\theta_i|^q \right)^{1/q}, \qquad q \in \{1, 2\}. \tag{3.21}$$

The L_1 norm sums absolute magnitudes, while the L_2 norm gives greater relative influence to large entries. Magnitude criteria require neither labels nor additional forward or backward passes after training. Their limitation is that parameter scale is not always directly comparable across layers or operations. A small parameter can still be influential when it multiplies a large activation or interacts with other weights.[12, 50, 45]

Loss-sensitivity criteria. Gradient-based criteria use the task loss to approximate the consequence of removal. Setting all parameters of group $\mathcal{G}_k$ to zero changes them by $\Delta\theta_i = -\theta_i$. A first-order Taylor approximation gives

$$\Delta\mathcal{L}_k \approx -\sum_{i \in \mathcal{G}_k} \frac{\partial \mathcal{L}}{\partial \theta_i} \theta_i, \qquad s_k^{\text{Taylor}} = \left| \sum_{i \in \mathcal{G}_k} \frac{\partial \mathcal{L}}{\partial \theta_i} \theta_i \right|. \tag{3.22}$$

This score combines the parameter value with current loss sensitivity and therefore requires representative data and a backward pass. Second-order methods additionally use curvature. For an individual parameter near a stationary point, where the first-order gradient is small, a diagonal approximation gives

$$\Delta\mathcal{L}_i \approx \frac{1}{2} H_{ii} \theta_i^2, \tag{3.23}$$

where H_{ii} is a diagonal entry of the loss Hessian. This Hessian-based approximation was used in early second-order pruning research to rank parameters by their estimated contribution to the loss. Curvature can provide richer information, but computing or approximating it is substantially more expensive than a grouped weight norm.[51, 52]

Activation and learned-mask criteria. Activation-based criteria measure how strongly a neuron, channel, or recurrent state responds over representative sam-

ples. Reconstruction criteria instead select structures whose removal changes an intermediate or final output as little as possible. Both approaches include information about the data distribution, but their result depends on the calibration set. A further class introduces trainable gates or masks and optimizes them jointly with the network. Sparsity penalties then encourage gates or complete parameter groups to approach zero. Examples include L_0-motivated gate regularization and group penalties for structured sparsity.[53, 47, 48]

Local and global ranking. The ranking scope determines how scores are converted into a retained set. Local pruning assigns a target K'_ℓ to each layer ℓ and ranks only within that layer,

$$\mathcal{R}_\ell = \mathrm{TopK}_{K'_\ell}\left(\{s_{\ell,k}\}_{k=1}^{K_\ell}\right). \tag{3.24}$$

It provides direct control over the width of each layer and prevents an entire sensitive or small layer from disappearing. However, the layer-wise pruning fractions must be selected in advance and may retain weak groups in one layer while discarding stronger groups in another.

Global pruning combines all candidates in one ranking,

$$\mathcal{R} = \mathrm{TopK}_{K'}\left(\bigcup_\ell \{s_{\ell,k}\}_{k=1}^{K_\ell}\right). \tag{3.25}$$

This allows the remaining capacity to be distributed according to the scores instead of fixed layer quotas. It can also over-prune a layer when raw scores are not comparable across layers. Normalization, minimum retained widths, and dependency constraints are therefore common safeguards. Both local and global selection are ranking heuristics rather than proofs that the resulting subnetwork is optimal.[44, 45]

Pruning schedules and optimization strategies

The classic pruning workflow trains a dense model, removes low-ranked parameters or groups, and fine-tunes the retained network,

$$\boldsymbol{\theta}_0 \xrightarrow{\text{train}} \boldsymbol{\theta}^* \xrightarrow{\text{score and prune}} (\boldsymbol{m}, \boldsymbol{\theta}^*) \xrightarrow{\text{fine-tune}} \widehat{\boldsymbol{\theta}}_{\boldsymbol{m}}. \tag{3.26}$$

The first training phase supplies task-adapted parameters from which importance is estimated. Pruning produces an immediate perturbation of the learned mapping.

Fine-tuning then adapts the surviving parameters to the reduced model capacity. This train–prune–fine-tune framework remains a strong practical baseline because its stages are interpretable and can be applied to an existing trained model.[12]

One-shot pruning performs the removal once at the target pruning fraction. It is computationally economical and reproducible, but a large single change may be difficult to recover from. Iterative pruning removes smaller fractions over several rounds and alternates them with optimization. The gradual process updates the ranking after the network has adapted, but requires more training and introduces additional choices for the number of rounds and the pruning schedule. Recovery can use conventional fine-tuning with a reduced learning rate, weight rewinding to an earlier training state, or learning-rate rewinding while retaining the final surviving weights.[54, 55]

Pruning is not restricted to a fully trained network. The principal strategy families can be distinguished by when and how the sparse structure is selected:

Pruning at initialization seeks an informative mask before ordinary weight training. SNIP, for example, ranks connections by their loss sensitivity at initialization and removes them in a single step.[56] A related hypothesis is that a randomly initialized dense network may already contain a sparse subnetwork that can be trained to comparable accuracy when its original initialization is retained.[54] These approaches address the cost of first training a dense model, but they solve a different problem from compressing a specific trained estimator whose behaviour is already known.

Dynamic sparse training begins with a sparse topology and changes the connectivity during optimization. Sparse evolutionary training removes low-magnitude connections and introduces new ones while maintaining a sparse parameter budget.[57] RigL combines magnitude-based removal with gradient information for regrowth and keeps the training cost tied to the selected density.[58] Such methods can reduce the cost of training itself. Their practical benefit depends on efficient sparse forward and backward kernels, which are not automatically available on all training platforms.

Learned-mask and penalty-based methods incorporate structure selection into the optimization objective. A generic regularized form is

$$\underset{\boldsymbol{\theta},\,\boldsymbol{\alpha}}{\text{minimize}} \quad \mathcal{L}(\boldsymbol{\theta} \odot g(\boldsymbol{\alpha})) + \lambda \sum_{k=1}^{K} \Omega(\alpha_k)\,, \tag{3.27}$$

Table 3.4: Principal strategies for obtaining a pruned or sparse neural network. The categories describe different optimization procedures and do not imply identical deployment representations.

Strategy	Structure selected	Main principle	Principal trade-off
Post-training pruning	After dense training	Rank trained parameters, remove candidates, and fine-tune or rewind	Reuses a trained model, but dense pretraining cost remains
Pruning at initialization	Before ordinary training	Estimate connection sensitivity or search for a trainable initial subnetwork	Avoids dense training of the final mask, but reliable early importance estimation is difficult
Gradual pruning during training	Across a prescribed schedule	Increase sparsity while weights continue adapting	Smoother transition, but schedule and final density become training hyperparameters
Dynamic sparse training	Throughout sparse training	Maintain a sparse budget while repeatedly removing and regrowing connections	Can reduce dense training cost, but requires sparse training support and topology updates
Learned masks or gates	Jointly with the weights	Optimize continuous gates and sparsity penalties, then discretize the result	Data-driven selection, but adds optimization variables and regularization choices
Penalty-based structured learning	During training	Apply group penalties so complete structures approach zero	Couples structure discovery with training, but penalty strength controls the accuracy–sparsity balance

where $g(\boldsymbol{\alpha})$ represents differentiable gates and Ω encourages sparsity. Group penalties can be applied to complete rows, columns, channels, or recurrent states so that deployment-relevant structures vanish together.[53, 47] Compared with a fixed post-training criterion, these methods allow importance and weights to co-adapt. They also introduce an additional regularization path and can require thresholding or model reconstruction after optimization.

The pruning fraction remains a design parameter rather than a universal property of a network. Increasing it strengthens the potential resource reduction but raises the risk of removing information that cannot be recovered. The selected model must therefore be assessed in terms of predictive performance and deployment-relevant quantities such as parameter count, persistent storage, intermediate-state memory, operation count, and measured inference time.

Software frameworks and deployment implications

Software libraries support different portions of the pruning workflow. Table 3.5 lists representative options. Their APIs simplify mask creation, scheduling, or comparison, but a pruning mask alone does not guarantee a physically smaller exported network.

PyTorch provides reusable pruning parametrizations and ranking utilities, while TensorFlow Model Optimization provides scheduled magnitude pruning and conversion paths for deployment.[59, 60, 49] Neural Network Intelligence exposes multiple pruning algorithms and separates mask generation from graph-level model speed-up.[61] ShrinkBench focuses on reproducible evaluation across pruning methods and models.[44]

For structured pruning, explicit model surgery is often required even when a framework can identify candidate groups. The retained rows and columns must be copied into newly dimensioned layers, coupled paths must share compatible indices, and downstream layers must be adapted. This produces a compact dense model that can use conventional kernels without storing masks or sparse indices. The specific one-shot, magnitude-based LSTM procedure used in this work and its custom PyTorch reconstruction are described in Section 7.4.

Table 3.5: Representative pruning support in common software ecosystems. Capabilities refer to the documented pruning workflow and must be matched to the target operator and deployment backend.

Framework	Main pruning support	Useful role	Deployment consideration
PyTorch pruning utilities	Mask-based local and global pruning with unstructured and structured criteria	Rapid implementation of standard criteria and custom pruning methods	Masks preserve the original module dimensions until explicit model reconstruction or sparse execution is applied
TensorFlow Model Optimization Toolkit	Magnitude pruning with schedules, Keras integration, model stripping, and supported structured sparsity patterns	Training-time pruning and export into the TensorFlow Lite ecosystem	Compression and acceleration depend on the exported representation and backend support
Neural Network Intelligence	Collection of pruners, evaluators, masks, and model-speedup utilities	Comparative experimentation and dependency-aware compression workflows	Supported operators and graph transformations must be checked for the chosen architecture
ShrinkBench	Standardized pruning experiments, metrics, and reproducible baselines	Method comparison and research evaluation	It is primarily an evaluation framework rather than an embedded inference runtime

3.5.2 Quantization

Quantization reduces the numerical precision used to store or process selected quantities in a neural network. Instead of retaining every value in a high-precision floating-point format, a quantizer maps it to one of a finite number of representable levels. This can reduce persistent model storage, memory traffic, intermediate-state memory, and arithmetic cost. The realized benefit depends on which tensors are quantized and whether the target processor provides kernels that operate directly on the selected low-precision format. Quantization is therefore both a numerical approximation problem and a model–hardware interface problem.[62, 63]

Quantizable quantities and precision boundaries

A neural layer combines stored parameters and dynamic tensors. For a dense neuron, the unquantized calculation is

$$z_j = \sum_{i=1}^{m} w_{ji} x_i + b_j, \qquad y_j = f(z_j), \tag{3.28}$$

where w_{ji} denotes a weight, b_j a bias, x_i an input activation, z_j the pre-activation, and y_j the output activation. This is the same neuron mapping introduced in Equation (3.7). Its quantities do not have to share one numerical format. The following precision boundaries are commonly distinguished:

- **Weights** are static after training and often account for most of the persistent model storage. Weight-only quantization can therefore reduce flash or file size without changing the representation of dynamic activations.
- **Biases** are stored parameters but are commonly retained at higher precision. In integer-only kernels they can be represented as higher-width integers matched to the product of the weight and activation scales.
- **Input and output activations** are generated at runtime. Their quantization can reduce intermediate memory and enables low-precision matrix operations, but their ranges depend on the input data and may require calibration or dynamic scale estimation.
- **Accumulators** collect many products. They normally use a wider type, such as INT32 for INT8 operands, to avoid overflow before the result is requantized.

- **Recurrent states** are dynamic activations that are fed back into the next update. Quantizing them changes not only one layer output but also the numerical state from which later predictions are computed.

The nonlinear function f is an operation rather than a stored tensor. Its input and output may be quantized, while the operation itself may be evaluated through integer arithmetic, a lookup table, a polynomial approximation, or a floating-point routine. Consequently, the label "quantized network" is incomplete unless the precision of weights, biases, activations, states, accumulators, and arithmetic is stated explicitly.

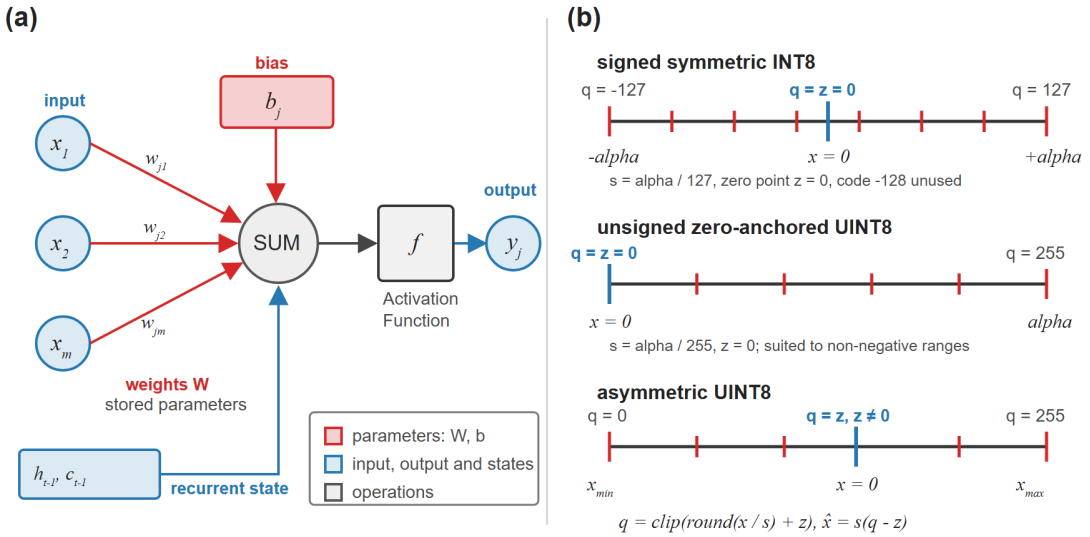


Figure 3.7: Quantizable quantities and representative uniform 8-bit grids. Panel (a) distinguishes stored weights and biases, dynamic inputs, outputs and recurrent states, and the summation and activation operations of a neural computation. From top to bottom, panel (b) shows signed symmetric INT8 with $z = 0$, unsigned zero-anchored UINT8 with $z = 0$ for non-negative values, and asymmetric UINT8 with $z \neq 0$. Labels above each grid denote integer codes q, whereas labels below denote their reconstructed real values x. The signed symmetric example uses the exactly balanced code range -127 to 127 and leaves the INT8 code -128 unused.

Figure 3.7 illustrates that quantization can be applied at several different locations. If tensor k contains N_k values represented by b_k bits, the ideal raw storage requirement is

$$M_{\text{raw}} = \sum_{k=1}^{K} \frac{N_k b_k}{8} + M_{\text{meta}}, \tag{3.29}$$

where M_{meta} contains scales, zero points, tensor descriptors, alignment, and any retained higher-precision parameters. Converting one FP32 tensor to INT8 ideally reduces the raw storage of that tensor by a factor of four. It does not imply a fourfold reduction of the complete model if other parameters, metadata, or code remain unchanged.

Uniform affine quantization

Uniform affine quantization maps real-valued quantities onto a uniformly spaced set of integer codes using a scale and a zero point. For a real value x, bit width b, scale $s > 0$, zero point z, and integer limits $q_{\min}$ and $q_{\max}$, quantization and reconstruction are defined as

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right), \qquad \widehat{x} = s(q - z). \tag{3.30}$$

The scale defines the distance between neighbouring reconstruction levels, while the zero point z specifies the integer code assigned to the real value $x = 0$. Accordingly, real zero is represented by $q = z$ irrespective of the position of zero within the integer code range.[13, 62]

Signed symmetric quantization is designed for real-valued ranges centred on zero. It sets $z = 0$ and chooses equal positive and negative limits. For signed b-bit quantization,

$$q_{\max} = 2^{b-1} - 1, \qquad q_{\min} = -q_{\max}, \qquad s = \frac{\alpha}{q_{\max}}, \tag{3.31}$$

where $\alpha = \max_i |x_i|$ if the complete observed range is retained. At $b = 8$, this produces the symmetric set $\{-127, \ldots, 0, \ldots, 127\}$. The underlying signed INT8 type also contains -128, but leaving this code unused gives equal-magnitude positive and negative limits. This convention is not mandatory. Other implementations use the full two's-complement interval and account for its one-code asymmetry.

Unsigned zero-anchored quantization applies to quantities that are known to be non-negative. For UINT8, $q_{\min} = 0$, $q_{\max} = 255$, $z = 0$, and $s = \alpha/255$, so the real interval $[0, \alpha]$ uses the complete code range. This configuration is suitable, for example, for the output of a ReLU activation.

Asymmetric quantization maps a shifted or unbalanced real interval $[x_{\min}, x_{\max}]$

to the complete integer range. Its parameters can be selected as

$$s = \frac{x_{\max} - x_{\min}}{q_{\max} - q_{\min}}, \tag{3.32}$$

$$z = \operatorname{clip}\left(\operatorname{round}\left(q_{\min} - \frac{x_{\min}}{s}\right), q_{\min}, q_{\max}\right). \tag{3.33}$$

For a range that is not centred on zero, the resulting zero point is generally non-zero. This offset uses the available codes efficiently, although the associated correction terms can increase kernel complexity. Signed symmetric quantization is therefore well suited to weights distributed approximately around zero, whereas unsigned zero-anchored and asymmetric mappings address non-negative and shifted ranges, respectively.

Rounding, clipping, and range selection

Quantization introduces two principal numerical errors. Values inside the selected interval are rounded to the nearest grid point. With nearest-integer rounding and no clipping,

$$|x - \widehat{x}| \leq \frac{s}{2}. \tag{3.34}$$

Values outside the interval are clipped to its lower or upper boundary and can exceed this bound. Expanding the interval reduces clipping but increases s, which makes the grid coarser and increases rounding error. Narrowing the interval has the opposite effect. Range selection therefore balances clipping of rare extremes against resolution in the densely populated part of the distribution.[62, 64]

The simplest choice uses the observed minimum and maximum. It avoids clipping of the calibration values but is sensitive to outliers. Percentile-based calibration deliberately excludes a small tail of the distribution. Error-based methods select lower and upper thresholds by minimizing a local reconstruction objective such as

$$\underset{\ell,u}{\text{minimize}} \quad \frac{1}{N}\sum_{i=1}^{N}\left(x_i - \widehat{x}_i(\ell, u)\right)^2. \tag{3.35}$$

Entropy-based criteria compare the original and quantized distributions. These local objectives do not necessarily predict the final task error because sensitivity differs between layers. More advanced post-training methods therefore include bias correction, cross-layer equalization, reconstruction losses, or learned rounding decisions.[65, 62]

Weights are fixed after training, so their ranges can be determined directly from the stored tensors. Activation ranges depend on the input data. Static activation quantization collects representative statistics before deployment and stores fixed quantization parameters. Dynamic quantization calculates activation scales during inference. Dynamic ranges can adapt to each input or time window but add runtime operations and may complicate deterministic timing.

Quantization granularity

The scale and zero point can be shared by different portions of a tensor. Granularity controls the compromise between representation accuracy, metadata, and kernel complexity:

Table 3.6: Principal quantization granularities and their implementation consequences. The term channel includes a matrix output row when each row produces one output component.

Granularity	Shared parameters	Main advantage	Main consequence
Per-tensor	One scale and zero point for a complete tensor	Minimal metadata and simple kernels	Channels with small ranges may use only a small part of the grid
Per-channel or per-row	One parameter set for every output channel or matrix row	Adapts to different output ranges and often reduces weight error	Requires multiple scales and channel-aware reconstruction or requantization
Per-group or per-block	One parameter set for a defined group of values	Intermediate compromise between local adaptation and metadata	Group shape must match packing and kernel support
Mixed precision	Different bit widths or formats for selected tensors and operators	Protects sensitive quantities while compressing less sensitive ones	Requires a deployment backend that supports every selected combination

Per-tensor quantization is straightforward because one scale can be factored out of a complete matrix operation. Per-channel weight quantization uses a separate scale for every output component. It improves the representation when rows have

different magnitudes and is compatible with many integer kernels because each accumulator output can be rescaled independently. Per-channel activation quantization is more difficult because input-channel scales occur inside the summation. Per-group methods provide finer adaptation but increase the number of parameters and can require specialized packing.[66, 62]

Integer arithmetic and requantization

A reduced storage type does not by itself define the arithmetic path. Consider quantized weights and input activations with parameters (s_w, z_w) and (s_x, z_x). An integer matrix operation forms

$$A_j^{\text{int}} = q_{b,j} + \sum_i \left(q_{w,ji} - z_w\right)\left(q_{x,i} - z_x\right), \tag{3.36}$$

where A_j^{int} is commonly accumulated in INT32. The corresponding real-valued pre-activation is approximately

$$a_j \approx s_w s_x A_j^{\text{int}}. \tag{3.37}$$

If the next activation uses parameters (s_y, z_y), the accumulator is requantized according to

$$q_{y,j} = \text{clip}\left(\text{round}\left(\frac{s_w s_x}{s_y} A_j^{\text{int}}\right) + z_y, q_{y,\min}, q_{y,\max}\right). \tag{3.38}$$

The bias can be stored as a higher-width integer with scale $s_b = s_w s_x$. With per-channel weight scales, s_w and s_b become output-dependent. Integer-only inference keeps the multiplication, accumulation, bias addition, and requantization in integer-compatible forms. Quantization-aware model export must preserve these scale relationships.[13]

Lower bit widths can reduce data movement and permit more parallel arithmetic operations, but these gains require suitable instructions and kernels.[67] A weight-only model that converts each integer weight back to floating point inside the innermost loop reduces persistent storage but may add conversion and scaling operations. Likewise, quantize and dequantize boundaries between unsupported operators can remove an expected speed advantage. Runtime, RAM, and energy improvements must therefore be measured on the selected backend.

Post-training and training-aware strategies

Quantization approaches can be organized by the point at which the numerical constraints enter model development:

Table 3.7: Principal quantization strategies and the information required to construct the deployed model. PTQ denotes post-training quantization and QAT denotes quantization-aware training.

Strategy	Quantized quantities	Main procedure	Principal trade-off
Weight-only PTQ	Stored weights, with other quantities retained at higher precision	Quantize a trained model directly from its weight tensors	Simple and effective for storage, but does not provide complete integer arithmetic
Static PTQ	Weights and selected activations	Use representative calibration data to fix activation ranges before deployment	Low inference overhead, but quality depends on representative ranges
Dynamic PTQ	Weights and activations with runtime activation scales	Determine dynamic tensor ranges during inference	Adapts to changing inputs, but adds scale-estimation overhead
Quantization-aware training	Simulated low-precision weights and activations during training or fine-tuning	Insert fake quantizers so the model adapts to the numerical grid	Can preserve accuracy at lower bit widths, but requires training data and additional optimization
Quantized training	Selected forward, backward, gradient, or optimizer quantities	Perform parts of the training computation itself at low precision	Can reduce training cost, but is distinct from inference-only compression and requires stable low-precision optimization

Post-training quantization starts from a trained floating-point model. Weight-only PTQ requires no activation data. Static full-integer PTQ normally requires a representative calibration set to estimate activation ranges. Dynamic PTQ evaluates selected activation parameters at runtime. PTQ is attractive when the original

training pipeline is expensive or unavailable, but aggressive bit-width reduction can exceed the error that the fixed parameters can tolerate.[64, 65]

Quantization-aware training inserts simulated quantizers into the forward path. A fake quantizer applies quantization followed by reconstruction while retaining floating-point tensors for gradient-based optimization,

$$x_{\mathrm{fq}} = s\left[\mathrm{clip}\left(\mathrm{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right) - z\right]. \tag{3.39}$$

The derivative of the rounding operation is zero or undefined, so QAT commonly uses a straight-through estimator that approximates the gradient through the quantizer. The weights can then adapt to quantization noise, clipping thresholds, and learned step sizes. This generally increases development effort but can be important for low-bit weights and activations.[13, 68, 62]

Numerical formats and recurrent networks

Integer quantization is only one form of reduced precision. FP16 and bfloat16 retain floating-point exponents and are often used where floating-point accelerators are available. INT8 uses a fixed affine interpretation supplied by scales and zero points. Four-bit, ternary, and binary representations can compress more strongly but provide fewer levels and place greater demands on training methods and specialized kernels. Power-of-two quantization restricts s to 2^k so that rescaling can be implemented through shifts, at the cost of a less flexible grid. Mixed-precision schemes assign different formats or bit widths to tensors according to sensitivity and backend support.[63, 62]

Recurrent networks require particular care because the hidden and cell states connect successive time steps. Weight quantization introduces a fixed perturbation to every recurrent update. Activation or state quantization additionally feeds a quantized dynamic state back into the next update. This does not imply that error must grow without bound because gates and nonlinearities can attenuate perturbations. It does mean that a single-step comparison is insufficient to establish long-horizon equivalence. Continuous replay without state resets is required to detect systematic drift, state-dependent error, or isolated deviations.

Weight-only recurrent quantization is a conservative precision boundary. It targets persistent matrix storage while preserving activation functions and recurrent states at higher precision. Full-integer recurrent inference can reduce arithmetic and state memory further, but requires compatible implementations of gate ac-

cumulation, sigmoid and hyperbolic-tangent functions, cell-state updates, and requantization. The best choice therefore depends on whether flash, RAM, latency, or energy is the active constraint.

Software support and deployment

Software frameworks cover different parts of the quantization workflow. Their high-level options are useful for model preparation, but the exported numerical graph and the target kernels remain decisive:

Table 3.8: Representative software support for quantization. Exact operator coverage, data types, and acceleration depend on the selected release and deployment backend.

Framework	Main support	Useful role	Deployment consideration
PyTorch and torchao	Weight-only, static, dynamic, mixed-precision, and QAT workflows	Model-side experimentation and fake-quantized training	Efficient execution depends on exported operators and available low-precision kernels
TensorFlow Model Optimization and Lite	PTQ, representative calibration, QAT, and integer-model conversion	End-to-end conversion into the TensorFlow Lite ecosystem	Full-integer acceleration requires supported operators and a representative calibration set
ONNX Runtime	Static and dynamic PTQ, QDQ and quantized operators, calibration, and debugging	Framework-neutral conversion and tensor-level comparison	Execution-provider support determines the useful formats and operators
STM32Cube.AI and custom C	Vendor conversion or explicit parameter export into microcontroller firmware	Integration with the final embedded application and direct measurement	Unsupported recurrent paths can require custom kernels and explicit scale handling

Current PyTorch tooling distinguishes weight-only, dynamic, static, and QAT workflows, while TensorFlow Lite provides post-training and training-aware conversion paths.[69, 70] ONNX Runtime represents quantized graphs through quan-

tized operators or explicit quantize–dequantize nodes and provides static and dynamic calibration workflows.[71] Vendor tools can shorten microcontroller integration when the architecture and operators are supported. A custom implementation provides a transparent precision boundary but places responsibility for scaling, accumulation width, state handling, and validation on the developer.

The application in Chapter 7 instantiates only one part of this design space. It uses symmetric per-row INT8 post-training quantization for the recurrent weight matrices and retains biases, activations, recurrent states, the MLP head, and arithmetic in FP32. The exact transformation and its manual C export are described in Section 7.5. This separation keeps the present subsection general and leaves the application-specific numerical boundary with the experimental method.

3.6 Embedded Hardware and Model-Hardware Co-Design

Embedded AI in battery applications adds a hardware constraint layer to model development. A network that performs well on a workstation must still fit the flash and RAM budget of the target controller, meet the required update interval, preserve numerical stability, and reproduce the intended online state handling in firmware.[43, 10, 11] Existing studies show that neural-network-based SOC and SOH estimators can be executed on microcontroller-class hardware and on dedicated edge devices for battery applications.[72, 73, 74, 75] However, an embedded demonstration is not defined by model inference alone. The complete execution chain also contains signal acquisition, preprocessing, scaling, recurrent-state storage, post-processing, communication, and the remaining BMS software. These components compete for the same memory, processor time, and energy budget.

These constraints are particularly important for stand-alone systems. They cannot rely on continuous cloud access, external computing, or rapid maintenance. Battery-state estimates must remain available locally despite limited energy and imperfect measurements, while controller resources remain available for protection, communication, and safety-critical BMS functions. Accuracy, robustness, and embedded feasibility are therefore connected requirements. Together, they determine whether an estimator supports reliable autonomous operation rather than only a low error in an offline evaluation.

The available computing platforms span different levels of programmability, spe-

cialization, and proximity between processing and memory. Table 3.9 summarizes the principal hardware classes relevant to neural-network training and inference. General-purpose processors provide flexibility, whereas increasingly specialized architectures improve throughput or energy efficiency by adapting arithmetic units, data paths, memory access, or all three to neural operations.[76, 77]

These labels are not mutually exclusive. A TPU is a domain-specific ASIC, an NPU is a broader term for neural acceleration, and DSP or vector units may be integrated into an MCU or another processor. Neuromorphic processors use event-driven computation, while analog and in-memory concepts move arithmetic closer to the stored parameters to reduce costly data transfers.[78, 79] For deployment, the decisive properties are therefore the supported operators and numerical formats, the memory hierarchy, data movement, programmability, and integration constraints rather than the hardware label alone.

Within this spectrum, conventional BMS implementations commonly execute estimation and control tasks on Arm Cortex-M microcontrollers. This remains a suitable approach for compact models and moderate update rates because the complete algorithm can be implemented within one familiar firmware environment. Hardware-supported edge AI extends this design space. Representative examples include the STM32N6 with the Neural-ART accelerator, selected NXP MCX N devices with the eIQ Neutron NPU, the Renesas RA8P1 with an Arm Ethos-U55 NPU, Alif Semiconductor Ensemble devices with optional Ethos-U55 acceleration, and Silicon Labs EFR32xG26/PG26 devices with AI/ML acceleration.[80, 81, 82, 83, 84] These controller families show that neural acceleration is becoming available within the resource and integration class of embedded systems rather than only in application processors or external accelerators.

The presence of an NPU does not by itself make it the preferable target for every battery estimator. The effective benefit depends on whether the exported operators, numerical formats, tensor dimensions, activation functions, and recurrent operations are supported by the accelerator and its software toolchain. Unsupported operations remain on the CPU or require graph transformations, and data transfers across precision or memory boundaries can offset part of the expected gain. General-purpose Cortex-M execution can therefore remain attractive for small recurrent estimators when transparency, deterministic state handling, and low integration effort are more important than maximum arithmetic throughput. Dedicated acceleration becomes more relevant as the model size, update rate, or number of concurrently executed AI functions increases.

Table 3.9: Functional overview of computing platforms for neural-network workloads. The categories describe their dominant implementation principle and can overlap within one device.

Hardware class	Technical principle	Main strength and typical role
CPU and GPU	General-purpose sequential or massively parallel programmable processing	Flexible model development, training, and high-throughput inference
DSP	Processor optimized for repeated signal and vector operations	Signal preprocessing and compact neural inference in embedded systems
MCU	Integrated processor, memory, and peripherals with tightly limited resources	Deterministic control and low-power TinyML within the BMS firmware
NPU	Dedicated neural accelerator integrated into an edge or application processor	Efficient inference for supported operators and numerical formats
TPU	Domain-specific tensor accelerator, commonly realized as an ASIC	High-throughput tensor operations for training or inference
FPGA	Reconfigurable logic and application-specific data paths	Deterministic low latency, custom precision, and adaptable operator pipelines
ASIC	Fixed hardware designed for a defined application or operation set	High efficiency and throughput for stable, high-volume workloads
Neuromorphic processor	Event-driven neuron and synapse structures inspired by biological information processing	Low-power event processing and spiking neural networks, mainly in specialized applications and research
Analog or in-memory computing	Matrix operations performed in or close to the memory array	Reduced data movement and potentially high energy efficiency, with continuing challenges in precision and device variability

This research does not compare accelerator technologies or develop a custom neural processor. For neural estimator deployment, it focuses on the model and software level under a defined MCU-class BMS target. The investigated choices include input representation, network architecture, structured pruning, numerical precision, recurrent-state handling, memory demand, and measured execution behaviour. The wider hardware landscape remains relevant because these choices determine how effectively a model can use current and future computing platforms.

Model-hardware co-design consequently links architecture and compression decisions to the complete target implementation. Structured pruning is useful when the resulting smaller dense tensors reduce both storage and executable operations. Quantization is useful when the selected precision reduces memory traffic or enables a supported low-precision kernel. Neither method guarantees a proportional reduction in measured latency or energy independently of memory layout, compiler behaviour, operator implementation, and processor architecture. The state of the art therefore supports embedded neural battery-state estimation in principle, while also showing why model size, numerical representation, online state handling, and measured target behaviour must be assessed together.[12, 13, 85, 86]

3.7 Research Gap and Contribution Scope

The state of research is uneven across the three requirement dimensions introduced in Figure 2.1. Practical BMS measurement and protection hardware is mature. Model-based and data-driven estimators can both reach low errors in controlled studies. Embedded demonstrations show that neural estimators can run on microcontrollers. These results are often reported separately, however, which makes it difficult to connect model accuracy, disturbed-operation behaviour, and measured embedded execution.

The first gap concerns representation. A complex history-dependent battery state does not necessarily require a large architecture if the input representation makes the relevant history accessible. The second gap concerns evaluation. Clean-condition error alone does not reveal how direct, hybrid, model-based, and data-driven estimators react to the same sensor and timing disturbances. The third gap concerns deployment. Parameter compression has to be traced through model export, stateful streaming execution, linked memory, and measured runtime before its practical benefit is known.

The three contribution chapters address these gaps in sequence. Chapter 5 studies temporal representation for a compact MLP. Chapter 6 compares estimator families under a shared robustness protocol and a matched embedded-performance assessment. Chapter 7 connects structured pruning and weight quantization to continuous recurrent inference and measured microcontroller behaviour.

Chapter 4

Experimental Basis and Hardware Ecosystem

The experimental basis of this work combines laboratory battery datasets, embedded execution targets, and the Custom BMS Platform for battery management. This combined setup makes it possible to study battery-related artificial intelligence not only as an offline modelling problem, but also as an embedded systems problem shaped by sensing, communication, firmware partitioning, and hardware constraints. The resulting experimental ecosystem forms the common basis for the ageing-estimation work, the robustness benchmark, and the embedded implementation studies.

Part of this experimental work was conducted within MG-FARM, which develops smart renewable-energy microgrids for the electrification of agricultural farms.[87, 88, 89] The project was funded through the LEAP-RE Joint Call 2021 for collaborative African–European research and innovation on renewable energy.[90] The overarching LEAP-RE programme received European Union Horizon 2020 funding under Grant Agreement No. 963530.[91] The Technische Universität Berlin subproject was funded by the German Federal Ministry of Education and Research under grant No. 03SF0684A.[92, 1]

4.1 Data Sources and Ageing Campaigns

Two battery datasets with different cell chemistries, ageing protocols, and application contexts are used. The first dataset uses NMC cells and supports the ageing-estimation study in Chapter 5. The second uses LFP cells and forms the experimental basis for both the robustness benchmark in Chapter 6 and the embedded SOC/SOH study in Chapter 7. Across both datasets, the directly measured quantities are voltage, current, temperature, and time. Dataset-side processing derives

the SOC and SOH labels used for training and evaluation. The contribution chapters then construct their own causal features from these measurements. Table 4.1 compares the two campaigns.

Table 4.1: Key properties of the two battery datasets used in this work. The NMC dataset supports the SOH ageing-estimation study in Chapter 5, while the LFP dataset provides the basis for Chapters 6 and 7.

Property	NMC Dataset	LFP Dataset
Cell type	INR18650 NMC	JGCFR18650 LFP
Nominal capacity	2,6 A h	1,8 A h
Nominal voltage	3,7 V	3,2 V
Number of cells	14	15
Campaign type	Cyclic ageing	DoE cyclic ageing
Varied parameters	$\mathrm{SOC}_{\mathrm{mean}}$, DOD, C-rate, temperature	C_{chg}, C_{dis}, DOD
Temperature	35 °C / 45 °C	25 °C (constant)
Data source	TU Berlin	TU Berlin / MG-FARM[93]
Used in chapters	Chapter 5	Chapters 6 and 7

4.1.1 NMC Dataset for Stationary-Storage Ageing Estimation

The first dataset consists of cyclic ageing tests on Li-ion INR18650 NMC cells carried out at Technische Universität Berlin. The cells have a nominal capacity of 2,6 A h, a nominal voltage of 3,7 V, a maximum charging voltage of 4,2 V, and a discharge cut-off at 2,75 V. In total, 14 cells are investigated across seven ageing scenarios, with two cells per scenario. The scenarios systematically vary four operating parameters: mean state of charge, depth of discharge, discharge C-rate, and operating temperature. Real-world stationary-storage trajectories vary with the application, renewable generation, local demand, system sizing, and energy-management strategy, so representative reference profiles differ between use cases.[94] The parametric design therefore covers distinct controlled operating windows rather than one universal charging pattern and separates the influence of the four varied parameters. Charging follows a CC-CV procedure, while discharging uses constant current. Periodic capacity tests inserted at defined intervals track the SOH progres-

sion over time and supply the capacity-based SOH reference labels used for model training.

Table 4.2: Ageing scenarios of the NMC stationary-storage dataset and the systematic variation of mean SOC, DOD, discharge C-rate, and operating temperature.

Scenario	Cell No.	SOC_{mean}	DOD	$C\text{-}Rate_{dis}$	T (°C)
1	1 and 2	50%	100%	1C	35
2	3 and 4	20%	30%	1C	35
3	5 and 6	50%	30%	1C	35
4	7 and 8	80%	30%	1C	35
5	9 and 10	50%	100%	2C	35
6	11 and 12	50%	100%	1C	45
7	13 and 14	50%	100%	2C	45

The sampling intervals depend on the respective process step, so the measurements are resampled to 1 s before feature engineering. Scenario 2 contains premature test termination and irregular measurements for cells 3 and 4. These data remain part of the training pool because they represent the type of incomplete and non-ideal records that can also occur in stationary-storage operation. Complete cells are nevertheless withheld for independent testing, as described in Chapter 5.

The state of health is defined as the ratio of the current cell capacity to the nominal capacity,[17, 14]

$$\mathrm{SOH}(t) = \frac{C(t)}{C_{\mathrm{N}}}, \tag{4.1}$$

where $C(t)$ is determined from periodic capacity tests in which the cell is fully charged and then discharged under controlled conditions. Because capacity tests exist only at discrete time points, the SOH label is linearly interpolated onto the continuous measurement timeline. Figure 4.1 shows the resulting SOH progression for all 14 cells over both equivalent full cycles and absolute time in days. Temperature has a pronounced impact on degradation, whereas the interactions between the remaining parameters are less obvious from direct visual inspection alone.

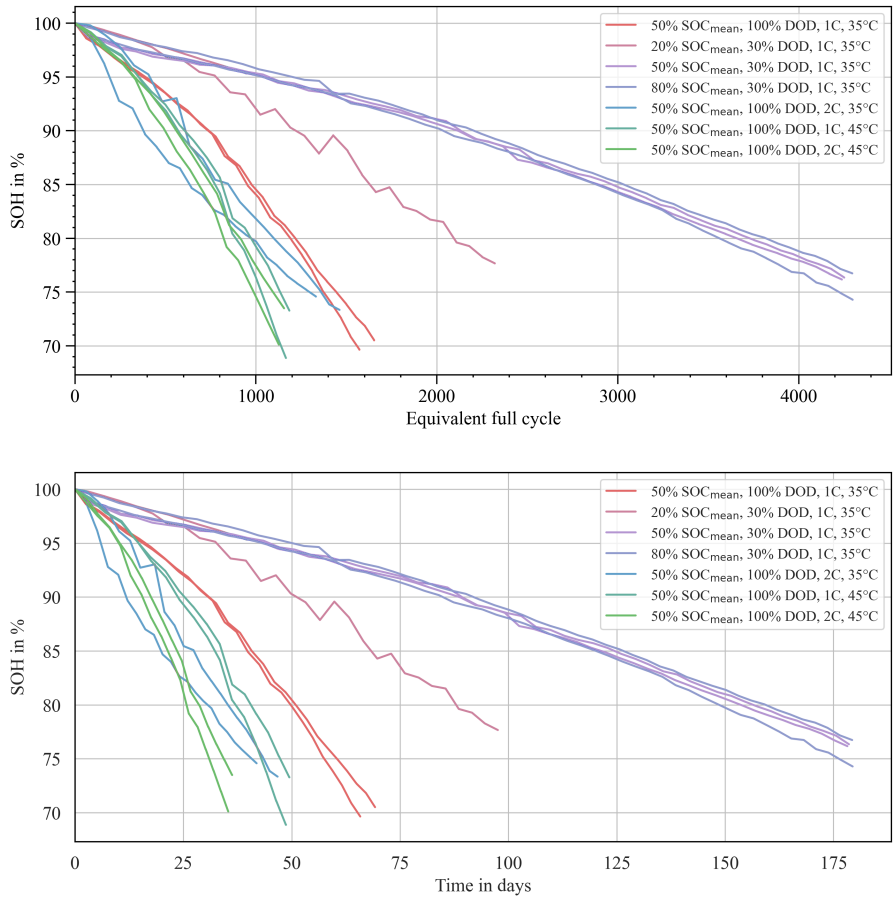


Figure 4.1: SOH progression of the 14 investigated NMC cells: top over equivalent full cycles and bottom over absolute time in days. Colors denote the seven ageing scenarios, with two cells per scenario.

This dataset is used exclusively in Chapter 5. The present section defines the cells, test scenarios, and SOH labels, while the contribution chapter concentrates on the cumulative-current and lag-sequence feature engineering.

4.1.2 LFP Dataset for Robustness Benchmarking and Embedded Deployment

The second dataset is the published LFP DoE Cycle Ageing Dataset recorded at Technische Universität Berlin within the MG-FARM project.[93] It uses commercially available JGCFR18650-1800mAh-3.2V LFP cells with a nominal capacity of 1,8 A h and a nominal voltage of 3,2 V. The ageing campaign follows a three-factor design-of-experiments scheme conducted at a constant ambient temperature of 25 °C. The three varied factors are the charge rate C_{chg} from 0.3 C to 1.0 C, the

discharge rate C_{dis} from 0.5 C to 3.0 C, and the depth of discharge DoD from 38% to 71%. A mixed full and fractional factorial plan assigns 15 test cells across eight operating points. Between ageing loops, dedicated check-up phases with capacity tests, open-circuit-voltage sweeps, and hybrid pulse-power characterisation are inserted. Stationary-storage load profiles derived from residential PV home storage are additionally applied so that the data contain both tightly controlled diagnostics and application-oriented dynamic behaviour. Raw measurements are available at 1 Hz and include voltage, current, terminal temperature, and auxiliary channels.

A design of experiments is used here to structure the ageing campaign so that several operating influences can be varied systematically within a manageable number of cells.[95] Instead of changing only one operating variable at a time, the DoE plan combines charge rate, discharge rate, and depth of discharge into defined operating points. This makes it possible to observe not only the isolated influence of one factor, but also differences that arise from their combined loading conditions. For the later SOC and SOH studies, this is important because the estimators are not trained and evaluated on one narrow cycling regime. They see cells that have aged under different current rates and cycling depths, while the check-up phases still provide comparable reference information across the campaign. The DoE structure therefore gives the dataset both experimental control and operational diversity, which is useful for robustness benchmarking and embedded estimator validation.

Figure 4.2 visualizes the three-factor operating-point design. Figure 4.3 shows a representative segment of the resulting trajectory structure, combining cyclic ageing blocks, check-up phases, and an approximately one-day stationary-storage load-profile segment. This profile provides one application-oriented example within the broader use-case-dependent profile space. Figure 4.4 shows the SOH spread across all cells and operating points over the full campaign duration and identifies the fixed training, validation, and independent holdout test sets used by the robustness benchmark.

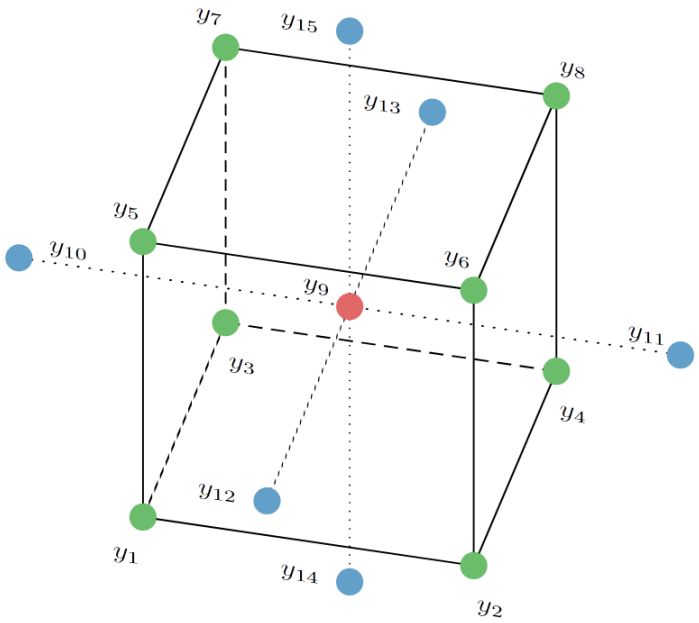


Figure 4.2: Design-of-experiments cube of the LFP ageing dataset. The three axes correspond to charge C-rate, discharge C-rate, and depth of discharge. The highlighted points define the factorial test matrix assigned to the 15 ageing cells.

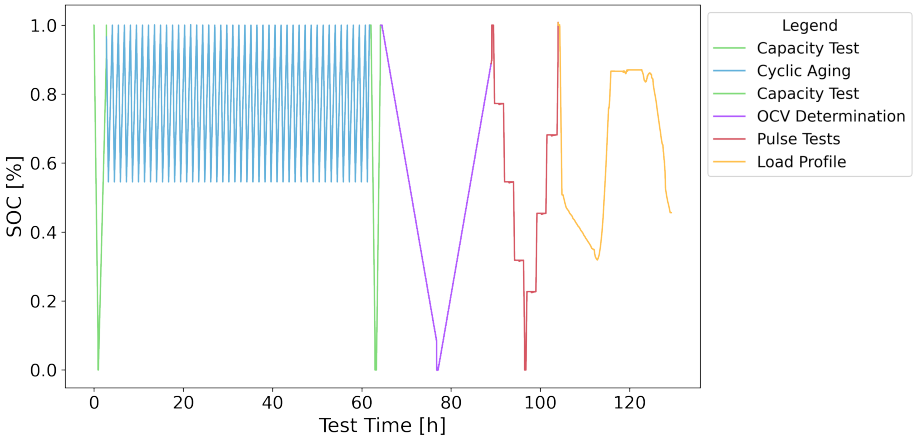


Figure 4.3: Representative SOC trajectory illustrating the experimental structure of the LFP campaign. The sequence combines cyclic ageing operation, diagnostic check-up blocks, and an approximately one-day stationary-storage load-profile segment that provides one application-oriented example within the broader use-case-dependent profile space.

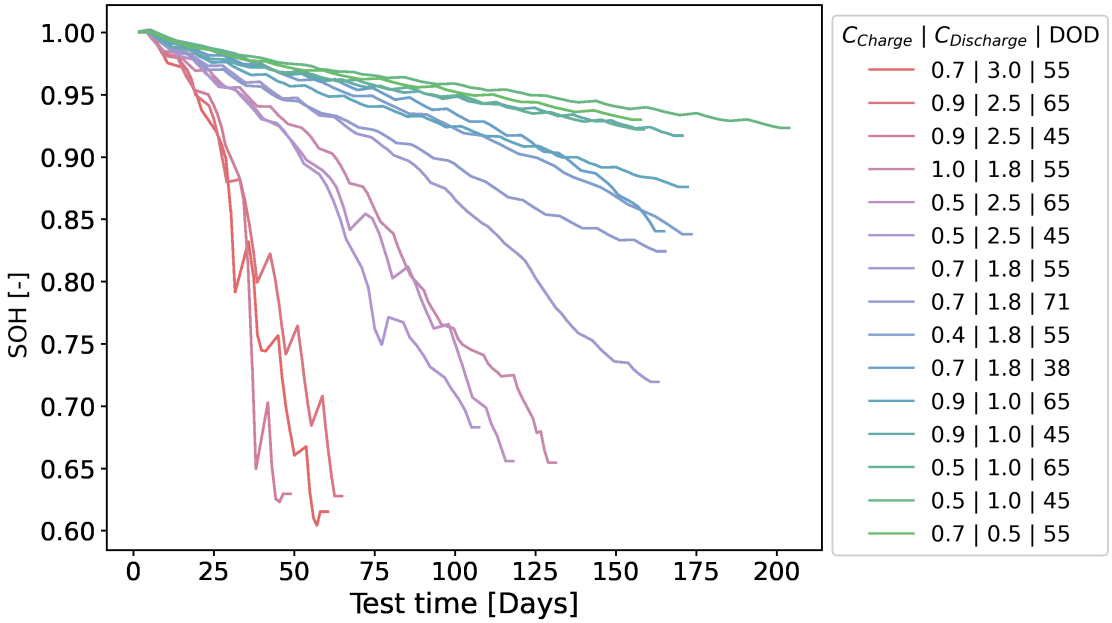


Figure 4.4: Measured SOH progression across the 15 LFP cells over the complete ageing campaign. Colors distinguish the fixed training, validation, and independent holdout test sets. The legend reports the charge C-rate, discharge C-rate, and depth of discharge assigned to each trajectory. Every SOH trajectory is constructed from discrete capacity check-ups and interpolated between the measured anchors.

The local upward movements in Figure 4.4 require a distinction between physical degradation and measured available capacity. The plotted points originate from repeated capacity check-ups and are not model predictions. A later check-up can yield a temporarily higher capacity because relaxation and reversible recovery, conditioning before the test, small temperature differences, current-integration uncertainty, and voltage cut-off detection all affect the measured result.[17] The available data do not isolate one of these effects as the unique cause of every increase. The correct interpretation is therefore a downward long-term ageing trend with locally non-monotonic measurement anchors. Linear interpolation carries these anchor variations into the displayed SOH trajectories.

The robustness study uses a fixed cell-disjoint split consisting of six training cells, three validation cells, and six independent holdout test cells. Neural training, scaler fitting, pruning, model selection, and the identification of the HECM parameter surfaces use only the training and validation sets. The holdout cells are reserved for the final nominal, disturbance, recovery, and embedded comparisons. Table 4.3 documents the complete assignment and the measured duration, SOH, temperature,

and load coverage of every trajectory.

Table 4.3: Measured coverage and exact cell-to-set assignment for the fixed training, validation, and independent holdout test split. The 95th-percentile absolute C-rate is calculated over each complete 1-Hz trajectory. Load classes are defined only for the holdout test set. Its high-load class contains one cell and is exploratory.

Cell	Split	Duration [d]	SOH range	T range [°C]	P95 $\lvert C_{\mathrm{rate}} \rvert$	Test-set load
C01	Training	198.6	0.922–1.000	24.3–29.4	1.01	–
C03	Training	153.3	0.921–1.000	25.9–30.7	1.00	–
C05	Training	166.1	0.917–1.000	25.2–30.6	1.00	–
C11	Training	113.2	0.656–1.000	25.6–42.8	2.48	–
C17	Training	158.7	0.719–1.000	25.8–34.9	1.75	–
C23	Training	167.8	0.837–1.000	26.1–37.2	1.74	–
C07	Validation	167.6	0.874–1.000	25.2–32.0	1.01	–
C19	Validation	160.4	0.840–1.000	26.3–33.9	1.75	–
C21	Validation	160.8	0.824–1.000	26.2–37.6	1.75	–
C09	Test	102.6	0.682–1.000	25.0–41.1	2.51	Middle
C13	Test	44.7	0.623–1.000	24.9–42.0	2.50	Middle
C15	Test	60.7	0.627–1.000	25.1–43.6	2.53	Middle
C25	Test	127.1	0.655–1.000	25.8–37.1	1.76	Low
C27	Test	152.0	0.930–1.000	26.0–31.9	0.70	Low
C29	Test	56.0	0.604–1.000	25.5–49.4	3.00	High*

The holdout set spans two descriptively low-load trajectories, three middle-load trajectories, and one exploratory high-load trajectory, classified by the measured 95th-percentile absolute C-rate. These labels summarize whole-cell load coverage and are not used as independent statistical samples. Figure 4.5 complements the tabulated split by showing the SOH and thermal envelopes and the fraction of each trajectory that lies in the fresh, mid-life, and aged SOH ranges. One low-load trajectory remains entirely within the fresh range. The robustness protocol therefore contains 16 available cell/SOH combinations rather than 18.

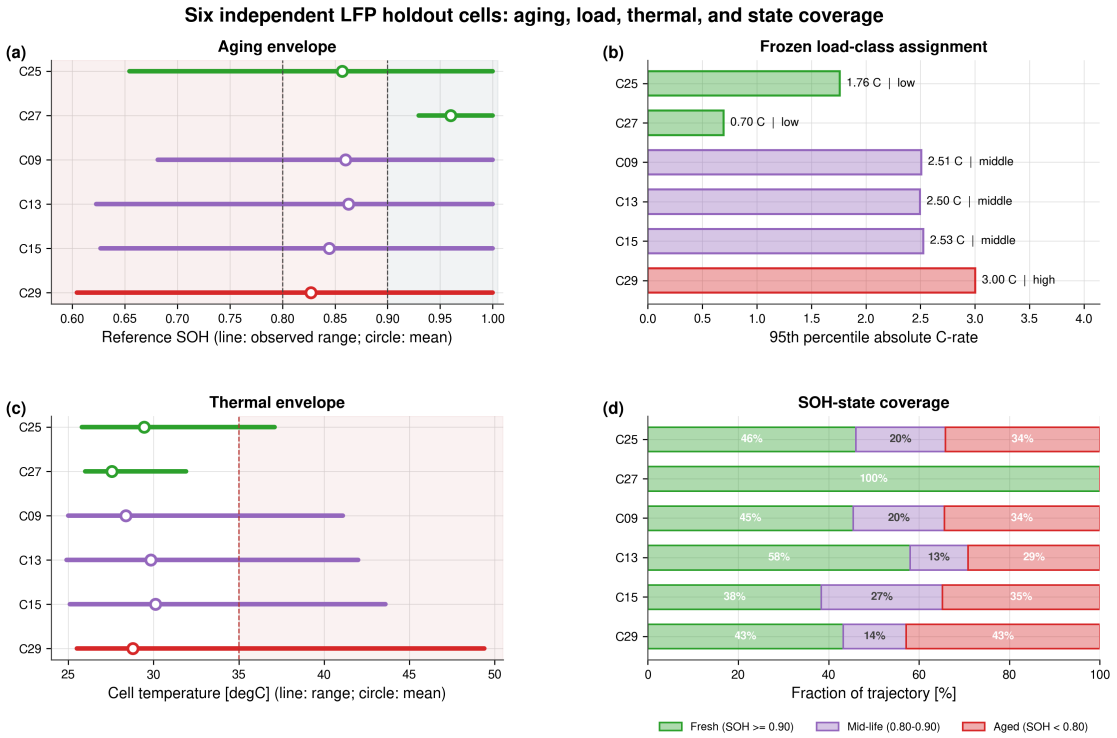


Figure 4.5: Measured coverage of the six-cell independent holdout test set. The panels show observed SOH, the descriptive load-class assignment, the thermal envelope, and full-trajectory SOH-state coverage. The high-load group contains one exploratory trajectory, while the low- and middle-load groups contain multiple cells.

4.1.3 SOC and SOH Reference-Label Construction

The LFP dataset contains measurements from which the labels for Chapters 6 and 7 are constructed. For a sample interval Δt_k, the signed transferred charge is

$$Q_k = \frac{I_k \Delta t_k}{3600}, \tag{4.2}$$

and the measured capacity of a dedicated capacity-test interval $\mathcal{T}_{\text{cap}}$ is

$$C_k = \sum_{j \in \mathcal{T}_{\text{cap}}} |Q_j| \,. \tag{4.3}$$

The embedded-AI study uses the conventional nominal-capacity definition

$$\text{SOH}_k = \frac{C_k}{C_{\text{nom}}}, \tag{4.4}$$

where C_{nom} is the nominal cell capacity. The robustness benchmark instead normalizes the capacity to the first measured reference capacity of its selected trajectory,

$$\mathrm{SOH}_{\mathrm{ref},k} = \frac{C_k}{C_{\mathrm{ref},0}}, \tag{4.5}$$

which sets the beginning of that trajectory to one and isolates relative capacity loss during the benchmark. The two definitions express the same capacity-based concept but use different denominators for their respective evaluation objectives.[4, 18]

The SOC label is derived from Coulomb counting with drift compensation,

$$\mathrm{SOC}(t) = \mathrm{SOC}(t_0) - \frac{1}{C_{\mathrm{app}}} \int_{t_0}^{t} I(\tau)\,\mathrm{d}\tau, \tag{4.6}$$

where C_{app} is the capacity reference used by the respective processing path. The upper reference is reset to SOC = 1 only after the voltage has remained near the upper limit for 300 s. This sustained condition captures a constant-voltage full-charge phase and prevents a short voltage excursion from being interpreted as a full cell. For the robustness trajectory, the equivalent discrete reference is

$$\mathrm{SOC}_{\mathrm{ref}} = 1 + \frac{Q_c}{C_{\mathrm{ref},0}}, \tag{4.7}$$

with $Q_c = 0$ at the sustained upper-voltage anchor and $Q_c = -C_{\mathrm{ref},0}$ at the lower-voltage anchor.

These quantities are reference labels rather than online estimator inputs. Their construction uses complete capacity-test intervals and confirmed full-charge phases that are unavailable during ordinary deployment. The feature pipelines in Chapters 6 and 7 use only causal measurement-derived information and are described in the respective contribution chapters.

4.2 Embedded Targets and Execution Chain

The embedded work relies on two complementary targets: the final Custom BMS Platform hardware and an STM32H755-based evaluation-board setup for rapid firmware iteration and controlled implementation tests. The integrated BMS hardware is used whenever sensing, balancing, protection, communication, telemetry, and on-board state estimation must be validated as one coupled system rather than

as isolated software modules, whereas the evaluation-board setup is used whenever estimator code, toolchain details, memory demand, and runtime behaviour have to be checked quickly before transfer to the full Custom BMS Platform.

In both cases, the deployment path follows the same practical sequence: state-estimation methods are first prototyped in Python or MATLAB, then rewritten as plain C modules, compiled into the STM32 firmware, and executed locally on the target hardware instead of off-board. During validation, the firmware records voltage, current, temperature, and relevant intermediate estimator variables. After each run, these traces are aligned with higher-fidelity host-side SOC and SOH reference labels. The workflow does not use a hardware-in-the-loop setup, which keeps the evaluation focused on firmware execution, measured board behaviour, and comparison against host-side reference traces. This split between evaluation-board work and integrated BMS operation is important for the present research because it separates pure implementation questions from questions of sensing, protection, and full system integration.

4.3 Custom BMS Platform

The custom laboratory hardware developed in this work is referred to simply as the Custom BMS Platform. It is a physically realised 4s laboratory BMS designed to acquire cell-voltage and temperature measurements, supervise electrical and thermal operating limits, support protective disconnection through the relay path, execute on-board SOC and SOH estimation, perform passive balancing, and provide the technical basis for controlled experiments with embedded battery intelligence. A central reason for the in-house design is adaptability. In contrast to a commercial off-the-shelf BMS, the Custom BMS Platform gives direct access to the sensing chain, firmware structure, interfaces, and estimator integration. State-estimation methods can therefore be exchanged and modified, then evaluated under repeatable measurement and execution conditions while the underlying hardware remains unchanged. The board should be understood less as a finished product and more as a configurable research platform for rapid iteration, customisation, and comparative testing.

4.3.1 System Objective and Laboratory Scope

The present implementation is a compact and reproducible 4s laboratory BMS for battery ageing studies and embedded state-estimation experiments at TU Berlin EET. The four-cell configuration was selected as a compromise between system relevance and experimental control. It retains the essential pack-level functions of individual cell monitoring, cell-wise balancing, shared current measurement, protective switching, and spatial temperature sensing, while keeping instrumentation, cell access, replacement, and repeated testing manageable in the laboratory. The platform is therefore intended for ageing tests, cyclic and calendar experiments, pack and general hardware tests, and representative charging and discharging profiles rather than direct qualification for large field-deployed battery racks. Exact field loads are not reproduced one-to-one. Instead, selected operating profiles are applied within a controlled setup that supports calibration, diagnostics, repeated validation, and the generation of well-documented datasets under repeatable conditions.

This laboratory orientation allows different estimators to be compared under the same measurement chain, including classical observers and data-driven or neural methods executed directly on the STM32 controller. One hardware base can support multiple estimator variants without rebuilding the complete BMS around each method. The validation campaigns use duty cycles derived from decentralized stationary-storage applications because these systems form the intended use case and combine alternating charging, discharging, and idle periods with long autonomous operating intervals and limited maintenance access. These application-derived profiles exercise accumulated SOC drift, thermal variation, and history-dependent SOH estimation while remaining repeatable. The recorded quantities include voltage drift, current ripple, temperature gradients, and embedded estimator outputs. The profiles reproduce selected electrical and thermal demands of the target application, but not the complete variability of field operation. Figure 4.6 shows the physical laboratory platform on which these functions are integrated. The resulting transfer limitations are discussed in Section 8.5.

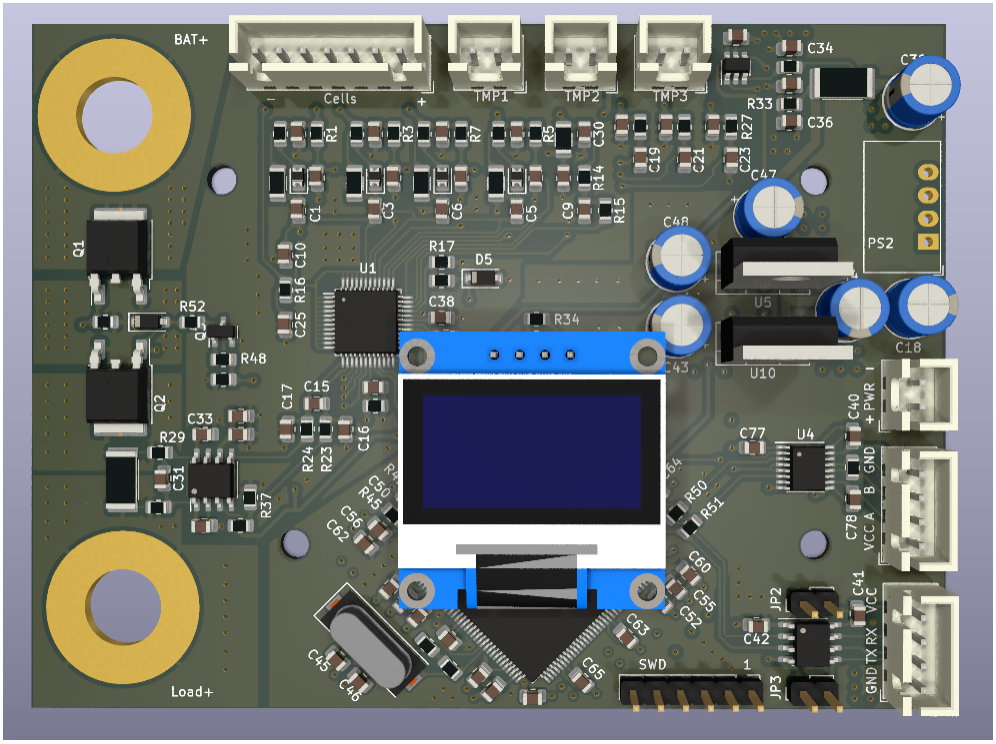


Figure 4.6: Board-level view of the Custom BMS Platform with integrated sensing, controller, communication, and auxiliary interfaces.

4.3.2 Hardware Architecture

The Custom BMS Platform can be understood as three tightly coupled hardware domains: the TLE9012DQU-based sensing and balancing stage, the current-measurement and protection interface, and the controller/communication island around the dual-core STM32H755. The board layout keeps the high-current zone, shunt, and relay path physically separated from the low-voltage controller and communication domain. This separation matters not only for safety, but also for electromagnetic compatibility and stable measurement quality. Figure 4.7 summarizes the signal and control paths between these domains.

The Infineon TLE9012DQU forms the measurement backbone of the pack. In the present 4s platform configuration, it supervises four series-connected cells with buffered RC filtering and provides cell-voltage and temperature acquisition together with passive balancing capability. The active front-end includes balancing circuitry, while unused monitor inputs are terminated according to the 12-to-4 cell adaptation of the reference design so that measurement integrity is preserved on the active channels. Temperature instrumentation is extended beyond the standard

TLE inputs by an NTC harness that can connect additional probes at cell terminals, busbars, or ambient reference points, which provides a finer spatial thermal picture during ageing and state-estimation tests. The controller domain uses an STM32H755ZITx dual-core microcontroller. This design choice is central because it creates a clean architectural separation between conventional BMS functionality and the estimator layer investigated in this work.

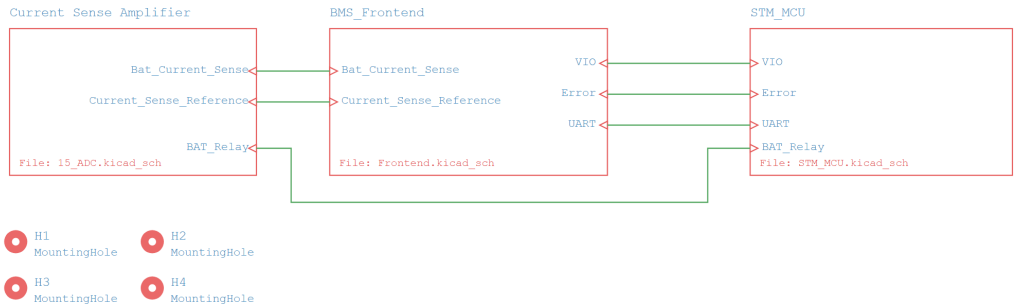


Figure 4.7: Reduced system-level block view of the Custom BMS Platform showing the interaction between current sensing, BMS front-end, and microcontroller domain.

4.3.3 Firmware and Execution Concept

The firmware concept is intentionally centred on the dual-core STM32 architecture. The Cortex-M4 core handles the conventional BMS tasks, including measurement acquisition, limit supervision, protective actuation, balancing coordination, communication, and general system management. The Cortex-M7 core is reserved for the computationally heavier state-estimation routines. This partition allows the timing of essential monitoring and control tasks to remain stable while the estimator layer is changed or its computational demand varies. Different SOC and SOH methods can be integrated, tested, and replaced on the M7 side without redesigning the complete BMS firmware structure.

The concept is useful wherever established BMS functions have to coexist with algorithms that evolve during development or differ between applications. It supports comparative evaluation of model-based, hybrid, and neural estimators, the testing of compressed model variants, and the integration of ageing diagnostics without changing the measurement and protection layer for every experiment. The same separation is relevant to modular stationary storage, remote microgrids, and laboratory ageing systems, where reliable local BMS operation must be maintained while

state-estimation methods are refined. The main value of the firmware architecture is therefore the reusable execution boundary it creates between deterministic BMS operation and customisable embedded state estimation.

4.3.4 Experimental Relevance for Embedded AI

The Custom BMS Platform is not only a monitoring board, but an experimental platform for comparing estimator classes under realistic embedded conditions. Its main value for this research lies in controllability and openness: the same measurement hardware can be reused while state-estimation methods are changed, refined, compressed, or replaced. This makes it possible to compare classical model-based observers and data-driven or neural approaches on one common embedded basis instead of distributing them across unrelated hardware environments. The Custom BMS Platform therefore closes the gap between algorithm development and embedded validation, and it was chosen precisely because it can be modified and extended in ways that would be difficult with a purchased closed BMS. At the same time, the 4s implementation remains deliberately compact: it serves as a reproducible laboratory baseline from which later transfer to larger stacks or a 16s/48 V variant can be reasoned without changing the overall control and estimator architecture.

Chapter 5

Temporal Feature Engineering for Neural SOH Estimation in Stationary Storage Systems

Chapter 4 established the datasets, reference labels, embedded targets, and hardware environment shared by the contribution studies. The present chapter begins the methodological progression with nominal ageing estimation. It moves from this common experimental basis to the construction and evaluation of one data-driven SOH estimator before the later chapters add disturbed measurements and microcontroller constraints. In this way, the chapter provides the first algorithmic step between the technical foundations and the deployment-oriented investigations.

The analysis develops the ageing-estimation study published by Rzepka et al. within this broader research argument.[1] The general differences between model-based and data-driven estimation, the role of Kalman filtering, and the foundations of MLP and recurrent architectures are already discussed in Chapter 3. They are therefore not repeated here. Section 5.1 positions the specific study relative to prior work, after which the chapter concentrates on its feature engineering, model configuration, evaluation, and results.

5.1 Study Objective and Relation to Prior Work

SOH estimation is important for stationary storage because storage units must remain available over long maintenance intervals and under changing renewable-power and load conditions.[96, 97] Classical capacity tests provide reliable diagnostic references, but they interrupt normal operation and are not continuously available. The estimator must therefore infer degradation from operational measurements acquired between these reference tests.

The general distinction between model-based and neural estimators is discussed in Sections 3.3 and 3.4.4. Within SOH estimation, model-based alternatives include Kalman-filter-based joint estimators, particle-filter approaches, and fractional-order ageing models. Reported estimation errors are about 1,0 % for dual adaptive Kalman filtering, 3,0 % for capacity-fade estimation, 4,0 % for power-fade estimation, and 3,1 % for remaining-capacity estimation with a fractional-order model.[30, 34, 35] These values were obtained with different cells, datasets, and error definitions and are therefore not directly comparable. They nevertheless demonstrate the achievable accuracy of model-based approaches when parameter identification, filter tuning, initial-state knowledge, and the underlying physical or empirical ageing description remain valid.

Measurement-driven SOH methods reduce the need for explicit battery parameterisation, but differ in the signals and operating intervals they require. The literature includes partial-voltage and incremental-capacity methods, autoregressive and support-vector approaches, fuzzy-entropy features, and combinations of deep learning with Kalman filtering. Reduced charging-voltage windows and hybrid learning-filter approaches have reported average errors below approximately 2,5 % and about 2,0 %, respectively.[98, 99, 100, 101, 102, 103, 104] At the same time, these studies show limitations that are relevant to stationary operation, including dependence on suitable charging intervals, feature validity, data quality, the observed degradation range, and transfer to changed operating conditions.[36, 37]

Recent hybrid and sequence-based models process voltage, current, and temperature histories directly or combine learned representations with physical constraints. Sequence-to-sequence degradation models have reported mean absolute percentage errors below 2,5 % for the capacity trajectory and 6,5 % for the remaining usable cycles when 20 % of the available data were used. With 50 % of the data, the reported errors decreased below 1,3 % and 5,0 %, respectively.[41, 42, 105, 106] These approaches demonstrate the value of temporal information, but they combine the effect of the input representation with the effect of the sequence architecture.

The present study isolates the representation question. A compact MLP is used deliberately to examine whether voltage, temperature, and current-throughput history can be transformed into an informative input without relying on a recurrent architecture. This choice does not imply that an MLP is universally preferable. It provides a transparent baseline for assessing how cumulative current and lagged measurements affect ageing estimation while retaining a comparatively sim-

ple model structure. The NMC cells, ageing scenarios, periodic capacity tests, resampling, irregular records, and SOH-label construction are documented in Section 4.1, Table 4.2, and Figure 4.1. The following section therefore begins directly with the feature engineering applied to those measurements and labels.

5.2 Temporal Feature Engineering and Data Preparation

The SOH definition, the construction of SOH labels from periodic capacity tests, and the resulting progression across all 14 NMC cells are described in Section 4.1 (Equation 4.1, Figure 4.1). The present section therefore concentrates on preparing the measurement-derived inputs and their temporal representation for supervised learning.

5.2.1 Cumulative Current

The preprocessing pipeline is built around the argument that instantaneous current alone is not sufficient as an explanatory variable for ageing estimation. Cumulative current is introduced as the central preprocessing feature because it captures the two quantities that are most relevant at this stage: the time dependence of the measurements and the cumulative throughput stress experienced by the cell. In its basic form, integrating the absolute current over time encodes not only the present loading level but also the prior electrochemical exposure and, therefore, a compact proxy for the cyclic age that is not visible from an instantaneous sample alone,

$$Q(t) = I_{\text{cumulative}}(t) = \int_0^t |I(\tau)|\, d\tau. \tag{5.1}$$

The two current directions are retained separately as

$$\begin{aligned} Q_{\text{pos}}(t) &= \int_0^t \max(I(\tau), 0)\ d\tau, \\ Q_{\text{neg}}(t) &= \int_0^t |\min(I(\tau), 0)|\ d\tau. \end{aligned} \tag{5.2}$$

The formulation is also physically motivated by the interaction between current and voltage within the temporal window represented by one lag sequence. Their joint evolution carries ageing-relevant information that is closer to an impedance-

like gradient than a single current reading. The preprocessing distinguishes the positive and negative current contributions because charging and discharging can induce different voltage gradients and different effective ageing signatures. The corresponding error reduction can be observed empirically, but because these effects are strongly collinear, the improvement cannot be attributed cleanly to one isolated mechanism alone. In this formulation, cumulative current becomes the feature that allows a simple feedforward network to access ageing-relevant temporal context without explicitly using a recurrent structure. Voltage and temperature are retained as further direct inputs, so the final feature set combines operating-state information with a compact representation of the prior load exposure.

5.2.2 Correlation Analysis

Temperature has a pronounced impact on degradation, which motivates the correlation analysis as a structured view of the relation between operating parameters and SOH.

To better understand which measured quantities should be retained for learning, the aligned data obtained after resampling and SOH-label interpolation are evaluated using the pairwise Pearson product-moment correlation coefficient. For two variables x and y with n aligned samples, the coefficient is

$$r_{xy} = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^{n}(x_i - \bar{x})^2}\sqrt{\sum_{i=1}^{n}(y_i - \bar{y})^2}}. \tag{5.3}$$

The calculation is applied pairwise to SOH, voltage, temperature, and the separate cumulative charging and discharging currents $Q_{\mathrm{pos}}(t)$ and $Q_{\mathrm{neg}}(t)$. Figure 5.1 shows the non-redundant lower triangular part of the resulting symmetric correlation matrix. Temperature and both cumulative-current variables show the clearest direct linear relation to SOH. Voltage, although itself related to temperature, does not show a comparably direct relation to SOH in this analysis.

The correlation analysis can only reveal linear dependencies and therefore serves only as a starting point for feature selection rather than as a substitute for the later model-based evaluation. Non-linear effects may still be relevant and can subsequently be captured by the neural network. Even with this limitation, the analysis provides a concrete justification for retaining temperature and cumulative-current descriptors as core inputs for the final model.

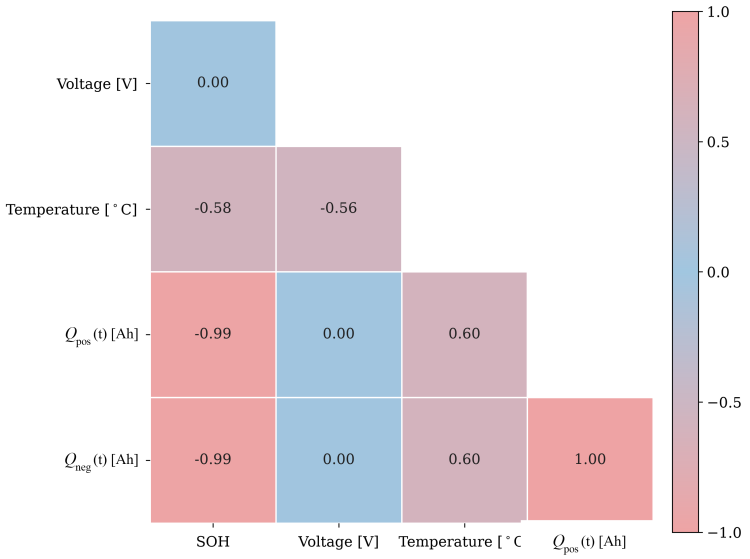


Figure 5.1: Pairwise Pearson correlation coefficients between SOH and the candidate input variables, including temperature and the separate cumulative currents $Q_{\mathrm{pos}}(t)$ and $Q_{\mathrm{neg}}(t)$. Only the non-redundant lower triangular part of the symmetric matrix is shown. The symmetric OKLAB scale maps zero correlation to blue and increasing absolute correlation magnitude toward red. Consequently, coefficients of -1 and $+1$ receive the same red endpoint colour, while the sign and exact value remain visible in the printed coefficient.

5.2.3 Formation of the Lag Sequence

Each learning sample is constructed as a lag sequence of voltage, temperature, and cumulative current, with the SOH at the current point in time used as the corresponding label, as illustrated in Figure 5.2. This transforms the continuous measurement stream into structured sliding-window samples that can be processed by a feedforward network without passing the complete life history of an individual cell directly into the model. Two parameters define the temporal representation. The time resolution Δt specifies the spacing between successive entries, while the number of lag steps L determines how many entries are retained. Their product, $L\Delta t$, gives the represented history duration.

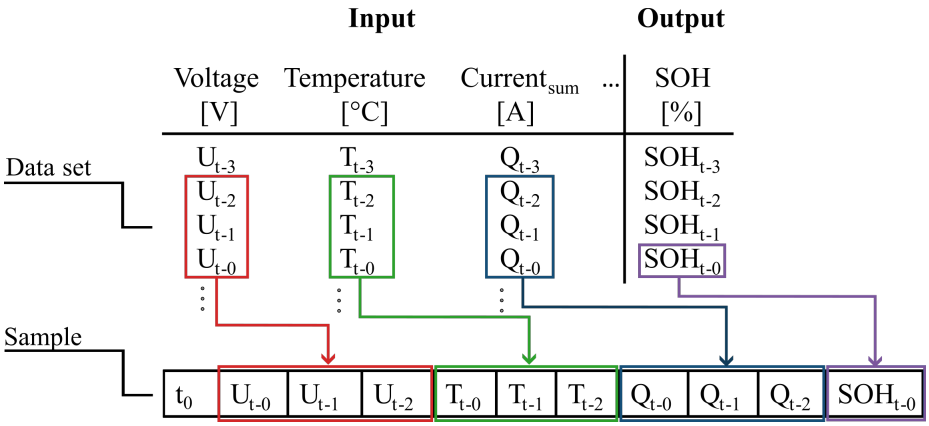


Figure 5.2: Illustration of lag-sequence construction from raw cell measurements into supervised MLP samples. Voltage, temperature, and cumulative current form the structured input history, while the SOH at the current time step serves as the corresponding label.

A configuration of $\Delta t = 30\,\text{min}$ and $L = 12$, corresponding to 6 h of history, is used as the initial search point. It provides a broad window relative to the approximately 20 min to 1 h charging and discharging intervals present in the investigated scenarios. The choice is subsequently evaluated rather than retained without comparison. The tested time resolutions are 1, 10, 30, and 60 min, combined with 8, 12, 16, and 20 lag steps. Among these combinations, 16 steps at a resolution of 10 min, representing 160 min of measurement history, yield the lowest error across the exemplary cells considered in Section 5.5.

The time resolution has to match the actual charging and discharging behaviour. If the spacing is too coarse, relevant dynamics disappear. If it is too fine, a fixed number of steps covers only a short operating interval, while extending the number of steps increases the input dimension and can make training less effective. If short pulses fall within one interval, they are represented only through the aggregated feature values of that interval. This design can also be linked to the sampling theorem: the spacing between data points has to remain compatible with the temporal structure of the charging and discharging cycles so that the network can still recognise the relevant periodic behaviour.[107] Because the raw measurements are resampled before sequence generation, the preprocessing step also acts like a low-pass filter and strongly attenuates high-frequency noise components, so noise is not treated as a dominant issue for the present estimation setup.

The lag sequence does not predict a future voltage, current, or temperature trajectory. Instead, the model learns a mapping between the recent operating history

and the SOH label at the end of that window. Similar local histories from different cells provide recurring combinations of electrical loading, temperature, and accumulated charge throughput from which ageing-related relationships can be learned. The objective is therefore not to identify a cell from a unique signal pattern, but to transfer relationships observed across the training cells to a previously unseen cell whose operating history remains within the represented data domain. Repeating this present-state estimate over time produces an SOH trajectory, whereas forecasting future degradation would require an additional prediction horizon and assumptions about future operation.

5.2.4 Training, Validation, and Test Splits

The available data are separated into training, validation, and test subsets, as summarised in Figure 5.3. The training split is used to adapt the network weights, while 20% of the available training sequences are reserved for validation. The validation split is used during training to monitor overfitting and generalisation behaviour and to support hyperparameter decisions without exposing the model to the final test cells. At the same time, several complete cells are withheld from the training process for the final performance assessment. In total, four out of fourteen cells are retained for this independent accuracy evaluation. This choice is important because the method is intended to generalise to previously unseen cells and not only to previously unseen windows extracted from known cells. The explicit separation of training, validation, and test data therefore makes the resulting performance assessment more robust and more informative.

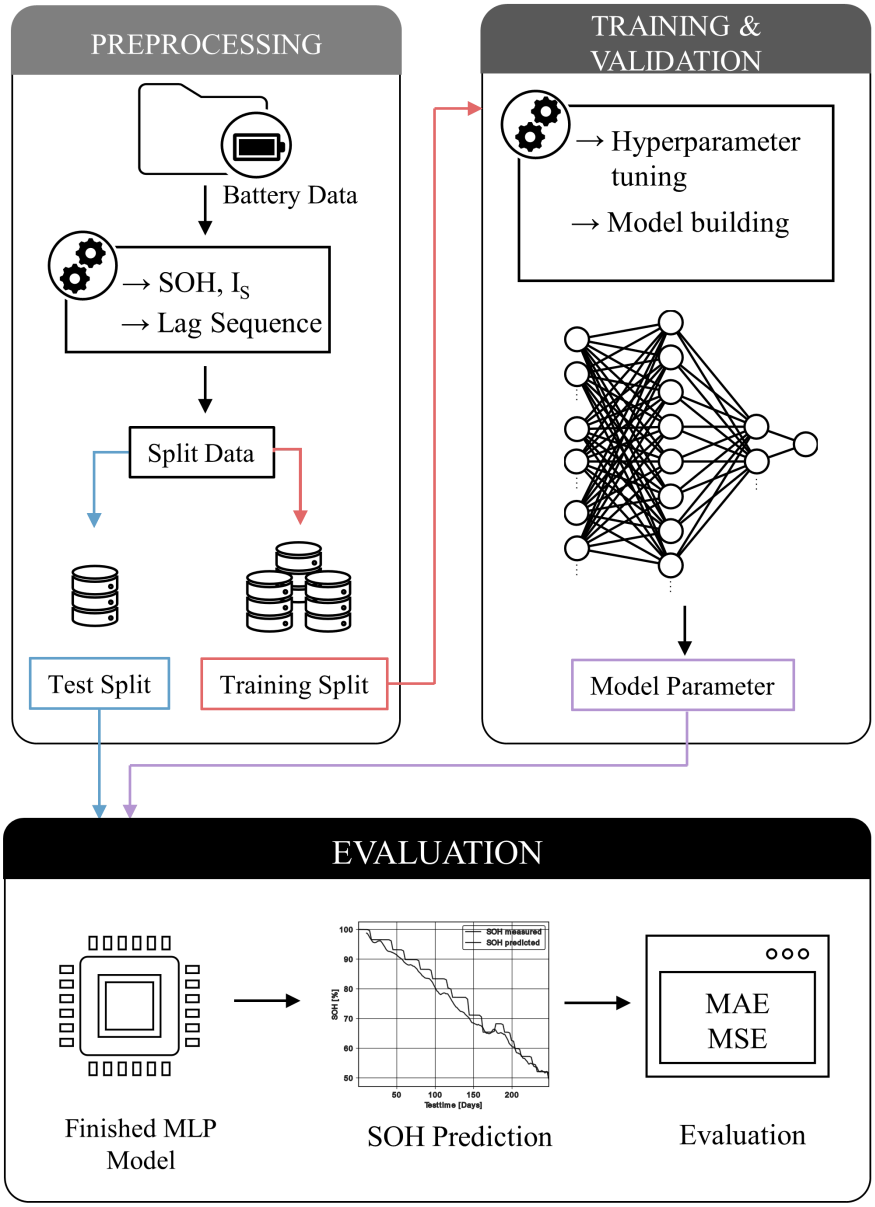


Figure 5.3: Workflow of the proposed MLP-based SOH estimation approach, from raw-cell preprocessing and data splitting through training and hyperparameter tuning to prediction and final evaluation.

5.2.5 Data Scaling

All input features are scaled to the interval between 0 and 1 before training.[108] This step is necessary because the numerical ranges of the features differ strongly: voltage remains in a narrow interval between 2,75 V and 4,2 V, whereas cumulative

current can rise to several thousand ampere-hours across the full ageing experiment. Uniform scaling improves numerical stability, supports convergence during optimisation, and avoids a situation in which large-magnitude inputs dominate the network weights while smaller but still relevant signals are effectively ignored.[108] The same scaling convention must be applied consistently to the training, validation, and test data so that the learned mapping is evaluated under comparable feature ranges and the resulting performance metrics remain interpretable across all splits.

5.3 MLP Selection and Architecture

The MLP is selected because the study examines whether explicit feature engineering can provide sufficient temporal context for SOH estimation without requiring an internally stateful model. After resampling and lag-sequence formation, every sample is a fixed-length tabular vector whose entries represent voltage, temperature, and cumulative-current values at defined time offsets. Fully connected layers can combine all of these feature–time entries directly. A convolutional architecture would impose a local and weight-sharing structure that is not motivated by the small set of heterogeneous measurement channels. An LSTM could instead learn an internal temporal representation, but this would make it more difficult to separate the contribution of recurrent memory from the contribution of the cumulative-current and lag features. It would also introduce additional state handling, parameters, and tuning effort.[109]

The feedforward architecture therefore provides a controlled way to evaluate the proposed input representation while retaining a comparatively direct training and inference path. Model flexibility is still adapted to the task through the lag-window design, network depth, node count, and training parameters. These quantities are searched efficiently with Hyperband rather than fixed solely from the initial architecture.[110]

The general feedforward principle is introduced in Section 3.4.2 and illustrated in Figure 3.2. The study-specific input and output structure is shown in Figure 5.4. Each position of the lag sequence contributes voltage, temperature, and cumulative-current information to the flattened input layer. The fully connected hidden layers combine these feature–time entries and produce one linear SOH estimate. The initial search configuration is intentionally small and uses one hidden layer with eight

nodes. Network depth and width are then selected through the hyperparameter search. In the tuned configuration, summarised in Table 5.1, the model grows into a deep dense architecture with nine hidden/output stages and a final linear regression output. The resulting structure remains conceptually feedforward, while its concrete layer widths are adapted to the ageing-estimation task.

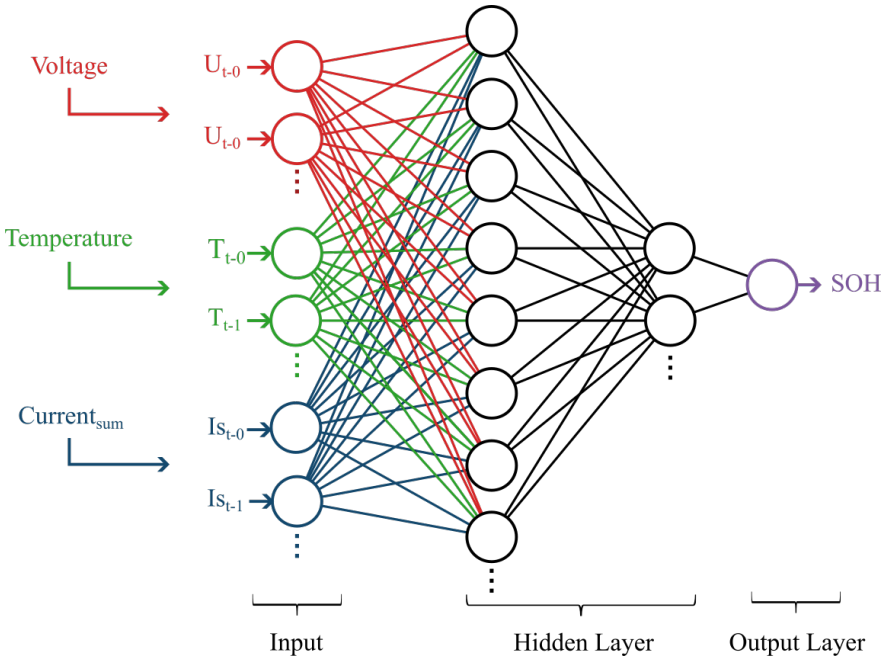


Figure 5.4: Study-specific MLP structure for SOH estimation. The flattened input layer contains voltage, temperature, and cumulative-current values from the lag sequence. These feature–time entries are fully connected to the dense hidden layers, and a single linear output provides the SOH estimate. The displayed node counts are schematic, while the tuned layer widths are reported in Table 5.1.

Table 5.1: Final tuned configuration of the SOH-estimation MLP after hyperparameter optimisation, listing the number of nodes and the activation function used in each layer of the network.

Layer	Nodes	Activation Function
0	128	ReLU
1	256	ReLU
2	256	ReLU
3	256	ReLU
4	128	ReLU
5	512	ReLU
6	128	ReLU
7	256	ReLU
8	256	ReLU
9	1	Linear

5.4 Evaluation Metrics and Hyperparameter Optimization

This section defines the error metrics used for training, validation, and final testing before summarising the hyperparameter tuning of the MLP. During training and validation, model quality is monitored primarily with the mean squared error,

$$\mathrm{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2, \tag{5.4}$$

where n is the number of evaluated samples, y_i is the measured SOH value of sample i, and $\hat{y}_i$ is the corresponding prediction. After training, the independent test cells are evaluated with both MSE and mean absolute error,

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|. \tag{5.5}$$

MAE is particularly useful for interpretation because it remains in the same unit as the SOH target. The stored SOH and prediction series use the normalized

0 to 1 representation, but the original evaluation scripts multiply both series by 100 before calculating the reported MAE and MSE values. The ageing-estimation results in this chapter are therefore reported primarily on the 0 to 100% SOH scale, with MAE in percentage points and MSE in squared percentage points. Where comparison with the robustness benchmark in Chapter 6 and the embedded-AI study in Chapter 7 benefits from the work-wide 0 to 1 convention, the corresponding normalized values are stated explicitly.

The hyperparameter search covers the number of lag steps, the temporal resolution of the lag sequence, network depth, number of nodes, batch size, kernel regularisation, optimizer type, and activation function. Within this set, the number of lag steps and the temporal resolution are the most method-specific parameters, because they define how much history is provided to the model and at which temporal granularity it is represented. The number of nodes controls the representational capacity of the network, whereas the network depth influences how strongly the model can build more abstract internal feature combinations across layers. Both parameters therefore affect the balance between flexibility, generalisation, and model complexity. Hyperband is used as the main search strategy because it distributes training resources adaptively, evaluates many candidate settings quickly, and allocates more epochs only to the most promising configurations.[110]

The study reports that ADAM outperforms RMSProp, Adadelta, and Adagrad for this task, that larger batch sizes reduce overfitting tendencies, and that explicit kernel regularisation does not lead to a meaningful improvement. Batch size also affects the training trade-off directly: larger batches are computationally efficient and converge reliably, whereas smaller batches can improve generalisation at the cost of longer or less stable optimisation. The optimizer analysis is complemented by an activation-function comparison, in which ReLU yields the lowest loss in the tested setup while sigmoid, tanh, and softmax perform worse. The models are trained on approximately 15,000 sequences with varying batch-size and epoch combinations, and larger batches converge to a comparable minimum while reducing overfitting tendencies.

5.5 SOH Estimation Results and Effects of Temporal Representation

This section evaluates how the two most influential sequence-design parameters, namely the number of lag steps and the temporal resolution, affect the final SOH-estimation performance. The result presentation and interpretation are kept together because the numerical errors depend directly on the chosen history length, the temporal aggregation, and the ageing trajectory of the evaluated cell. For the detailed analysis, Figure 5.5 compares three exemplary cells aged under different conditions. Four time resolutions are considered, namely 1 min, 10 min, 30 min, and 60 min, while the number of lag steps is varied between 8, 12, 16, and 20. All MAE values in this section are shown in percentage points of SOH to remain directly interpretable on the original 0 to 100% SOH scale.

Across the exemplary cells, the most favourable behaviour appears at intermediate temporal aggregation rather than at the shortest or longest tested sampling intervals. This matches the characteristic charge and discharge durations of the investigated scenarios, which typically fall in the range of approximately 20 min to 1 h. If the time resolution is too large, relevant behaviour is averaged out and important information is lost. If it is too small, the training process becomes less effective, the effective histories become unnecessarily long, and the resulting representation does not translate automatically into better prediction quality. The MAE matrix shows the smallest errors for 16 lag steps at a time resolution of 10 min, which corresponds to a history length of 160 min. Extending the sequence length beyond 20 steps is reported as disadvantageous because the resulting history becomes excessively long and does not improve prediction quality further.

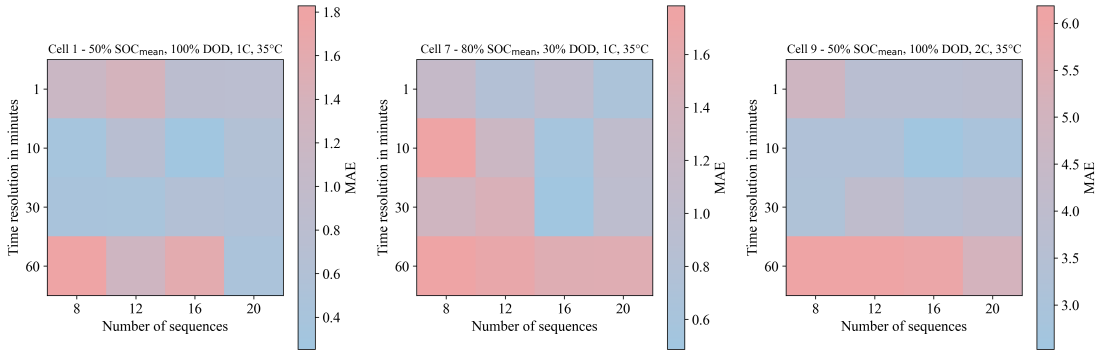


Figure 5.5: Matrix of MAE values shown in percentage points of SOH on the 0 to 100% SOH scale. The corresponding normalized MAE on a 0 to 1 SOH scale is obtained by scaling the displayed values accordingly. The colour scale indicates the magnitude of the prediction error and highlights that the most favourable operating region is found at intermediate temporal aggregation rather than at the shortest or longest tested interval.

Figure 5.6 compares the predicted and measured SOH trajectories. Cells 7 and 1 represent typical ageing behaviour and are reproduced well. The prediction for Cell 9 follows parts of its pronounced U-shaped progression, but the estimated SOH values show larger pointwise and systematic deviations from the capacity-based reference. Its atypical shape does not itself produce a high error. If that shape were reproduced quantitatively, the points would remain close to the identity line in Figure 5.7 and both MAE and MSE would be small. Cell 9 instead provides an example in which the model captures part of the qualitative development but generalises less accurately in absolute SOH.

Figure 5.7 complements the trajectory-based view with a direct comparison between predicted and measured SOH values. The distance of each point from the identity line is the individual prediction residual. MAE summarizes the average absolute magnitude of these residuals, whereas MSE gives greater weight to the larger deviations. For the favourable individual cells, the reported best errors are MSE values of $0.410\,\%^2$ and $0.118\,\%^2$ and MAE values of $0.487\,\%$ and $0.253\,\%$. These correspond to normalized MAE values of 0.00487 and 0.00253 on a 0 to 1 SOH scale. For Cell 9, the MAE increases to $2.528\,\%$ and the MSE to $8.327\,\%^2$, which quantifies the reduced prediction accuracy visible in the trajectory and scatter plots. Across the evaluated cases, the average MAE is $1.10\,\%$, corresponding to a normalized MAE of 0.0110, and the average MSE is $2.95\,\%^2$, corresponding to a normalized MSE of $2.95 \cdot 10^{-4}$. The prediction plots additionally show that

a post-rolling filter is applied to reduce visual noise and to make the comparison between measured and estimated SOH easier to interpret.

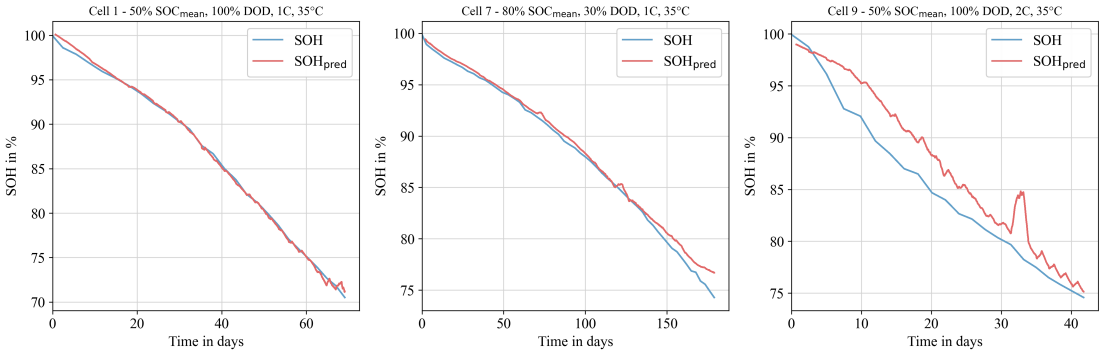


Figure 5.6: Comparison of predicted and measured SOH trajectories over time for three representative test cells. Cells 7 and 1 show the typical ageing progression most frequently observed in the dataset and are estimated accurately by the network, whereas Cell 9 was intentionally chosen as an atypical example with a pronounced U-shaped SOH curve. A post-rolling filter is applied to reduce visual noise and to make the agreement and deviation between measured and predicted trajectories easier to interpret.

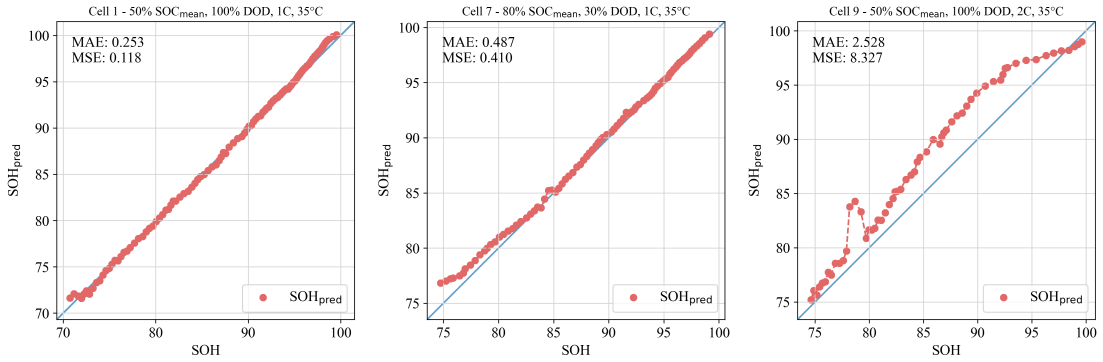


Figure 5.7: Scatter comparison between predicted and measured SOH values for the three representative test cells. Cells 7 and 1 remain close to the identity line and therefore exhibit low prediction errors. For Cell 9, the model follows parts of the qualitative SOH development, but the larger distances from the identity line show that the absolute SOH values are reproduced less accurately. The atypical trajectory alone does not cause the error. MAE and MSE quantify the residual deviations between prediction and reference. Error values are shown on the 0 to 100% SOH scale, with MAE in percentage points and MSE in squared percentage points.

The effect of removing the problematic cells 3 and 4 from the training basis is also examined. In that variant, the aggregate error improves only marginally to an MAE of 1.07 % (normalized MAE 0.0107) and an MSE of 2.52 %^{2} (normalized MSE $2.52 \cdot 10^{-4}$). At the same time, the behaviour on the atypical cell 9 becomes more unstable, which indicates that cleaner data alone do not replace broader data diversity. The result section therefore already makes a broader methodological point: the exact optimum for sequence length and time resolution depends strongly on the available dataset, and direct comparison with other literature remains difficult if the underlying cell data differ substantially.

5.6 Conclusions on Feature-Based SOH Estimation

The ageing-estimation study shows that a meaningful SOH estimate can be obtained with a comparatively simple feedforward model when the preprocessing and feature engineering are aligned with the underlying ageing problem. The central result is not only the achieved average MAE of 1.10 % and average MSE of 2.95 %^{2}, but the way in which this accuracy is reached: cumulative-current features and lag-sequence construction supply temporal and throughput-related context to an otherwise memoryless MLP. The chapter therefore demonstrates that input representation can be as important as model class, especially when the target variable evolves slowly and is only observed directly during periodic capacity tests.

The results also define the practical boundary of the approach. The best lag-sequence setting in the investigated dataset uses 16 lag steps with a time resolution of 10 min, corresponding to a history length of 160 min, while the generally useful settings are found at intermediate temporal resolutions rather than at the shortest or longest tested intervals. This optimum is not universal. It depends on the charge and discharge durations, the operating profiles, and the available ageing trajectories. The atypical behaviour of cell 9 and the limited benefit of removing cells 3 and 4 show that neural SOH estimation is strongly constrained by data coverage and label quality. Cleaner data can reduce some error, but it cannot replace a broad dataset that contains the relevant ageing modes.

Within the present work, this chapter should therefore not be read as a claim that feedforward MLPs are universally preferable to recurrent or hybrid models. Its role is to establish a first data-driven baseline in which ageing-relevant temporal information is represented explicitly while the estimator architecture itself remains

deliberately simple. Because the method relies on directly measured quantities such as voltage, temperature, and current-derived throughput features, the concept is not tied to one narrow cell technology in principle, but transfer to other chemistries or applications would require sufficiently rich training data and renewed validation of the lag-sequence settings.

This baseline role is important for the broader research argument. Once a workable representation for ageing information has been identified, the next question is no longer only how well the estimator performs under nominal conditions, but how different estimator classes behave under deployment-relevant disturbances such as bias, noise, missing samples, and initialization errors. This motivates the shift toward the robustness benchmark in the following chapter. Building on this robustness perspective, the later chapters extend the analysis to recurrent LSTM and MLP models for SOC and SOH estimation, where the aim is no longer only to compare estimator concepts at a general level, but to examine whether stateful recurrent models offer advantages when temporal information has to be processed causally over long streaming sequences and ultimately transferred to embedded microcontroller execution.

Chapter 6

Performance and Robustness Benchmarking Across Battery State-of-Charge Estimator Classes

The preceding chapter established how temporal representation can make a compact neural estimator sensitive to battery ageing. The present chapter moves from constructing one estimator to comparing different SOC-estimation principles under the same causal protocol. It combines a robustness benchmark under disturbed measurements with a target-hardware benchmark of the isolated SOC implementations. Within this comparison, a structurally pruned recurrent estimator is also evaluated under three alternative forms of recurrent-state handling. The chapter therefore provides the methodological link between temporal representation in Chapter 5 and the detailed neural model optimization in Chapter 7.

The general direct-measurement, model-based, hybrid, and data-driven foundations are introduced in Chapter 3. The LFP ageing campaign, reference-label construction, and cell-disjoint data split are documented in Chapter 4. This chapter therefore concentrates on the estimator implementations, the common evaluation protocol, the resulting disturbance mechanisms, and the practical embedded consequences.

6.1 Study Objective and Benchmark Scope

Reliable SOC estimation affects usable energy, operating limits, charge control, and degradation-aware battery management. Clean-condition accuracy alone cannot establish this reliability because practical BMS operation includes sensor calibration errors, random noise, missing measurements, timing irregularities, initialization mismatch, and transient signal corruption.[15, 26, 111, 7, 8, 9]

The benchmark addresses three linked questions. First, it examines whether nominal accuracy predicts behavior when measurements are corrupted or unavailable. Second, it identifies the structural mechanisms that govern degradation and recovery. Third, it determines how estimator architecture and recurrent-state handling affect embedded execution cost. The evaluation therefore comprises two complementary branches. The robustness branch characterizes nominal accuracy, resistance to error growth under disturbed inputs, and recovery after initialization mismatch. The hardware branch verifies numerical equivalence between software and microcontroller execution and measures inference latency, flash occupancy, and peak runtime RAM. For the neural representative, this branch additionally separates the cost of exact rolling-window recomputation from continuous hidden-state forwarding and periodic state reset.

Four concrete representatives are evaluated. The direct-measurement model (DM) uses fixed-capacity Coulomb counting. The hybrid direct-measurement model (HDM) retains the same integration core but adapts effective capacity through a causal SOH estimate. The hybrid equivalent-circuit model (HECM) combines a second-order RC observer with SOH-dependent parameter scheduling and voltage-residual correction. The data-driven model (DD) uses a recurrent neural estimator with multichannel temporal inputs. These representatives expose distinct update and correction mechanisms. Their results identify transferable class mechanisms without claiming that one implementation defines every possible member of its broader estimator family.

6.2 Estimator Portfolio and Benchmark Methodology

6.2.1 Estimator representatives and class boundaries

Figure 6.1 summarizes the causal benchmark chain from measured signals and online feature reconstruction to disturbance injection, estimator execution, and global or local evaluation. Table 6.1 relates the defining mechanisms of the broader classes to the representatives evaluated here. More complex corrections, richer sensing, higher-order physical models, and alternative sequence architectures can extend these class envelopes while also changing deployment cost.

Table 6.1: Estimator-class envelope and implementation scope. Literature in the second column supports the defining mechanisms and broader capability ranges, while the final column specifies the representative evaluated in this chapter.

Class	Defining mechanism and broader capability range	Representative used in this study
Direct measurement	Causal integration of measured current using an assumed capacity. Extensions can include coulombic efficiency, capacity adaptation, OCV or end-point anchoring, plausibility logic, and reset thresholds.[24, 74]	Fixed-capacity Coulomb counting with a sustained full-charge CV reset and no voltage-residual correction
Hybrid direct measurement	Integration core augmented by an estimated or learned state or parameter. Extensions can include learned capacity or efficiency, residual correction, multi-sensor fusion, and adaptive anchoring.[39, 40]	Same Coulomb-counting and reset logic as DM, with effective capacity supplied by the shared causal LSTM-SOH estimate
ECM observer	Coulomb-counted SOC propagation combined with a voltage model and feedback correction. Extensions can include higher RC order, hysteresis, thermal states, operating-point maps, and different nonlinear observers.[24, 7, 26]	Second-order RC model with EKF correction, charge and discharge SOC–SOH lookup surfaces, and shared causal SOH input
Data driven	Learned mapping from measured signals or their history to SOC. Implementations can use feed-forward, convolutional, recurrent, attention-based, physics-informed, raw-signal, or engineered-feature models.[112, 113, 74]	Structurally pruned rolling-window GRU using voltage, current, temperature, SOH, Q_c, derivatives, and Δt

All SOH-dependent representatives receive the same causal LSTM-SOH trace.

Differences therefore arise from how each SOC branch consumes the shared health context. DM is the low-complexity integration reference. HDM isolates the effect of learned capacity adaptation without introducing voltage feedback. HECM adds model-based voltage-residual correction to current-driven state propagation. DD represents compact temporal learning. The GRU is used because SOC estimation and disturbance recovery depend on recent signal history while its gated single-state structure remains lighter than a comparable LSTM. Since DD also consumes the engineered cumulative-charge feature Q_c, it is a data-driven sequence estimator with physics-informed inputs rather than a raw-signal-only neural model.

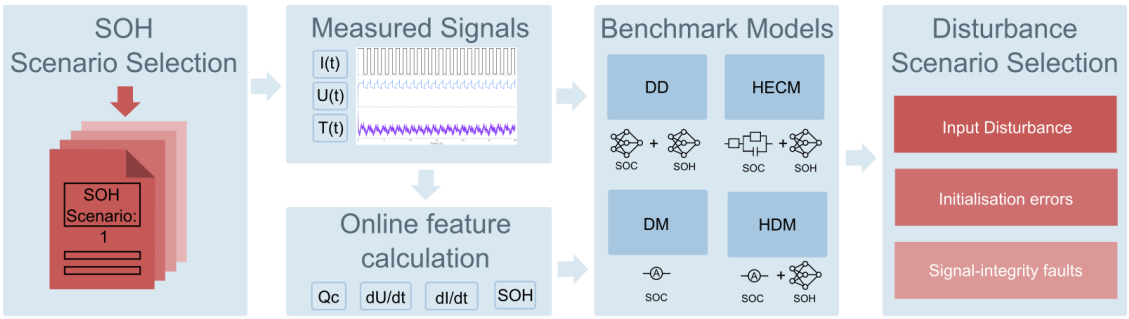


Figure 6.1: Methodology overview of the causal benchmark chain. Measured battery signals are converted into online auxiliary features, processed by the four estimator representatives, exposed to measurement and initialization interventions, and evaluated through global and local performance metrics.

6.2.2 SOC estimation methods

Direct-measurement model (DM)

The direct-measurement formulation used in DM, HDM, and as an explicit auxiliary input to DD follows the same causal cumulative-charge state Q_c. It is updated from the measured current stream as

$$Q_c(k) = Q_c(k-1) + \frac{I_k \Delta t_k}{3600\,\mathrm{s\,h^{-1}}}, \tag{6.1}$$

where I_k is given in amperes and Δt_k in seconds. The conversion factor produces an ampere-hour increment. The sign convention is fixed by the benchmark implementation, and the state is reset to $Q_c = 0$ after a sustained constant-voltage full-charge event. DM maps this charge state to SOC through the fixed nominal

capacity

$$\widehat{\mathrm{SOC}}_{\mathrm{DM}}(k) = \mathrm{clip}\left(1 + \frac{Q_c(k)}{C_{\mathrm{nom}}}, 0, 1\right). \tag{6.2}$$

Hybrid direct-measurement model (HDM)

HDM preserves the same Coulomb-counting core and changes only the capacity term. The online SOH estimate rescales the usable capacity according to

$$C_{\mathrm{eff}}(k) = C_{\mathrm{nom}}\widehat{\mathrm{SOH}}(k), \tag{6.3}$$

which yields

$$\widehat{\mathrm{SOC}}_{\mathrm{HDM}}(k) = \mathrm{clip}\left(1 + \frac{Q_c(k)}{C_{\mathrm{eff}}(k)}, 0, 1\right). \tag{6.4}$$

Both direct-measurement variants use identical reset logic. When the voltage remains above the configured near-maximum threshold for the required CV duration, the internal charge state is set to $Q_c = 0$ and the SOC estimate is anchored at full charge.

Hybrid equivalent-circuit model (HECM)

HECM uses a second-order RC model with state vector

$$\mathbf{x}_k = \begin{bmatrix} \widehat{\mathrm{SOC}}_k & U_{1,k} & U_{2,k} \end{bmatrix}^\top, \tag{6.5}$$

where U_1 and U_2 denote the polarization voltages of the two RC branches. The prediction step is

$$\mathbf{x}_k^- = \mathbf{A}_k \mathbf{x}_{k-1} + \mathbf{b}_k I_{k-1}, \tag{6.6}$$

with

$$\mathbf{A}_k = \mathrm{diag}\left(1, e^{-\Delta t_k/\tau_1}, e^{-\Delta t_k/\tau_2}\right), \qquad \mathbf{b}_k = \begin{bmatrix} \eta \Delta t_k / C_b \\ R_1\left(1 - e^{-\Delta t_k/\tau_1}\right) \\ R_2\left(1 - e^{-\Delta t_k/\tau_2}\right) \end{bmatrix}, \tag{6.7}$$

where $C_b = C_0\widehat{\mathrm{SOH}}$ is the SOH-scaled available capacity. The corrected terminal-voltage estimate is

$$\widehat{U}_k = \mathrm{OCV}\left(\widehat{\mathrm{SOC}}_k, \widehat{\mathrm{SOH}}_k, I_k\right) + U_{1,k} + U_{2,k} + R_i I_k. \tag{6.8}$$

The Kalman update uses the residual between $\widehat{U}_k$ and the measured terminal voltage to correct the internal state. State propagation uses the previous current sample, whereas the correction equation uses the current sample available at the update. The parameters R_i, R_1, R_2, τ_1, τ_2, OCV, and dOCV/dSOC are interpolated from preidentified lookup surfaces over SOC and SOH with separate charge and discharge parametrizations.

Data-driven SOC model (DD)

The data-driven SOC model is a GRU-based recurrent estimator with the online feature vector

$$\mathbf{z}_k = \begin{bmatrix} U_k & I_k & T_k & \widehat{\mathrm{SOH}}_k & Q_c(k) & \frac{\mathrm{d}U}{\mathrm{d}t}(k) & \frac{\mathrm{d}I}{\mathrm{d}t}(k) & \Delta t_k \end{bmatrix}^\top . \tag{6.9}$$

The deployed DD architecture originates from a GRU with 96 recurrent units. Structured pruning removes 29 complete units and all associated recurrent and downstream connections. The recurrent width is thereby reduced by 30.2% to 67 units. Short fine-tuning after pruning restores the weights of the retained structure. The resulting matrices remain dense, which allows direct execution with the hand-written floating-point C kernels without sparse-matrix support. All pruning, fine-tuning, and model-selection decisions use only the declared training and validation data. The independent holdout cells are not used to select the compact architecture.

The pruned GRU is followed by an MLP head with one hidden layer of size 96 and a sigmoid output. During optimization, the recurrent core is exposed to causal sequence segments of length

$$L_{\mathrm{SOC}} = 2024. \tag{6.10}$$

The cumulative-charge input is the same causal state defined in Equation (6.1). The derivative inputs are reconstructed online as

$$\frac{\mathrm{d}I}{\mathrm{d}t}(k) = \frac{I_k - I_{k-1}}{\Delta t_k}, \qquad \frac{\mathrm{d}U}{\mathrm{d}t}(k) = \frac{U_k - U_{k-1}}{\Delta t_k}. \tag{6.11}$$

These channels provide local transient information, while the explicit Δt_k input allows the network to distinguish signal changes from changes in sampling interval. In the primary robustness benchmark, each estimate is generated from the latest causal 2024-sample window and the recurrent state is reset between windows, matching training and validation. Pruning therefore defines the fixed deployment-

ready neural representative used throughout the cross-class comparison. It is not introduced as a separate benchmark factor against an unpruned control model.

6.2.3 Shared SOH estimation method

HDM, HECM, and DD receive the same LSTM-based SOH estimate. Its causal hourly input is constructed from five base signals, namely voltage, current, temperature, EFC, and cumulative charge. Mean, standard deviation, minimum, and maximum are calculated for every base signal. The network therefore receives a 20-dimensional hourly feature vector derived from five measured or causally reconstructed quantities and four aggregation statistics rather than 20 independently measured channels.[114, 115, 116, 1]

The SOH network first projects this vector through a two-layer feature-embedding block of width 128 with layer normalization, GELU activation, and dropout. A two-layer unidirectional LSTM with hidden size 112 processes the projected sequence. Three residual MLP blocks and a prediction head with hidden size 160 map the recurrent representation to an hourly SOH estimate. During execution, hidden and cell states are forwarded causally between hourly updates. The first output is anchored with the configured initial SOH, and subsequent estimates are held piecewise constant between updates.

The common SOH context enters the SOC branches differently. HDM uses it to scale effective capacity. HECM uses it in both capacity scaling and lookup scheduling. DD receives it as one feature within the multichannel recurrent input. A paired diagnostic comparison replaces the causal LSTM-SOH trace with the offline dataset ground-truth SOH while leaving the SOC branch, disturbed measurements, evaluation interval, and error calculation unchanged. This is not a deployable estimator configuration. It isolates whether SOC errors originate within the SOC branch or are inherited through estimated health information.

6.2.4 Model and target-hardware preparation

The software benchmark uses trained floating-point parameters and input scalers that are fixed before evaluation on the independent holdout test set. Neural training, hyperparameter selection, scaler fitting, structured pruning, and model selection use the declared training and validation sets. The HPPC-derived HECM lookup surfaces are identified from the same model-development sets and are fixed

without retuning before holdout evaluation. Figure 4.4 and Table 4.3 document this cell-disjoint development boundary.

Three execution modes are implemented for the same fixed pruned DD network. Exact rolling-window execution resets the hidden state and recomputes the latest 2024 samples for every output. This reproduces the sequence-to-one semantics used during training and in the primary robustness benchmark. Continuous-state execution processes each new sample once and forwards the updated hidden state to the next inference. Periodic-reset execution uses the same incremental calculation but clears the hidden state after every 2024 updates. The weights, feature scaling, and floating-point kernels remain identical in all three cases. Their comparison consequently isolates the effect of sequence recomputation and recurrent-state retention from changes in the trained model.

The target-hardware experiment evaluates the isolated SOC cores on a NUCLEO-H753ZI board with a 480 MHz STM32H753 Cortex-M7. The board receives the precomputed causal SOH trace used by the software reference so that the measured resource demand can be attributed to each SOC implementation. The shared SOH-LSTM is not executed on the microcontroller in this benchmark. Numerical equivalence and inference latency are measured directly on the board. The processor-integrated hardware cycle counter records every valid inference, and each holdout cell sequence is replayed three times. Flash occupancy is calculated from the compiled firmware regions stored in non-volatile program memory.

Peak runtime RAM combines statically allocated variable storage, maximum dynamic-memory allocation, and maximum temporary call-stack demand. Static storage is obtained from the compiled firmware, dynamic allocation is monitored during execution, and stack demand is measured with a high-water-mark pattern during representative on-device replays that exercise the complete estimator path. For DD, this includes valid inference after its rolling input window has been filled. An unused compilation-time memory reserve is excluded. Hardware predictions are compared both with the software implementation and with the dataset SOC reference.

6.2.5 Robustness benchmark design

The primary cross-model benchmark manipulates current, voltage, temperature, sample availability, and sampling time. These quantities are measured or reconstructed online by a BMS. A separate initialization branch acts on deployment-

accessible estimator states. Parameter mismatch is excluded from the common cross-model ranking and is examined through the supplementary HECM analysis defined in Section 6.2.7. Figure 6.2 groups the scenarios into input disturbances, initialization errors, and signal-integrity faults. Table 6.2 specifies their implementation.[9]

The disturbance magnitudes represent deployment-level sensing limits, adverse measurement corruption, and signal-integrity stress conditions.[19, 21, 23, 22, 117, 118] Random sensor noise models zero-mean fluctuations in current, voltage, and temperature. Systematic sensor errors comprise multiplicative current-gain error and additive offsets in current, voltage, and temperature. Voltage spikes and ADC quantization complete the input-disturbance branch. The remaining scenarios address initialization, measurement availability, and sampling-time integrity.

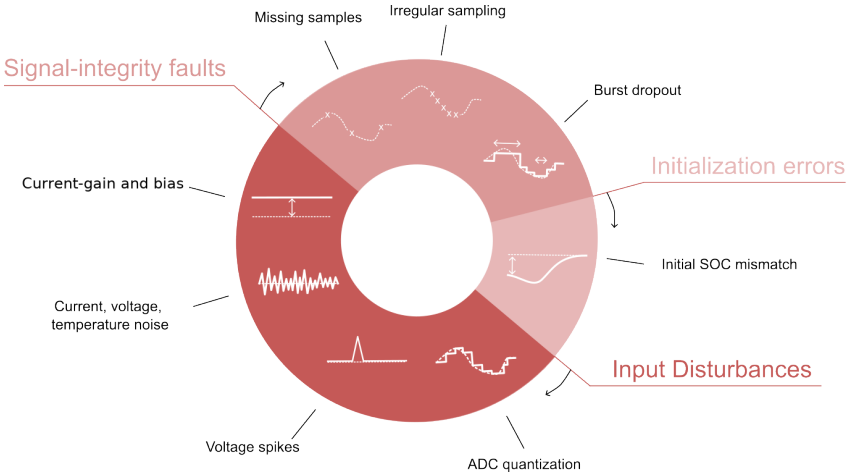


Figure 6.2: Taxonomy of the benchmark scenarios, grouped into input disturbances, initialization errors, and signal-integrity faults.

Table 6.2: Disturbance configurations and practical rationale. The numerical values are fixed benchmark settings, while the cited sources motivate the disturbance mechanisms and magnitude ranges.

Scenario	Benchmark setup	Practical origin and rationale
ADC quantization	$\Delta I = 0{,}01\,\mathrm{A}$, $\Delta U = 0{,}005\,\mathrm{V}$, and $\Delta T = 0{,}5\,^{\circ}\mathrm{C}$	ADC steps, sensor scaling, rounding, and coarse deployment-level discretization.[19, 21, 20]

Table 6.2: Disturbance configurations used in the robustness benchmark, continued.

Scenario	Benchmark setup	Practical origin and rationale
Current gain error	$I_{\text{meas}} = I(1 + g_I)$ with $g_I = \pm 0.5\%$, $\pm 1.5\%$, and $\pm 3.0\%$. Both signs are evaluated	Sensor sensitivity error, scale-factor miscalibration, drift, and an adverse upper stress level.[21, 23, 22]
Current noise	Gaussian noise with $\sigma_I = 0{,}02\,\text{A}$ and $0{,}10\,\text{A}$	Sensing-chain noise, EMI pickup, and low and high sensor-noise levels.[7, 8]
Voltage noise	Gaussian noise with $\sigma_U = 0{,}01\,\text{V}$	Wiring noise, front-end noise, poor filtering, and EMI.[7, 19, 117]
Temperature noise	Gaussian noise with $\sigma_T = 1{,}0\,^\circ\text{C}$	Sensor tolerance, conversion noise, thermal gradients, and sensor-to-cell mismatch.[20, 40]
Current offset	$I_{\text{meas}} = I + \Delta I$ with $\Delta I = \pm 0{,}05\,\text{A}$. Both signs are evaluated	Zero-current calibration residual and systematic current-sensor offset.[21, 23, 22]
Voltage offset	$U_{\text{meas}} = U + 0{,}02\,\text{V}$	Front-end offset, calibration residual, wiring contribution, and adverse sensing stress
Temperature offset	$T_{\text{meas}} = T + 3\,^\circ\text{C}$	Sensor placement error, calibration residual, and cell-to-sensor thermal mismatch
Initial SOC error	Initial mismatch of -0.10 SOC, with DD perturbed through online Q_c	Restart, storage, missing recent history, and weak LFP OCV anchoring.[7, 25]
Missing samples	Every 50th sample frozen and a separate random 2% loss case	Packet loss, blocked updates, logger dropouts, and periodic or random data freeze.[8, 118]
Irregular sampling	Uniform jitter of $\pm 0{,}1\,\text{s}$, $\pm 0{,}5\,\text{s}$, and $\pm 0{,}9\,\text{s}$ around the nominal 1 s grid	Scheduling jitter, bus latency, and asynchronous acquisition.[9, 8, 118]

Table 6.2: Disturbance configurations used in the robustness benchmark, continued.

Scenario	Benchmark setup	Practical origin and rationale
Burst dropout	One frozen 3600 s interval selected by maximum absolute net charge, with at least 12 h before and 24 h after the interval	Communication interruption, frozen task, stuck data stream, and severe measurement-availability stress.[9, 8]
Voltage spikes	Single-sample spikes of ±0,20 V every 1000 samples	EMI, switching disturbances, and corrupted voltage samples.[117]

6.2.6 Metrics and evaluation criteria

The benchmark synthesizes estimator behavior along three dimensions. Accuracy describes agreement with dataset SOC under nominal inputs. Robustness describes resistance to error growth under sustained or transient measurement corruption and signal unavailability. Recovery describes the re-establishment of a reliable SOC estimate after initialization mismatch. Global behavior is quantified through MAE, RMSE, bias, maximum error, and the 95th-percentile absolute error. The paired disturbance penalty is

$$\Delta\mathrm{MAE} = \mathrm{MAE}_{\mathrm{disturbed}} - \mathrm{MAE}_{\mathrm{baseline}}. \tag{6.12}$$

Local metrics include jump counts, output variance, absolute-error variance, excess rolling MAE, and event-aligned response. Where a disturbance mask exists, pre-disturbance, disturbed, and post-disturbance intervals are evaluated separately.

The canonical initialization experiment compares every perturbed trajectory with its cell-, window-, model-, SOH-, and seed-matched correctly initialized trajectory. Their absolute paired difference is the recovery signal, while error against dataset SOC remains the accuracy measure. A first qualifying entry occurs when the paired difference remains within a 0.02 SOC band for at least 300 s. Persistent recovery is the first subsequent point after which the paired difference remains inside the band for the remainder of the 24-h observation horizon. Runs without this endpoint are right-censored. First-entry time, later departure from the band, censoring, relapse fraction, and excess-error area provide complementary recovery diagnostics.

Each holdout cell is treated as one independent statistical unit. Protocol-defined SOH windows and random seeds are summarized within each cell before the six cells receive equal macro weight. Hierarchical bootstrap resampling with seeds nested inside cells and 10,000 repetitions provides 95% confidence intervals. Scenario effects are evaluated as paired cell-matched differences from baseline. Estimator pairs are compared using exact two-sided sign-flip tests, raw mean and median differences, paired standardized effect sizes, and Holm-adjusted p-values. The effect size quantifies the magnitude of a paired difference, the confidence interval expresses its uncertainty, the sign-flip test checks directional consistency across cells, and the Holm correction controls the family-wise error across multiple comparisons. With six independent cells, effect magnitudes and confidence intervals provide the primary evidence, while p-values support their interpretation.

All global accuracy, robustness, stratified, confidence-interval, and paired-test calculations begin when the rolling DD first provides a valid estimate. Every model is therefore scored over the same source interval and the same number of observations. The decision synthesis combines baseline MAE, RMSE, and 95th-percentile error for accuracy. Robustness gives equal weight to eight disturbance families. Recovery combines persistent recovery-or-censor time, excess-error area, and persistent censored fraction. Alternative equal-scenario and high-severity aggregations test how strongly the summary depends on the weighting rule.

6.2.7 HECM lookup-table sensitivity analysis

HECM is the only representative whose SOC calculation depends directly on identified resistance, time-constant, and OCV lookup surfaces. An observed penalty could therefore originate from the disturbed measurement, local lookup calibration uncertainty, or an interaction between both. A separate sensitivity analysis tests whether the HECM disturbance profile remains stable under local lookup variations and whether its larger penalties can still be attributed primarily to the manipulated measurements.

The nominal lookup surfaces are identified from the training and validation data and remain fixed during the cross-model benchmark. The resistance and time-constant surfaces are scaled independently by $\pm 10\%$, while the OCV surfaces are shifted by $\pm 10\,\mathrm{mV}$. Every one-at-a-time perturbation is crossed with the complete set of measurement and signal-integrity subcases and with the paired initialization experiment. For each disturbance, the interaction is the change in scenario ΔMAE

caused by the perturbed lookup relative to the nominal lookup. Since these variations apply only to HECM, they are evaluated separately and do not enter the cross-model robustness score.

6.3 Feature Reconstruction and Experimental Protocol

The LFP dataset, reference-label construction, complete data split, and holdout coverage are defined in Section 4.1. Every estimator is executed causally, and all required auxiliary quantities are reconstructed directly from the disturbed input stream. This applies to cumulative charge, EFC, current and voltage derivatives, and hourly SOH aggregates. Frozen or unavailable measurements therefore propagate consistently into every derived feature that depends on them.

The equivalent full-cycle feature is reconstructed from the absolute charge throughput according to

$$Q_{\text{through}}(k) = \sum_{j=1}^{k} \frac{|I_j|\Delta t_j}{3600}, \qquad \text{EFC}(k) = \frac{Q_{\text{through}}(k)}{C_{\text{nom}}}, \tag{6.13}$$

where C_{nom} is the nominal cell capacity used by the feature pipeline. EFC is an online model input rather than a scoring label. It is therefore recomputed from the current and timing stream of every clean or disturbed run.

All estimators are evaluated on every available cell and SOH-state combination in the independent holdout set. One representative 24-h window is fixed for each available fresh, mid-life, and aged state, giving 16 windows. Every candidate begins at the same measured full-charge condition with SOC $\geq$ 0.98 and $U \geq 3{,}58\,\text{V}$. Multiple eligible windows can exist within one cell and SOH state. Their measured operating conditions are represented by

$$\mathbf{z}_w = \left[\widetilde{\text{SOH}}_w \quad \overline{T}_w \quad P_{95}(T)_w \quad P_{95}(|C_{\text{rate}}|)_w \quad Q_{\text{through},w} \quad f_{\text{SOC}<0.2,w}\right]^{\top}. \tag{6.14}$$

The descriptors are median SOH, mean temperature, 95th-percentile temperature, 95th-percentile absolute C-rate, absolute charge throughput, and the fraction of samples below 20% SOC. Candidate values are centered by the descriptor median m_j and scaled by the median absolute deviation s_j. The selected observed window

is

$$w^* = \arg\min_{w} \sqrt{\sum_{j} \left(\frac{z_{w,j} - m_j}{s_j} \right)^2}. \tag{6.15}$$

This measured-feature medoid selects the actual window closest to the typical multivariate operating profile within each cell and SOH state. Model predictions and estimation errors do not enter the selection. The procedure therefore retains diversity across cells and ageing states without favoring one estimator.

The shared SOH-LSTM receives 192 h of undisturbed causal context before every scored interval. Standard scenarios use a 24-h evaluation window. The one-hour dropout uses 48 h from the same full-charge anchor so that sufficient post-gap behavior remains observable. Figure 6.3 summarizes the resulting selection and timing protocol.

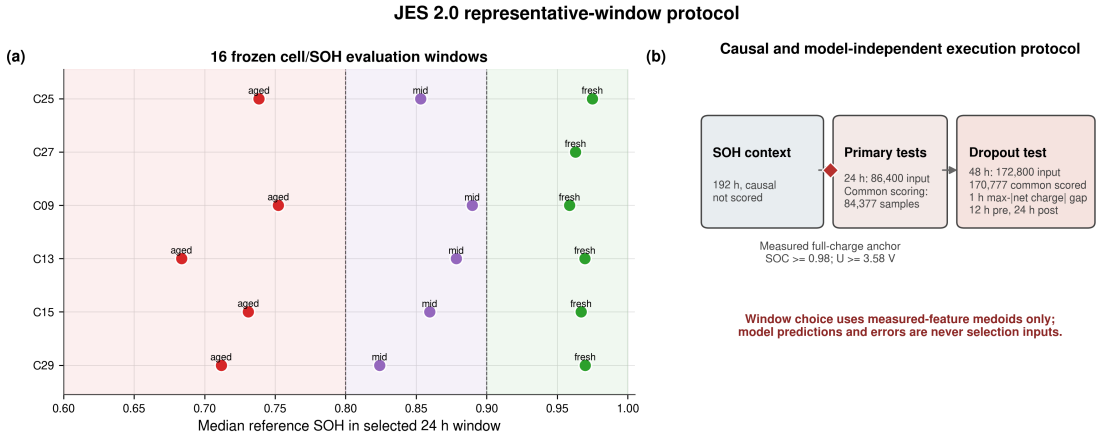


Figure 6.3: Representative-window protocol for the robustness benchmark. Sixteen cell/SOH windows are selected using measured-feature medoids. Every estimator is scored over the same interval after the causal SOH and rolling DD input contexts are available. Standard scenarios cover 24 h, while the dropout case uses 48 h and places its one-hour gap in the eligible interval with maximum absolute net charge.

The benchmark contains 20 scenario definitions. These comprise the nominal baseline, two current-noise levels, voltage and temperature noise, three signed current-gain magnitudes, one signed current-offset level, voltage and temperature offsets, ADC quantization, initial SOC mismatch, periodic and random sample loss, three timing-jitter levels, a one-hour burst dropout, and randomized-sign voltage spikes. Every current-gain magnitude and the 50 mA current-offset case contain matched positive and negative signs. Gaussian noise cases use 10 deterministic

seeds. Random sample loss, timing jitter, and spike cases use five seeds. Scenarios without a stochastic component are executed once. The complete cross-model campaign contains 7040 model-window evaluations. Appendix Figure A.1 provides the complete traceability matrix.

The primary DD evaluation preserves the trained rolling sequence-to-one semantics. Each prediction uses the latest 2024 samples and resets recurrent state between windows. Continuous-state streaming is treated as a separate deployment configuration. The burst-dropout start is selected from measured current as the eligible one-hour interval with the largest absolute integrated net charge. At least 12 h of observations must precede the gap and 24 h must follow it. Dataset SOC and estimator output do not influence this selection. Initialization mismatch is mapped to the accessible Coulomb-counting state in DM and HDM, the EKF SOC state in HECM, and a capacity-equivalent Q_c offset in DD.

6.4 Performance and Robustness Results

6.4.1 Baseline performance

Figure 6.4 summarizes nominal accuracy against dataset SOC over the common evaluation intervals of the 16 predefined windows. Equal weighting across the six holdout cells gives the lowest baseline MAE to DD at 0.0258 with a 95% confidence interval from 0.0189 to 0.0325. HECM follows at 0.0337 with an interval from 0.0246 to 0.0434, HDM at 0.0412 from 0.0301 to 0.0515, and DM at 0.0690 from 0.0499 to 0.0823. The RMSE ranking is identical at 0.0300, 0.0388, 0.0442, and 0.0740, respectively.

The ordering is consistent with the correction information available to the four implementations. DM propagates charge with fixed capacity and has no continuous correction path. HDM reduces ageing-related capacity mismatch through its SOH-dependent effective capacity. HECM additionally uses a voltage residual to correct its model-based state. DD maps the complete multichannel history to SOC and can represent nonlinear temporal relations that are absent from the other representatives. These mechanisms explain the observed order within the tested data without implying that architecture alone determines accuracy in every operating domain.

Cell- and SOH-window results reveal substantial variation across holdout trajectories, particularly for DM. All six pairwise mean differences follow the same ranking

direction, and five of the six paired 95% confidence intervals exclude zero. Exact sign-flip tests provide a complementary check of directional consistency. Their Holm-adjusted p-values remain above 0.05 with six independent cells. The practical interpretation therefore relies primarily on the observed effect magnitudes, uncertainty intervals, and consistent ranking direction across the available cells. Together, these statistics support a clear nominal ranking for the tested representatives while retaining the uncertainty expected from a six-cell holdout design.

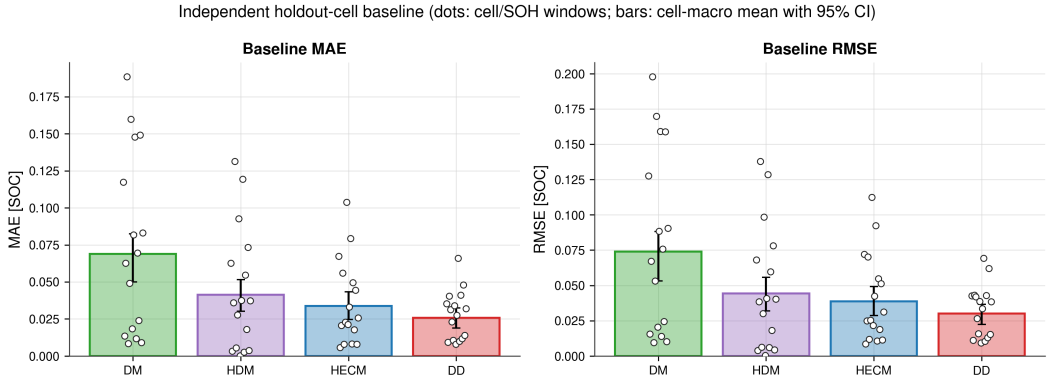


Figure 6.4: Baseline accuracy against dataset SOC across the independent holdout test set. Dots show cell and SOH-window values. Bars and error bars show the equal-weight cell-macro mean and hierarchical 95% confidence interval. DD has the lowest macro MAE and RMSE, while DM has the largest error and between-cell spread.

6.4.2 Current-gain sensitivity

Current-gain error is a systematic sensor error that persists over time and scales with instantaneous current. It differs from an additive offset, which shifts the measurement zero independently of current magnitude, and from zero-mean random noise. Figure 6.5 summarizes the matched signed sweep at ±0.5%, ±1.5%, and ±3.0%. For each magnitude and holdout cell, the adverse-direction summary uses the larger MAE change from the two signs because one direction can partly compensate an existing baseline error.

Degradation increases monotonically with gain-error magnitude for every estimator. At 3%, cell-macro MAE rises by 0.0108 for DM, 0.0101 for HDM, 0.0060 for HECM, and 0.0038 for DD. DM and HDM integrate the scaled current directly, so the error accumulates with transferred charge. HECM is exposed through current-driven propagation, but its voltage-residual correction can counter part of the state

deviation. DD receives current together with voltage, temperature, cumulative charge, derivatives, and temporal context. Its smaller observed penalty is therefore consistent with a lower dependence on one systematically scaled channel.

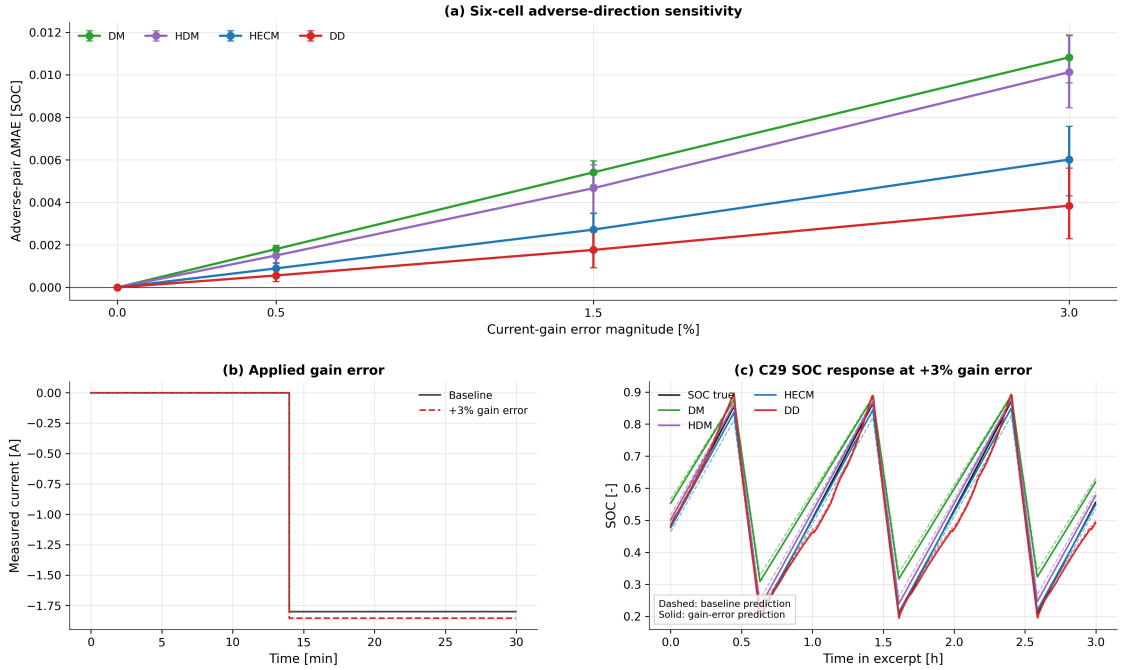


Figure 6.5: Current-gain sensitivity. Panel (a) reports six-cell adverse-direction ΔMAE for every matched signed gain pair with hierarchical 95% confidence intervals. Panel (b) compares representative current before and after the +3% gain error. Panel (c) shows SOC trajectories from the representative high-load holdout trajectory over three charge and discharge intervals.

The continuous high-load lifecycle analysis in Figure 6.6 complements the independent 24-h windows. The gain-error contribution changes over life because signed Coulomb-counting error interacts with current direction and operating state. The penalty often grows between full charges, while a full-charge anchor can reduce or restructure it. The process is therefore neither unrestricted monotonic drift nor a complete reset. Across this single full-life replay, the adverse 3% gain error raises MAE by 0.0102 for DM, 0.0072 for HDM, 0.0037 for HECM, and 0.0048 for DD. HECM has the smallest contribution in this trajectory. This mechanism-focused diagnostic does not replace the six-cell window comparison.

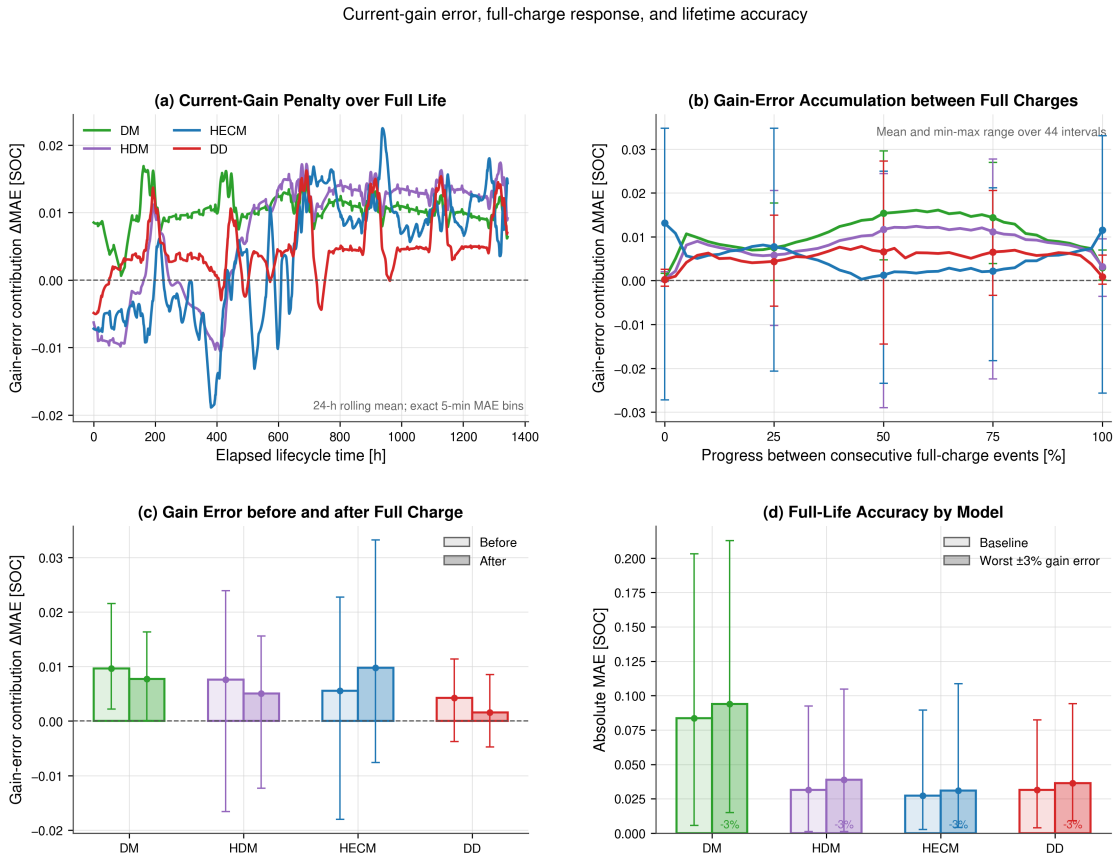


Figure 6.6: Current-gain diagnostic over the continuous high-load holdout trajectory. The panels report the rolling 24-h gain-error contribution over elapsed life, its normalized development between consecutive full-charge events, its value immediately before and after those events, and the full-life baseline and adverse-gain MAE. Full-charge anchoring changes accumulated error but does not eliminate the lifetime penalty uniformly.

6.4.3 Additive sensor offsets

The signed current-offset experiment shifts measured current by ±0,05 A and retains the larger paired ΔMAE within each cell before equal-weight aggregation. DD has the smallest adverse cell-macro penalty at 0.0401 with a 95% confidence interval from 0.0226 to 0.0556. HECM follows at 0.1763 from 0.1329 to 0.2142, DM at 0.2467 from 0.2253 to 0.2693, and HDM at 0.2727 from 0.2546 to 0.2911. The adverse sign differs between models and cells, showing why one fixed sign would not represent offset sensitivity reliably.

The integration-based estimators accumulate the persistent zero-current error

throughout the window. The recurrent DD limits but does not eliminate the systematic shift. Additive current offset produces the largest global cell-macro disturbance penalty for all four representatives. Figure 6.15 places this result beside the remaining disturbance families.

Voltage and temperature offsets produce a different model pattern. A +3 °C temperature offset increases DD MAE by 0.0074, while DM, HDM, and HECM remain at or below 0.0002. A +0,02 V voltage offset increases DD by 0.0038 and HDM by 0.0005, leaves DM effectively unchanged, and changes HECM by −0.0040 over the finite windows. DM does not use temperature and uses voltage primarily for physical anchoring. HDM receives both channels indirectly through the slowly updated SOH service. HECM responds through its voltage residual, where a fixed offset can partly compensate an existing trajectory error. DD consumes absolute voltage and temperature as direct learned inputs. The negative HECM value therefore represents finite-window compensation relative to its baseline rather than a physical improvement.

6.4.4 Noise sensitivity

Random sensor noise differs from the persistent gain and offset errors because it introduces zero-mean fluctuations. Its global penalties are smaller and depend on the disturbed channel and estimator structure. At $\sigma_I = 0{,}10$ A, HECM has the largest mean macro penalty at $\Delta\text{MAE} = 0.0038$, although its confidence interval includes zero. DM changes by −0.0009, while HDM and DD remain close to zero.

Every disturbed current sample enters cumulative charge in DM and HDM, but positive and negative increments partly cancel during integration. This operation smooths rapid zero-mean fluctuations in the resulting SOC trajectory even though every sample contributes to the state. HECM uses disturbed current directly in state propagation and in its voltage-prediction path, which provides a plausible transmission mechanism for its larger mean response. DD receives raw current and its derivative at every recurrent step. These direct channels explain the visibly larger short-term DD output variation in Figure 6.7. Its rolling multichannel context distributes the response across the input sequence. At the tested level, this local transmission does not become a persistent global bias, so its window-wide ΔMAE remains close to zero.

Voltage noise at $\sigma_U = 0{,}01$ V produces penalties of 0.0014 for DM and 0.0013 for HDM, while HECM and DD remain below 0.0003. The small response of the direct-

measurement branches can arise through voltage-dependent full-charge anchoring rather than continuous SOC propagation. Temperature noise is neutral for DM and small for the SOH-aware pipelines because temperature enters through learned or parameter-dependent paths rather than through the direct charge update.

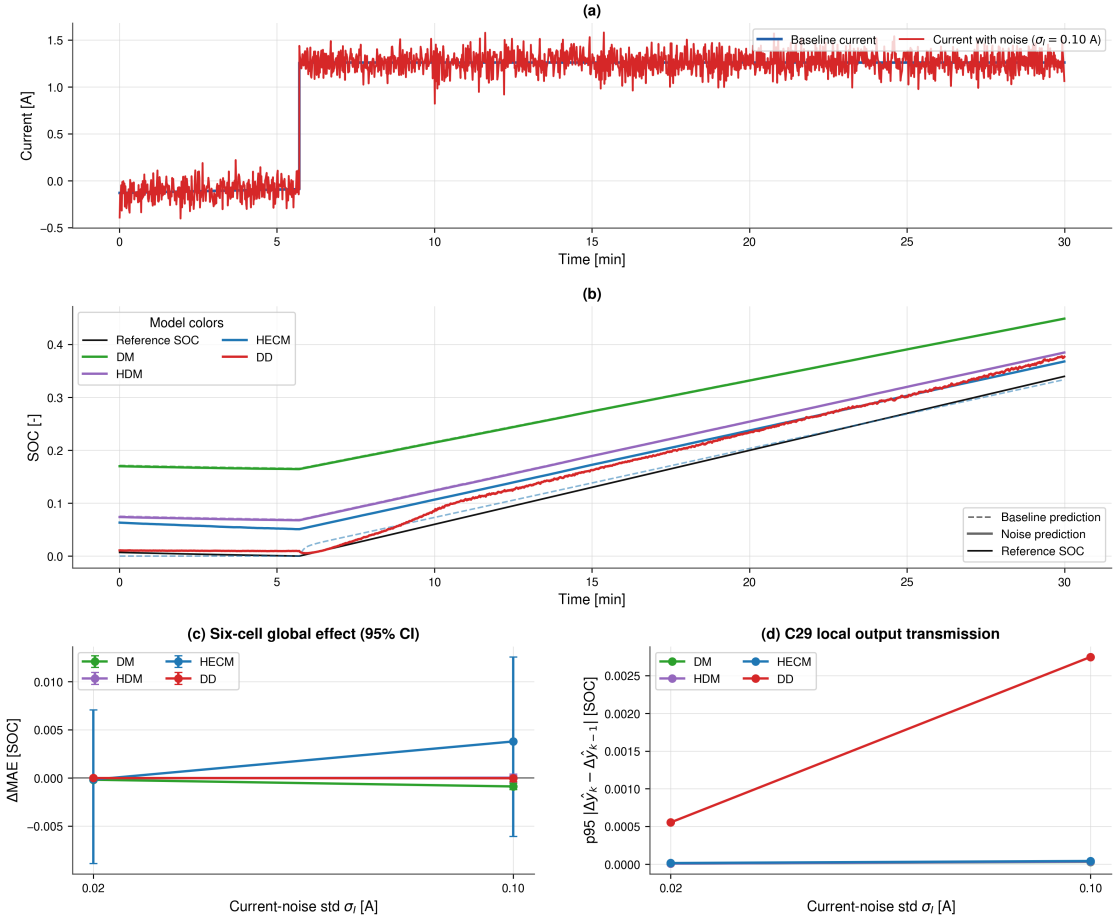


Figure 6.7: Noise sensitivity under additive current noise. Panels (a) and (b) compare current and SOC over a reset-free 30-min example. Panel (c) reports the cell-macro ΔMAE with hierarchical 95% confidence intervals. Panel (d) quantifies local output transmission. DD shows the largest short-term transmission because raw current and its derivative enter the recurrent input directly, while integration in DM and HDM smooths zero-mean sample-level fluctuations.

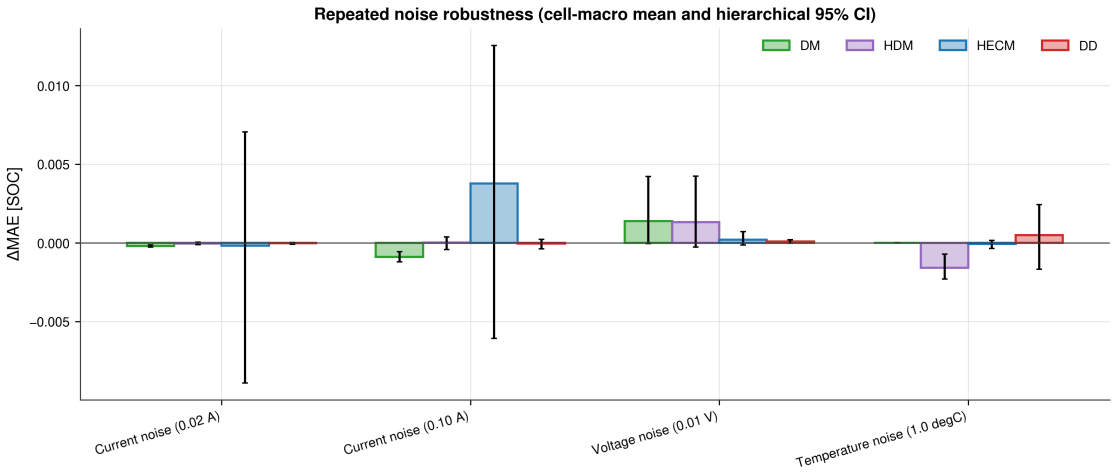


Figure 6.8: Repeated noise robustness across the holdout test set. Bars show cell-macro ΔMAE and error bars show hierarchical 95% confidence intervals with random seeds nested within cells. HECM is most affected by high current noise, while most other channel and model combinations remain close to baseline relative to the between-cell uncertainty.

Figures 6.7 and 6.8 together show that zero-mean sensor noise changes window-wide accuracy only slightly for most model and channel combinations. The local current-noise analysis nevertheless demonstrates that aggregate ΔMAE must be interpreted together with short-term output transmission, since rapid fluctuations can remain weakly represented by a window-wide metric.

6.4.5 Initialization error and recovery

The recovery experiment maps the same −0.10 SOC-equivalent mismatch, corresponding to ten percentage points, to the Coulomb-counting states in DM and HDM, the EKF SOC state in HECM, and a Q_c offset of $-0.10C_{\mathrm{nom}}$ in DD. Figure 6.9 reports the paired trajectory differences from the common evaluation start, while recovery time remains referenced to the original intervention.

HECM provides the fastest persistent recovery with a cell-macro mean of 1.20 h, followed by DD at 2.60 h. DD enters the 2% recovery band temporarily in several runs but has a higher relapse fraction than HECM at 0.78 compared with 0.17. DM and HDM generally do not recover persistently within the observation horizon, and both have a persistent-censor fraction of 0.89. HECM also has the smallest excess-error area at 0.078 SOC h, compared with 0.116 SOC h for DD.

The observed recovery follows the available correction mechanisms. DM and

HDM propagate the imposed offset until a physical charge anchor is reached because neither has continuous output feedback. HECM uses its voltage residual to correct the explicit SOC state and can therefore re-anchor persistently. DD does not propagate SOC as one isolated state. Its multichannel rolling input allows the prediction to reduce dependence on the disturbed cumulative-charge feature. The changing interaction between that feature and the recurrent context also explains why an initial entry into the recovery band can be followed by relapse. Recovery time, relapse behavior, censoring, and excess-error area together identify HECM as the strongest recovery implementation under this protocol.

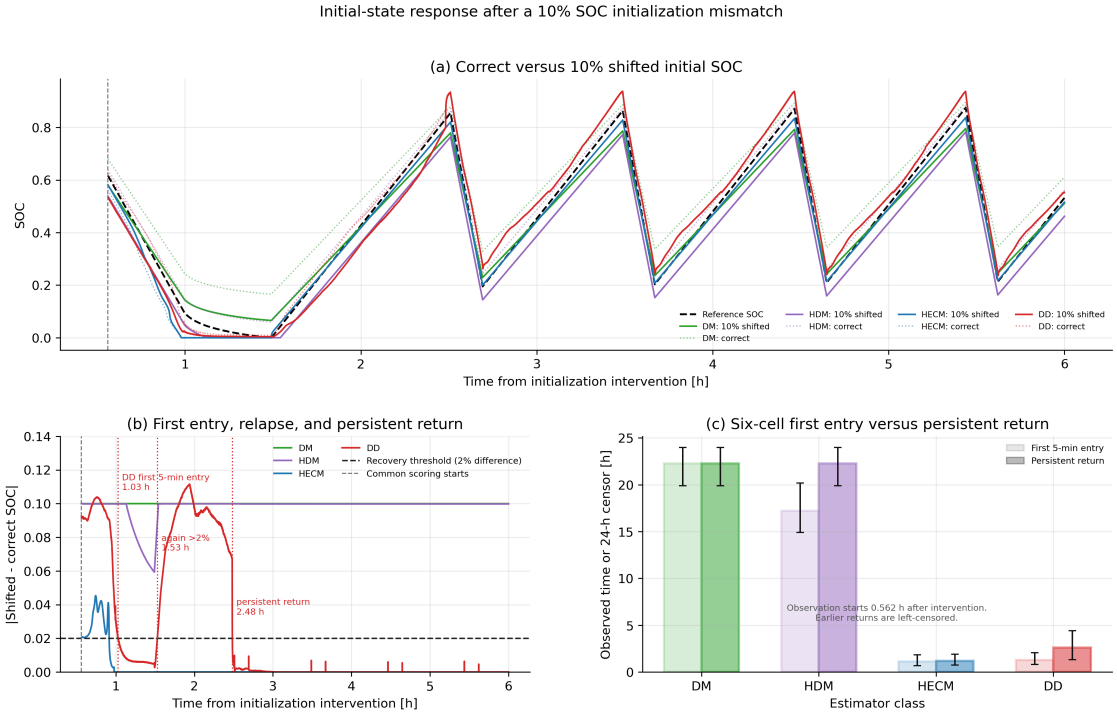


Figure 6.9: Response to a ten-percentage-point SOC-equivalent initialization mismatch. Panels (a) and (b) show correctly and incorrectly initialized trajectories and their absolute paired differences for a representative mid-life window. Panel (c) reports observed first qualifying entry and persistent recovery-or-censor time across the holdout test set with hierarchical 95% confidence intervals.

6.4.6 Missing samples and signal integrity

Figure 6.10 compares periodic and random sample loss with timing jitter. Losing measurement information is substantially more consequential than changing its sampling time. Periodic freezing of every 50th sample increases MAE by 0.0072,

0.0065, 0.0028, and 0.0011 for DM, HDM, HECM, and DD, respectively. Random 2% loss produces increases of 0.0080, 0.0081, 0.0039, and 0.0026. By contrast, even $\pm 0{,}9\,\mathrm{s}$ timing jitter remains within approximately 6×10^{-4} SOC of baseline for every representative, with confidence intervals generally spanning zero.

DM and HDM depend structurally on uninterrupted charge integration. A missing or frozen current sample removes or distorts charge information that cannot be reconstructed retrospectively by their update equations. HECM can counter part of the error through voltage feedback. DD can draw on the surrounding multichannel sequence rather than on the affected sample alone. Timing jitter preserves the measurements themselves, and every implementation rebuilds its time-dependent quantities from the disturbed sampling intervals. This explains why timing perturbation is less consequential than actual information loss under the tested protocol.

The one-hour burst dropout creates a longer availability problem. Figure 6.11 shows the transition into and out of the missing-input interval together with the resulting cell-macro penalty. Relative to the duration-matched 48-h baseline, macro ΔMAE is largest for HECM at 0.0239 with a 95% confidence interval from 0.0155 to 0.0366. HDM follows at 0.0209, DM at 0.0050, and DD at -0.0093. The DD interval from -0.0288 to 0.0099 includes zero. Its negative mean therefore does not demonstrate improved accuracy. It indicates that no persistent global degradation is resolved under this protocol.

DM and HDM retain the corrupted charge balance until a physical anchor because the missing interval cannot be recovered by integration alone. HECM receives valid measurements after the gap, but the flat LFP voltage relation limits how strongly its observer can infer the missing charge and correct the state deviation. In DD, frozen raw and derivative values are progressively removed from the finite rolling window after measurements resume. New voltage, temperature, current, and timing information then reduces dependence on the corrupted cumulative-charge feature. This mechanism is consistent with its smaller persistent effect while the finite-window confidence interval limits a stronger conclusion.

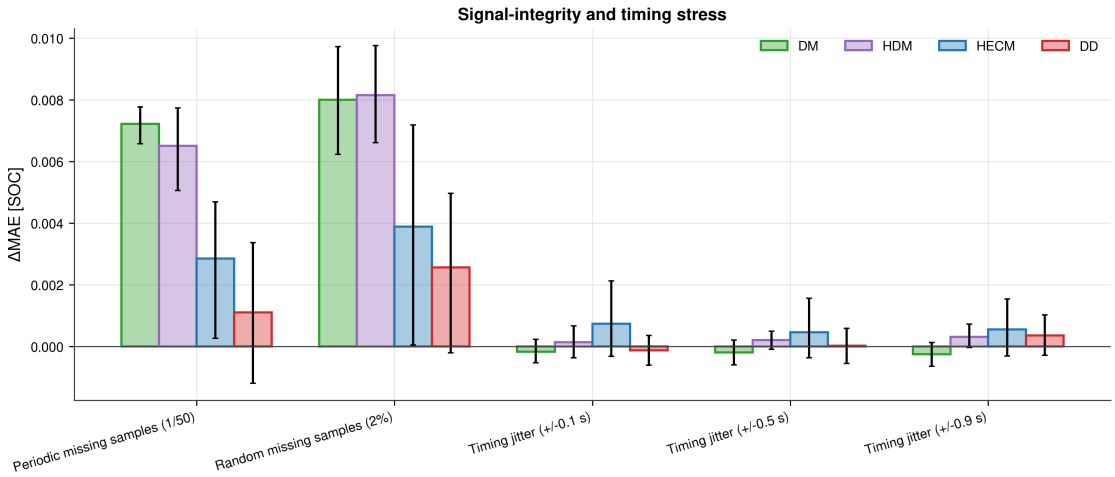


Figure 6.10: Six-cell signal-integrity and timing stress. Bars show cell-macro ΔMAE and hierarchical 95% confidence intervals for periodic and random sample loss and three timing-jitter levels. Sample loss dominates timing jitter under the tested protocol.

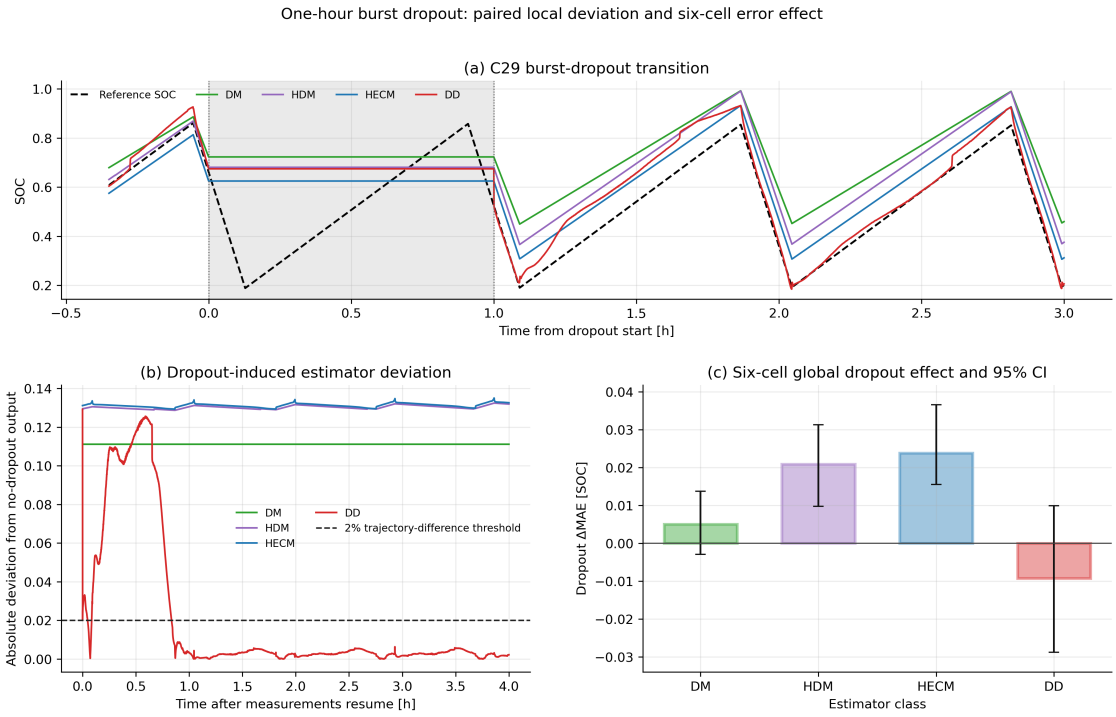


Figure 6.11: One-hour burst-dropout transition. Panel (a) shows estimator output around the shaded missing-input interval. Panel (b) reports absolute deviation from the duration-matched no-dropout output after measurements resume. Panel (c) gives the six-cell global ΔMAE with hierarchical 95% confidence intervals.

6.4.7 Spike sensitivity

Single voltage spikes demonstrate why local and global metrics must be considered together. Full-window ΔMAE remains below 4×10^{-4} SOC for every estimator and is effectively zero for DM, HDM, and DD. Event-aligned analysis nevertheless identifies a reproducible DD transient. Its mean spike-sample penalty is approximately 7.7×10^{-3} SOC across cells, compared with near-zero direct penalties for the other representatives. Figures 6.12 and 6.13 show that this response varies across cells and SOH states and decays after the event.

DM and HDM do not use voltage as a continuous SOC-correction signal, so an isolated spike away from the full-charge threshold has little direct effect. HECM processes voltage through its observer residual, which filters one inconsistent measurement through the physical model and covariance weighting. DD receives the impulse simultaneously through absolute voltage and $\mathrm{d}U/\mathrm{d}t$, and both disturbed values enter its recurrent input context. This provides a plausible mechanism for the larger but short-lived response. A channel-removal ablation would be required to isolate the individual contribution of each voltage feature.

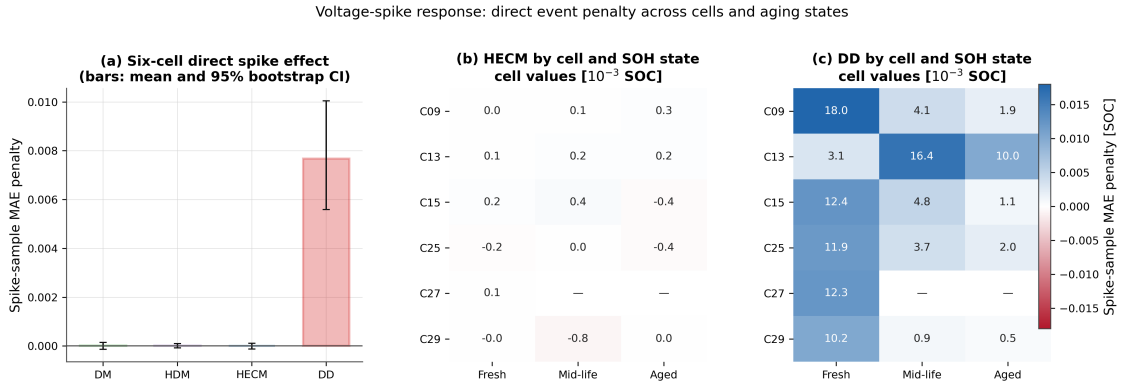


Figure 6.12: Voltage-spike sensitivity across six cells. Panel (a) reports the mean event-sample MAE penalty with a hierarchical 95% confidence interval. Panels (b) and (c) resolve HECM and DD penalties by cell and SOH state. The event-aligned metric exposes the local DD response that is obscured by full-window ΔMAE.

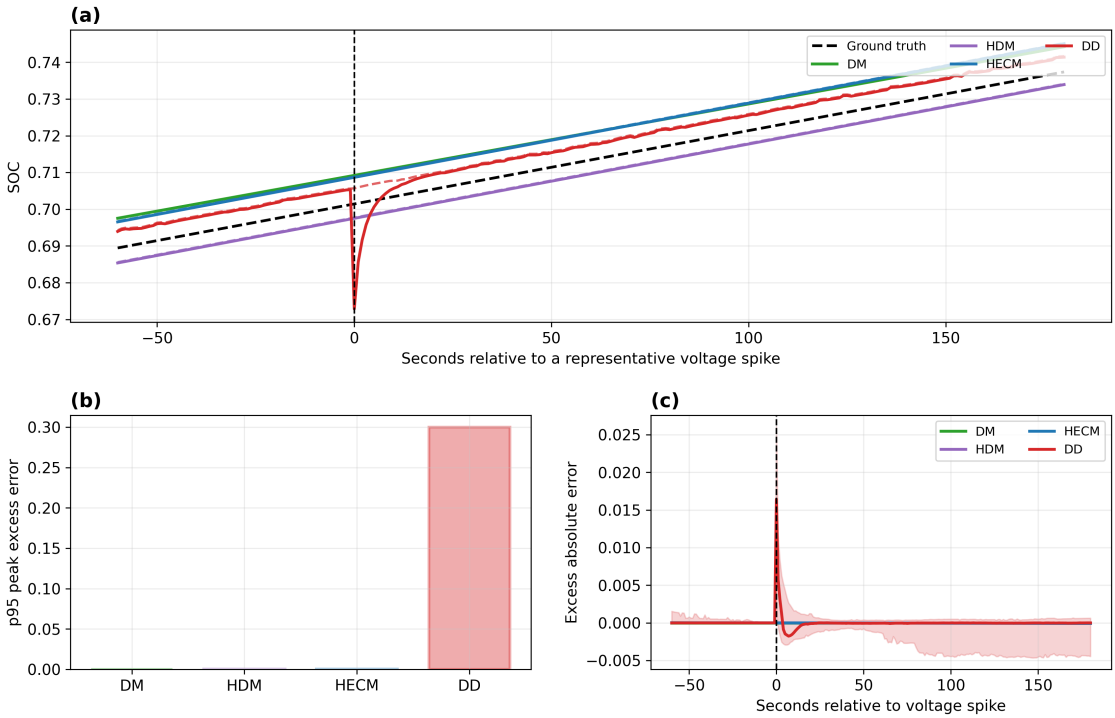


Figure 6.13: Event-aligned voltage-spike response. Panel (a) compares representative SOC trajectories with dataset SOC. Panel (b) reports the model-wise 95th percentile peak excess error. Panel (c) shows mean excess absolute error aligned across repeated spikes. The local DD response decays rapidly and is therefore weakly represented by full-window MAE.

6.4.8 ADC quantization

Figure 6.14 compares representative raw and quantized current and voltage signals and summarizes the six-cell ΔMAE. The tested steps of $\Delta I = 0{,}01\,\mathrm{A}$, $\Delta U = 0{,}005\,\mathrm{V}$, and $\Delta T = 0{,}5\,^{\circ}\mathrm{C}$ preserve local signal shape but affect the representatives differently. DM and HDM increase by 0.0038 and 0.0035 MAE. HECM changes by −0.0018 and DD by +0.0002. Every confidence interval overlaps zero.

The small response of DM and HDM is consistent with repeated current-rounding errors entering cumulative charge. HECM combines quantized current with voltage-residual correction, while DD embeds all quantized channels in a multichannel temporal context. Their signed changes remain inside the uncertainty range and do not indicate physical improvement or exact cancellation. At the selected coarse resolution, ADC quantization produces smaller penalties than systematic current error, sample loss, and burst dropout.

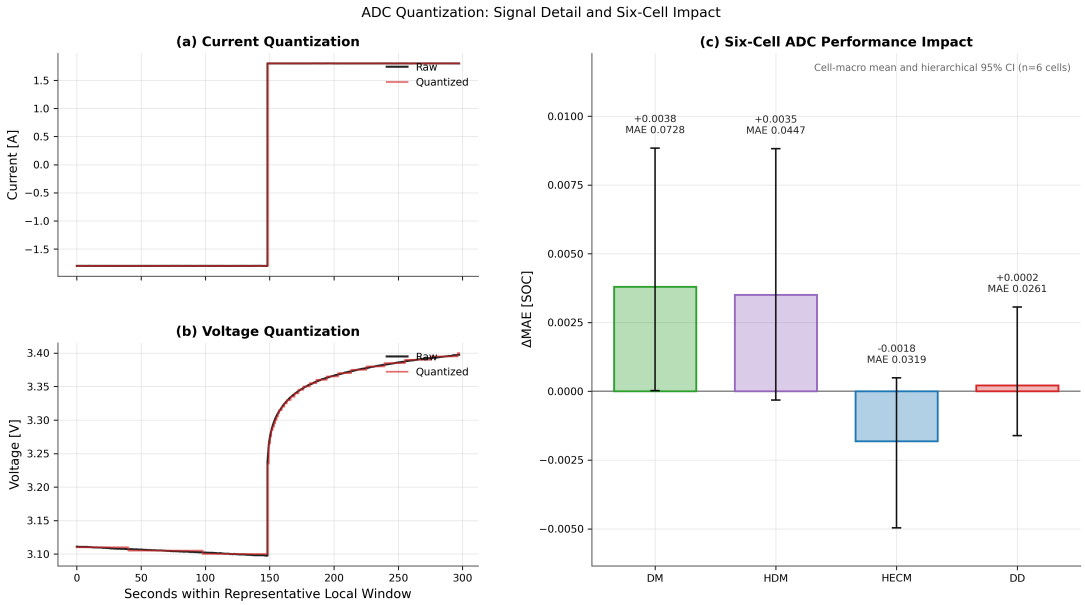


Figure 6.14: ADC quantization. Panels (a) and (b) compare representative raw and quantized current and voltage signals. Panel (c) reports six-cell ΔMAE with hierarchical 95% confidence intervals and the resulting quantized-signal MAE.

6.4.9 Diagnostic comparison with ground-truth SOH

The primary comparison supplies the same causally estimated LSTM-SOH trace to HDM, HECM, and DD. This is the health information available to the complete causal pipelines during online operation. The diagnostic comparison replaces it with dataset ground-truth SOH derived from capacity check-ups and interpolated between measured anchors. Its purpose is to determine whether SOC errors originate predominantly in the SOC branch or are inherited through the estimated SOH input.

Ground-truth SOH reduces baseline MAE by 0.0370 for HDM and 0.0166 for HECM, while DD MAE increases by 0.0049 on average. The pronounced HDM improvement follows from the direct use of SOH to scale effective capacity. HECM also benefits because SOH enters capacity scaling and lookup parametrization. DD uses SOH as one learned feature among several interacting inputs. Replacing that feature with its ground truth therefore does not impose a monotonic relation between SOH accuracy and SOC error.

The model-specific direction persists for most noise, sample-loss, and dropout cases. At 3% current-gain error, ground-truth SOH improves HDM and HECM and

changes DD by +0.0091. For temperature noise, the ground-truth-minus-LSTM difference changes from its baseline value by 0.0016 for HDM and less than 10^{-4} for HECM and DD. SOH-input quality consequently explains part of the baseline differences, particularly for HDM and HECM, but does not account for every disturbance response. The LSTM-SOH runs remain the primary deployment-representative results, while the ground-truth SOH runs provide diagnostic attribution.

6.4.10 HECM lookup-table sensitivity

Within the tested local perturbation range, lookup uncertainty is not the dominant source of the observed HECM measurement-disturbance penalties. Across all 22 measurement and signal-integrity subcases, the largest absolute lookup and disturbance interaction is 0.006116 MAE. It occurs in the additive current-offset test, but its cell-bootstrap interval includes zero. This is small relative to the HECM current-offset penalty of 0.1763 obtained with the nominal lookup. The large offset response is therefore attributable primarily to the systematic measurement error rather than to the tested parameter-surface variations.

The lookup tables remain relevant to baseline calibration and recovery. Five of the six lookup perturbations retain mean persistent recovery-or-censor times between 1.18 and 1.41 h, close to the nominal 1.20 h. Under the remaining resistance perturbation, one low-load holdout trajectory stays outside the recovery band for the complete observation horizon and raises the equal-weight mean to 5.10 h. The analysis therefore answers the primary disturbance-attribution question negatively while identifying recovery as a calibration-sensitive boundary. Appendix Figure A.3 and Table A.1 provide the complete results.

6.4.11 Robustness synthesis across representatives

Figure 6.15 condenses 18 measurement-corruption and signal-integrity rows and shows how degraded-input behavior changes the nominal ranking. Additive current offset creates the largest global penalties for DM, HDM, and HECM, while DD remains less affected. Current-gain sensitivity increases with magnitude for every representative. Periodic and random sample loss particularly affect DM and HDM, and the one-hour dropout produces the largest non-offset point penalty for HECM. Timing jitter remains nearly neutral. Voltage spikes reveal a distinct DD transient through event alignment despite their small full-window MAE.

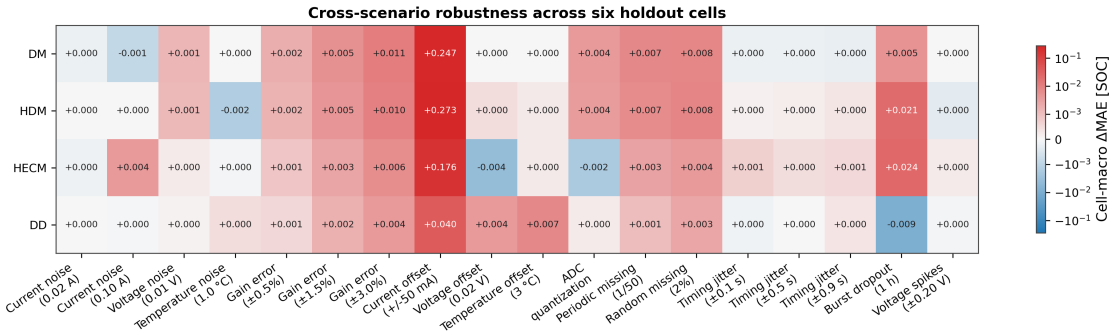


Figure 6.15: Cell-macro ΔMAE across 18 measurement-corruption and signal-integrity rows. Current-gain and current-offset rows use adverse directions from matched signed sweeps. Positive values denote degradation relative to each model's paired baseline, while small negative values arise from finite-window compensation. Initialization recovery is evaluated separately.

Figure 6.16 combines relative accuracy, family-balanced robustness, and recovery and illustrates how deployment priorities alter the combined assessment. Under the family-balanced definition, robustness scores are 0.452 for DM, 0.351 for HDM, 0.360 for HECM, and 0.967 for DD. Equal weighting per scenario gives 0.549, 0.476, 0.552, and 0.793. A highest-level-plus-offset variant gives 0.573, 0.469, 0.562, and 0.693. DD leads all three aggregations. The changing score distances and the changing order of DM and HECM demonstrate the influence of the aggregation rule. The scenario-level heatmap remains necessary for identifying the disturbance mechanisms behind these relative summaries.

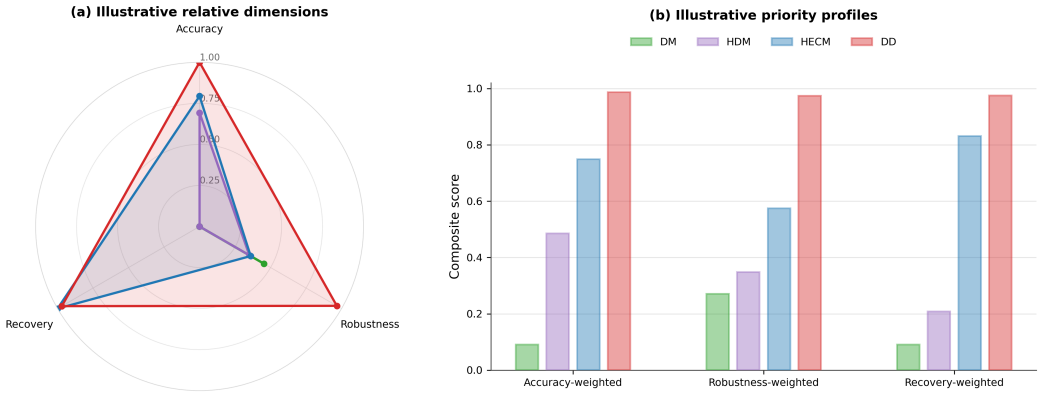


Figure 6.16: Decision-oriented synthesis for the four representatives. Panel (a) compares min–max-normalized accuracy, family-balanced robustness across eight disturbance families, and observed persistent recovery. Panel (b) applies three illustrative priority profiles that assign 60% to one dimension and 20% to each remaining dimension. The summary must be interpreted together with the scenario-level evidence in Figure 6.15.

6.4.12 Embedded performance and recurrent execution modes

The hardware experiment first verifies numerical equivalence before comparing speed and memory. Across the six nominal cell sequences, mean hardware and software MAE remains below 4×10^{-4} percentage points for every SOC core, as shown in Figure 6.17. These differences are orders of magnitude smaller than the estimator-to-dataset errors. The float32 C implementations therefore reproduce their software references sufficiently closely for deployment measurements.

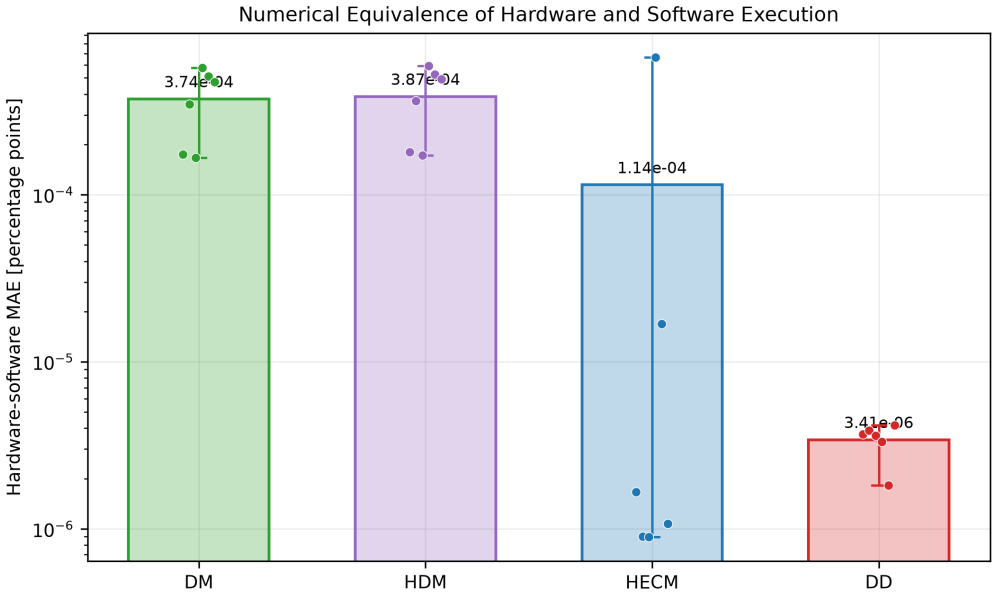


Figure 6.17: Numerical equivalence of STM32 and software execution over six cell sequences. Bars show mean hardware-to-software MAE in percentage points, dots show cells, and whiskers show the cell range. The logarithmic scale emphasizes that implementation differences are negligible relative to estimator error.

DM and HDM have sub-microsecond median latency at 0.171 and 0.165 μs. HECM requires 23.9 μs, while continuous-state DD requires 424.9 μs. Figure 6.18 shows the complete distributions. The ordering follows the operations performed at each update. DM and HDM require a small number of scalar calculations. HECM additionally performs parameter lookup and observer matrix operations. DD evaluates recurrent matrix products and its MLP output head. At the nominal 1-Hz update rate, even the largest median occupies less than 0.043% of one sampling period. This result applies to the isolated estimator task and quantifies the execution margin available for other BMS functions.

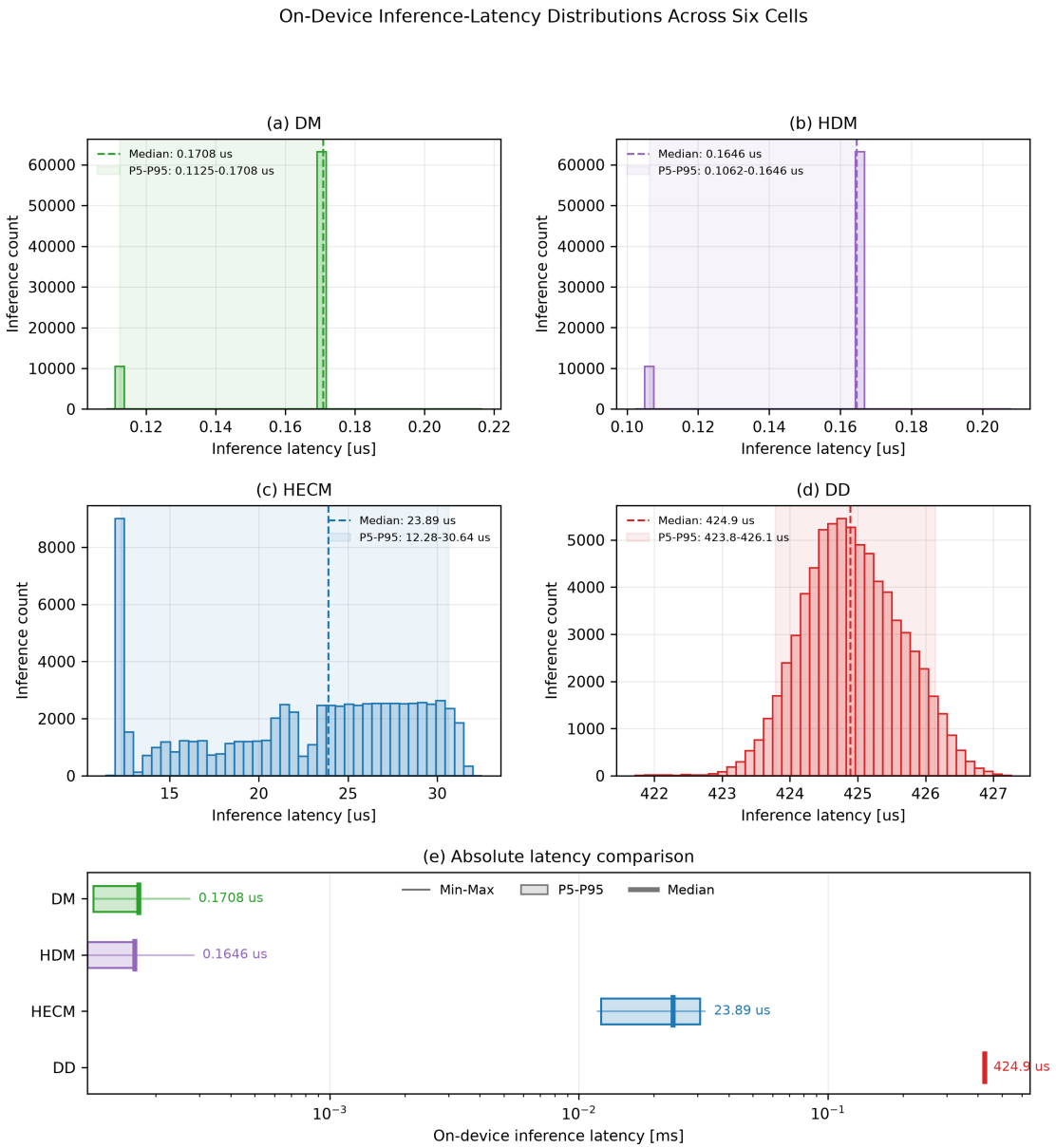


Figure 6.18: On-device SOC-core latency distributions across six cells and three replay rounds. Panels (a) to (d) show per-inference distributions. Panel (e) compares the minimum, 5th percentile, median, 95th percentile, and maximum on a logarithmic scale. DD denotes continuous-state inference in this cross-model comparison.

The compiled DM, HDM, HECM, and continuous-state DD firmware occupies 27.0, 27.1, 470.1, and 115.9 KiB of flash. Measured peak runtime RAM is 2.4, 2.4, 2.5, and 4.1 KiB, respectively. Figure 6.19 compares both quantities. HECM flash is dominated by its parameter surfaces, whereas neural parameters account for most of the DD flash demand. All four isolated SOC implementations fit within

the selected microcontroller and leave resource margin for additional BMS tasks.

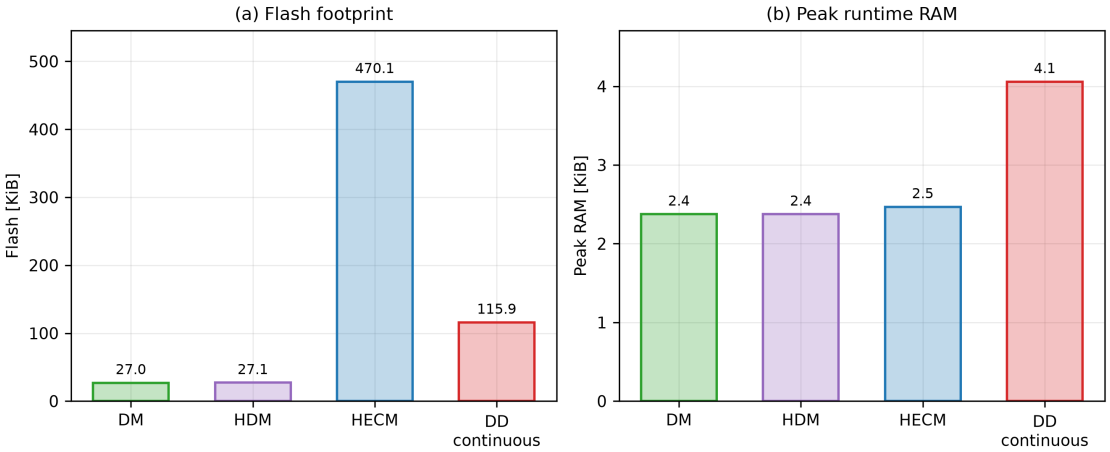


Figure 6.19: Embedded memory requirements of the isolated STM32 SOC implementations. Panel (a) reports compiled flash occupancy. Panel (b) reports peak runtime RAM, combining statically allocated variable storage, maximum dynamic allocation, and call-stack demand. DD denotes continuous-state execution.

The DD execution-mode comparison in Figure 6.20 makes the deployment trade-off explicit. The same pruned network weights are used in every mode. Exact rolling-window inference preserves the primary software semantics but requires approximately 724 ms median inference time and 67.3 KiB peak runtime RAM because the complete 2024-sample sequence is evaluated for every output. This occupies 72.4% of the one-second sampling period. Continuous-state execution evaluates only the new recurrent step and reduces latency to approximately 0.425 ms and peak RAM to 4.1 KiB while retaining similar nominal error over the six sequences. Periodically resetting the recurrent state has comparable memory and latency but creates much larger transient errors. The mean of the cellwise maximum dataset-SOC errors is approximately 29 percentage points, and individual cells reach approximately 39 percentage points. Each reset removes the temporal context accumulated before that point, whereas continuous forwarding preserves it.

Continuous-state forwarding therefore provides the practical DD hardware configuration. Its recurrent semantics differ from the rolling-window benchmark and must be validated as a distinct deployment mode. Appendix Figure A.2 provides the full latency distributions for all three modes.

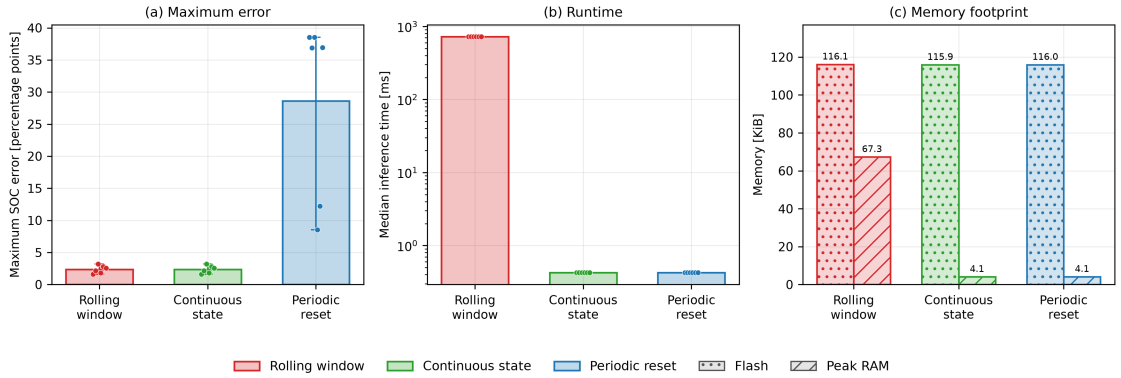


Figure 6.20: DD inference-mode trade-off on the STM32. The panels compare maximum dataset-SOC error, median latency, compiled flash occupancy, and measured peak runtime RAM for exact rolling-window, continuous-state, and periodically reset recurrent execution. Cell points show that periodic resets, rather than continuous forwarding, create the dominant accuracy loss.

6.5 Discussion

6.5.1 Structural interpretation across estimator classes

The four implementations expose distinct relationships between update mechanism, available correction information, and disturbance response. DM has the largest nominal error and reacts strongly to persistent current-gain and current-offset errors and to missing samples. Its current-integration structure has no voltage-residual correction. A practical full-charge anchor limits accumulated drift, but the lifecycle analysis shows that this anchor restructures the gain-error contribution without removing it uniformly. HDM adapts the capacity denominator through learned SOH and reduces capacity mismatch. It retains the same charge-integration core, so its responses to systematic current errors, missing charge information, and quantization remain structurally similar to DM.

HECM combines current-driven propagation with a voltage residual and has the strongest observed recovery profile under the tested initialization mapping. Additive current offset produces its largest global disturbance penalty. High current noise and burst dropout are the most consequential remaining cases because corrupted or frozen current affects state propagation and observer correction. The supplementary lookup analysis shows that the largest local lookup and disturbance interaction is 0.00612 MAE, compared with a 0.1763 HECM penalty from the ad-

verse current offset itself. The offset response is therefore governed primarily by the corrupted input within the tested lookup range, although absolute accuracy and re-anchoring remain dependent on lookup calibration.

DD has the lowest nominal cell-macro error and the smallest adverse penalties under both systematic current-error tests. It is also comparatively resilient to isolated sample loss. Its multichannel recurrent context can combine current, voltage, temperature, derived quantities, and their temporal development rather than relying on one update equation. This broader information path provides a plausible mechanism for the observed nominal and disturbance performance. Event-aligned voltage spikes nevertheless expose a recurrent transient that full-window MAE hides. Absolute voltage, the voltage derivative, and the rolling recurrent context provide a plausible transmission path and identify a specific target for feature ablation and disturbance-aware training.

The ground-truth SOH comparison demonstrates why an auxiliary state estimator must be considered part of the complete SOC pipeline. Part of the HDM and HECM baseline error is inherited through estimated SOH. DD instead uses SOH as one feature within a learned multivariate interaction. Improving one auxiliary input therefore benefits a physically defined capacity relation more directly than it benefits a learned mapping whose output depends on several correlated features. The limited change in several paired disturbance penalties further shows that the shared SOH estimator is not the dominant cause of every robustness response.

At class level, the representative results identify transferable mechanisms rather than a universal ranking. Direct-measurement methods provide transparent and deterministic updates with minimal computational and memory demand. Their update structure primarily propagates accumulated charge and offers no continuous mechanism for reconstructing missing charge information or correcting systematic integration errors. Reliable current calibration, uninterrupted sampling, and physical anchoring are therefore central to their robustness. Adding learned SOH information can reduce capacity mismatch in a hybrid direct-measurement method, but it does not by itself add voltage feedback or temporal correction to the underlying integration core.

Model-based ECM observers extend this structure with a physically defined correction mechanism. The voltage residual allows HECM to re-anchor its internal SOC state and is consistent with its strong observed recovery. This correction does not provide the same breadth of measurement history as a learned multichannel sequence representation. Its effectiveness remains constrained by operating-point ob-

servability, the flat OCV–SOC relation of LFP cells, calibrated parameter surfaces, and disturbances that affect both state propagation and observer correction. The data-driven representative instead combines several measured and derived channels over time. The results support the value of this context without establishing that every data-driven estimator will outperform every physical model.

Temporal context is not exclusively beneficial. The voltage-spike experiment shows that a localized disturbance can propagate through selected features and recurrent state. The periodic-reset experiment shows that removing accumulated context can create large transient errors. The same mechanism that allows DD to combine information across time can therefore transmit disturbances or become unavailable through unsuitable state handling. Structural pruning makes this context model smaller by reducing the recurrent width, but it does not remove these temporal failure mechanisms. Robust data-driven deployment requires representative training coverage, disturbance-aware evaluation, careful feature selection, and a state strategy that matches the intended embedded execution mode.

6.5.2 Accuracy, robustness, and recovery as separate requirements

Nominal MAE describes average agreement with dataset SOC under clean inputs. It does not reveal the DD voltage-spike transient, HECM re-anchoring behavior, shared-SOH coupling, or the different responses to random noise, systematic sensor errors, and unavailable measurements. These observations demonstrate why accuracy, robustness, and recovery must be treated as separate estimator properties. A low clean-condition error remains necessary, but it does not predict which failure mechanism will dominate in practical operation.

The decision scores provide useful relative summaries, but their changing distances and order demonstrate that no one weighting can replace scenario-level evidence. Estimator selection should begin with the sensing risks, availability constraints, recovery requirements, and fault-handling strategy of the target BMS. The selected model can then be improved through current calibration, observer adaptation, disturbance-aware training, feature modification, filtering, reset logic, or fallback behavior according to the weakness revealed by the relevant scenario.

6.5.3 Implications for BMS deployment

Direct-measurement methods are attractive when transparency, deterministic execution, and minimal memory are primary requirements, provided that reliable current calibration, uninterrupted sampling, and physical anchoring can be ensured. Hybrid methods can add adaptive health information or learned correction to an existing physical core. Their benefit depends on the part of the underlying estimator that is modified. Model-based ECM observers provide physically interpretable feedback and strong re-anchoring potential at the cost of parameter identification and larger flash demand. Data-driven methods offer broad flexibility for representing nonlinear dependencies and temporal context, but require representative training data and explicit validation of feature-mediated transients and recurrent-state behavior.

The STM32 results show that hardware resources do not create a fundamental deployment barrier for any of the four isolated SOC cores on the selected microcontroller. Every implementation fits into available memory and completes its update within the one-second sampling interval. This does not make computational constraints irrelevant because the resource profiles differ substantially. DM and HDM are the smallest and fastest implementations. HECM remains fast but requires 470.1 KiB of flash due primarily to its parameter surfaces. Continuous-state DD uses more computation and runtime memory than the other representatives, yet its approximately 0.425 ms median latency and 4.1 KiB peak runtime RAM demonstrate practical recurrent neural estimation in the tested environment.

The DD implementation separates two complementary deployment decisions. Structured pruning reduces recurrent width before firmware export and produces a compact dense network that does not require specialized sparse kernels. Recurrent execution semantics then determine how often this fixed network is evaluated and how its temporal context is retained. Exact rolling-window execution repeatedly evaluates the complete sequence, while continuous-state forwarding calculates only one recurrent step. Periodic resets preserve low computational cost but discard temporal information and create large transients. The large reduction from rolling-window to continuous-state resource demand is therefore caused by implementation semantics rather than by another compression step. Model architecture and execution strategy must be designed together, and the selected state semantics must be validated against the required estimation behavior.

Table 6.3 relates the measured estimator profiles to dominant deployment prior-

ities. The recommendations identify the strongest observed representative for one priority within the stated protocol rather than a universal class winner.

Table 6.3: Recommendation map derived from the performance and robustness benchmark. Every row identifies the strongest observed representative for one dominant deployment priority and the principal trade-off that remains.

Deployment priority	Main supporting evidence	Main trade-off
Lowest nominal SOC error: DD	Lowest cell-macro MAE at 0.0258 and RMSE at 0.0300	Exact rolling-window execution is the most computationally demanding mode
Strongest recovery from initialization mismatch: HECM	Lowest observed persistent recovery-or-censor time and fewer first-entry relapses than DD	Recovery is compared after every estimator provides valid output
Lowest adverse systematic current-error penalties: DD	ΔMAE = 0.0038 at 3% gain error and 0.0401 at 50 mA offset	Temperature-offset and local voltage-spike sensitivity remain visible
Smallest periodic and random sample-loss penalty: DD	ΔMAE = 0.0011 and 0.0026	The duration-matched dropout effect remains highly variable across cells
Lowest deterministic compute cost: DM and HDM	Sub-microsecond SOC-core latency and approximately 27 KiB flash	Integration remains sensitive to systematic current error and missing charge information
Practical fast neural deployment: continuous-state DD	Approximately 0.425 ms median latency and 4.1 KiB peak runtime RAM	Recurrent semantics differ from the primary rolling-window benchmark

6.5.4 Limitations and future extensions

The conclusions are grounded in four specified implementations and six independent holdout cells from one controlled LFP ageing campaign. Alternative reset logic, ECM structures, neural architectures, input features, and training data can produce different profiles. The low- and middle-load groups contain multiple cells, while the high-load group contains one exploratory trajectory. One low-load trajectory contributes no mid-life or aged window. The reported confidence intervals quantify the available between-cell variation within this design. Transfer to other chemistries, cell formats, pack configurations, thermal environments, and field duty

cycles requires corresponding cell-disjoint validation.

The disturbance campaign concentrates on the BMS measurement chain, signal availability and timing, and deployment-accessible initialization states. It does not cover time-varying calibration drift, interacting sensor faults, pack-level coupling, or supervisory BMS failures. The HECM sensitivity study uses local one-at-a-time resistance, time-constant, and OCV variations rather than probability distributions or arbitrary model mismatch. The initialization experiment maps an SOC-equivalent mismatch into each estimator's accessible state. Its recovery times therefore characterize operational re-anchoring under class-specific interventions rather than convergence of identical latent states.

The hardware profile covers isolated SOC firmware with SOH supplied as an external causal trace. It does not yet include execution of the shared SOH-LSTM, energy measurement, pack communication, protection tasks, or concurrent BMS scheduling. These functions form the next system-level validation stage. The detailed embedded neural study in Chapter 7 complements this cross-class benchmark by examining pruning, quantization, and direct SOC and SOH execution in greater depth.

6.6 Conclusion

This chapter establishes a common robustness and embedded-performance benchmark using representatives from direct-measurement, hybrid direct-measurement, model-based ECM, and data-driven SOC estimation. Each implementation exposes a characteristic update and correction mechanism under the same causal and cell-disjoint protocol. The resulting evidence connects these mechanisms with the strengths and weaknesses that become relevant when an estimator moves from nominal software evaluation to practical BMS operation.

Direct-measurement methods provide transparent operation, deterministic execution, and very low resource demand. Their dependence on integrated current makes reliable sensing, uninterrupted charge information, and physical anchoring central to practical robustness. Hybridization through learned health information can improve ageing-related capacity adaptation without automatically eliminating the sensitivities of the integration core. Model-based ECM observers add physically interpretable voltage-feedback correction and strong re-anchoring capability. These benefits require calibrated parameters, additional flash storage, and a phys-

ical model that remains valid across the intended operating range.

The data-driven family shows strong potential for future BMS applications. The tested recurrent representative combines the lowest nominal error with strong results across several disturbance families. This demonstrates the value of learning nonlinear temporal relationships from multichannel battery data. Its deployment path also shows how this capability can be made compatible with embedded execution. Structured pruning reduces the GRU width from 96 to 67 units before export. Continuous-state forwarding then reduces median inference latency from approximately 724 ms for exact rolling-window recomputation to 0.425 ms and peak runtime RAM from 67.3 KiB to 4.1 KiB without a relevant loss of nominal accuracy on the evaluated sequences. Periodic reset does not retain this behavior because removing hidden context creates pronounced transients. Compact recurrent estimation is therefore feasible on the selected microcontroller, but its practical quality depends on both model preparation and stateful execution. Representative training coverage, disturbance-aware evaluation, feature selection, and recurrent-state handling remain essential to use this potential reliably.

The broader conclusion is that nominal MAE alone cannot establish suitability for BMS deployment. Accuracy must be evaluated together with disturbance-specific robustness, recovery behavior, numerical equivalence, inference latency, flash occupancy, and runtime RAM. The benchmark therefore does more than compare four implementations. It reveals mechanisms that determine class-specific behavior and provides a practical basis for improving a selected estimator through sensing design, physical-model adaptation, disturbance-aware training, or embedded optimization.

Chapter 7

Efficient Neural SOC and SOH Estimation for Embedded Systems

The preceding contribution chapters addressed two conditions that must be met before a battery-state estimator can be deployed. Chapter 5 examined how relevant temporal information can be represented for learning, while Chapter 6 established a common basis for evaluating estimator behaviour beyond nominal operation. The remaining step is physical realization on the target controller. This chapter therefore moves the investigation from estimator development and validation to model–hardware interaction. It completes the methodological path from data representation through comparative assessment to embedded execution.

The chapter develops the embedded-AI study available as an SSRN preprint and integrates its deployment results with the wider research framework.[3] It applies the recurrent-network and compression foundations from Chapter 3 to the LFP data and execution chain introduced in Chapter 4. Section 7.1 defines the specific deployment objective and its position within prior embedded-estimation work. The following sections then trace the models from feature processing and training through pruning, quantization, C implementation, streaming evaluation, and measured microcontroller performance.

7.1 Embedded Deployment Objective

Online battery management requires SOC and SOH estimates that remain accurate while meeting strict memory, timing, and energy budgets. This combination is not guaranteed by a low offline test error alone. The estimator must preserve its recurrent state over long streams, use the same feature scaling as during model development, fit into flash and SRAM, and complete every update before the next sample arrives.[111, 119, 18, 15, 26]

Well-calibrated model-based observers and recent sequence models can both provide high accuracy under their respective evaluation conditions.[38, 25, 120, 121] Hybrid studies further combine parameter identification, learned correction, filtering, and recurrent prediction.[122, 123] Compact battery estimators have also been demonstrated on microcontrollers.[73, 72, 74, 114, 11] However, these implementations commonly focus on one state variable, one model family, one compression method, or one target platform. This makes it difficult to distinguish the effect of compression from differences in models, temporal execution, export paths, and hardware measurement.

The application setting is modular stationary storage in remote microgrids within the MG-FARM programme.[87, 88, 124, 89] Each storage module must operate autonomously on an STM32H753ZI-class controller. Against this setting, the study evaluates the same stateful LSTM + MLP estimators for both SOC and SOH in matched Base, Pruned, and Quantized variants. Its objective is to determine whether structured hidden-unit pruning and post-training recurrent-weight quantization reduce flash footprint, RAM demand, inference time, and the corresponding timing-derived energy estimate while retaining estimation accuracy and stable sequential behaviour. A traceable PyTorch-to-C path with explicit state and scaling handling connects continuous replay to linked-firmware memory and measured kernel time. The contribution is therefore a comparative deployment benchmark across two battery states and two compression routes rather than a new recurrent cell or compression algorithm.

7.2 Feature Engineering and Online Processing

All experiments use the LFP DoE Cycle Ageing Dataset and the reference labels described in Section 4.1.[93, 125] The present section defines only the model-specific feature vectors, scaling, state handling, and causal SOH post-processing.

The SOC estimator consumes the six-dimensional feature vector

$$\mathbf{x}_t^{\mathrm{SOC}} = \Big[U(t),\, I(t),\, T(t),\, Q_c(t),\, \tfrac{\mathrm{d}U}{\mathrm{d}t}(t),\, \tfrac{\mathrm{d}I}{\mathrm{d}t}(t)\Big], \tag{7.1}$$

The derivative channels are causal timestamp-aware backward differences. For

sample $k > 0$,

$$\Delta t_k = \max(t_k - t_{k-1}, \Delta t_{\min}), \qquad \Delta t_{\min} = 10^{-6}\,\mathrm{s}, \tag{7.2}$$

$$\left.\frac{\mathrm{d}U}{\mathrm{d}t}\right|_k = \frac{U_k - U_{k-1}}{\Delta t_k}, \qquad \left.\frac{\mathrm{d}I}{\mathrm{d}t}\right|_k = \frac{I_k - I_{k-1}}{\Delta t_k}. \tag{7.3}$$

Only the preceding sample and timestamp are required, so the online calculation has constant cost. For the reported STM32 benchmark, however, the derivative and cumulative channels were prepared on the host and transmitted as part of the complete feature vector. The measured neural-kernel time consequently excludes sensor acquisition and feature construction. This boundary isolates the effect of model compression from that of the feature front end.

The SOH estimator uses a slower-varying feature stack,

$$\mathbf{x}_t^{\mathrm{SOH}} = \Big[t,\, U(t),\, I(t),\, T(t),\, \mathrm{EFC}(t),\, Q_c(t)\Big], \tag{7.4}$$

with cumulative timestamp t, equivalent full cycles $\mathrm{EFC}(t)$, and cumulative charge state $Q_c(t)$. The EFC definition based on absolute charge throughput is given in Equation 6.13. All channels are robustly normalised with the centre m_k and scale s_k fitted on the training data,

$$\tilde{x}_{t,k} = \frac{x_{t,k} - m_k}{s_k}. \tag{7.5}$$

For a nonconstant channel, the default RobustScaler uses the training-set median as m_k and the interquartile range as s_k. The fitted centre and scale values are stored with the model and exported unchanged to the C implementation. This reduces the influence of extreme training values and keeps host-side replay and MCU execution numerically aligned.[126, 127]

The SOC and SOH labels follow the Coulomb-counting and capacity-check-up procedures in Section 4.1. Their definitions are given in Equations 4.6 and 4.4. Sustained constant-voltage full-charge phases and periodic complete capacity tests provide information that is unavailable during ordinary online operation. These quantities are therefore targets for training and scoring, not estimator inputs. During deployment, only the measurement-derived feature stream and the internal hidden and cell states are available. Both states are propagated sample by sample without sliding windows or resets at artificial segment boundaries.

The displayed SOH trajectory passes through two causal post-processing stages.

Each raw prediction $\hat{y}_t$ is first aligned to the initial label by $\hat{y}_t^{(0)} = (y_0/\hat{y}_0)\hat{y}_t$. Stage 1 restricts positive and negative proposal changes to

$$\ell_t = \min\left(10^{-4}|y_{t-1}^{(1)}|, 10^{-5}\right) \tag{7.6}$$

and then applies an exponential moving average with $\alpha_1 = 0.02$. Stage 2 processes this complete intermediate trajectory with $\alpha_2 = 10^{-6}$ and limits only downward changes,

$$e_t^{(2)} = (1-\alpha_2)e_{t-1}^{(2)} + \alpha_2 y_t^{(1)}, \qquad y_t^{(2)} = \max\left(e_t^{(2)}, y_{t-1}^{(2)} - 2\times 10^{-8}\right). \tag{7.7}$$

For a linear exponential moving average sampled every Δt, the equivalent time constant and cutoff frequency are

$$\tau = -\frac{\Delta t}{\ln(1-\alpha)}, \qquad f_\mathrm{c} = \frac{1}{2\pi\tau}. \tag{7.8}$$

At 1 Hz, stage 1 has $\tau = 49{,}5\,\mathrm{s}$, $f_\mathrm{c} = 3{,}22\,\mathrm{mHz}$, and an EMA-only 90 % response time of about 114 s. Stage 2 corresponds to $\tau = 11.57$ days, $f_\mathrm{c} = 1.59\times10^{-7}$ Hz, and a 90 % response time of 26.65 days. These frequency values apply only to the linear averaging elements. The rate limiters are nonlinear and cannot be represented by one cutoff frequency. Figure A.6 later separates the influence of compression from the accuracy and delay introduced by these stages.

Figure 7.1 summarizes the stateful LSTM + MLP structure used for both tasks.

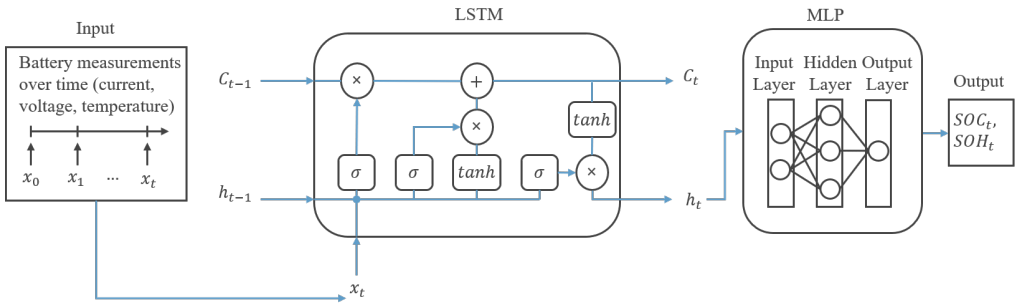


Figure 7.1: Stateful LSTM + MLP estimator structure used for both SOC and SOH estimation. Preprocessed feature streams are processed step by step by a single-layer LSTM whose hidden and cell states are propagated across time. A lightweight MLP head then maps the hidden representation to either SOC or SOH. The SOC instance uses hidden size 64 with an MLP width of 64 and a sigmoid output, the SOH instance hidden size 128 with an MLP width of 128 and a linear output. This structure is the basis for the later pruning, quantization, and MCU export steps.

7.3 LSTM Architecture and Training Setup

Both SOC and SOH estimators employ the LSTM update defined in Equation 3.12, but with task-specific feature sets and sizes, as summarized in Figure 7.2. The SOC model uses a single-layer LSTM with hidden size $H = 64$ that consumes a six-dimensional feature vector $\{U, I, T, Q_c, \mathrm{d}U/\mathrm{d}t, \mathrm{d}I/\mathrm{d}t\}$. Its MLP head contains one ReLU-activated hidden layer of width 64 and a sigmoid output neuron that constrains the estimate to $[0, 1]$. The SOH model uses the same overall structure with hidden size $H = 128$ and an MLP hidden width of 128. It consumes the feature vector $\{t, U, I, T, \mathrm{EFC}, Q_c\}$ and ends in a linear output neuron because the slowly varying SOH target does not benefit from a saturating output activation.

In contrast to the hourly aggregated SOH estimator of Chapter 6, both models operate directly on the sample-wise 1 Hz stream. The hidden and cell states are propagated continuously between samples. The implementation follows the standard PyTorch gate order and reproduces the same matrix operations, state update, and activation functions in C.[32, 59] The application-specific architecture in Figure 7.1 therefore extends the generic cell in Figure 3.3 by adding the six-channel feature input and the task-specific MLP output heads.

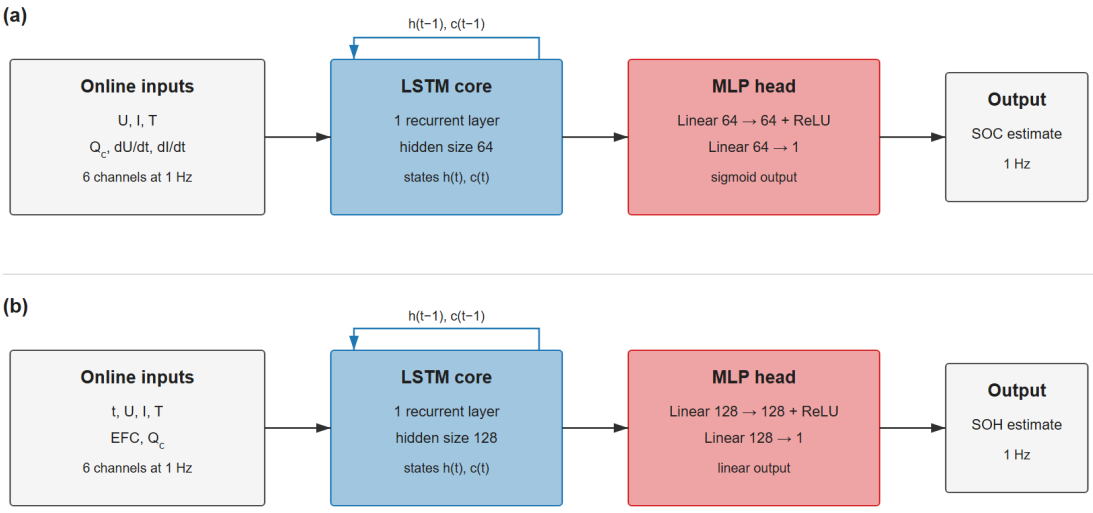


Figure 7.2: Task-specific architectures of the two embedded estimators. (a) The SOC estimator combines six online input channels with a single-layer LSTM of hidden size 64 and an MLP head with sigmoid output. (b) The SOH estimator processes its six online input channels with a single-layer LSTM of hidden size 128 and an MLP head with linear output. Both models operate statefully on the sample-wise 1 Hz stream and propagate their hidden and cell states between consecutive updates.

For input dimension $D = 6$, hidden size H, and MLP width M, the weight-related multiply–accumulate count of one streaming update is

$$N_{\mathrm{MAC}}(H) = 4H(D + H) + HM + M. \tag{7.9}$$

This count excludes bias additions, activation functions, indexing, and memory traffic. If a fraction p of the recurrent channels is removed, the retained dimension is $H_p = (1 - p)H$. The corresponding analytical reduction follows as

$$\begin{aligned} R_{\mathrm{MAC}}(H, p) &= 1 - \frac{N_{\mathrm{MAC}}((1-p)H)}{N_{\mathrm{MAC}}(H)} \\ &= \frac{4(2p - p^2)H^2 + p(4D + M)H}{4H^2 + (4D + M)H + M}, \\ \lim_{H\to\infty} R_{\mathrm{MAC}}(H, p) &= 2p - p^2. \end{aligned} \tag{7.10}$$

The quadratic recurrent contribution is reduced more strongly than the linear input

and MLP terms. At the evaluated $p = 0.30$, the large-model limit is 51 %. The actual integer dimensions yield a reduction from 22,080 to 12,124 MACs for SOC and from 85,120 to 46,208 MACs for SOH, corresponding to 45.1 % and 45.7 %. Figure A.9 visualizes the finite-size effect and the general dependence on pruning fraction.

The single-layer LSTM + MLP architecture provides a compact and traceable basis for comparing the two compression routes. Its recurrent update can be reproduced directly in C, and each task-specific Base architecture remains fixed while pruning or quantization is applied. The central question is therefore how compression changes the accuracy and embedded resource use of stand-alone recurrent SOC and SOH estimators. The hourly SOH context model in Chapter 6 has a different purpose. It supplies common health information for comparing several SOC-estimator families, whereas the models in this chapter directly produce the state estimates evaluated on the microcontroller. The results of the two chapters consequently do not constitute an architecture ranking.

Both SOC and SOH models are trained with mean-squared-error loss against the corresponding reference label at each training sample, while validation additionally tracks MAE, RMSE, and R^2. Model parameters, configurations, and optimiser states are retained so that the subsequent compression and export stages can be reproduced deterministically. Training is carried out on NVIDIA A100-class GPU hardware in mixed precision (FP16), and both tasks use the AdamW optimiser, cosine-annealing warm restarts, gradient clipping, and mini-batch training.[128, 129, 130, 33] For SOC, the learning rate is 1.5×10^{-4}, the batch size is 512, and the sequence chunk length is 2024. For SOH, the learning rate is 2.0×10^{-4}, the batch size is 128, and the chunk length is 2048. At the 1 Hz sampling rate, the two chunk lengths represent approximately 34 minutes of consecutive operation. They provide a common temporal scale long enough to include extended load changes while keeping backpropagation memory bounded. Although deployment is fully stateful and step-wise, training uses truncated backpropagation through time over these chunks. Within each chunk, the hidden and cell states are propagated exactly as in deployment. The task-specific learning rates and batch sizes are fixed settings of the reported training runs rather than universal recommendations. For both tasks, the exported parameter set is selected by the lowest validation RMSE, independently of the later test trajectory. Its suitability for the present benchmark is then assessed by the complete held-out streaming evaluation.

The base models are first evaluated in streaming mode on the full ageing tra-

jectory of a representative LFP cell and serve as the numerical reference for all subsequent experiments. The same trained parameter sets are then pruned, quantized, and exported to STM32 firmware. Identical feature streams ensure that accuracy, flash, RAM, latency, and the timing-derived energy proxy refer to the same stateful sequence. Figure 7.3 summarizes this end-to-end path from training to embedded benchmarking.

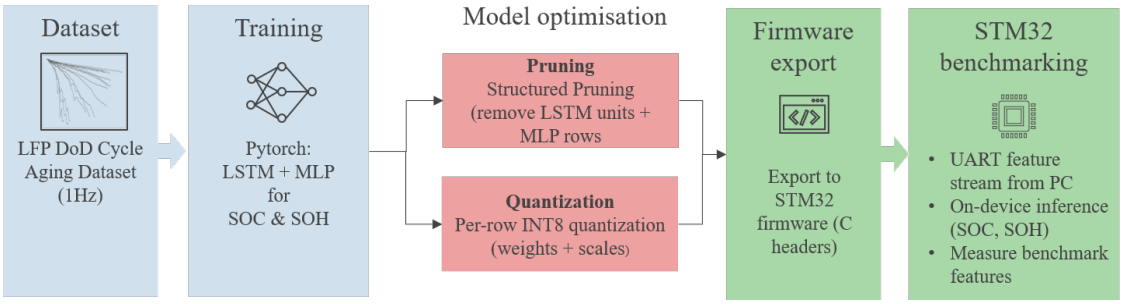


Figure 7.3: End-to-end deployment pipeline from the LFP ageing dataset and PyTorch training to pruning, quantization, C-header export, and final STM32 benchmarking. The same trained SOC and SOH parameter sets form the basis of the floating-point, pruned, and quantized deployments, which keeps the later comparisons numerically consistent.

7.4 Structured Pruning Configuration

The grouped magnitude criterion in Equation 3.21 and the local ranking principle in Equation 3.24 are instantiated here as one-shot structured post-training pruning. The implementation removes complete LSTM channels so that the exported tensors remain dense and the recurrent state becomes smaller. This structure directly matches the manually implemented C kernel and avoids an irregular element-wise sparsity mask. For hidden unit h, the implemented score combines the L2 norms of its input and recurrent rows across all four gates,

$$s_h = \sum_{g \in \{i,f,o,g\}} \left(\|W_{\text{ih}}^{(g)}[h,:]\|_2 + \|W_{\text{hh}}^{(g)}[h,:]\|_2 \right), \tag{7.11}$$

which follows the gate-grouping principle used in structured recurrent-network pruning.[47, 131, 132]

After sorting the scores, the retained index set $\mathcal{R}$ contains the H' highest-ranked units. The score itself uses only the input and recurrent gate rows. Once a channel is removed, the same index must also be removed from both LSTM bias vectors,

the recurrent input dimension, the hidden and cell states, and the input dimension of the MLP head. The dense reduced tensors are formed by

$$\begin{aligned} \widetilde{W}_{\mathrm{ih}}^{(g)} &= W_{\mathrm{ih}}^{(g)}[\mathcal{R}, :], \\ \widetilde{W}_{\mathrm{hh}}^{(g)} &= W_{\mathrm{hh}}^{(g)}[\mathcal{R}, \mathcal{R}], \\ \widetilde{W}_{\mathrm{MLP},1} &= W_{\mathrm{MLP},1}[:, \mathcal{R}]. \end{aligned} \tag{7.12}$$

The same indices therefore remove gate rows, recurrent columns, and the associated MLP inputs. Figure 7.4 makes the scoring and slicing process numerically traceable through a reduced example with $H = 3$ and $D = 2$. The displayed coefficients are illustrative rather than weights taken from the trained estimators, while the arrangement of the gate blocks and all removal operations follow the implementation. For each candidate channel, the L2 saliency score is the sum of its four input-weight row norms and four recurrent-weight row norms. A small score therefore indicates a channel with low aggregate weight magnitude. In the example, channel h_2 has the smallest score of 0.15, compared with 4.46 for h_1 and 5.23 for h_3. Selecting h_2 removes the highlighted row from every gate block and the corresponding recurrent and MLP input columns. The retained set $\mathcal{R} = \{h_1, h_3\}$ then forms new dense tensors with $H' = 2$.

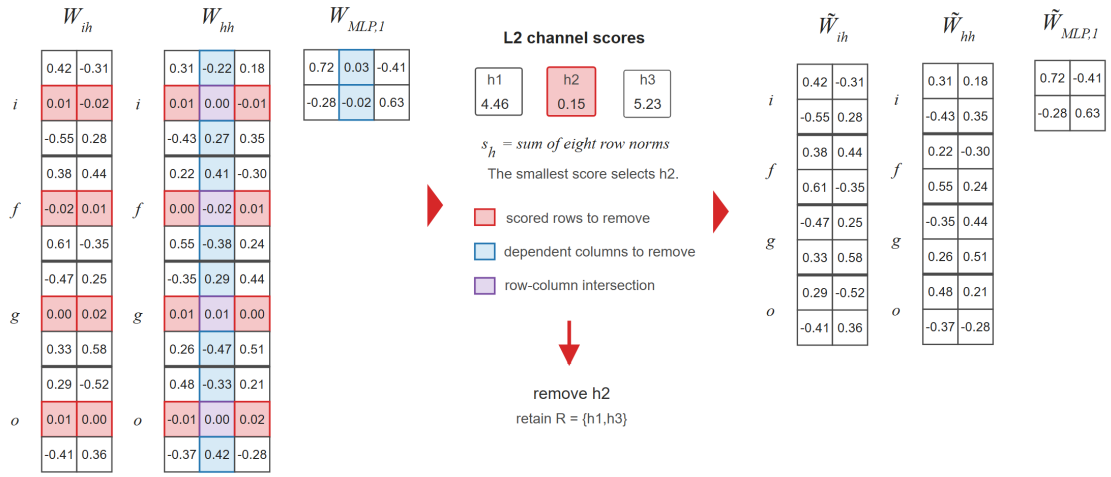


Figure 7.4: Illustrative numerical example of gate-consistent LSTM pruning. Red cells identify the rows included in the L2 saliency score, blue cells identify dependent columns that must be removed with the selected channel, and purple cells belong to both groups. Removing the lowest-scoring channel h_2 and retaining $\mathcal{R} = \{h_1, h_3\}$ produces the smaller dense tensors shown on the right. The numerical values are provided to make the scoring principle transparent, while the evaluated SOC and SOH estimators apply the same operations to their full hidden dimensions.

SOC is reduced from 64 to 45 units and SOH from 128 to 90 units, in both cases approximately 30 %. This fraction is one moderate operating point rather than the result of an exhaustive rate search. It was chosen to provide a substantial dimensional reduction while retaining useful accuracy. Determining an optimum would require separate pruning and fine-tuning runs for several candidate fractions.

The transformation is a one-shot structured removal followed by brief post-pruning fine-tuning. It is not an iterative pruning schedule and does not use pruning-aware training. The L2 score was selected because it is deterministic once the model is trained, groups the four gate rows consistently, and maps directly to smaller dense arrays. Gradient-based criteria would additionally require training data and a backward pass, while activation-based scores would depend on a calibration stream and a temporal aggregation rule. No experimental ranking of these alternatives was performed, so the L2 criterion is treated as a reproducible implementation choice rather than a universally optimal method. Figures A.4 and A.10 document the selected channels and the scope of this rationale.

7.5 INT8 Quantization and Manual STM32 Export

The general quantization design space is introduced in Section 3.5.2 and summarized in Figure 3.7. The implementation evaluated here retains the Base topology but changes the stored representation of the two recurrent matrices W_{ih} and W_{hh}.[13, 66, 85] Each row r of either FP32 matrix is mapped independently,

$$s_r = \max\left(\frac{\max_k |(W^{\mathrm{fp32}})_{r,k}|}{127}, \varepsilon_q\right), \tag{7.13}$$

$$(W^{\mathrm{int8}})_{r,k} = \mathrm{clip}\left(\mathrm{round}\left(\frac{(W^{\mathrm{fp32}})_{r,k}}{s_r}\right), -127, 127\right), \tag{7.14}$$

$$(\widehat{W})_{r,k} = s_r (W^{\mathrm{int8}})_{r,k}. \tag{7.15}$$

The lower bound ε_q protects an all-zero row. Although the signed INT8 data type spans -128 to 127, the symmetric quantizer uses only $\{-127, \ldots, 127\}$ and leaves -128 unused. This keeps zero exact and gives equal positive and negative ranges. With nearest-integer rounding and the row maximum as scale, finite values do not clip and satisfy

$$\left|(W^{\mathrm{fp32}})_{r,k} - (\widehat{W})_{r,k}\right| \leq \frac{s_r}{2}. \tag{7.16}$$

Figure 7.5 shows the row-wise transformation and the resulting export representation. Every matrix row receives an individual FP32 scale. The recurrent weights are stored as INT8 codes, while scales and biases are exported separately for the manually implemented kernel.

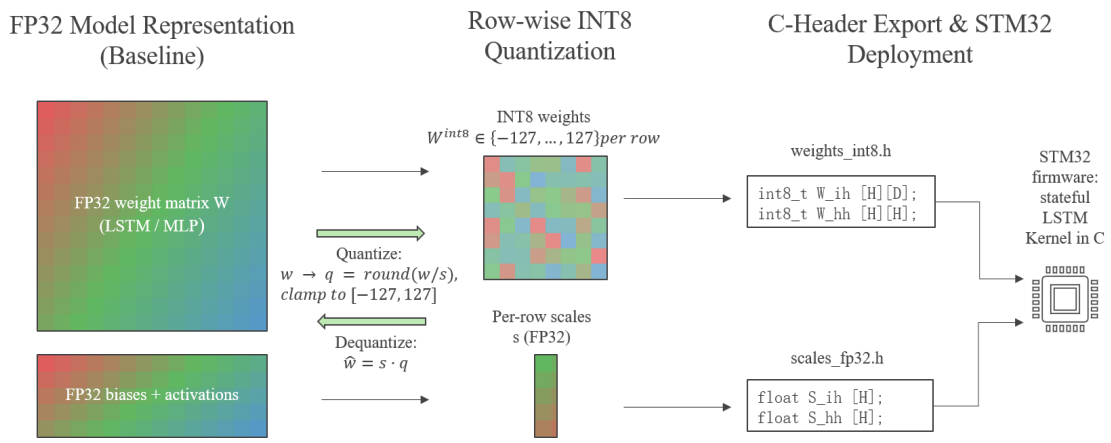


Figure 7.5: Row-wise weight-only INT8 quantization and embedded export used for the recurrent matrices. Floating-point matrix rows are represented by integer codes and individual floating-point scales. Biases and activations remain in floating point, while the stored recurrent weight matrices require fewer bytes.

The complete mixed-precision gate pre-activation can be written as

$$\mathbf{a}_t = \mathbf{s}_{\text{ih}} \odot \left(W_{\text{ih}}^{\text{int8}} \mathbf{x}_t\right) + \mathbf{s}_{\text{hh}} \odot \left(W_{\text{hh}}^{\text{int8}} \mathbf{h}_{t-1}\right) + \mathbf{b}_{\text{fp32}}. \tag{7.17}$$

In the inspected C kernels, each INT8 recurrent weight is converted to FP32 and multiplied by its row scale inside the accumulation. Only W_{ih} and W_{hh} use INT8 storage. The scales, biases, hidden and cell states, gate activations, MLP weights and arithmetic, and final output remain FP32. The recurrent matrices account for about 80 % of the raw Base constants, so this boundary targets the dominant persistent storage while avoiding an additional quantization source in the recurrent state. It is therefore a weight-only mixed-precision implementation, not a full-integer network, and it cannot deliver the RAM and arithmetic benefits of activation and state quantization. Figure A.11 makes this precision boundary explicit.

The exported parameters are written as C headers and integrated into manually implemented stateful LSTM + MLP firmware. This route exposes all scale operations and state updates and avoids ambiguity in generated support for continuous recurrent state.[133, 134, 135] Streaming checks against the source implementation typically produce MAE differences below 10^{-4}.

7.6 Embedded Benchmark Methodology and Evaluation Metrics

The hardware benchmark uses an STM32 Nucleo-144 development board with an STM32H753ZI microcontroller, 2 MB flash, and 1 MB SRAM.[136, 11] For each task, the Base FP32 model, the structured-pruned FP32 model, and the INT8-weight mixed-precision model receive the same ordered feature stream. The recurrent states are not reset during the complete replay.

The host analysis produces trajectory errors, distributions, and local views. It also divides the naturally ordered output into ten equal-count segments without resetting the hidden or cell state. Target MAE and P95 are calculated in every segment, together with the mean absolute difference between each compressed output and Base. This separates a general increase in task difficulty later in the sequence from a compression-specific drift. The corresponding evidence is collected in Figure A.5.

7.6.1 Transient Input-Buffer and Report-Loss Tests

Two bounded software experiments extend the nominal replay. First, randomly unavailable output reports at rates of 1, 5, and 10 % are replaced by the most recent valid estimate while the underlying inference continues. The same hold-last policy is tested for ten deterministic gaps of 60, 600, and 3600 samples. This experiment addresses report availability rather than a disturbance of the recurrent computation.

The second experiment inserts a one-sample fault before inference and follows its propagation through the recurrent state. Each exported C model processes a continuous 8000-sample segment from one representative evaluation trajectory at 1 Hz. After at least 2048 clean warm-up samples, 30 independent events are distributed over the stream. Ten affect voltage, ten current, and ten temperature. Bits are indexed from zero at the least significant position. Bit 22 is therefore the highest-order bit of the 23-bit FP32 fraction field. It is flipped for one input value and one sample only, leaving sign and exponent unchanged. Clean and disturbed branches start from the same recurrent state and then receive the same 60 unmodified inputs without resetting the LSTM.

With $k = 0$ denoting the affected sample and $k = 1, \ldots, 60$ the clean continuation, the window contains 61 values and spans a 60-s recovery horizon. The

disturbed-to-clean deviation is

$$d_{e,k} = 100 \left| \hat{y}_{e,k}^{\text{fault}} - \hat{y}_{e,k}^{\text{clean}} \right| \tag{7.18}$$

in percentage points. Peak deviation, the residual after 60 s, and the first sustained return below $\max(0.1 d_{e,\text{peak}}, 10^{-4}\,\text{pp})$ characterize propagation independently of the ordinary target error. The associated accuracy change is

$$\Delta\text{MAE}_{61,e} = \frac{100}{61} \sum_{k=0}^{60} \left(\left| \hat{y}_{e,k}^{\text{fault}} - y_{e,k} \right| - \left| \hat{y}_{e,k}^{\text{clean}} - y_{e,k} \right| \right). \tag{7.19}$$

A positive value means that the fault increases target MAE. A negative value can occur when an imperfect disturbed prediction happens to move closer to the label.

7.6.2 STM32 Measurement Scope and Evaluation Indicators

Feature vectors are sent by UART to dedicated SOC and SOH firmware. End-to-end latency covers transmission, scheduling, and return of the estimate. Neural-kernel time is measured around the LSTM and MLP with the data watchpoint and trace cycle counter and averaged over 10,000 streaming updates. This interval includes instructions and memory accesses executed by the kernels but excludes UART transfer, host processing, sensor acquisition, and host-side feature construction.

Six-feature scaling is inside the timed SOC call. For SOH, scaling occurs immediately before the measured interval in Base and Pruned, while the Quantized forward path includes it. This small affine cost was not separated and is absent from the static operation counts. The inspected projects use the hard-float interface and Cortex-M7 floating-point unit with compiler optimization disabled. The results therefore describe transparent reference kernels rather than a production-optimized integer library.

For normalized label y_n and estimate $\hat{y}_n$, the signed percentage-point error is $e_n = 100(\hat{y}_n - y_n)$. The reported metrics are

$$\text{MAE} = \frac{1}{N} \sum_{n=1}^{N} |e_n|, \qquad \text{RMSE} = \sqrt{\frac{1}{N} \sum_{n=1}^{N} e_n^2}, \tag{7.20}$$
$$\text{P95} = Q_{0.95}\left(\{|e_n|\}_{n=1}^{N} \right),$$

where P95 is the absolute-error threshold below which 95 % of samples lie. MAE and RMSE include every sample.

Flash is read from the linked firmware image. RAM combines initialized and zero-initialized static storage with the largest observed streaming stack use. Energy was not measured separately for each model. The reported quantity is the timing-derived proxy

$$E_{\text{est}} = P_{\text{assumed}} t_{\text{inf}}, \qquad P_{\text{assumed}} = 0{,}5\,\text{W}. \tag{7.21}$$

The power value is a representative assumption for the complete development board. Hence, relative changes in E_{est} are identical to changes in kernel time. The proxy cannot separate processor arithmetic, flash and SRAM access, UART activity, regulator losses, or other board consumers.

Table 7.1: STM32 engineering KPIs for the SOC deployments under full-stream replay. The table separates end-to-end latency from DWT-measured kernel time. Energy is the proxy from Equation 7.21, not an independent power measurement.

Model	Latency [ms]	Inference [ms]	Flash [KiB]	RAM [KiB]	E_{est} [mJ]
Base FP32	12.70	1.40	106.37	4.93	0.70
Pruned FP32	12.20	0.80	66.83	4.03	0.40
Quant INT8	18.48	6.99	53.54	3.96	3.49

Table 7.2: STM32 engineering KPIs for the SOH deployments under full-stream replay. Energy is estimated with the same constant-power timing proxy used for SOC.

Model	Latency [ms]	Inference [ms]	Flash [KiB]	RAM [KiB]	E_{est} [mJ]
Base FP32	34.88	22.73	351.14	8.69	11.37
Pruned FP32	24.69	12.72	199.14	6.96	6.36
Quant INT8	41.23	29.21	154.82	6.70	14.60

7.7 Streaming Prediction Accuracy

The evaluation starts with full-sequence streaming replay over the complete ageing trajectory. Figures 7.6 and 7.7 therefore provide the central long-horizon view of

how the dense, pruned, and quantized deployments behave under identical stateful replay conditions. The PC reference replay and the exported STM32 firmware process the same ordered feature stream with the same stateful update logic, so the reported values are full-stream averages over the same trajectory rather than short-window averages from selected excerpts. Differences between variants therefore reflect pruning, quantization, and numerical representation rather than a change of evaluation data.

For SOC, the tracking remains tight across the entire dynamic range. The residuals stay centred around zero, and the quantized model overlaps the full-precision baseline so closely that the two traces are often visually indistinguishable. The full-stream SOC MAE values are 2,68 % for Base, 2,34 % for Pruned, and 2,79 % for Quantized. These are percentage-point errors on the 0–100 % SOC scale. The corresponding RMSE values are 3,81 %, 3,13 %, and 3,88 %. Thus, recurrent-weight quantization retains the useful stateful behaviour of the original network over the evaluated sequence. Pruned produces the lowest aggregate error for this particular trained model and data split, although some dynamic sections show a larger deviation from Base.

For SOH, degradation must be inferred from a much slower signal. The capacity tests that define the labels are not presented to the estimator, whereas open-circuit-voltage sweeps, pulse tests, profile sections, and ordinary operating data remain in the causal input stream. The displayed trajectories pass through the complete post-processing chain defined around Equation 7.7, including initial alignment, the first EMA with its symmetric step limiter, and the second EMA with the downward-rate restriction. After this processing, the full-stream SOH MAE values are 0,85 % for Base, 1,46 % for Pruned, and 1,41 % for Quantized. The respective RMSE values are 1,15 %, 1,87 %, and 1,89 %. All three variants reproduce the long-term loss of capacity, but both compressed versions have a higher aggregate error than Base in this evaluation.

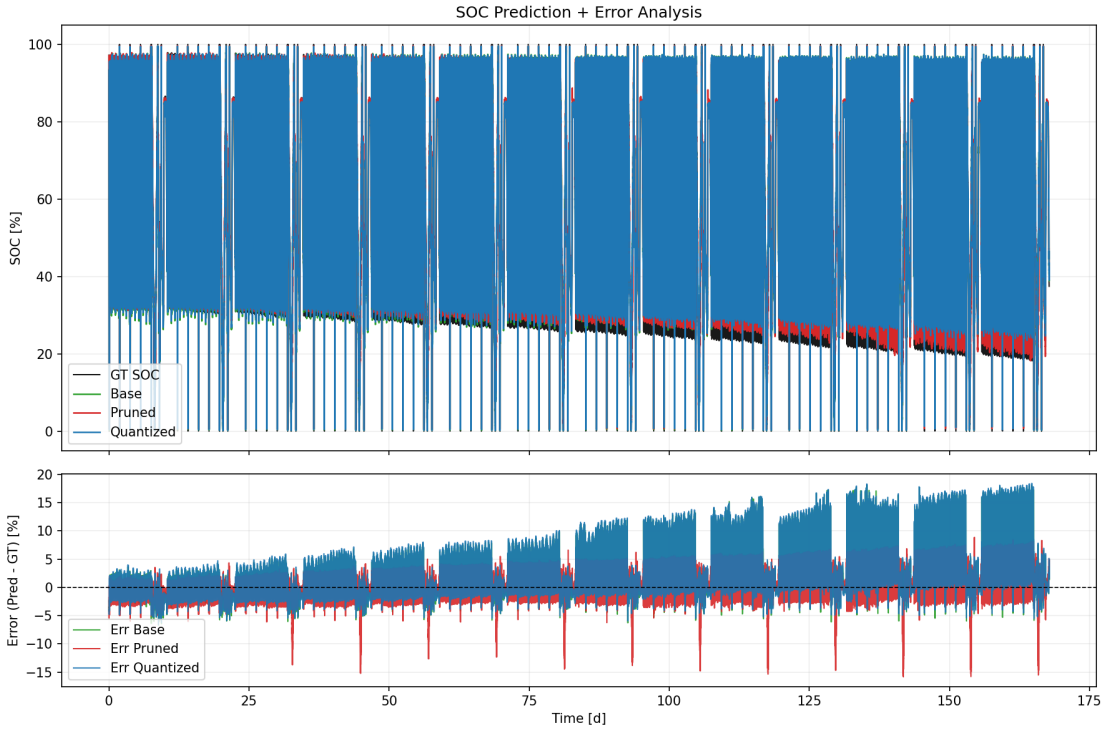


Figure 7.6: Streaming SOC dashboard for the base, pruned, and quantized SOC deployments over the full ageing trajectory. The upper panel compares the three stateful predictions with the SOC reference label, while the lower panel shows the instantaneous prediction error. The near-overlap of the dense and quantized traces is one of the central observations of the chapter.

7.8 Temporal Stability and SOH Filter Interaction

The complete output sequence is divided into ten consecutive parts of equal sample count to distinguish ageing-related changes in task difficulty from a steadily growing compression error. Recurrent states continue across every boundary. For SOC, the MAE of Base rises from 1.116 percentage points in the first part to 4.538 percentage points in the final part. Quantized changes from 1.234 to 4.600 percentage points over the same interval. Despite this common rise in target error, the mean absolute difference between Quantized and Base decreases from 0.332 to 0.271 percentage points. The corresponding Pruned-to-Base difference increases from 0.924 to 2.592 percentage points, even though Pruned has the smallest MAE over the complete trajectory. The SOH segment errors vary with the ageing phase, but neither compressed estimator develops a continuously increasing distance from

Base. Figure A.5 contains the complete segment-wise comparison.

The filter study evaluates every SOH model before post-processing, after stage 1, and after the final stage using an identical input order. Stage 1 yields MAE values of 1.677, 1.644, and 1.926 percentage points for Base, Pruned, and Quantized. The final values for this controlled sequence are 1.476, 1.695, and 1.028 percentage points. Accordingly, post-processing can alter both the numerical error and the ordering of the variants. Figure A.6 shows this interaction together with the response characteristics calculated in Equation 7.8. The final filter can reduce short-term variation substantially, but its EMA-only 90 % response time of 26.65 days at 1 Hz must be considered together with the reported accuracy.

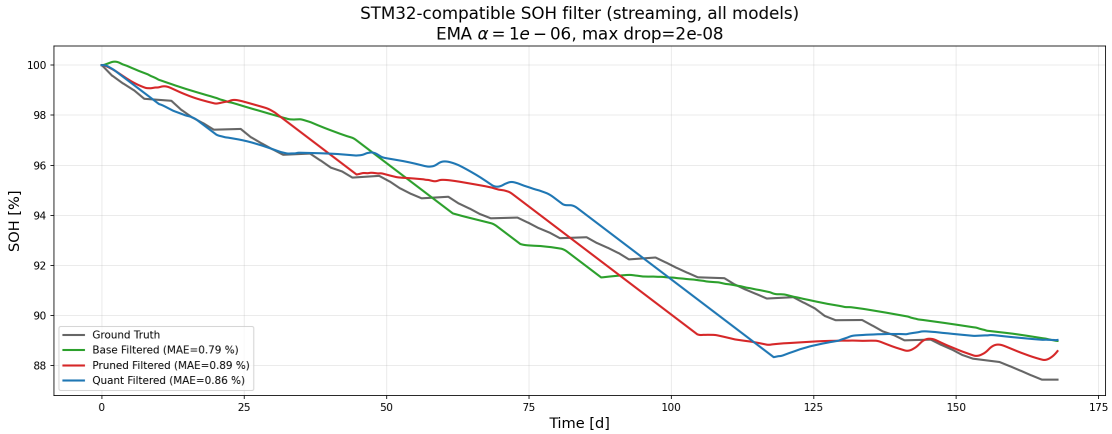


Figure 7.7: Streaming SOH dashboard with causal filtering for the three deployment variants. The upper panel shows the reference SOH together with the base, pruned, and quantized predictions after online smoothing, while the lower panel displays the corresponding prediction errors. The figure highlights the much slower degradation dynamics and the stronger role of causal post-processing in the SOH task.

7.9 Error Distribution and Local Transient Behaviour

Figures 7.8 and 7.9 analyse the statistical distribution of the prediction errors. For SOC, the distribution is narrow and approximately symmetric. The median absolute errors are 1,81 % for Pruned, 1,89 % for Base, and 2,03 % for Quantized. The pruning diagnostics in Figure A.4 verify that the 19 removed channels are the lowest-ranked channels under the defined L2 saliency criterion and compare the central recurrent-weight distributions before and after removal. These observations establish the implemented selection and the lower error of this particular Pruned

model, but they do not prove a regularisation mechanism or statistical superiority across training initialisations. For SOH, the distribution is broader, which is consistent with the greater difficulty of estimating capacity fade from operational data. Nevertheless, most errors remain within an approximately ±2% band around the reference trajectory.

Global metrics alone would still hide local failures. The chapter therefore also zooms into a high-dynamic pulse segment between 30k and 40k seconds (Figure 7.10) and into a representative check-up sequence between 657k and 897k seconds (Figure 7.11). In the pulse region, the quantized model follows the reference transients with high fidelity and captures voltage-recovery phases accurately, whereas the pruned model smooths some sharper local features. In the check-up phase, all variants remain stable over the extended charge and discharge interval. These local views complement the segment-wise temporal analysis rather than replacing it.

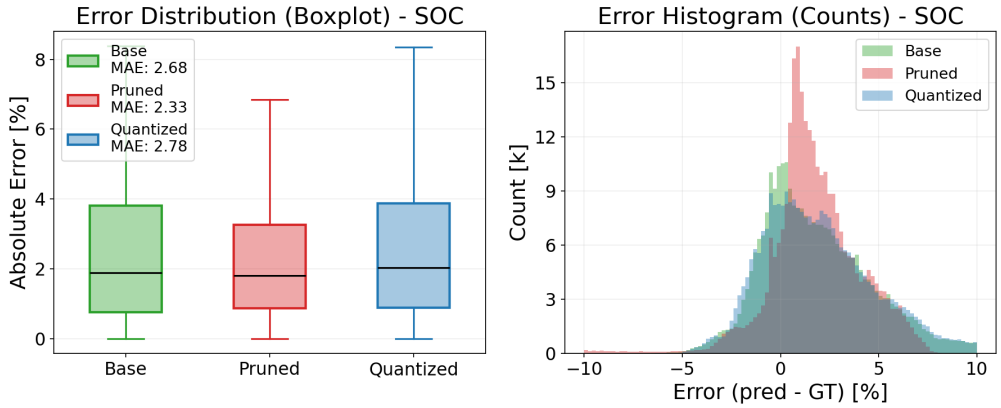


Figure 7.8: SOC error distribution for the embedded SOC deployments. The absolute-error boxplots on the left show that the Pruned model has the smallest median and interquartile range. The signed-error histograms on the right show the distribution relative to the SOC reference and the substantial overlap between the three variants.

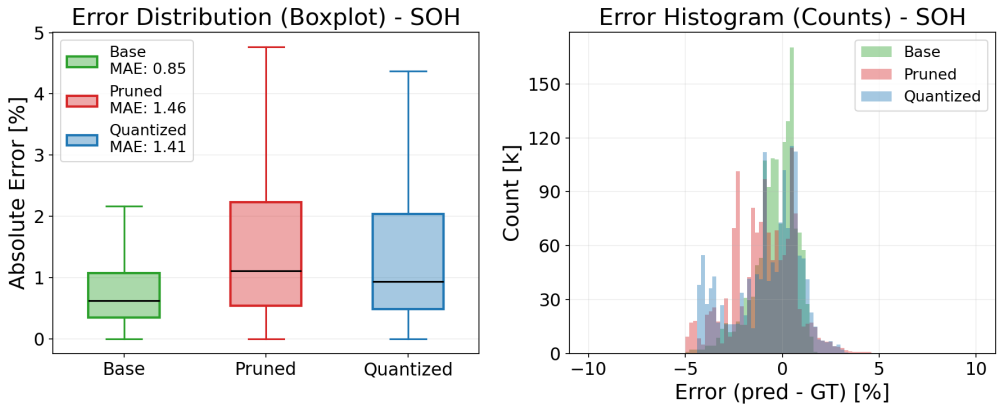


Figure 7.9: SOH error distribution for the embedded SOH deployments. The absolute-error boxplots on the left compare the median and interquartile range of the three variants. The signed-error histograms on the right show the broader and less regular error structure produced by the slowly varying SOH task.

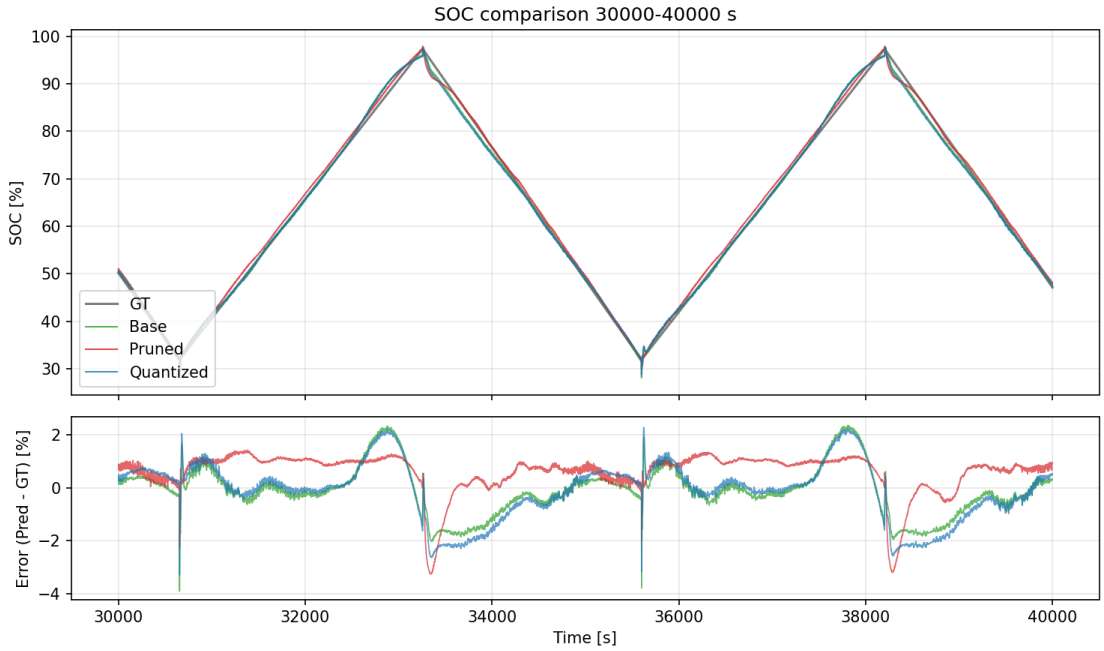


Figure 7.10: Detailed pulse-region analysis of embedded SOC streaming replay. The figure focuses on a highly dynamic load segment and shows how the three deployment variants capture fast transients and voltage-recovery behaviour under identical stateful inference conditions.

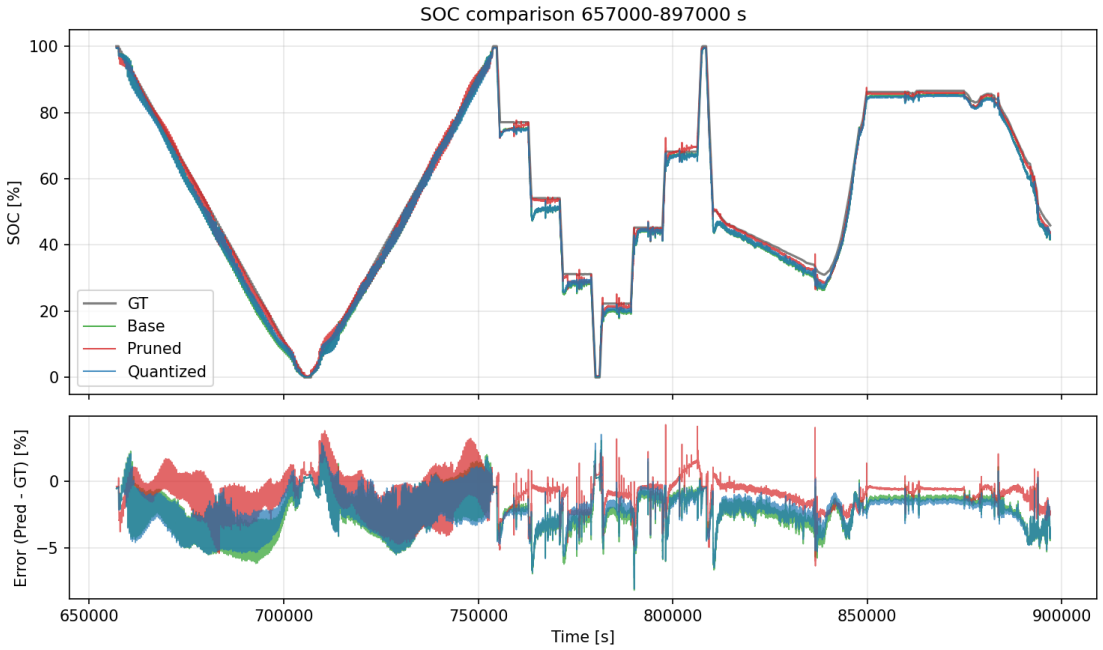


Figure 7.11: Detailed check-up-region analysis of embedded SOC streaming replay. The figure covers a representative diagnostic interval with long integration horizons and illustrates the stability of the stateful deployments during extended charge/discharge behaviour.

7.10 Software-Level Fault-Sensitivity Results

Holding the last valid estimate during randomly unavailable output reports has almost no effect on aggregate accuracy for report-loss rates up to 10 %. In every model, the resulting MAE change remains below 0.001 percentage points. Longer deterministic report gaps expose a clearer SOC effect. Ten gaps of 3600 samples each increase SOC MAE by approximately 0.05 percentage points, whereas the slowly changing SOH estimate remains almost unaffected. Since inference continues beneath the reporting layer, this part of the test characterises output availability and not corruption of the recurrent calculation.

The one-sample input-buffer experiment produces a distinct transient response. Across the SOC variants, median peak deviations between the faulted and clean branches lie between 3.29 and 3.82 percentage points. Their P95 values range from 16.18 to 23.81 percentage points. Every one of the 30 events per model satisfies the recovery criterion within 60 s, with median recovery times between 10.0 and 12.5 s. Quantized behaves similarly to Base, while Pruned combines the smallest median

peak with the largest upper-tail peak.

For SOH, median peak deviations range from 0.83 to 1.59 percentage points and P95 values from 3.37 to 5.85 percentage points. Within the specified 60-s horizon, 93.3 % of Base events, 90.0 % of Pruned events, and 86.7 % of Quantized events recover. The corresponding median times among recovered events are 9, 7, and 5 s. The target-related $\Delta\mathrm{MAE}_{61}$ values remain much smaller than the instantaneous peaks. For SOC, their medians range from 0.046 to 0.079 percentage points and their P95 values from 1.42 to 1.59 percentage points. For SOH, the median range is -0.0009 to 0.0012 percentage points and the P95 range is 0.053 to 0.122 percentage points. A negative event value only means that the altered prediction happened to approach an already imperfect target value. It is not an improvement in fault tolerance. Figure A.8 and Table A.7 provide the model-specific results.

The test shows that a single corrupted measurement can temporarily influence the recurrent state after the affected input has disappeared. No persistent SOC divergence or non-finite output occurs under the defined events. The analysis is deliberately bounded to input-buffer corruption and temporary report loss. It does not establish robustness against damaged weights, recurrent-state memory faults, communication failures, or physical sensor faults.

7.11 Resource Efficiency, Latency, and Trade-Off Summary

The KPI tables already show that pruning and quantization affect SOC and SOH differently, but Figures 7.12 and 7.13 condense the same trade-off visually across model size, memory, and timing. For each deployment, the model complexity can first be estimated from the parameter count under idealized storage assumptions, namely four bytes per FP32 weight and one byte per INT8 weight. The measured flash footprint is then taken from the STM32 linker output and is systematically larger because it also includes code, constant tables, and runtime support, a gap that is particularly visible for the larger SOH firmware. The RAM view in Figure 7.12 is based on the measured static data sections plus worst-case stack usage during streaming, and therefore approximates the true runtime requirement of the deployed kernels more realistically than a pure parameter-memory count.

For SOC, pruning reduces flash from 106.37 KiB to 66.83 KiB and kernel time from 1,40 ms to 0,80 ms. The associated E_{est} changes from 0,70 mJ to 0,40 mJ.

Quantized requires only 53.54 KiB of flash, but its mixed-precision kernel increases inference time to 6,99 ms and therefore raises the proxy to 3,49 mJ. For SOH, pruning changes flash from 351.14 KiB to 199.14 KiB, kernel time from 22,73 ms to 12,72 ms, and E_{est} from 11,37 mJ to 6,36 mJ. Quantized reaches the smallest SOH flash footprint of 154.82 KiB, while its 29,21 ms kernel time gives $E_{\text{est}} = 14{,}60$ mJ. These energy values contain no additional power information beyond the measured inference time.

The end-to-end latency distributions in Figure 7.13 reveal the same hierarchy. For SOC, the mean host-observed values are 12,20 ms for Pruned, 12,70 ms for Base, and 18,48 ms for Quantized. The corresponding SOH values are 24,69 ms, 34,88 ms, and 41,23 ms. The larger absolute Base–Pruned separation for SOH follows from its larger recurrent model. Its Base update contains 85,120 model MACs, compared with 22,080 for SOC, so removing approximately 30 % of the hidden channels saves more execution time in absolute terms. Relative to their respective Base kernels, however, the pruning reductions are similar at 42.9 % for SOC and 44.0 % for SOH. All end-to-end values remain below roughly 20 ms for SOC and 50 ms for SOH, which is comfortably inside typical stationary-storage BMS timing limits.[4, 18]

The analytical operation counts clarify why storage reduction and runtime do not necessarily follow the same ratio. Base and Quantized have unchanged topologies and therefore both require 22,080 model MACs for SOC and 85,120 for SOH. In the implemented mixed-precision expressions, however, applying the FP32 row scales introduces 17,920 additional source-level multiplications for SOC and 68,608 for SOH. Counting one MAC and one extra multiplication as one operation gives the same static Quantized-to-Base ratio of approximately 1.81 for both tasks. The measured time ratios differ considerably at 4.99 for SOC and 1.29 for SOH. Figure A.12 compares these quantities directly. Integer-to-float conversion, indexing, loops, load and store activity, nonlinear functions, cache behaviour, and the absence of an optimised integer dot-product kernel are not represented by the simple count. Since no operation-level or memory-bandwidth profiling was performed, these implementation details explain plausible contributors but not a measured causal decomposition.

Flash footprints in this chapter were audited from the available firmware ELF builds as the sum of flash-backed loadable segment sizes, including RAM initializers, divided by 1024 for KiB. Timing and RAM retain the previously recorded benchmark values. Build identity with those historical measurements has not been established; combined resource scores therefore provide a cross-record comparison

rather than a newly measured matched-build benchmark.

For a compact comparison across competing objectives, variant v is assigned the dimensionless score

$$U_v(\mathbf{w}) = w_A \frac{\mathrm{MAE}_v}{\mathrm{MAE}_{\mathrm{Base}}} + w_F \frac{F_v}{F_{\mathrm{Base}}} + w_R \frac{R_v}{R_{\mathrm{Base}}} + w_E \frac{E_{\mathrm{est},v}}{E_{\mathrm{est,Base}}}, \qquad \sum_j w_j = 1, \quad (7.22)$$

where all weights are non-negative. Here, F and R denote measured flash and RAM, while E_{est} is the timing-derived proxy. The main summary uses $w_A = w_F = w_R = w_E = 0.25$. In this equal-priority case, U is the arithmetic mean of the four ratios, Base is exactly one, and a smaller value indicates a more favourable combined operating point. The value should not be read as a physical efficiency or accuracy metric. It states how a model ranks for a chosen set of priorities.

To test whether that conclusion depends on equal weighting, the weights are varied over all 1771 non-negative combinations on a 0.05 grid whose sum is one. Pruned is the preferred SOC variant in 93.79 % of these combinations and Quantized in 6.21 %, while Base is never first. For SOH, Pruned ranks first in 53.36 %, Quantized in 17.11 %, and Base in 29.53 %. Thus, the SOC recommendation is stable across most priorities. The SOH choice depends more strongly on the application, with Base becoming attractive when accuracy dominates and Quantized when flash is assigned the largest importance. Figure A.7 shows both the priority sweeps and the complete ranking distribution.

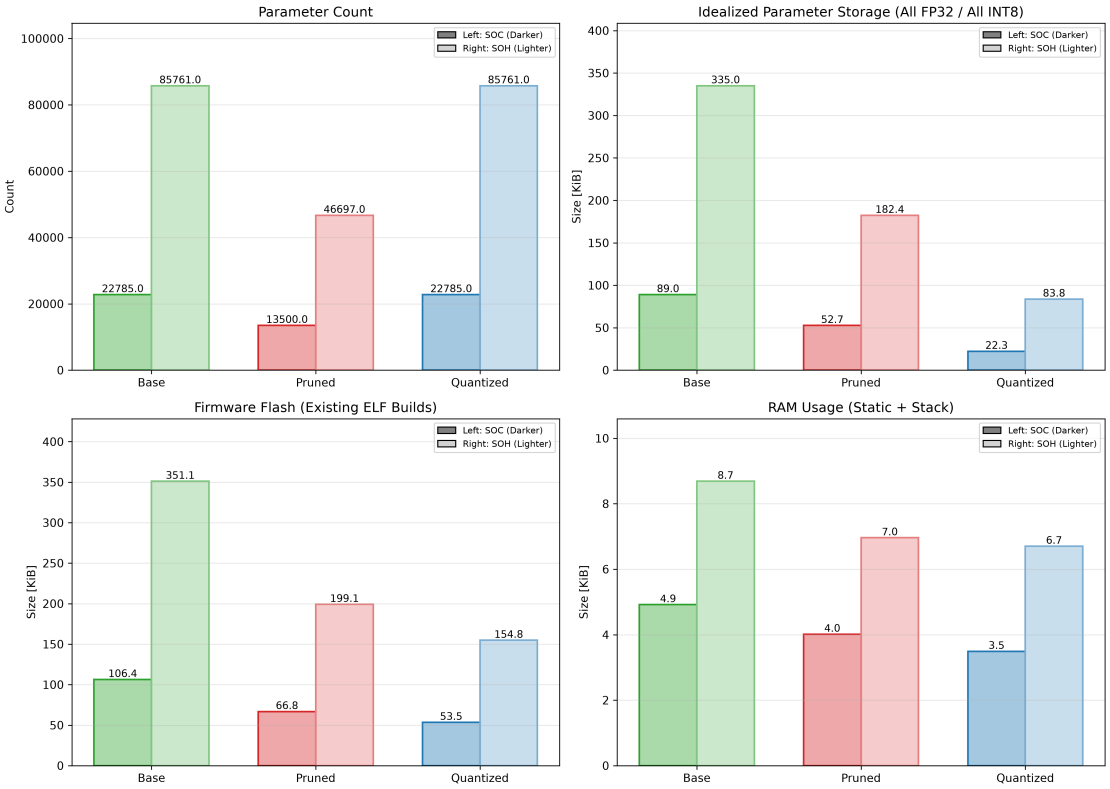


Figure 7.12: Resource landscape of the embedded SOC and SOH variants. The figure compares parameter count, idealized parameter storage assuming four bytes per parameter for Base and Pruned and one byte for Quantized, firmware flash extracted consistently from the available ELF builds, and previously recorded RAM usage. Flash is the sum of flash-backed loadable segment sizes, including initialization data for RAM. The all-INT8 estimate is idealized because the deployed quantized model retains FP32 parameters and scales. These available builds are not asserted to be identical to the historical timing builds; RAM values are unchanged.

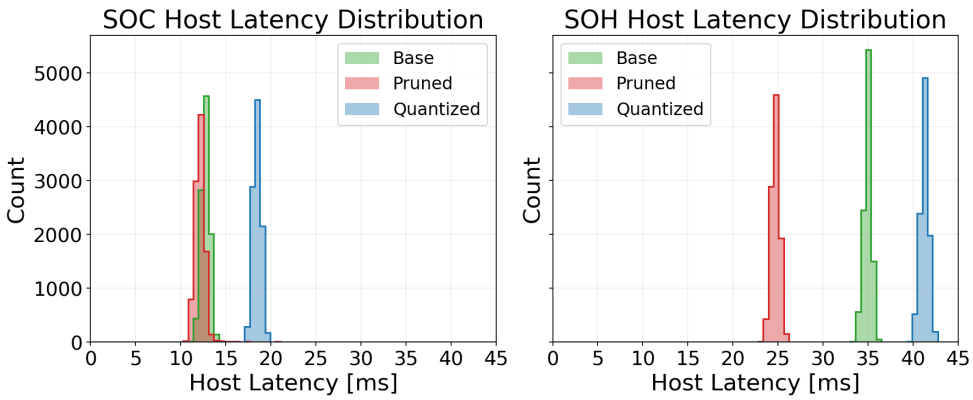


Figure 7.13: End-to-end host-latency distributions for repeated STM32H753ZI streaming measurements. Each observation includes UART communication and the on-device estimator update. The larger absolute Base–Pruned separation for SOH reflects the larger recurrent kernel, while the Quantized variants include the overhead of the mixed-precision weight reconstruction.

Table 7.3: Composite pruning and quantization summary. Values are reported as SOC/SOH. ΔMAE is given in percentage points of full scale. Flash, RAM, and E_{est} are relative changes with respect to the corresponding Base model. The utility score uses the equal weights in Equation 7.22, with lower values indicating the more favourable combined operating point.

Variant	ΔMAE [pp]	ΔFlash [%]	ΔRAM [%]	ΔE_{est} [%]	U
Pruned	−0.35 / +0.61	−37 / −43	−18 / −20	−43 / −44	0.72 / 0.91
Quantized	+0.10 / +0.56	−50 / −56	−20 / −23	+399 / +29	1.83 / 1.04

7.12 Discussion of Embedded Compression Results

7.12.1 Compression Behaviour of SOC and SOH Estimators

Pruning and quantization produce different outcomes for the two estimation tasks. In SOC estimation, Pruned achieves an MAE that is 0.35 percentage points below Base, while Quantized is 0.11 percentage points above it. The saliency and weight-distribution analysis confirms that the specified L2 procedure removed the intended low-ranked channels. It does not establish why the Pruned model has the lower test error. Only one trained model and one predefined split are compared for each task,

since the main objective is the embedded comparison of the compression routes. Repeated initialisations, cross-validation, and a matched fine-tuning control for the unpruned model would be required before interpreting the observed difference as a statistically stable training effect.

For SOH, both compressed variants show a bounded accuracy penalty. Interpretation requires particular care because the target changes slowly, the label is anchored by periodic capacity tests, and the reported online trajectory passes through two strong causal filter stages. The controlled comparison demonstrates that filtering can change both MAE and the order of Base, Pruned, and Quantized. Raw-network error, filtered error, and response delay therefore describe different properties and should be reported together when a post-processed SOH signal is used for BMS decisions.

7.12.2 Embedded Operating Points

The variants define distinct engineering operating points and do not follow a single accuracy order. Base offers the simplest numerical path but occupies the most flash. Structured pruning reduces the recurrent dimension and the input width of the MLP head. For SOC, this lowers flash from 106.37 KiB to 66.83 KiB and inference time from 1,40 ms to 0,80 ms. For SOH, the respective values change from 351.14 KiB to 199.14 KiB and from 22,73 ms to 12,72 ms. Within this common FP32 kernel structure, analytical and measured ratios agree closely. Pruned retains 54.9 % of the Base MACs and 57.2 % of its time for SOC, compared with 54.3 % and 56.0 % for SOH.

Quantized occupies only 53.54 KiB for SOC and 154.82 KiB for SOH, making it attractive when non-volatile memory is the binding constraint. Yet the retained FP32 activations, recurrent states, MLP, and accumulations prevent the model from becoming a fully integer implementation. Each recurrent INT8 value is converted and combined with its row scale inside the accumulation. The static counts alone predict the same 1.81 operation ratio for SOC and SOH, while the measured time ratios are 4.99 and 1.29. The difference cannot be attributed to one cause without cycle-level and memory-bandwidth profiling. It can include conversions, indexing, memory behaviour, nonlinear functions, and compiler-generated loop code. Scale hoisting, fused scale and bias handling, DSP-oriented dot products, or activation and state quantization are possible routes toward a more efficient kernel.

7.12.3 Latency, Utility Score, and BMS Deployment

The distinction between communication-dominated system latency and pure on-device inference time is important for interpreting the results. The UART-based measurements report end-to-end latencies of 12,20 ms, 12,70 ms, and 18,48 ms for the pruned, dense, and quantized SOC variants. The corresponding SOH latencies are 24,69 ms, 34,88 ms, and 41,23 ms. The actual neural computation is substantially shorter. Tables 7.1 and 7.2 show that dense and pruned SOC inference remains below 1,5 ms, and even the quantized SOC kernel stays below 10 ms. The SOH updates require more time because of the larger hidden state and post-processing, with 12,72 ms for pruning, 22,73 ms for the dense model, and 29,21 ms for quantization. These margins are compatible with the low-rate sampling and diagnostic time scales typical of stationary-storage BMS operation.[4, 18]

The utility score in Table 7.3 combines normalized accuracy, flash, RAM, and timing-proxy terms. It does not replace hard requirements, but it exposes whether the benefits of a compression variant outweigh its penalties under explicit priorities. With equal weights, SOC Pruned reaches $U = 0.72$, whereas SOC Quantized reaches $U = 1.83$ because its timing overhead outweighs its flash reduction. The SOH values of 0.91 for Pruned and 1.04 for Quantized describe a closer trade-off. The weight sensitivity adds the information that the SOC recommendation remains stable over most priority combinations, while the preferred SOH variant changes when accuracy or flash is emphasised. This distinction makes the score useful as a transparent decision aid rather than as a universal model ranking.

7.12.4 Limitations and Future Extensions

The quantitative evidence belongs to one LFP ageing campaign at 25 °C, the selected SOC and SOH architectures, and one STM32H753ZI implementation. Removing the same hidden channels or storing the same recurrent matrices as INT8 gives predictable changes in parameter storage for a fixed topology. Retaining estimation accuracy is more dependent on the cell chemistry, operating range, label construction, and training coverage. Results from other battery systems therefore support the general feasibility of compact estimators, but not the direct transfer of the MAE values or the preferred compression level.[72, 116, 1]

The MAC relation is independent of the microcontroller, whereas measured flash and timing depend on the clock, compiler settings, floating-point and signal-processing support, memory organisation, and cache. The energy numbers in this

chapter are obtained from the constant 0,5 W assumption in Equation 7.21. They are not component-level power measurements and cannot reveal the separate contributions of computation, flash, SRAM, UART, or regulators.

The two compression routes are post-training methods. Structured pruning is performed once and followed by short fine-tuning, while quantization starts from the original FP32 model. Quantization-aware training and iterative or pruning-aware training could adapt the parameters to their constrained representation.[66, 137] They require a separate training comparison and are outside the present benchmark.

Host-side derivative and cumulative-feature construction deliberately isolates neural inference in the timing comparison. The transient input-buffer analysis adds a bounded test of fault propagation through the recurrent state, but it is not a complete safety assessment. Further work should include variable sampling, causal derivative filtering, corrupted recurrent states and parameters, communication errors, and physical sensor or MCU faults. Moving the complete pipeline to the custom BMS platform would also add scheduler interaction, sensing-chain behaviour, and pack-level communication. On the model side, relevant extensions include additional battery systems and operating ranges, repeated training runs, combined pruning and quantization, and comparisons with compact GRUs, transformer-style models, and hybrid observers.[32, 138, 25, 40]

7.13 Embedded Deployment Conclusions and Outlook

This chapter evaluates two transparent compression paths for stateful LSTM + MLP SOC and SOH estimators under a common STM32 benchmark. Base, Pruned, and Quantized are compared using the same continuous feature stream, C implementation boundary, error definitions, linked-firmware memory, and cycle-counter timing. Energy per inference is represented only by the constant-power timing proxy E_{est}.

For SOC, Pruned reduces flash by 40.9 %, RAM by approximately 18 %, and inference time by 42.9 %, while its MAE is 0.35 percentage points below Base for the evaluated model and split. Quantized reduces flash by 50.2 % and RAM by approximately 20 %, but raises inference time by 399 % and MAE by 0.11 percentage points. For SOH, Pruned saves 45.5 % flash, approximately 20 % RAM, and 44.0 % inference time, with an MAE increase of 0.61 percentage points. Quantized saves 58.8 % flash and approximately 23 % RAM, while inference time rises

by 28.5 % and MAE by 0.56 percentage points. Under the fixed-power assumption, the percentages for E_{est} are identical to those for inference time.

The continuous ten-part analysis reveals no quantization-specific monotonic divergence from Base. Following one-sample input-buffer faults, all SOC events recover under the defined criterion within 60 s. For SOH, 6.7–13.3 % of events, depending on the model, do not meet that criterion inside the same horizon. The SOH comparison further demonstrates that compression and post-processing delay cannot be assessed independently because the two-stage filter changes both error magnitude and model ordering.

For the present firmware, structured pruning is the more balanced operating point, whereas recurrent-weight quantization is most attractive when flash dominates and its mixed-precision timing overhead is acceptable. This conclusion is bounded to the evaluated data, models, implementation, and build configuration. Further work should add externally measured board power, kernel profiling, wider battery and operating coverage, repeated training runs, direct comparisons with compression-aware training, and internal as well as hardware-facing fault classes. Joint pruning and quantization remains a promising route for combining a smaller recurrent state with compact weight storage.

Chapter 8

General Discussion, Conclusion, and Outlook

8.1 Integrated Synthesis of the Research Questions

The global result of this work can be read along the three research questions introduced in Chapter 1. The first question asks how comparatively simple neural models can represent time-dependent ageing without immediately requiring large recurrent architectures. Chapter 5 answers this through representation rather than model complexity: cumulative-current information and lag-sequence history inject degradation-relevant temporal context into an otherwise memoryless MLP. The resulting average SOH error of about 1,10 % shows that the structure of the input can be as important as the structure of the network, especially for slowly evolving ageing targets.

The second question asks how estimator quality changes under realistic measurement and operating conditions. Chapter 6 shows that the answer cannot be obtained from a clean-condition MAE alone. Across the six independent holdout cells, the data-driven representative achieves the lowest nominal cell-macro SOC MAE at 0.0258, followed by the hybrid equivalent-circuit model at 0.0337, the SOH-adaptive direct-measurement model at 0.0412, and fixed-capacity Coulomb counting at 0.0690. The disturbance tests change the relative distances and, in individual scenarios, the ordering. Additive current offset produces the largest error increase across the tested sensor faults, whereas the initialization test gives the fastest strict recovery to the equivalent-circuit observer at 1.20 h and then to the recurrent estimator at 2.60 h. These results demonstrate that accuracy, resistance to disturbed inputs, and recovery are distinct estimator properties. The accompanying STM32 measurements further show that all four isolated SOC cores meet the one-second update interval, although their flash, peak runtime RAM, and latency

differ substantially.

The third question asks what remains of neural estimation once embedded hardware is treated as part of the problem. Chapter 7 shows that stateful recurrent SOC and SOH estimators can be executed on the STM32 target with comfortable timing margins, but that compression changes the evaluation. Structured pruning is the most balanced lever for the SOC estimator because it reduces flash, RAM, inference time, and the corresponding timing-derived energy indicator, with an equal-weight utility score of $U = 0.72$. INT8 weight quantization reduces non-volatile memory more strongly, but the current mixed-precision path reconstructs effective floating-point weights during inference and raises SOC kernel time, which contributes to $U = 1.83$. The embedded result is thus not merely that a neural estimator can be deployed, but that model architecture, compression method, firmware implementation, and measured hardware behaviour must be selected together.

Across these questions, the research moves from representation design to disturbed-operation validation and finally to measured embedded execution. The custom BMS platform and the shared measurement chain provide the system context that makes this path practically meaningful. Without this layer, the work would remain an offline comparison of algorithms. With it, estimator design can be connected to sensing, firmware, memory, timing, and energy constraints.

8.2 Findings Across BMS Requirements

Table 8.1: Synthesis of the contribution chapters with respect to the three requirement dimensions used throughout this work.

Research component	Performance	Hardware / embedded	Deployment logic
Neural ageing estimation	SOH accuracy through representation quality	indirect relevance	lag-sequence and feature design for compact models
Performance and robustness benchmark	nominal accuracy, disturbance sensitivity, and recovery	numerical equivalence, flash, peak runtime RAM, inference latency, and recurrent execution modes	matched multi-scenario and on-device evaluation across estimator classes
Embedded AI implementation	retained accuracy after compression	flash, RAM, and latency measured on MCU, with a timing-based energy proxy	pruning, quantization, export, and firmware validation
BMS platform and datasets	measurement basis for evaluation	sensing chain and controller architecture	reproducible integration and estimator exchange

The central result of Table 8.1 is that estimator suitability emerges only from the joint assessment of the three requirement dimensions introduced in Figure 2.1. No single dimension can substitute for the others. The performance dimension is not limited to mean error: it also includes robustness against disturbed measurements, recovery after state inconsistency, and the ability to retain useful behaviour when the input stream is incomplete or corrupted. This is why the robustness chapter is

not just another accuracy experiment, but the point where nominal model ranking is deliberately challenged.

The hardware dimension changes the same comparison again. A model that is attractive in software must still fit the flash and RAM budget, meet the update-rate requirement, and avoid excessive energy demand. The embedded chapter therefore evaluates retained accuracy after pruning or quantization rather than unconstrained floating-point accuracy alone. This is a different criterion from the one used during model training.

The deployment dimension links these two views. It includes the feature pipeline, recurrent-state handling, model export, firmware implementation, and the ability to exchange estimators on a reproducible measurement chain. This work therefore treats deployment as a design requirement in its own right, not as a final formatting step after a model has already been selected.

8.3 Transferable Design Lessons

Four design lessons can be transferred from the individual chapters to BMS estimator design more generally.

The first lesson is *feature representation before model complexity.* Chapter 5 shows that neural networks do not have to become large merely because the battery state is history-dependent. With cumulative-current features and lag-sequence inputs, a compact MLP can exploit temporal and throughput-related information even though the estimator itself has no recurrent state. This principle also applies to recurrent models: an LSTM or GRU benefits from meaningful causal input channels just as much as a feedforward model does. Chapter 7 adds the complementary result that more expressive time-dependent models can still be made embedded-feasible when their architecture is structured well enough for pruning and quantization. The design message is therefore not MLP versus LSTM. It is to design the representation first, choose only as much model complexity as the task requires, and then optimize the selected architecture for deployment.

The second lesson is that *single-metric accuracy is not enough for deployment decisions.* A clean-condition MAE is a necessary reference, but it does not reveal why an estimator fails. Chapter 6 shows that there is no universally best estimator class under all tested conditions. Current-gain error, additive sensor offsets, noise, missing samples, burst dropout, irregular sampling, voltage spikes, ADC quantiza-

tion, and initialization mismatch each probe a different weakness of the estimation chain. The practical value of the robustness benchmark is therefore not a fixed leaderboard. It is a way to identify the weak points of a chosen estimator so that they can be addressed through feature design, retraining, observer tuning, filtering, reset logic, or fault handling before deployment.

The third lesson is that *compression and execution semantics must be matched to the active embedded constraint.* Pruning and quantization are useful for different reasons. Structured pruning removes recurrent channels and downstream matrix rows, so it can reduce flash, RAM, inference time, and energy at the same time. Quantization reduces the weight storage more strongly, but the present STM32 implementation shows that it can increase inference time when INT8 weights have to be reconstructed through software-side scaling during every update. The robustness benchmark adds that resource demand also depends on how a recurrent model is executed. Continuous state forwarding reduces repeated computation and temporary memory demand relative to exact rolling-window recomputation, whereas periodic reset can damage estimation behavior by discarding temporal context. If flash memory is the limiting resource, quantization can be the decisive step. If latency, runtime memory, or energy is limiting, pruning and continuous state forwarding can be the more balanced options. The most promising direction is therefore to coordinate architecture, compression, and recurrent execution rather than selecting each one independently.

The fourth lesson is that *deployment constraints have to shape model design from the beginning.* The estimator architecture, feature pipeline, firmware implementation, and target hardware are coupled. A very complex model may look attractive in an offline benchmark, but it is harder to export, optimize, validate, and maintain on a microcontroller than a compact or structurally regular model. Model design should therefore include the expected BMS target from the start: available sensor signals, sampling rate, memory budget, arithmetic support, update frequency, and firmware integration path. Validation should also not end with an offline test. The estimator should be replayed on the target controller or in a hardware-facing test stand where timing, memory, communication, and numerical behaviour can be measured together. The Custom BMS Platform is important for this reason: it provides the environment in which state-estimation software can be tested as deployable BMS functionality rather than only as a model file.

8.4 Position Within the State of Research and Practical Implications

Compared with much of the battery-estimation literature, the contribution of this research is not only a new model or a single accuracy result. Many existing studies focus on one dimension at a time: a neural architecture under curated signals, an observer under a specific fault assumption, or an embedded implementation after the model has already been chosen. The present work connects these layers. It links representation design, disturbance-based benchmarking, compression, MCU measurement, and BMS hardware integration into one deployment-oriented evaluation chain.

This positioning matters because BMS hardware is becoming more capable, with stronger microcontrollers, embedded-AI support, and tighter interaction between sensing ICs and application processors. These developments make neural and hybrid estimators more realistic for BMS deployment, but they do not remove the need for rigorous validation. More capable hardware expands the design space, but it does not make a nominal MAE sufficient.

The practical implication is a clear design order. Estimator selection should begin with the target use case and the expected sensing conditions, not with an architecture preference. The feature representation should then be designed for the target state, the estimator should be tested under clean and disturbed scenarios, the compression strategy should be matched to the resource bottleneck, and the final model should be verified on the intended controller or an equivalent embedded path. The present research therefore contributes less a single winning estimator than a structured way to judge estimator classes under BMS-relevant conditions.

8.5 Limitations and Generalization Boundaries

The results must be interpreted within the boundary conditions of the studies. The ageing-estimation chapter uses an NMC cyclic-ageing dataset, whereas the robustness and embedded chapters use an LFP design-of-experiments campaign. Their numerical results are therefore complementary rather than directly interchangeable. For another cell chemistry, cell format, temperature range, or operating profile, the same evaluation procedure can be applied, but the baseline error, disturbance response, estimator ranking, and accuracy retained after compression may change.

The corresponding models must therefore be trained and evaluated with representative measurements and reference values from the intended application. The robustness benchmark uses six independent holdout cells and 16 predeclared 24-hour evaluation windows. This design supports cell-level comparisons across different load and ageing conditions, but it does not represent every member of the estimator classes, every possible fault sequence, or pack-level operation. Its class-level conclusions therefore concern the observed structural strengths and weaknesses of the selected representatives rather than a universal ranking.

The embedded results are also specific to the evaluated microcontroller platform. They refer to STM32H7 hardware, hand-written recurrent kernels, and the selected measurement setup for latency and energy. Transferring an estimator to another microcontroller changes the available memory, arithmetic support, execution speed, and energy characteristics rather than the underlying BMS estimation task. Accuracy, flash, RAM, latency, and energy must therefore be evaluated again after porting. A memory-limited controller may require a smaller hidden state, structured pruning, or compact weight storage. Efficient integer instructions can make quantization more attractive, whereas a controller with sufficient memory and floating-point throughput may benefit less from compression. The microcontroller alone does not reduce the estimation error. Lower error primarily requires representative training data and reference values, suitable features, appropriate model capacity, and retraining after compression. More capable hardware can support these measures indirectly by permitting a larger model or higher-precision arithmetic. Other controllers, vendor toolchains, memory hierarchies, or fixed-point kernels could therefore change the quantitative balance between pruning and quantization.

The Custom BMS Platform provides controlled and repeatable conditions, but this experimental strength also defines the boundary of its evidence. Field-deployed systems introduce broader cell-to-cell variation, changing load profiles, seasonal and spatial temperature gradients, sensor and connector drift, electromagnetic interference, communication interruptions, and longer intervals without maintenance. These effects can shift the feature distributions seen during training, change the validity of SOC and SOH references, amplify measurement-related estimator errors, and alter processor load through additional communication and diagnostic tasks. Laboratory agreement therefore demonstrates functional integration and supports comparative evaluation, but it does not by itself establish long-term field reliability. Transfer to autonomous operation requires longer campaigns with pack-scale

electrical and thermal coupling, environmental variation, representative communication loads, and maintenance conditions that cannot be reproduced fully in the present setup.

The transferable result is therefore the evaluation logic, not every numerical value. Estimator classes should be compared under matched nominal, disturbed, recovery, and embedded conditions, and the final judgement should be based on how the method behaves across this complete chain.

8.6 Overall Conclusion

This research examined AI-based battery-state estimation from the perspective of a practical BMS rather than from the perspective of an isolated offline model. The work began with the observation that SOC and SOH estimators are only useful in operation if they satisfy more than one requirement at the same time. They must represent the relevant battery history, remain reliable under imperfect measurements, fit within embedded hardware limits, and be transferable into a reproducible firmware and validation chain. The central conclusion is therefore that deployment-oriented battery intelligence is not defined by the lowest clean-data error, but by the behaviour of the full estimator path from data representation to hardware execution.

The contribution chapters address this path step by step. The ageing-estimation study shows that simple models can become time-aware when the input representation contains physically meaningful history, as demonstrated by the MLP-based SOH estimator with cumulative-current and lag-sequence features. The performance and robustness benchmark then shows why such accuracy claims have to be stress-tested under realistic measurement disturbances. Its results also show why estimator classes should be compared through their mechanisms rather than through a nominal ranking alone. Direct integration is economical but cannot correct accumulated error by itself. Equivalent-circuit feedback supports recovery when its voltage model remains informative. Recurrent data-driven estimation can use a broader temporal and multichannel context, which gives it substantial potential for low nominal error and resistance to several disturbance types while leaving identifiable sensitivities to local transients and state handling. The on-device branch confirms that representatives of all four tested approaches can run within the selected STM32 memory and timing limits. It also demonstrates that a

pruned recurrent estimator changes from a costly rolling-window implementation to a fast and memory-efficient implementation when hidden state is forwarded continuously, while periodic state resets produce pronounced transients. The embedded-AI study extends this argument by demonstrating that model quality after pruning, quantization, export, and on-device measurement is the relevant quantity for deployment. The experimental and hardware basis in Chapter 4 makes this chain concrete through accessible sensing, firmware partitioning, and estimator integration.

Taken together, the work contributes a requirement-based way of evaluating battery-state estimators. The purpose is not to declare one estimator family universally superior. Direct-measurement, model-based, hybrid, and data-driven approaches each have useful operating regions and failure modes. The contribution is instead to show how these methods can be compared in a BMS-relevant way: first by choosing a suitable representation, then by extending the evaluation beyond clean-condition accuracy to disturbance and recovery behaviour, then by selecting pruning, quantization, or their combination according to the actual hardware bottleneck, and finally by validating the result on the controller. This evaluation chain is the main outcome of the research.

8.7 Outlook

Future work should first broaden the empirical basis. The evaluation chain should be applied to additional cells, chemistries, ageing states, temperature ranges, and operating profiles to test the relationships between accuracy, robustness, recovery, and compressibility beyond the present trajectories. This would separate general estimator behaviour from dataset-specific effects and quantify the training coverage required before data-driven estimators can be trusted outside the original ageing and load conditions.

A second direction is to move the disturbance protocols closer to the embedded platform. Bias, noise, missing samples, timing irregularities, and spikes should be injected into the live firmware and measurement chain of the Custom BMS Platform rather than only into host-side replay data. This would connect the robustness and embedded-deployment analyses and reveal interactions among disturbed signals, compressed models, firmware scheduling, communication overhead, and controller timing.

A third direction is architecture and compression co-design. Future work should compare MLPs, GRUs, LSTMs, compact transformer-style models, and neural-augmented equivalent-circuit models under the same accuracy, robustness, memory, latency, and energy criteria because the benefit of compression depends on the architecture and firmware kernel. Combining pruning and quantization is particularly relevant: pruning reduces state size and operation count, while quantization reduces the remaining persistent weight memory. This comparison could turn the qualitative design lesson developed here into quantitative selection rules for embedded BMS estimator architectures.

Finally, the Custom BMS Platform should be extended toward larger pack configurations and longer autonomous operation. This includes higher cell counts, pack-level thermal and electrical effects, longer cycling experiments, and eventually online or continual adaptation on the controller. For long-lived decentralized and mobile systems, such adaptation would keep the estimator maintainable as the battery ages and the operating environment changes. The long-term objective is therefore a BMS in which AI-based state estimation is an integrated, testable, and resource-aware part of the battery system rather than an offline add-on. Across the research questions, the central conclusion is that dependable battery-state estimation requires data representation, disturbance behaviour, embedded implementation, and BMS hardware to be designed and validated as one system.

Bibliography

[1] Florian Rzepka, Philipp Hematty, Mano Schmitz, and Julia Kowal. "Neural Network Architecture for Determining the Aging of Stationary Storage Systems in Smart Grids". In: *Energies* 16.17 (Aug. 22, 2023), p. 6103. DOI: 10.3390/en16176103. URL: https://www.mdpi.com/1996-1073/16/17/6103 (visited on 12/04/2025) (cited on pp. 2, 3, 49, 65, 89, 150).

[2] Florian Rzepka, Qinnan Chen, and Julia Kowal. "Performance and Robustness Benchmarking Across Battery State-of-Charge Estimator Classes for Embedded Applications". Manuscript submitted to Engineering Applications of Artificial Intelligence. 2026 (cited on p. 3).

[3] Florian Rzepka and Julia Kowal. *Embedded Artificial Intelligence in Battery Management Systems: Pruning and Quantization for Efficient State-of-Charge and State-of-Health Estimation*. SSRN. Feb. 24, 2026. DOI: 10.2139/ssrn.6298526. URL: https://ssrn.com/abstract=6298526 (visited on 07/23/2026) (cited on pp. 3, 123).

[4] Gregory L. Plett. *Battery Management Systems: Volume 1: Battery Modeling*. Artech House Power Engineering and Power Electronics. Boston London: Artech House, 2015. 1 p. (cited on pp. 5, 9–12, 58, 145, 150).

[5] Long Zhou, Xin Lai, Bin Li, Yi Yao, Ming Yuan, Jiahui Weng, and Yuejiu Zheng. "State Estimation Models of Lithium-Ion Batteries for Battery Management System: Status, Challenges, and Future Trends". In: *Batteries* 9.2 (Feb. 13, 2023), p. 131. DOI: 10.3390/batteries9020131. URL: https://www.mdpi.com/2313-0105/9/2/131 (visited on 06/24/2026) (cited on pp. 5, 11).

[6] Ming Jiang, Dongjiang Li, Zonghua Li, Zhuo Chen, Qinshan Yan, Fu Lin, Cheng Yu, Bo Jiang, Xuezhe Wei, Wensheng Yan, and Yong Yang. "Advances in Battery State Estimation of Battery Management System in Elec-

tric Vehicles". In: *Journal of Power Sources* 612 (Aug. 2024), p. 234781. DOI: 10.1016/j.jpowsour.2024.234781. URL: https://linkinghub.elsevier.com/retrieve/pii/S037877532400733X (visited on 06/24/2026) (cited on pp. 5, 11).

[7] Xue Li, Jiuchun Jiang, Caiping Zhang, Le Yi Wang, and Linfeng Zheng. "Robustness of SOC Estimation Algorithms for EV Lithium-Ion Batteries against Modeling Errors and Measurement Noise". In: *Mathematical Problems in Engineering* 2015 (2015), pp. 1–14. DOI: 10.1155/2015/719490. URL: http://www.hindawi.com/journals/mpe/2015/719490/ (visited on 03/23/2026) (cited on pp. 6, 83, 85, 92).

[8] F. Naseri, C. Barbu, A.C.S. Jensen, P. Setoodeh, J.A. Romero Baena, N. Hevele, and E. Schaltz. "Toward Reliable Wireless BMS: Robust Battery State Estimation Under Noise and Uncertainty". In: *IEEE Transactions on Vehicular Technology* (2025), pp. 1–12. DOI: 10.1109/TVT.2025.3638613. URL: https://ieeexplore.ieee.org/document/11271354/ (visited on 03/23/2026) (cited on pp. 6, 83, 92, 93).

[9] Franziska Berger, Dominik Joest, Elias Barbers, Katharina Quade, Ziheng Wu, Dirk Uwe Sauer, and Philipp Dechent. "Benchmarking Battery Management System Algorithms - Requirements, Scenarios and Validation for Automotive Applications". In: *eTransportation* 22 (Dec. 2024), p. 100355. DOI: 10.1016/j.etran.2024.100355. URL: https://linkinghub.elsevier.com/retrieve/pii/S2590116824000456 (visited on 03/23/2026) (cited on pp. 6, 83, 91–93).

[10] Ji Lin, Ligeng Zhu, Wei-Ming Chen, Wei-Chen Wang, and Song Han. "Tiny Machine Learning: Progress and Futures". In: *IEEE Circuits and Systems Magazine* 23.3 (Aut. 2023), pp. 8–34. DOI: 10.1109/MCAS.2023.3302182. arXiv: 2403.19076 [cs]. URL: http://arxiv.org/abs/2403.19076 (visited on 12/02/2025) (cited on pp. 7, 23, 43).

[11] Mina Naguib, Phillip Kollmeyer, Carlos Vidal, Josimar Duque, Oliver Gross, and Ali Emadi. "Microprocessor Execution Time and Memory Use for Battery State of Charge Estimation Algorithms". In: WCX SAE World Congress Experience. Detroit & Online, Michigan, United States, Mar. 29, 2022, pp. 2022-01–0697. DOI: 10.4271/2022-01-0697. URL: https://saemobilus.sae.org/papers/microprocessor-execution-time-memory-use-battery-state-charge-estim

ation-algorithms-2022-01-0697 (visited on 12/03/2025) (cited on pp. 7, 23, 43, 124, 135).

[12] Song Han, Jeff Pool, John Tran, and William J. Dally. "Learning Both Weights and Connections for Efficient Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 28. 2015, pp. 1135–1143. URL: https://proceedings.neurips.cc/paper/2015/hash/ae0eb3eed39d2bcef4622b2499a05fe6-Abstract.html (visited on 08/05/2026) (cited on pp. 7, 24, 28, 30, 46).

[13] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018, pp. 2704–2713. DOI: 10.1109/CVPR.2018.00286. URL: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html (visited on 08/05/2026) (cited on pp. 7, 36, 39, 41, 46, 133).

[14] Peter Kurzweil and Otto K. Dietlmeier. *Elektrochemische Speicher: Superkondensatoren, Batterien, Elektrolyse-Wasserstoff, Rechtliche Rahmenbedingungen*. Wiesbaden: Springer Fachmedien Wiesbaden, 2018. DOI: 10.1007/978-3-658-21829-4. URL: http://link.springer.com/10.1007/978-3-658-21829-4 (visited on 04/14/2026) (cited on pp. 9, 10, 51).

[15] Prashant Shrivastava, P. Amritansh Naidu, Sakshi Sharma, Bijaya Ketan Panigrahi, and Akhil Garg. "Review on Technological Advancement of Lithium-Ion Battery States Estimation Methods for Electric Vehicle Applications". In: *Journal of Energy Storage* 64 (Aug. 2023), p. 107159. DOI: 10.1016/j.est.2023.107159. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X2300556X (visited on 01/14/2025) (cited on pp. 10, 83, 123).

[16] M. Kassem, J. Bernard, R. Revel, S. Pélissier, F. Duclaud, and C. Delacourt. "Calendar Aging of a Graphite/LiFePO4 Cell". In: *Journal of Power Sources* 208 (June 2012), pp. 296–305. DOI: 10.1016/j.jpowsour.2012.02.068. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775312004284 (visited on 04/14/2026) (cited on p. 11).

[17] Anup Barai, Kotub Uddin, Matthieu Dubarry, Limhi Somerville, Andrew McGordon, Paul Jennings, and Ira Bloom. "A Comparison of Methodolo-

gies for the Non-Invasive Characterisation of Commercial Li-ion Cells". In: *Progress in Energy and Combustion Science* 72 (May 2019), pp. 1–31. DOI: 10.1016/j.pecs.2019.01.001. URL: https://linkinghub.elsevier.com/retrieve/pii/S0360128518300996 (visited on 04/14/2026) (cited on pp. 11, 51, 55).

[18] M. Berecibar, I. Gandiaga, I. Villarreal, N. Omar, J. Van Mierlo, and P. Van Den Bossche. "Critical Review of State of Health Estimation Methods of Li-ion Batteries for Real Applications". In: *Renewable and Sustainable Energy Reviews* 56 (Apr. 2016), pp. 572–587. DOI: 10.1016/j.rser.2015.11.042. URL: https://linkinghub.elsevier.com/retrieve/pii/S1364032115013076 (visited on 12/02/2025) (cited on pp. 11, 58, 123, 145, 150).

[19] *BQ79616 16-Series Battery Monitor, Balancer, and Integrated Hardware Protector*. 2023. URL: https://www.ti.com/lit/ds/symlink/bq79616.pdf?ts=1775312350051&ref_url=https%253A%252F%252Fwww.ti.com%252Fproduct%252FBQ79616 (visited on 08/05/2026) (cited on pp. 11, 91, 92).

[20] *10s Battery Pack Monitoring, Balancing, and Comprehensive Protection, 50-A Discharge Reference Design*. URL: https://www.ti.com/lit/ug/tiduar8c/tiduar8c.pdf (visited on 08/05/2026) (cited on pp. 11, 91, 92).

[21] *Design Considerations of Current Sensing With BQ769x2 Family for Battery Management System*. URL: https://www.ti.com/lit/an/sluab10a/sluab10a.pdf?ts=1775276566199 (visited on 08/05/2026) (cited on pp. 11, 91, 92).

[22] *TLE4972 High Precision Coreless Current Sensor for Automotive Applications*. URL: https://www.infineon.com/assets/row/public/documents/24/49/infineon-tle4972-ae35d5-datasheet-en.pdf?fileId=8ac78c8c7c72fb9a017c7e1109e20a17 (visited on 08/05/2026) (cited on pp. 11, 91, 92).

[23] *TMCS1108 3% Functional Isolation Hall-Effect Current Sensor With ±100-V Working Voltage*. URL: https://www.ti.com/lit/ds/symlink/tmcs1108.pdf (visited on 08/05/2026) (cited on pp. 11, 91, 92).

[24] Zeeshan Ahmad Khan, Prashant Shrivastava, Syed Muhammad Amrr, Saad Mekhilef, Abdullah A. Algethami, Mehdi Seyedmahmoudian, and Alex Stojcevski. "A Comparative Study on Different Online State of Charge Estimation Algorithms for Lithium-Ion Batteries". In: *Sustainability* 14.12 (June 17, 2022), p. 7412. DOI: 10.3390/su14127412. URL: https://www.mdpi.com/2071-1050/14/12/7412 (visited on 01/14/2025) (cited on pp. 12, 85).

[25] Yizhao Gao, Gregory L. Plett, Guodong Fan, and Xi Zhang. "Enhanced State-of-Charge Estimation of LiFePO4 Batteries Using an Augmented Physics-Based Model". In: *Journal of Power Sources* 544 (Oct. 2022), p. 231889. DOI: 10.1016/j.jpowsour.2022.231889. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775322008758 (visited on 01/14/2025) (cited on pp. 12, 14, 92, 124, 151).

[26] Feng Guo, Luis D. Couto, Grietus Mulder, Khiem Trad, Guangdi Hu, Odile Capron, and Keivan Haghverdi. "A Systematic Review of Electrochemical Model-Based Lithium-Ion Battery State Estimation in Battery Management Systems". In: *Journal of Energy Storage* 101 (Nov. 2024), p. 113850. DOI: 10.1016/j.est.2024.113850. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X24034364 (visited on 01/14/2025) (cited on pp. 12, 14, 83, 85, 123).

[27] M.S. Hossain Lipu, M.A. Hannan, Aini Hussain, Afida Ayob, Mohamad H.M. Saad, Tahia F. Karim, and Dickson N.T. How. "Data-Driven State of Charge Estimation of Lithium-Ion Batteries: Algorithms, Implementation Factors, Limitations and Future Trends". In: *Journal of Cleaner Production* 277 (Dec. 2020), p. 124110. DOI: 10.1016/j.jclepro.2020.124110. URL: https://linkinghub.elsevier.com/retrieve/pii/S095965262034155X (visited on 04/14/2026) (cited on pp. 12, 14).

[28] Xiao Sun, Long Zuo, Mingkang Zhang, Yanzhi Su, Qiang Fu, and Jiahui Jiang. "Equivalent Circuit Models for Lithium-Ion Batteries: A Comprehensive Review". In: *Electronics* 15.9 (May 6, 2026), p. 1968. DOI: 10.3390/electronics15091968. URL: https://www.mdpi.com/2079-9292/15/9/1968 (visited on 06/24/2026) (cited on p. 12).

[29] Davide Pio Laudani, Davide Milillo, Michele Quercio, Francesco Riganti Fulginei, and Lorenzo Sabino. "A Comprehensive Review of Equivalent Circuit Models and Neural Network Models for Battery Management Systems". In: *Batteries* 12.1 (Jan. 22, 2026), p. 37. DOI: 10.3390/batteries12010037. URL: https://www.mdpi.com/2313-0105/12/1/37 (visited on 06/24/2026) (cited on p. 12).

[30] Jiabo Li, Min Ye, Kangping Gao, Xinxin Xu, Meng Wei, and Shengjie Jiao. "Joint Estimation of State of Charge and State of Health for Lithium-ion Battery Based on Dual Adaptive Extended Kalman Filter". In: *Interna-*

tional Journal of Energy Research 45.9 (July 2021), pp. 13307–13322. DOI: 10.1002/er.6658. URL: https://onlinelibrary.wiley.com/doi/10.1002/er.6658 (visited on 04/14/2026) (cited on pp. 14, 21, 66).

[31] Feng Zhao, Yun Guo, and Baoming Chen. "A Review of Lithium-Ion Battery State of Charge Estimation Methods Based on Machine Learning". In: *World Electric Vehicle Journal* 15.4 (Mar. 25, 2024), p. 131. DOI: 10.3390/wevj15040131. URL: https://www.mdpi.com/2032-6653/15/4/131 (visited on 06/24/2026) (cited on p. 14).

[32] Zachary C. Lipton, John Berkowitz, and Charles Elkan. *A Critical Review of Recurrent Neural Networks for Sequence Learning*. Version 4. 2015. DOI: 10.48550/ARXIV.1506.00019. URL: https://arxiv.org/abs/1506.00019 (visited on 12/02/2025). Pre-published (cited on pp. 17, 127, 151).

[33] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *Proceedings of the 30th International Conference on Machine Learning*. Vol. 28. Proceedings of Machine Learning Research. 2013, pp. 1310–1318. URL: https://proceedings.mlr.press/v28/pascanu13.html (visited on 08/05/2026) (cited on pp. 17, 129).

[34] Yalan Bi, Yilin Yin, and Song-Yul Choe. "Online State of Health and Aging Parameter Estimation Using a Physics-Based Life Model with a Particle Filter". In: *Journal of Power Sources* 476 (Nov. 2020), p. 228655. DOI: 10.1016/j.jpowsour.2020.228655. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775320309599 (visited on 04/14/2026) (cited on pp. 21, 66).

[35] Jinpeng Tian, Rui Xiong, and Quanqing Yu. "Fractional-Order Model-Based Incremental Capacity Analysis for Degradation State Recognition of Lithium-Ion Batteries". In: *IEEE Transactions on Industrial Electronics* 66.2 (Feb. 2019), pp. 1576–1584. DOI: 10.1109/TIE.2018.2798606. URL: https://ieeexplore.ieee.org/document/8270363/ (visited on 04/14/2026) (cited on pp. 21, 66).

[36] Bin Gou, Yan Xu, and Xue Feng. "State-of-Health Estimation and Remaining-Useful-Life Prediction for Lithium-Ion Battery Using a Hybrid Data-Driven Method". In: *IEEE Transactions on Vehicular Technology* 69.10 (Oct. 2020), pp. 10854–10867. DOI: 10.1109/TVT.2020.3014932. URL: https://ieeexplore.ieee.org/document/9162518/ (visited on 04/14/2026) (cited on pp. 21, 66).

[37] Tsuyoshi Oji, Yanglin Zhou, Song Ci, Feiyu Kang, Xi Chen, and Xiulan Liu. "Data-Driven Methods for Battery SOH Estimation: Survey and a Critical Analysis". In: *IEEE Access* 9 (2021), pp. 126903–126916. DOI: 10.1109/ACCESS.2021.3111927. URL: https://ieeexplore.ieee.org/document/9535528/ (visited on 04/14/2026) (cited on pp. 21, 66).

[38] Yizhao Gao, Kailong Liu, Chong Zhu, Xi Zhang, and Dong Zhang. "Co-Estimation of State-of-Charge and State-of- Health for Lithium-Ion Batteries Using an Enhanced Electrochemical Model". In: *IEEE Transactions on Industrial Electronics* 69.3 (Mar. 2022), pp. 2684–2696. DOI: 10.1109/TIE.2021.3066946. URL: https://ieeexplore.ieee.org/document/9384220/ (visited on 01/14/2025) (cited on pp. 21, 124).

[39] Jiangwei Shen, Wensai Ma, Jian Xiong, Xing Shu, Yuanjian Zhang, Zheng Chen, and Yonggang Liu. "Alternative Combined Co-Estimation of State of Charge and Capacity for Lithium-Ion Batteries in Wide Temperature Scope". In: *Energy* 244 (Apr. 2022), p. 123236. DOI: 10.1016/j.energy.2022.123236. URL: https://linkinghub.elsevier.com/retrieve/pii/S0360544222001396 (visited on 01/14/2025) (cited on pp. 21, 85).

[40] Chunyu Wang, Naxin Cui, Zhongrui Cui, Haitao Yuan, and Chenghui Zhang. "Fusion Estimation of Lithium-Ion Battery State of Charge and State of Health Considering the Effect of Temperature". In: *Journal of Energy Storage* 53 (Sept. 2022), p. 105075. DOI: 10.1016/j.est.2022.105075. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X22010775 (visited on 01/14/2025) (cited on pp. 21, 85, 92, 151).

[41] Le Xu, Zhongwei Deng, Yi Xie, Xianke Lin, and Xiaosong Hu. "A Novel Hybrid Physics-Based and Data-Driven Approach for Degradation Trajectory Prediction in Li-Ion Batteries". In: *IEEE Transactions on Transportation Electrification* 9.2 (June 2023), pp. 2628–2644. DOI: 10.1109/TTE.2022.3212024. URL: https://ieeexplore.ieee.org/document/9911693/ (visited on 04/14/2026) (cited on pp. 21, 66).

[42] Hui Pang, Longxing Wu, Jiahao Liu, Xiaofei Liu, and Kai Liu. "Physics-Informed Neural Network Approach for Heat Generation Rate Estimation of Lithium-Ion Battery under Various Driving Conditions". In: *Journal of Energy Chemistry* 78 (Mar. 2023), pp. 1–12. DOI: 10.1016/j.jechem.2022.11.036.

URL: https://linkinghub.elsevier.com/retrieve/pii/S2095495622006349 (visited on 04/14/2026) (cited on pp. 21, 66).

[43] Ruth Cordova-Cardenas, Daniel Amor, and Álvaro Gutiérrez. "Edge AI in Practice: A Survey and Deployment Framework for Neural Networks on Embedded Systems". In: *Electronics* 14.24 (Dec. 11, 2025), p. 4877. DOI: 10.3390/electronics14244877. URL: https://www.mdpi.com/2079-9292/14/24/4877 (visited on 06/24/2026) (cited on pp. 23, 43).

[44] Davis Blalock, Jose Javier Gonzalez Ortiz, Jonathan Frankle, and John Guttag. "What Is the State of Neural Network Pruning?" In: *Proceedings of Machine Learning and Systems*. Vol. 2. 2020. URL: https://arxiv.org/abs/2003.03033 (visited on 08/05/2026) (cited on pp. 23, 24, 29, 32).

[45] Hongrong Cheng, Miao Zhang, and Javen Qinfeng Shi. "A Survey on Deep Neural Network Pruning: Taxonomy, Comparison, Analysis, and Recommendations". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.12 (Dec. 2024), pp. 10558–10578. DOI: 10.1109/TPAMI.2024.3447085. URL: https://doi.org/10.1109/TPAMI.2024.3447085 (visited on 07/25/2026) (cited on pp. 23, 24, 26, 28, 29).

[46] Yang He and Lingao Xiao. "Structured Pruning for Deep Convolutional Neural Networks: A Survey". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 46.5 (May 2024), pp. 2900–2919. DOI: 10.1109/TPAMI.2023.3334614. URL: https://doi.org/10.1109/TPAMI.2023.3334614 (visited on 07/25/2026) (cited on pp. 23, 25).

[47] Wei Wen, Chunpeng Wu, Yandan Wang, Yiran Chen, and Hai Li. "Learning Structured Sparsity in Deep Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 29. 2016, pp. 2074–2082. URL: https://arxiv.org/abs/1608.03665 (visited on 08/05/2026) (cited on pp. 25, 29, 32, 130).

[48] Zhuang Liu, Jianguo Li, Zhiqiang Shen, Gao Huang, Shoumeng Yan, and Changshui Zhang. "Learning Efficient Convolutional Networks through Network Slimming". In: *Proceedings of the IEEE International Conference on Computer Vision*. Oct. 2017, pp. 2736–2744. DOI: 10.1109/ICCV.2017.298. URL: https://openaccess.thecvf.com/content_iccv_2017/html/Liu_Learning_Efficient_Convolutional_ICCV_2017_paper.html (visited on 07/27/2026) (cited on pp. 25, 29).

[49] TensorFlow Model Optimization. *Pruning Comprehensive Guide*. 2026. URL: https://www.tensorflow.org/model_optimization/guide/pruning/comprehensive_guide (visited on 07/27/2026) (cited on pp. 26, 32).

[50] Trevor Gale, Erich Elsen, and Sara Hooker. *The State of Sparsity in Deep Neural Networks*. Feb. 2019. DOI: 10.48550/arXiv.1902.09574. arXiv: 1902.09574 [cs.LG]. URL: https://arxiv.org/abs/1902.09574 (visited on 07/27/2026) (cited on p. 28).

[51] Yann LeCun, John S. Denker, and Sara A. Solla. "Optimal Brain Damage". In: *Advances in Neural Information Processing Systems 2*. Morgan-Kaufmann, 1989, pp. 598–605. URL: https://proceedings.neurips.cc/paper/1989/hash/6c9882bbac1c7093bd25041881277658-Abstract.html (visited on 07/27/2026) (cited on p. 28).

[52] Pavlo Molchanov, Arun Mallya, Stephen Tyree, Iuri Frosio, and Jan Kautz. "Importance Estimation for Neural Network Pruning". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. June 2019, pp. 11264–11272. DOI: 10.1109/CVPR.2019.01152. URL: https://openaccess.thecvf.com/content_CVPR_2019/html/Molchanov_Importance_Estimation_for_Neural_Network_Pruning_CVPR_2019_paper.html (visited on 07/27/2026) (cited on p. 28).

[53] Christos Louizos, Max Welling, and Diederik P. Kingma. "Learning Sparse Neural Networks through L_0 Regularization". In: *International Conference on Learning Representations*. 2018. URL: https://openreview.net/forum?id=H1Y8hhg0b (visited on 07/27/2026) (cited on pp. 29, 32).

[54] Jonathan Frankle and Michael Carbin. "The Lottery Ticket Hypothesis: Finding Sparse, Trainable Neural Networks". In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=rJl-b3RcF7 (visited on 07/27/2026) (cited on p. 30).

[55] Alex Renda, Jonathan Frankle, and Michael Carbin. "Comparing Rewinding and Fine-Tuning in Neural Network Pruning". In: *International Conference on Learning Representations*. 2020. URL: https://openreview.net/forum?id=S1gSj0NKvB (visited on 07/27/2026) (cited on p. 30).

[56] Namhoon Lee, Thalaiyasingam Ajanthan, and Philip H. S. Torr. "SNIP: Single-Shot Network Pruning Based on Connection Sensitivity". In: *International Conference on Learning Representations*. 2019. URL: https://op

enreview.net/forum?id=B1VZqjAcYX (visited on 07/27/2026) (cited on p. 30).

[57] Decebal Constantin Mocanu, Elena Mocanu, Peter Stone, Phuong H. Nguyen, Madeleine Gibescu, and Antonio Liotta. "Scalable Training of Artificial Neural Networks with Adaptive Sparse Connectivity Inspired by Network Science". In: *Nature Communications* 9 (June 2018), p. 2383. DOI: 10.1038/s41467-018-04316-3. URL: https://www.nature.com/articles/s41467-018-04316-3 (visited on 07/27/2026) (cited on p. 30).

[58] Utku Evci, Trevor Gale, Jacob Menick, Pablo Samuel Castro, and Erich Elsen. "Rigging the Lottery: Making All Tickets Winners". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 2943–2952. URL: https://proceedings.mlr.press/v119/evci20a.html (visited on 07/27/2026) (cited on p. 30).

[59] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zach DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. 2019, pp. 8024–8035. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (visited on 08/05/2026) (cited on pp. 32, 127).

[60] PyTorch Contributors. *Pruning Tutorial*. 2026. URL: https://docs.pytorch.org/tutorials/intermediate/pruning_tutorial.html (visited on 07/27/2026) (cited on p. 32).

[61] Microsoft. *Overview of NNI Model Pruning*. 2023. URL: https://nni.readthedocs.io/en/stable/compression/pruning.html (visited on 07/27/2026) (cited on p. 32).

[62] Markus Nagel, Marios Fournarakis, Rana Ali Amjad, Yelysei Bondarenko, Mart van Baalen, and Tijmen Blankevoort. *A White Paper on Neural Network Quantization*. June 15, 2021. DOI: 10.48550/arXiv.2106.08295. arXiv: 2106.08295 [cs.LG]. URL: https://arxiv.org/abs/2106.08295 (visited on 07/27/2026). Pre-published (cited on pp. 34, 36, 37, 39, 41).

[63] Amir Gholami, Sehoon Kim, Zhen Dong, Zhewei Yao, Michael W. Mahoney, and Kurt Keutzer. *A Survey of Quantization Methods for Efficient Neural Network Inference*. June 21, 2021. DOI: 10.48550/arXiv.2103.13630. arXiv: 2103.13630 [cs.CV]. URL: https://arxiv.org/abs/2103.13630 (visited on 07/27/2026). Pre-published (cited on pp. 34, 41).

[64] Ron Banner, Yury Nahshan, and Daniel Soudry. "Post Training 4-Bit Quantization of Convolutional Networks for Rapid-Deployment". In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/c0a62e133894cdce435bcb4a5df1db2d-Abstract.html (visited on 07/27/2026) (cited on pp. 37, 41).

[65] Markus Nagel, Rana Ali Amjad, Mart van Baalen, Christos Louizos, and Tijmen Blankevoort. "Up or Down? Adaptive Rounding for Post-Training Quantization". In: *Proceedings of the 37th International Conference on Machine Learning*. Vol. 119. Proceedings of Machine Learning Research. PMLR, 2020, pp. 7197–7206. URL: https://proceedings.mlr.press/v119/nagel20a.html (visited on 07/27/2026) (cited on pp. 37, 41).

[66] Raghuraman Krishnamoorthi. *Quantizing Deep Convolutional Networks for Efficient Inference: A Whitepaper*. Version 1. 2018. DOI: 10.48550/ARXIV.1806.08342. URL: https://arxiv.org/abs/1806.08342 (visited on 12/02/2025). Pre-published (cited on pp. 39, 133, 151).

[67] Mark Horowitz. "1.1 Computing's Energy Problem and What We Can Do about It". In: *2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers*. IEEE, 2014, pp. 10–14. DOI: 10.1109/ISSCC.2014.6757323. URL: https://doi.org/10.1109/ISSCC.2014.6757323 (visited on 07/27/2026) (cited on p. 39).

[68] Steven K. Esser, Jeffrey L. McKinstry, Deepika Bablani, Rathinakumar Appuswamy, and Dharmendra S. Modha. "Learned Step Size Quantization". In: *International Conference on Learning Representations*. 2020. URL: https://openreview.net/forum?id=rkgO66VKDS (visited on 07/27/2026) (cited on p. 41).

[69] PyTorch. *Quantization Overview*. PyTorch. Mar. 25, 2026. URL: https://docs.pytorch.org/ao/stable/contributing/quantization_overview.html (visited on 07/27/2026) (cited on p. 42).

[70] TensorFlow. *Post-Training Quantization*. Google. Aug. 3, 2022. URL: https://www.tensorflow.org/model_optimization/guide/quantization/post_training (visited on 07/27/2026) (cited on p. 42).

[71] Microsoft. *Quantize ONNX Models*. ONNX Runtime. URL: https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html (visited on 07/27/2026) (cited on p. 43).

[72] Spyridon Giazitzis, Maciej Sakwa, Sonia Leva, Emanuele Ogliari, Susheel Badha, and Filippo Rosetti. "A Case Study of a Tiny Machine Learning Application for Battery State-of-Charge Estimation". In: *Electronics* 13.10 (May 16, 2024), p. 1964. DOI: 10.3390/electronics13101964. URL: https://www.mdpi.com/2079-9292/13/10/1964 (visited on 05/19/2025) (cited on pp. 43, 124, 150).

[73] Giulia Crocioni, Danilo Pau, Jean-Michel Delorme, and Giambattista Gruosso. "Li-Ion Batteries Parameter Estimation With Tiny Neural Networks Embedded on Intelligent IoT Microcontrollers". In: *IEEE Access* 8 (2020), pp. 122135–122146. DOI: 10.1109/ACCESS.2020.3007046. URL: https://ieeexplore.ieee.org/document/9133084/ (visited on 05/19/2025) (cited on pp. 43, 124).

[74] Danilo Pietro Pau and Alberto Aniballi. "Tiny Machine Learning Battery State-of-Charge Estimation Hardware Accelerated". In: *Applied Sciences* 14.14 (July 18, 2024), p. 6240. DOI: 10.3390/app14146240. URL: https://www.mdpi.com/2076-3417/14/14/6240 (visited on 05/19/2025) (cited on pp. 43, 85, 124).

[75] Muh-Tian Shiue, Yang-Chieh Ou, Chih-Feng Wu, Yi-Fong Wang, and Bing-Jun Liu. "Design and Implementation of a Decentralized Node-Level Battery Management System Chip Based on Deep Neural Network Algorithms". In: *Electronics* 15.2 (Jan. 9, 2026), p. 296. DOI: 10.3390/electronics15020296. URL: https://www.mdpi.com/2079-9292/15/2/296 (visited on 06/24/2026) (cited on p. 43).

[76] Vivienne Sze, Yu-Hsin Chen, Tien-Ju Yang, and Joel S. Emer. "Efficient Processing of Deep Neural Networks: A Tutorial and Survey". In: *Proceedings of the IEEE* 105.12 (Dec. 2017), pp. 2295–2329. DOI: 10.1109/JPROC.2017.2761740. URL: https://doi.org/10.1109/JPROC.2017.2761740 (cited on p. 44).

[77] Norman P. Jouppi et al. "In-Datacenter Performance Analysis of a Tensor Processing Unit". In: *Proceedings of the 44th Annual International Symposium on Computer Architecture*. ISCA '17. Association for Computing Machinery, 2017, pp. 1–12. DOI: 10.1145/3079856.3080246. URL: https://doi.org/10.1145/3079856.3080246 (cited on p. 44).

[78] Mike Davies et al. "Loihi: A Neuromorphic Manycore Processor with On-Chip Learning". In: *IEEE Micro* 38.1 (Jan. 2018), pp. 82–99. DOI: 10.1109/MM.2018.112130359. URL: https://doi.org/10.1109/MM.2018.112130359 (cited on p. 44).

[79] Abu Sebastian, Manuel Le Gallo, Riduan Khaddam-Aljameh, and Evangelos Eleftheriou. "Memory Devices and Applications for In-Memory Computing". In: *Nature Nanotechnology* 15.7 (July 2020), pp. 529–544. DOI: 10.1038/s41565-020-0655-z. URL: https://doi.org/10.1038/s41565-020-0655-z (cited on p. 44).

[80] *STM32N6*. June 25, 2026. URL: https://www.st.com/en/microcontrollers-microprocessors/stm32n6-series.html (visited on 08/05/2026) (cited on p. 44).

[81] *MCX N Series Microcontrollers*. URL: https://www.nxp.com/products/processors-and-microcontrollers/arm-microcontrollers/general-purpose-mcus/mcx-arm-cortex-m/mcx-n-series-microcontrollers:MCX-N-SERIES (visited on 08/05/2026) (cited on p. 44).

[82] *Renesas Sets New MCU Performance Bar with 1-GHz RA8P1 Devices with AI Acceleration: RA8P1*. URL: https://www.renesas.com/en/about/newsroom/renesas-sets-new-mcu-performance-bar-1-ghz-ra8p1-devices-ai-acceleration (visited on 08/05/2026) (cited on p. 44).

[83] *Alif Semiconductor: Ensemble E1/E3/E5/E7*. URL: https://alifsemi.com/products/ensemble/ (visited on 08/05/2026) (cited on p. 44).

[84] *Introducing Our Highest-Performance Wireless SoCs and MCU: EFR32xG26 / PG26*. URL: https://www.silabs.com/wireless/efr32xg26 (visited on 08/05/2026) (cited on p. 44).

[85] Jonas Schulze, Nils Strassenburg, and Tilmann Rabl. "PQ Bench: Benchmarking Pruning and Quantization Techniques". In: *Proceedings of the Workshop on Data Management for End-to-End Machine Learning*. SIGMOD/PODS '25: International Conference on Management of Data. Berlin Germany: ACM, June 22, 2025, pp. 1–6. DOI: 10.1145/3735654.3735940. URL: https

://dl.acm.org/doi/10.1145/3735654.3735940 (visited on 12/05/2025) (cited on pp. 46, 133).

[86] Hamza A. Abushahla, Dara Varam, Ariel Justine N. Panopio, and Mohamed I. AlHajri. *Neural Network Quantization for Microcontrollers: A Comprehensive Survey of Methods, Platforms, and Applications*. Jan. 7, 2026. DOI: 10.48550/arXiv.2508.15008. arXiv: 2508.15008 [cs.LG]. URL: http://arxiv.org/abs/2508.15008 (visited on 06/24/2026). Pre-published (cited on p. 46).

[87] LEAP-RE. *MG FARM Smart Stand-Alone Micro-Grids as a Solution for Agriculture Farms Electrification*. URL: https://www.leap-re.eu/mg-farm/ (visited on 08/05/2026) (cited on pp. 49, 124).

[88] Microenergy. *MG-FARM Powering Sustainable Agriculture with Smart Micro-Grids*. URL: https://mg-farm.microenergy-consulting.com/ (visited on 08/05/2026) (cited on pp. 49, 124).

[89] Technische Universität Berlin. *MG Farm Smart Microgrids as a Solution for Agriculture Farms Electrification*. URL: https://www.tu.berlin/eet/forschung/projekte/mg-farm (visited on 08/05/2026) (cited on pp. 49, 124).

[90] LEAP-RE. *LEAP-RE Joint Call 2021 for AU–EU Collaborative Research and Innovation Projects on Renewable Energy*. Apr. 13, 2021. URL: https://leap-energy.eu/news-124-applications-received-during-1st-phase-of-the-leap-re-call-for-projects (visited on 07/23/2026) (cited on p. 49).

[91] European Commission. *Long-Term Joint EU–AU Research and Innovation Partnership on Renewable Energy*. CORDIS. DOI: 10.3030/963530. URL: https://cordis.europa.eu/project/id/963530 (visited on 07/23/2026) (cited on p. 49).

[92] EnArgus. *LEAP-RE: Verbundvorhaben MG-Farm, Teilvorhaben: Intelligente Microgrids als Lösung für die Elektrifizierung von landwirtschaftlichen Betrieben*. Project funding reference 03SF0684A. URL: https://enargus.de/ (visited on 07/23/2026) (cited on p. 49).

[93] Florian Rzepka. *Design of Experiments for Cyclic Aging of LFP 18650 Cells: Impact of Charge/Discharge C-Rates, Depth of Discharge, and Temperature on Battery Degradation*. Technische Universität Berlin, Nov. 13, 2025. DOI: 10.14279/DEPOSITONCE-24800. URL: https://depositonce.tu-berlin.de/handle/11303/25972 (visited on 12/02/2025) (cited on pp. 50, 52, 124).

[94] Daniel Kucevic, Benedikt Tepe, Stefan Englberger, Anupam Parlikar, Markus Mühlbauer, Oliver Bohlen, Andreas Jossen, and Holger Hesse. "Standard Battery Energy Storage System Profiles: Analysis of Various Applications for Stationary Energy Storage Systems Using a Holistic Simulation Framework". In: *Journal of Energy Storage* 28 (Apr. 2020), p. 101077. DOI: 10.1016/j.est.2019.101077. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X19309016 (visited on 12/15/2025) (cited on p. 50).

[95] Karl Siebertz, David Van Bebber, and Thomas Hochkirchen. *Statistische Versuchsplanung*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2017. DOI: 10.1007/978-3-662-55743-3. URL: http://link.springer.com/10.1007/978-3-662-55743-3 (visited on 12/03/2025) (cited on p. 53).

[96] Younes Sahri, Youcef Belkhier, Salah Tamalouzt, Nasim Ullah, Rabindra Nath Shaw, Md. Shahariar Chowdhury, and Kuaanan Techato. "Energy Management System for Hybrid PV/Wind/Battery/Fuel Cell in Microgrid-Based Hydrogen and Economical Hybrid Battery/Super Capacitor Energy Storage". In: *Energies* 14.18 (Sept. 11, 2021), p. 5722. DOI: 10.3390/en14185722. URL: https://www.mdpi.com/1996-1073/14/18/5722 (visited on 04/14/2026) (cited on p. 65).

[97] Mohamed S. Soliman, Youcef Belkhier, Nasim Ullah, Abdelyazid Achour, Yasser M. Alharbi, Ahmad Aziz Al Alahmadi, Habti Abeida, and Yahya Salameh Hassan Khraisat. "Supervisory Energy Management of a Hybrid Battery/PV/Tidal/Wind Sources Integrated in DC-microgrid Energy Storage System". In: *Energy Reports* 7 (Nov. 2021), pp. 7728–7740. DOI: 10.1016/j.egyr.2021.11.056. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352484721012014 (visited on 04/14/2026) (cited on p. 65).

[98] Daniel Ioan Stroe, Vaclav Knap, and Erik Schaltz. "State-of-Health Estimation of Lithium-Ion Batteries Based on Partial Charging Voltage Profiles". In: *ECS Transactions* 85.13 (June 19, 2018), pp. 379–386. DOI: 10.1149/08513.0379ecst. URL: https://iopscience.iop.org/article/10.1149/08513.0379ecst (visited on 04/14/2026) (cited on p. 66).

[99] Daniel-Ioan Stroe and Erik Schaltz. "SOH Estimation of LMO/NMC-based Electric Vehicle Lithium-Ion Batteries Using the Incremental Capacity Analysis Technique". In: *2018 IEEE Energy Conversion Congress and Exposition (ECCE)*. 2018 IEEE Energy Conversion Congress and Exposition (ECCE).

Portland, OR, USA: IEEE, Sept. 2018, pp. 2720–2725. DOI: 10.1109/ECCE.2018.8557998. URL: https://ieeexplore.ieee.org/document/8557998/ (visited on 04/14/2026) (cited on p. 66).

[100] Yu Shi, Shakeel Ahmad, Qing Tong, Tuti M. Lim, Zhongbao Wei, Dongxu Ji, Chika M. Eze, and Jiyun Zhao. "The Optimization of State of Charge and State of Health Estimation for Lithium-ions Battery Using Combined Deep Learning and Kalman Filter Methods". In: *International Journal of Energy Research* 45.7 (June 10, 2021), pp. 11206–11230. DOI: 10.1002/er.6601. URL: https://onlinelibrary.wiley.com/doi/10.1002/er.6601 (visited on 04/14/2026) (cited on p. 66).

[101] Seongwoo Kim, Pyeong-Yeon Lee, Miyoung Lee, Jonghoon Kim, and Woonki Na. "Improved State-of-health Prediction Based on Auto-Regressive Integrated Moving Average with Exogenous Variables Model in Overcoming Battery Degradation-Dependent Internal Parameter Variation". In: *Journal of Energy Storage* 46 (Feb. 2022), p. 103888. DOI: 10.1016/j.est.2021.103888. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X2101553X (visited on 04/14/2026) (cited on p. 66).

[102] Wei Xiong, Yimin Mo, and Cong Yan. "Online State-of-Health Estimation for Second-Use Lithium-Ion Batteries Based on Weighted Least Squares Support Vector Machine". In: *IEEE Access* 9 (2021), pp. 1870–1881. DOI: 10.1109/ACCESS.2020.3026552. URL: https://ieeexplore.ieee.org/document/9311733/ (visited on 04/14/2026) (cited on p. 66).

[103] Xin Sui, Shan He, Alejandro Gismero, Remus Teodorescu, and Daniel-Ioan Stroe. "Robust Fuzzy Entropy-Based SOH Estimation for Different Lithium-Ion Battery Chemistries". In: *2022 IEEE Energy Conversion Congress and Exposition (ECCE)*. 2022 IEEE Energy Conversion Congress and Exposition (ECCE). Detroit, MI, USA: IEEE, Oct. 9, 2022, pp. 1–8. DOI: 10.1109/ECCE50734.2022.9947 URL: https://ieeexplore.ieee.org/document/9947792/ (visited on 04/14/2026) (cited on p. 66).

[104] Sungwoo Jo, Sunkyu Jung, and Taemoon Roh. "Battery State-of-Health Estimation Using Machine Learning and Preprocessing with Relative State-of-Charge". In: *Energies* 14.21 (Nov. 2, 2021), p. 7206. DOI: 10.3390/en14217206. URL: https://www.mdpi.com/1996-1073/14/21/7206 (visited on 04/14/2026) (cited on p. 66).

[105] Jungsoo Kim, Jungwook Yu, Minho Kim, Kwangrae Kim, and Soohee Han. "Estimation of Li-ion Battery State of Health Based on Multilayer Perceptron: As an EV Application". In: *IFAC-PapersOnLine* 51.28 (2018), pp. 392–397. DOI: 10.1016/j.ifacol.2018.11.734. URL: https://linkinghub.elsevier.com/retrieve/pii/S2405896318334542 (visited on 04/14/2026) (cited on p. 66).

[106] Hicham Chaoui, Chinemerem C. Ibe-Ekeocha, and Hamid Gualous. "Aging Prediction and State of Charge Estimation of a LiFePO 4 Battery Using Input Time-Delayed Neural Networks". In: *Electric Power Systems Research* 146 (May 2017), pp. 189–197. DOI: 10.1016/j.epsr.2017.01.032. URL: https://linkinghub.elsevier.com/retrieve/pii/S037877961730041X (visited on 04/14/2026) (cited on p. 66).

[107] Fernando Puente León and HolgerBG Jäkel. *Signale Und Systeme:* DE GRUYTER, Sept. 25, 2015. DOI: 10.1515/9783110403862. URL: https://www.degruyterbrill.com/document/doi/10.1515/9783110403862/html (visited on 04/14/2026) (cited on p. 70).

[108] Tim Salimans and Diederik P. Kingma. "Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 29. 2016, pp. 901–909. URL: https://arxiv.org/abs/1602.07868 (visited on 08/05/2026) (cited on pp. 72, 73).

[109] Hans Weytjens, Enrico Lohmann, and Martin Kleinsteuber. "Cash Flow Prediction: MLP and LSTM Compared to ARIMA and Prophet". In: *Electronic Commerce Research* 21.2 (June 2021), pp. 371–391. DOI: 10.1007/s10660-019-09362-7. URL: https://link.springer.com/10.1007/s10660-019-09362-7 (visited on 04/14/2026) (cited on p. 73).

[110] Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization". Version 4. In: (2016). DOI: 10.48550/ARXIV.1603.06560. URL: https://arxiv.org/abs/1603.06560 (visited on 04/14/2026) (cited on pp. 73, 76).

[111] Jiaqi Yao and Julia Kowal. "Towards a Smarter Battery Management System: A Critical Review on Deep Learning-Based State of Charge Estimation of Lithium-Ion Batteries". In: *Energy and AI* 21 (Sept. 2025), p. 100585. DOI:

10.1016/j.egyai.2025.100585. URL: https://linkinghub.elsevier.com/retrieve/pii/S266654682500117X (visited on 03/23/2026) (cited on pp. 83, 123).

[112] Pratik Mondal, Divyakumar Bhavsar, Kanupriya Mittal, and Mayank Mittal. "Estimating State-of-Charge in Lithium-Ion Batteries Through Deep Learning Techniques: A Comparative Evaluation". In: *IEEE Access* 12 (2024), pp. 78773–78786. DOI: 10.1109/ACCESS.2024.3408220. URL: https://ieeexplore.ieee.org/document/10546296/ (visited on 03/23/2026) (cited on p. 85).

[113] Yahia Mazzi, Hicham Ben Sassi, Ahmed Gaga, and Fatima Errahimi. "State of Charge Estimation of an Electric Vehicle's Battery Using Tiny Neural Network Embedded on Small Microcontroller Units". In: *International Journal of Energy Research* 46.6 (May 2022), pp. 8102–8119. DOI: 10.1002/er.7713. URL: https://onlinelibrary.wiley.com/doi/10.1002/er.7713 (visited on 05/19/2025) (cited on p. 85).

[114] M. Adib Kamali, Angela C. Caliwag, and Wansu Lim. "Novel SOH Estimation of Lithium-Ion Batteries for Real-Time Embedded Applications". In: *IEEE Embedded Systems Letters* 13.4 (Dec. 2021), pp. 206–209. DOI: 10.1109/LES.2021.3078443. URL: https://ieeexplore.ieee.org/document/9426956/ (visited on 05/19/2025) (cited on pp. 89, 124).

[115] Bin Gou, Yan Xu, and Xue Feng. "An Ensemble Learning-Based Data-Driven Method for Online State-of-Health Estimation of Lithium-Ion Batteries". In: *IEEE Transactions on Transportation Electrification* 7.2 (June 2021), pp. 422–436. DOI: 10.1109/TTE.2020.3029295. URL: https://ieeexplore.ieee.org/document/9216038/ (visited on 01/14/2025) (cited on p. 89).

[116] Weihan Li, Neil Sengupta, Philipp Dechent, David Howey, Anuradha Annaswamy, and Dirk Uwe Sauer. "Online Capacity Estimation of Lithium-Ion Batteries with Deep Long Short-Term Memory Networks". In: *Journal of Power Sources* 482 (Jan. 2021), p. 228863. DOI: 10.1016/j.jpowsour.2020.228863. URL: https://linkinghub.elsevier.com/retrieve/pii/S0378775320311678 (visited on 01/14/2025) (cited on pp. 89, 150).

[117] Orazio Aiello. "Electromagnetic Susceptibility of Battery Management Systems' ICs for Electric Vehicles: Experimental Study". In: *Electronics* 9.3 (Mar. 19, 2020), p. 510. DOI: 10.3390/electronics9030510. URL: https://www.mdpi.com/2079-9292/9/3/510 (visited on 04/04/2026) (cited on pp. 91–93).

[118] Mengxi Liu, Daniel Geißler, Sizhen Bian, Bo Zhou, and Paul Lukowicz. *Assessing the Impact of Sampling Irregularity in Time Series Data: Human Activity Recognition As A Case Study*. Jan. 25, 2025. DOI: 10.48550/arXiv.2501.15330. arXiv: 2501.15330 [eess]. URL: http://arxiv.org/abs/2501.15330 (visited on 04/04/2026). Pre-published (cited on pp. 91, 92).

[119] A. M. S. M. H. S. Attanayaka, J. P. Karunadasa, K. T. M. U. Hemapala, and Department of Electrical Engineering, University of Moratuwa, Moratuwa, Sri Lanka. "Estimation of State of Charge for Lithium-Ion Batteries - A Review". In: *AIMS Energy* 7.2 (2019), pp. 186–210. DOI: 10.3934/energy.2019.2.186. URL: http://www.aimspress.com/article/10.3934/energy.2019.2.186 (visited on 12/02/2025) (cited on p. 123).

[120] Wenjie Sun, Bangyu Zhou, Huan Xu, Wenjiao Yao, Yuanjun Guo, and Zhile Yang. "State of Charge Estimation of Sodium-Ion Batteries Based on N-BEATS Neural Network". In: *2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT)*. 2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT). Jiaxing, China: IEEE, Nov. 10, 2023, pp. 1395–1401. DOI: 10.1109/ACAIT60137.2023.10528411. URL: https://ieeexplore.ieee.org/document/10528411/ (visited on 01/13/2025) (cited on p. 124).

[121] Wenjie Sun, Huan Xu, Bangyu Zhou, Yuanjun Guo, Yongbing Tang, Wenjiao Yao, and Zhile Yang. "State-of-Charge Estimation of Sodium-Ion Batteries: A Fusion Deep Learning Approach". In: *Journal of Energy Storage* 91 (June 2024), p. 111527. DOI: 10.1016/j.est.2024.111527. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X24011125 (visited on 01/13/2025) (cited on p. 124).

[122] Shunli Wang, Chao Wang, Paul Takyi-Aninakwa, Siyu Jin, Carlos Fernandez, and Qi Huang. "An improved parameter identification and radial basis correction-differential support vector machine strategies for state-of-charge estimation of urban-transportation-electric-vehicle lithium-ion batteries". en. In: *Journal of Energy Storage* 80 (Mar. 2024), p. 110222. DOI: 10.1016/j.est.2023.110222. URL: https://linkinghub.elsevier.com/retrieve/pii/S2352152X23036216 (visited on 08/05/2026) (cited on p. 124).

[123] Shunli Wang, Fan Wu, Paul Takyi-Aninakwa, Carlos Fernandez, Daniel-Ioan Stroe, and Qi Huang. "Improved singular filtering-Gaussian process regression-long short-term memory model for whole-life-cycle remaining ca-

pacity estimation of lithium-ion batteries adaptive to fast aging and multi-current variations". en. In: *Energy* 284 (Dec. 2023), p. 128677. DOI: 10.1016/j.energy.2023.128 URL: https://linkinghub.elsevier.com/retrieve/pii/S0360544223020716 (visited on 08/05/2026) (cited on p. 124).

[124] A. Aqachtoul, K. Karam, A. Elamrani, Y. Alidrissi, M. Najib, A. Rochd, and M. Bakhouya. "MGFARM: An IoT and Edge-Driven Framework for Smart and Sustainable Agricultural Practices". In: *Connected Objects, Artificial Intelligence, Telecommunications and Electronics Engineering*. Ed. by Youssef Mejdoub, Abdelkebir Elamri, and Mustapha Kardouchi. Vol. 1584. Cham: Springer Nature Switzerland, 2026, pp. 419–425. DOI: 10.1007/978-3-032-01536-5_64. URL: https://link.springer.com/10.1007/978-3-032-01536-5_64 (visited on 12/03/2025) (cited on p. 124).

[125] JGNE. *JGCFR18650-1800mAh-3.2V Product Specification*. JGNE. URL: https://www.goldencellpower.com/product-item/18650-1800mah-3-2v-lifepo4-cells/ (visited on 08/05/2026) (cited on p. 124).

[126] Peter J. Huber and Elvezio M. Ronchetti. *Robust Statistics*. 1st ed. Wiley Series in Probability and Statistics. Wiley, Jan. 29, 2009. DOI: 10.1002/9780470434697. URL: https://onlinelibrary.wiley.com/doi/book/10.1002/9780470434697 (visited on 12/02/2025) (cited on p. 125).

[127] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Andreas Müller, Joel Nothman, Gilles Louppe, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. "Scikit-Learn: Machine Learning in Python". Version 4. In: (2012). DOI: 10.48550/ARXIV.1201.0490. URL: https://arxiv.org/abs/1201.0490 (visited on 12/02/2025) (cited on p. 125).

[128] Ilya Loshchilov and Frank Hutter. "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations*. 2019. URL: https://openreview.net/forum?id=Bkg6RiCqY7 (visited on 08/05/2026) (cited on p. 129).

[129] Ilya Loshchilov and Frank Hutter. "SGDR: Stochastic Gradient Descent with Warm Restarts". In: *International Conference on Learning Representations*. 2017. URL: https://openreview.net/forum?id=Skq89Scxx (visited on 08/05/2026) (cited on p. 129).

[130] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. "Mixed Precision Training". In: *International Conference on Learning Representations*. 2018. URL: https://arxiv.org/abs/1710.03740 (visited on 08/05/2026) (cited on p. 129).

[131] Ekaterina Lobacheva, Nadezhda Chirkova, Alexander Markovich, and Dmitry Vetrov. "Structured Sparsification of Gated Recurrent Neural Networks". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 34.04 (Apr. 3, 2020), pp. 4989–4996. DOI: 10.1609/aaai.v34i04.5938. URL: https://ojs.aaai.org/index.php/AAAI/article/view/5938 (visited on 12/08/2025) (cited on p. 130).

[132] Sharan Narang, Erich Elsen, Gregory Diamos, and Shubho Sengupta. "Exploring Sparsity in Recurrent Neural Networks". In: *International Conference on Learning Representations*. 2017. URL: https://arxiv.org/abs/1704.05119 (visited on 08/05/2026) (cited on p. 130).

[133] *ONNX Runtime: High-Performance Inference Engine*. URL: https://onnxruntime.ai/ (visited on 08/05/2026) (cited on p. 134).

[134] *NVIDIA TensorRT: High-Performance Deep Learning Inference*. URL: https://developer.nvidia.com/tensorrt (visited on 08/05/2026) (cited on p. 134).

[135] *X-CUBE-AI: STM32Cube Expansion Package for Artificial Intelligence*. URL: https://www.st.com/en/embedded-software/x-cube-ai.html (visited on 08/05/2026) (cited on p. 134).

[136] *STM32 Nucleo-144 Boards*. STMicroelectronics, Dec. 2, 2025. URL: https://www.st.com/resource/en/data_brief/nucleo-h753zi.pdf (visited on 12/02/2025) (cited on p. 135).

[137] Wei Wen, Yuxiong He, Samyam Rajbhandari, Minjia Zhang, Wenhan Wang, Fang Liu, Bin Hu, Yiran Chen, and Hai Li. "Learning Intrinsic Sparse Structures within Long Short-Term Memory". In: *International Conference on Learning Representations*. 2018. URL: https://arxiv.org/abs/1709.05027 (visited on 08/05/2026) (cited on p. 151).

[138] S. Singh and P.R. Budarapu. "Deep Machine Learning Approaches for Battery Health Monitoring". In: *Energy* 300 (Aug. 2024), p. 131540. DOI: 10.1016/j.energy.2024.131
URL: https://linkinghub.elsevier.com/retrieve/pii/S0360544224013136 (visited on 01/13/2025) (cited on p. 151).

Appendix

A Supplementary Robustness and Embedded-Compression Analyses

This appendix contains supplementary tables, additional hardware documentation, experiment matrices, and publication-specific material that would interrupt the flow of the main text if it were placed directly in the core chapters.

A.1 Supplementary Robustness-Benchmark Material

The following figures and tables provide the complete scenario traceability, HECM lookup sensitivity, DD execution-mode detail, and numerical summaries supporting Chapter 6. SOC errors are reported on the normalized 0 to 1 scale unless percentage points are stated explicitly.

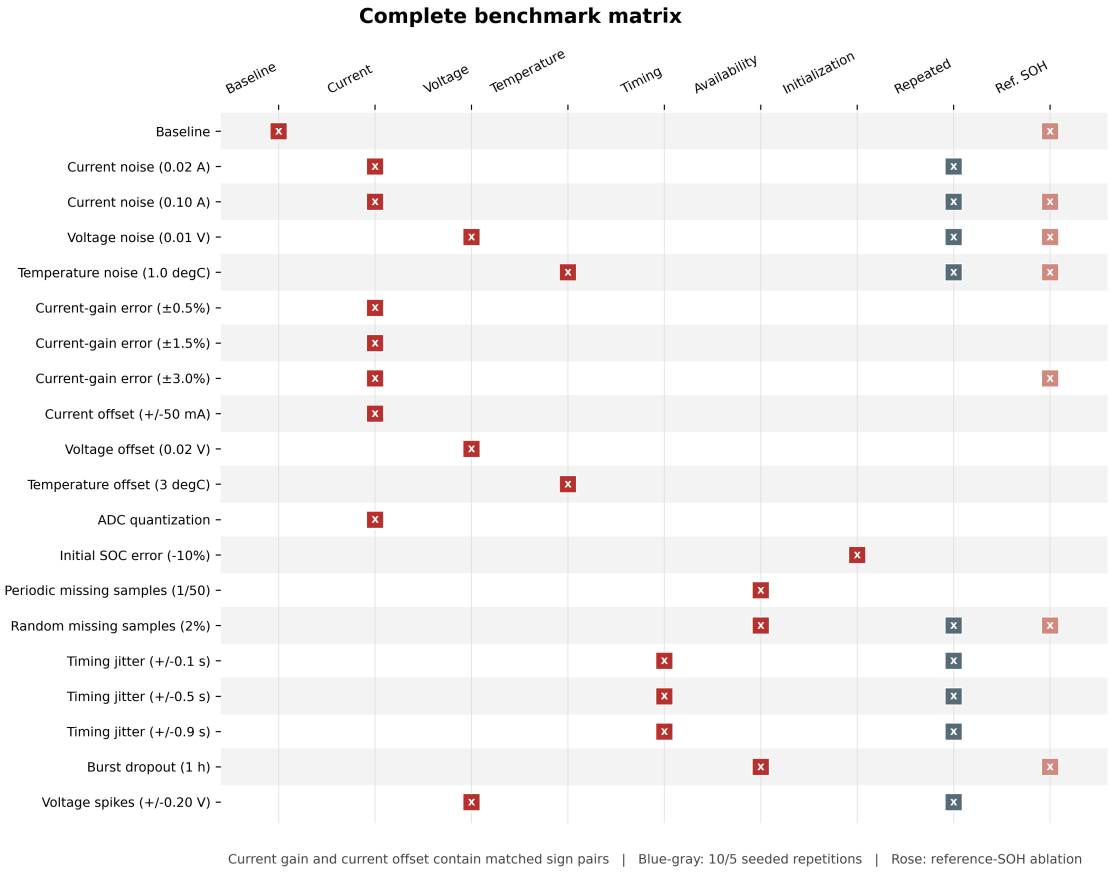


Figure A.1: Complete 20-scenario benchmark matrix. Red markers identify manipulated measurements, timing, availability, or initialization states. Blue-gray markers denote repeated stochastic cases. Rose markers identify paired ground-truth-SOH diagnostic runs. Current-gain magnitudes and the additive current-offset case contain matched positive and negative sign levels.

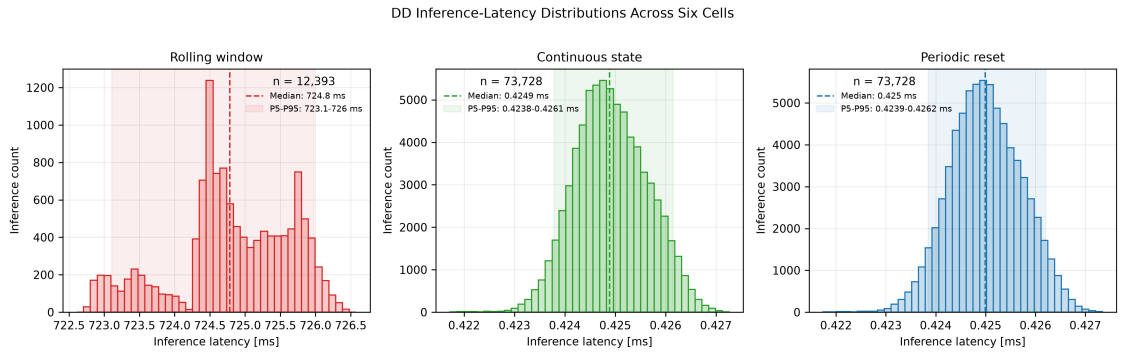


Figure A.2: DD latency distributions over the six hardware cell sequences. Exact rolling-window execution is shown separately from continuous-state and periodically reset execution because their scales and recurrent semantics differ substantially.

A.1.1 HECM lookup-table sensitivity

The cross-model benchmark evaluates one fixed HECM implementation whose lookup surfaces are identified exclusively from training and validation data. The supplementary HECM-only analysis tests whether its disturbance response depends strongly on local lookup calibration. The R_i, R_1, and R_2 surfaces are jointly scaled by -10% or $+10\%$. The τ_1 and τ_2 surfaces are likewise scaled by -10% or $+10\%$, while the OCV surfaces are shifted by $-10\,\mathrm{mV}$ or $+10\,\mathrm{mV}$. These six one-at-a-time perturbations and the nominal lookup are evaluated across 22 measurement and signal-integrity subcases and the paired initialization intervention. All original seeds and 16 frozen windows are retained. The complete design comprises 9184 HECM executions.

Main-campaign disturbances use their lookup-matched baseline, the one-hour dropout uses a lookup-matched 48-h baseline, and initialization recovery uses a separate paired baseline. For every disturbance, the lookup interaction is the scenario ΔMAE under the perturbed lookup minus the corresponding ΔMAE under the nominal lookup. Results are summarized within cells before the same equal-weight cell bootstrap used by the primary benchmark is applied.

Local lookup changes shift baseline HECM MAE between 0.0314 and 0.0373, compared with 0.0337 for the nominal lookup. Across the 22 disturbance subcases, the largest absolute interaction is 0.006116 MAE. It occurs for the $-0{,}05\,\mathrm{A}$ current offset under $+10\%$ time-constant scaling, and its cell-bootstrap interval includes zero. This interaction is small relative to the nominal adverse current-offset penalty

of 0.1763 MAE. The largest current-gain interaction is 0.000443 MAE.

Initialization recovery exposes a separate boundary. Five perturbations retain mean persistent recovery-or-censor times between 1.18 and 1.41 h, close to the nominal 1.20 h. Scaling the resistance surfaces by −10% increases the equal-weight mean to 5.10 h because one low-load holdout trajectory remains outside the recovery band for the 24-h horizon. The analysis supports local stability of most measurement-disturbance responses but does not establish lookup independence of baseline accuracy or observer re-anchoring.

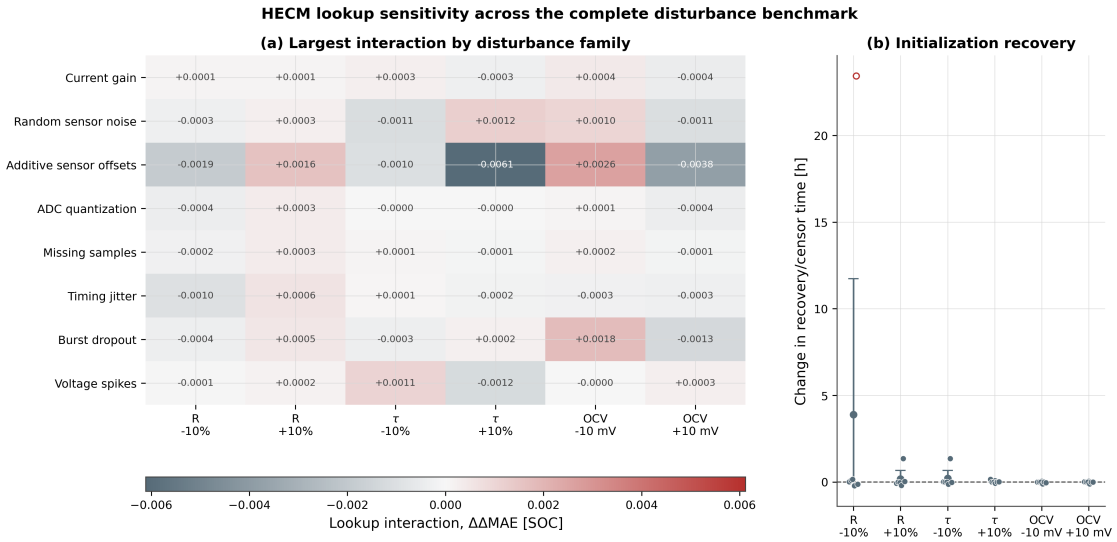


Figure A.3: HECM lookup sensitivity across the complete disturbance benchmark. Panel (a) reports the largest lookup interaction in each robustness family under local resistance, time-constant, and OCV perturbations. Panel (b) reports the corresponding change in persistent initialization recovery-or-censor time. Open red points identify cell means containing a censored recovery window.

Table A.1: HECM lookup sensitivity across the complete disturbance benchmark. The largest robustness interaction is selected from 22 measurement and signal-integrity subcases after equal-weight cell aggregation. Bracketed counts report subcases whose 95% interaction interval excludes zero. Recovery is the persistent recovery-or-censor time from the paired initialization analysis, with the equal-weight censored-cell percentage in brackets.

Lookup condition	Baseline MAE	Max. robust $\lvert\Delta\Delta\text{MAE}\rvert$ [CI]	Source subcase	Recovery/censor [h] [censored]
Nominal lookup	0.0337	Reference [–]	–	1.20 [0%]
Resistance −10%	0.0351	0.00191 [6/22]	Current offset +50 mA	5.10 [17%]
Resistance +10%	0.0327	0.00162 [5/22]	Current offset +50 mA	1.38 [0%]
Time constants −10%	0.0343	0.00113 [6/22]	Current noise 0.10 A	1.41 [0%]
Time constants +10%	0.0335	0.00612 [5/22]	Current offset −50 mA	1.22 [0%]
OCV −10 mV	0.0314	0.00260 [6/22]	Current offset +50 mA	1.18 [0%]
OCV +10 mV	0.0373	0.00378 [5/22]	Current offset +50 mA	1.19 [0%]

A.1.2 Supplementary numerical results

Table A.2: Six-cell baseline cell-macro statistics with hierarchical 95% confidence intervals.

Estimator	MAE [SOC]	RMSE [SOC]	P95 error [SOC]
DM	0.0690 [0.0499, 0.0823]	0.0740 [0.0532, 0.0881]	0.1088 [0.0768, 0.1307]
HDM	0.0412 [0.0301, 0.0515]	0.0442 [0.0319, 0.0557]	0.0649 [0.0449, 0.0846]
HECM	0.0337 [0.0246, 0.0434]	0.0388 [0.0286, 0.0493]	0.0644 [0.0468, 0.0821]
DD	0.0258 [0.0189, 0.0325]	0.0300 [0.0223, 0.0366]	0.0508 [0.0375, 0.0619]

Table A.3: Exact paired baseline-MAE comparisons across the six-cell holdout test set. Differences are defined as estimator B minus estimator A. Holm correction is applied across the six model-pair tests.

Estimator A	Estimator B	Mean difference	95% CI	d_z	$p_{\text{exact}}/p_{\text{Holm}}$
DM	HDM	-0.0278	[-0.0381, -0.0171]	-1.93	0.0313 / 0.1875
DM	HECM	-0.0352	[-0.0470, -0.0224]	-2.00	0.0313 / 0.1875
DM	DD	-0.0431	[-0.0558, -0.0286]	-2.29	0.0313 / 0.1875
HDM	HECM	-0.0075	[-0.0162, 0.0015]	-0.61	0.1563 / 0.1875
HDM	DD	-0.0154	[-0.0243, -0.0084]	-1.39	0.0313 / 0.1875
HECM	DD	-0.0079	[-0.0176, -0.0005]	-0.68	0.0938 / 0.1875

Table A.4: Selected cell-macro scenario penalties relative to paired baseline. Current-gain and current-offset values use adverse directions from matched signed sweeps.

Scenario	DM	HDM	HECM	DD
Current noise, $\sigma_I = 0{,}10\,\text{A}$	-0.0009	+0.0000	+0.0038	-0.0000
Current gain, adverse 3%	+0.0108	+0.0101	+0.0060	+0.0038
Current offset, adverse 50 mA	+0.2467	+0.2727	+0.1763	+0.0401
Voltage offset, +0,02 V	+0.0000	+0.0005	-0.0040	+0.0038
Temperature offset, +3 °C	+0.0000	+0.0002	+0.0002	+0.0074
ADC quantization	+0.0038	+0.0035	-0.0018	+0.0002
Random missing samples, 2%	+0.0080	+0.0081	+0.0039	+0.0026
Timing jitter, ±0,9 s	-0.0003	+0.0003	+0.0006	+0.0003
Burst dropout, 1 h	+0.0050	+0.0209	+0.0239	-0.0093
Voltage spikes, ±0,20 V	+0.0000	-0.0003	+0.0002	-0.0001

Table A.5: Sensitivity of the illustrative normalized robustness score to the aggregation rule.

Estimator	Family balanced	Equal scenario	Highest level plus offsets
DM	0.452	0.549	0.573
HDM	0.351	0.476	0.469
HECM	0.360	0.552	0.562
DD	0.967	0.793	0.693

Table A.6: Canonical paired initialization recovery and target-hardware summary. Recovery values are equal-weight cell means over the common evaluation interval. Endpoints already satisfied when all estimators first provide valid output are conservative upper bounds.

Metric	DM	HDM	HECM	DD
First 300-s entry or censor time [h]	22.28	17.21	1.12	1.29
Persistent recovery or censor time [h]	22.28	22.28	1.20	2.60
Recovery excess-error AUC [SOC h]	1.735	1.738	0.078	0.116
First-entry censored fraction	0.89	0.67	0.00	0.00
Persistent-recovery censored fraction	0.89	0.89	0.00	0.00
Relapse fraction after first hold	0.00	0.22	0.17	0.78
Median SOC-core latency [μs]	0.171	0.165	23.9	424.9[a]
Flash [KiB]	27.0	27.1	470.1	115.9[a]
Peak runtime RAM [KiB]	2.4	2.4	2.5	4.1[a]

[a]Continuous-state DD. Exact rolling-window execution requires approximately 724 ms and 67.3 KiB peak RAM.

A.2 Supplementary Analyses for Embedded Model Compression

This appendix provides the detailed evidence behind the compression, temporal, filtering, utility, and software-fault analyses discussed in Chapter 7.

A.2.1 Diagnostics of the Structured-Pruning Step

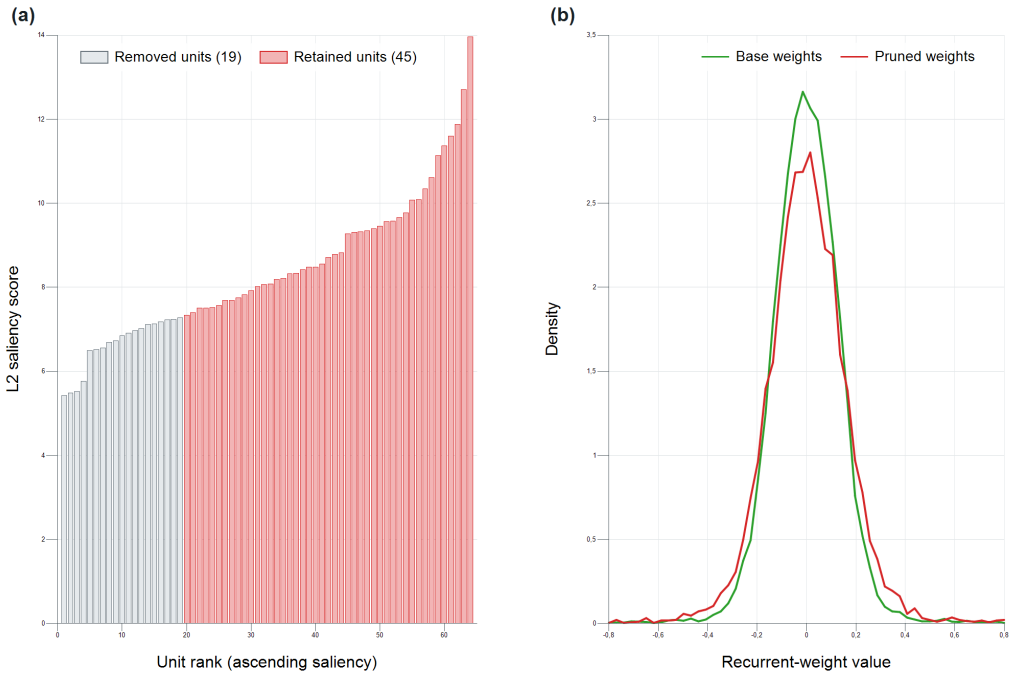


Figure A.4: Diagnostics of the one-shot SOC pruning step. Panel (a) orders hidden channels by the L2 gate-group saliency from Equation 7.11. The 19 lowest-ranked channels are removed, leaving 45 channels. Panel (b) compares the central range of the recurrent-weight distributions before and after this structural selection.

A.2.2 Long-Horizon Behaviour of the Compressed Models

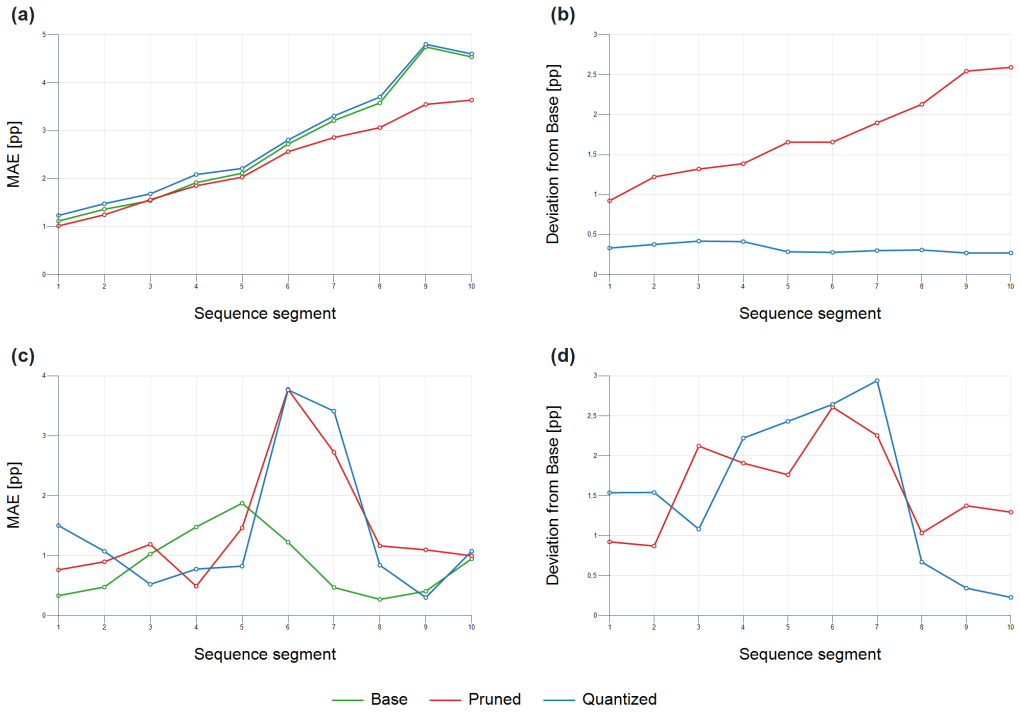


Figure A.5: Segment-wise errors over ten consecutive parts of the natural replay sequence. Recurrent states continue across all boundaries. Target MAE and P95 describe accuracy against the reference labels, while the model-to-Base deviation isolates the relative movement of each compressed output.

A.2.3 Interaction Between SOH Compression and Filtering

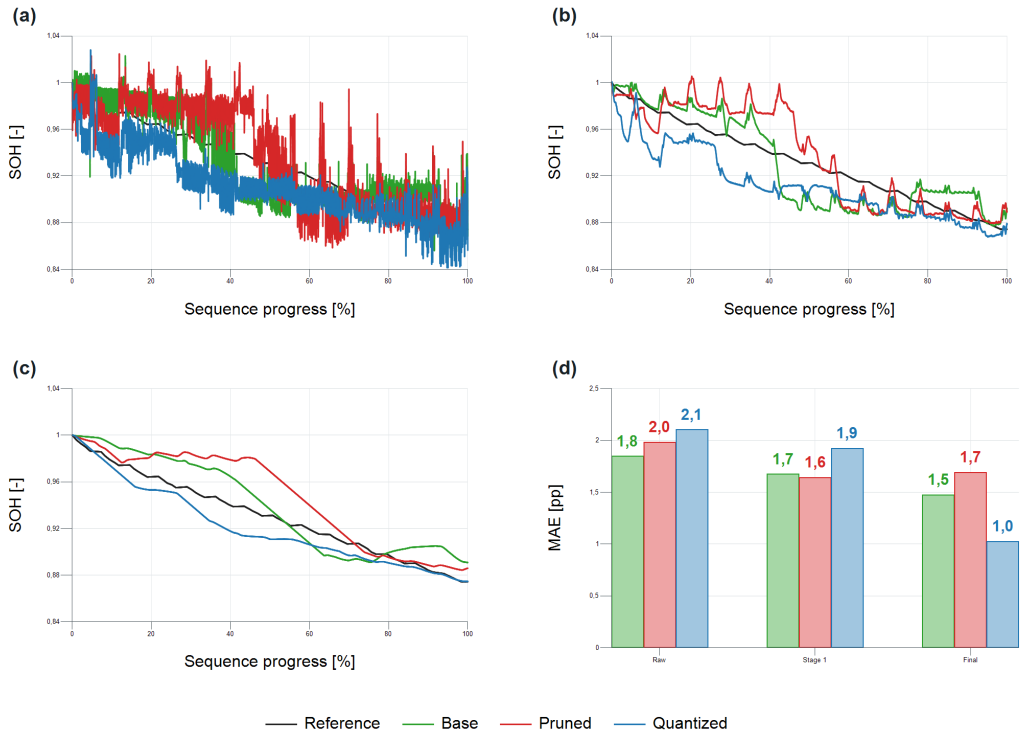


Figure A.6: SOH output and error at successive stages of the causal post-processing chain for Base, Pruned, and Quantized. Comparing raw, stage-1, and final trajectories shows that filtering affects absolute accuracy, ordering between variants, and temporal response.

A.2.4 Sensitivity of the Composite Utility Score

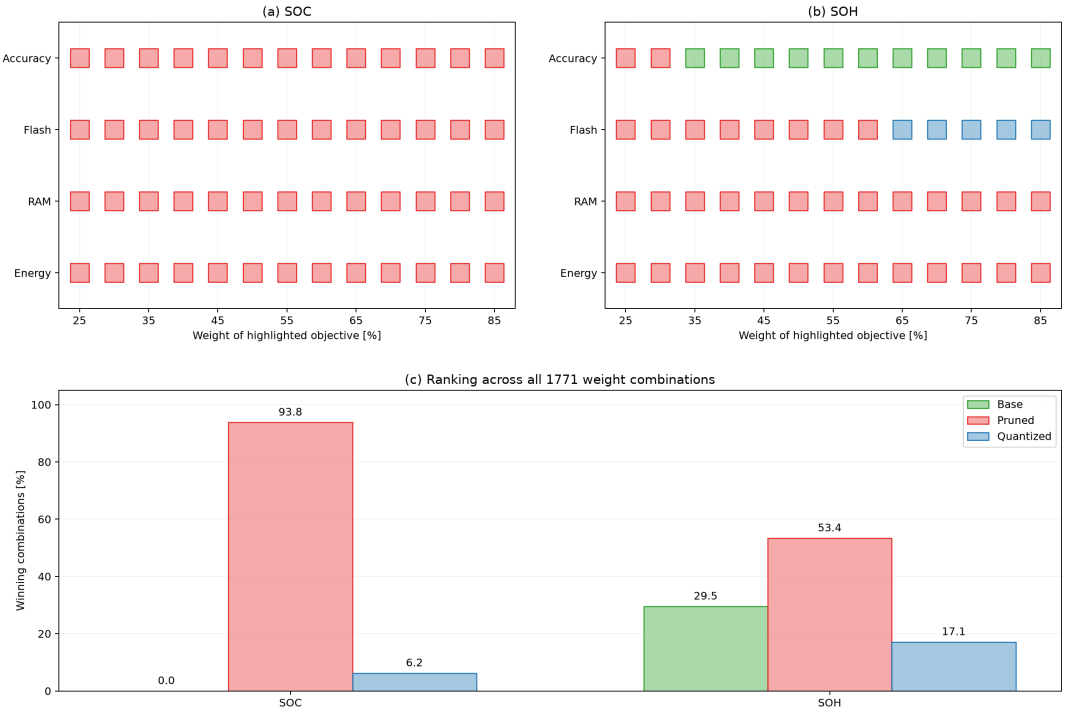


Figure A.7: Sensitivity of the composite utility ranking to different application priorities. The priority sweeps increase one weight while distributing the remaining importance equally. The ranking summary covers all 1771 weight combinations on the 0.05 simplex grid.

A.2.5 Transient Input-Buffer Fault Sensitivity

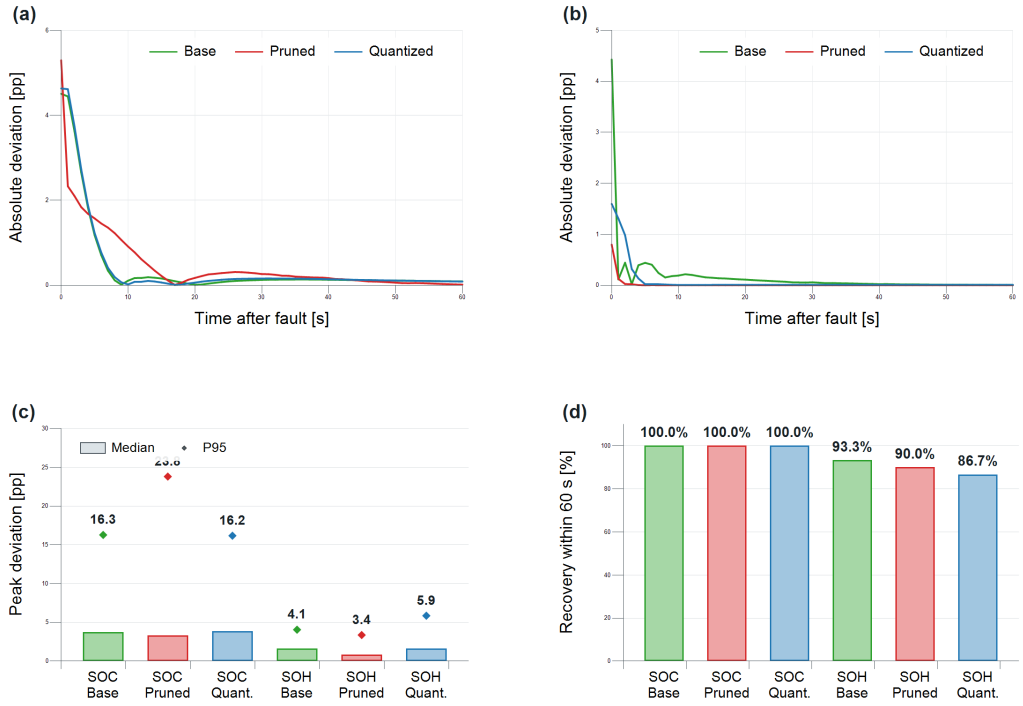


Figure A.8: Transient response of the clean and faulted model branches after a one-sample FP32 fraction-bit change in voltage, current, or temperature. The panels report peak disturbed-to-clean deviation, recovery within the following 60 s, and the target-error change over the complete 61-sample event window.

Table A.7: Median and P95 of the event-wise change in target MAE over each 61-sample fault window. All entries are percentage points.

Task	Model	Median ΔMAE_{61}	P95 ΔMAE_{61}
SOC	Base	0.0740	1.4235
SOC	Pruned	0.0464	1.5867
SOC	Quantized	0.0785	1.4314
SOH	Base	−0.0009	0.1224
SOH	Pruned	−0.00004	0.0533
SOH	Quantized	0.0012	0.1055

A.2.6 Model-Complexity Scaling Under Structured Pruning

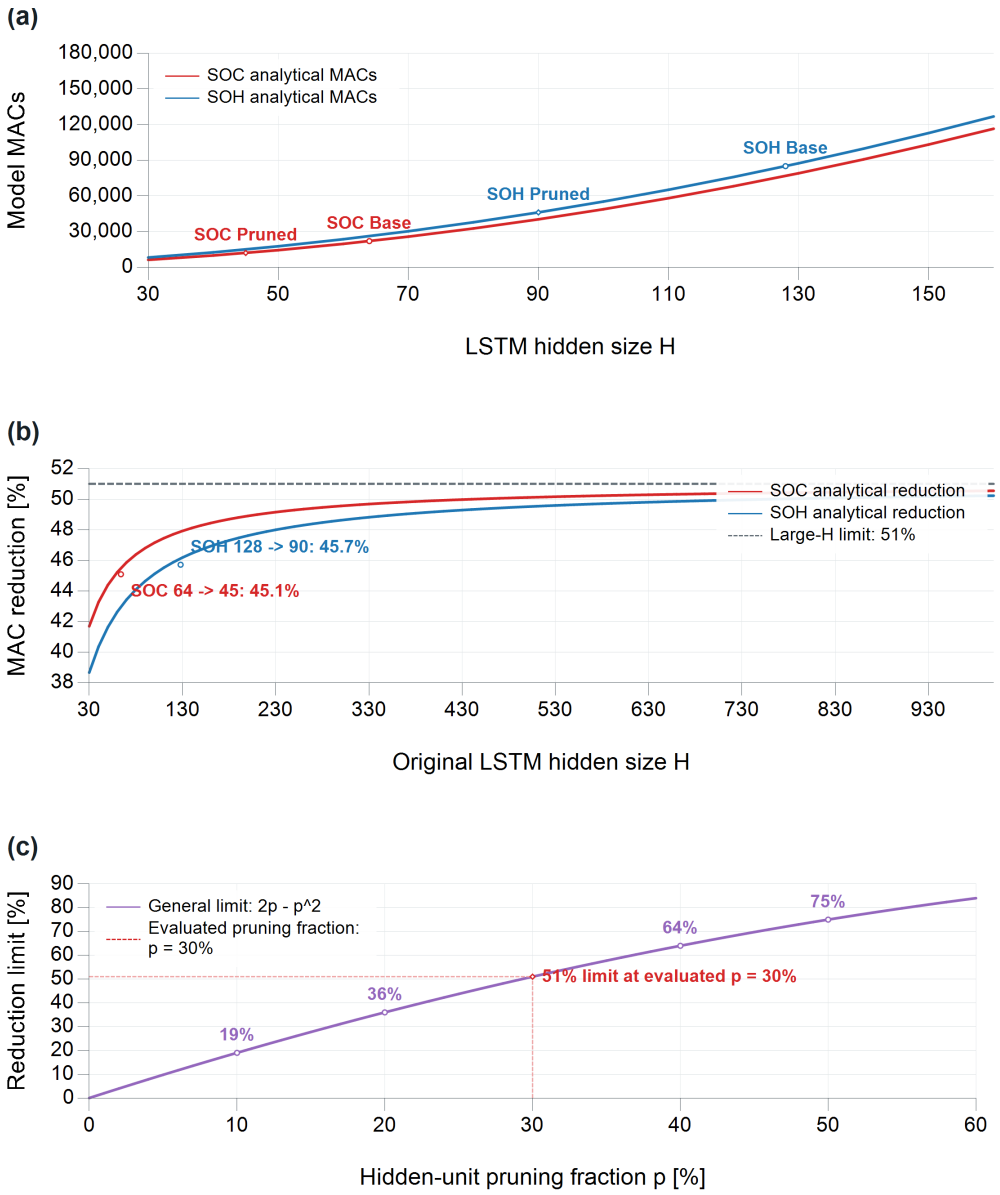
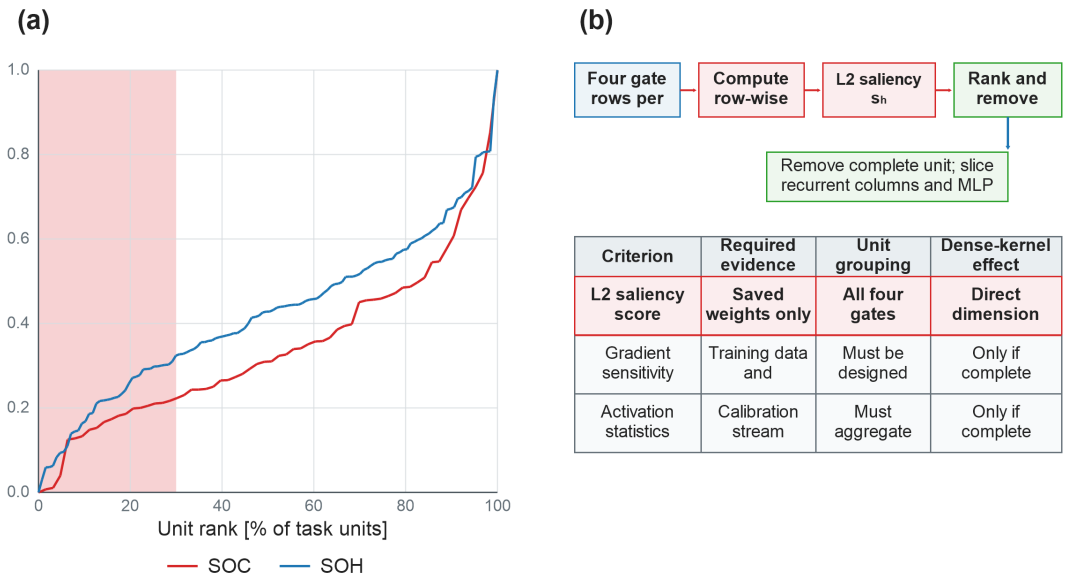


Figure A.9: Analytical MAC count and relative reduction for structured hidden-channel pruning. The finite-size curves follow Equation 7.10. Their large-model limits are $2p - p^2$, and the evaluated 30 % pruning configuration is marked for the SOC and SOH architectures.

A.2.7 Scope of the ℓ_2-Norm Pruning Criterion



Criterion	Required evidence	Unit grouping	Dense-kernel effect
L2 saliency score	**Saved weights only**	**All four gates**	**Direct dimension**
Gradient sensitivity	Training data and	Must be designed	Only if complete
Activation statistics	Calibration stream	Must aggregate	Only if complete

Figure A.10: Definition and methodological scope of the L2 gate-group saliency used for hidden-channel selection. The diagram distinguishes the implemented deterministic criterion from gradient- and activation-based alternatives that would require additional data-dependent evaluation.

A.2.8 Precision Boundary of the Quantized Network

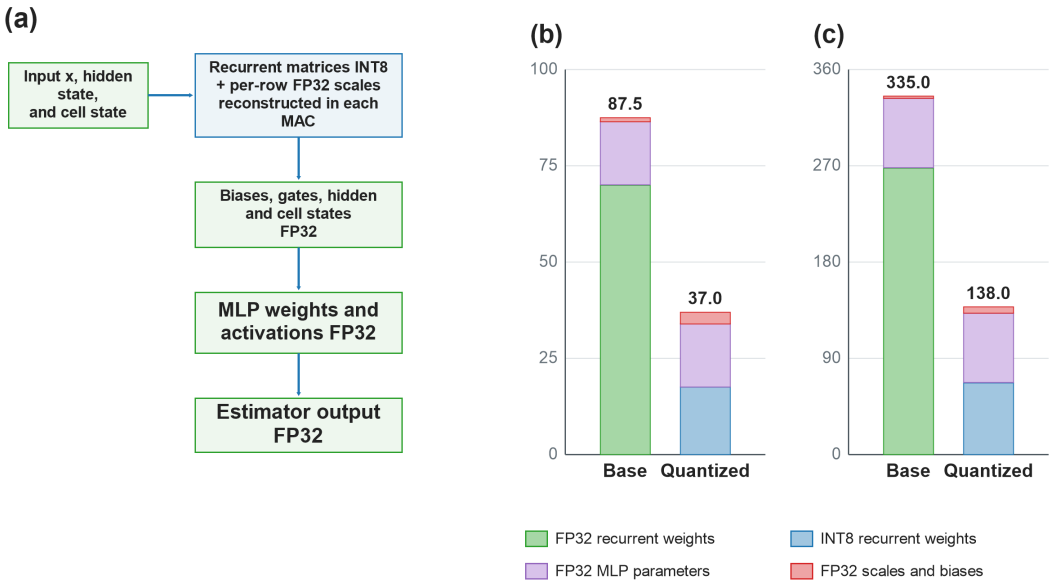


Figure A.11: Numerical path of the weight-only quantized implementation and composition of the stored model constants. Only the recurrent matrices use INT8 storage. Row scales, biases, recurrent states, activations, MLP parameters, and output processing remain in FP32.

A.2.9 Static Counts and Measured Quantized Runtime

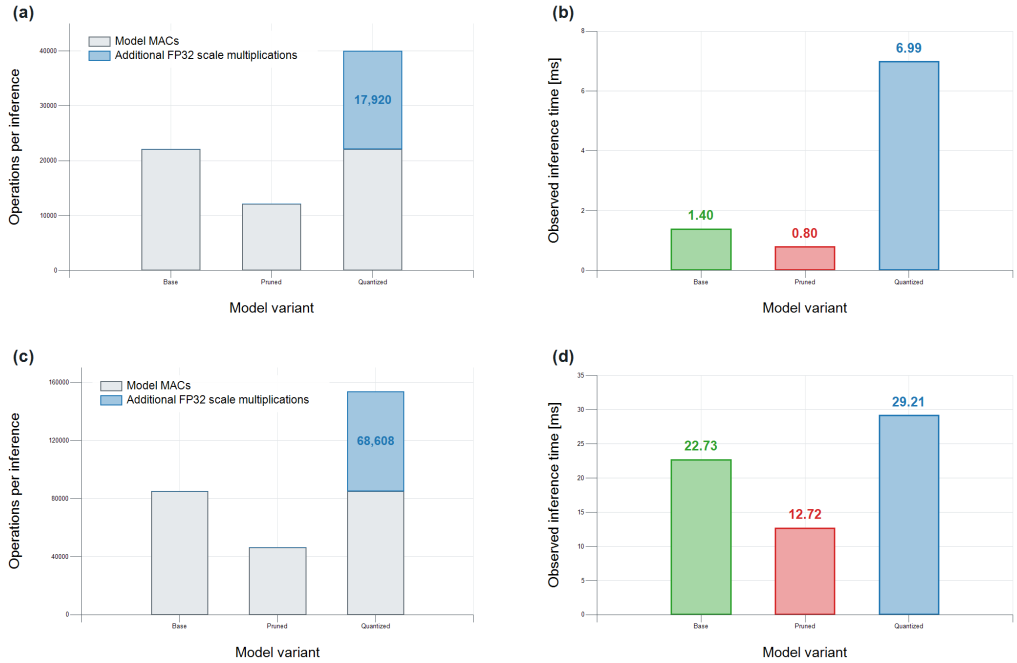


Figure A.12: Comparison of model MACs, additional source-level FP32 scale multiplications, and measured cycle-counter kernel times. The static count captures the explicit mixed-precision arithmetic but excludes conversion, indexing, loop, memory, activation-function, cache, and compiler effects.